비밀	기술 표준 ENGINEERING STANDARD	표준번호
작성팀		ES95486-02
전자네트워크개발팀		
제정일	기술 규격 ENGINEERING SPECIFICATON	PAGE
2025-10-16		270
제 목	UDS 진단통신 통합 사양서	

목 차

23		EESP0612	1. Routine 상세 명세 1) OTA-Ready와 UpdateEnable Condition의 Bootloader Optional 변경 2) App 업데이트 목적의 CPD의 Parameter 추가 3) OTA1.0 SwapActiveArea의 NRC24 조건 추가 4) Routine control의 Security access 필요내용 설명 보완 2. 진단사양서 3.1절의 Table에서 암전류(D0, D1) 세션을 삭제하고 DiagnosticSessionControl의 Subfunction에서도 삭제 또한 Safety 세션은 DiagnosticSessionControl에서만 명세하고 3.1절에서 삭제. 3. RequestDownload에 NRC24 조건 추가 4. Response format에 OEUK 13 파라메터 추가, OEUK의 권한은 ASK+Rollback임을 명세		2025-10-16	고인준
차수		EO NO.	변경내역	변경페이지	개정일자	작 성 자

원본보관처 표준통합관리시스템 (AUTOONE)	참고자료	구분	작성	검토1	검토2	승인
		결재	도록 사양 개 고인준 state 를 확 추가 정의	김정환	정명규	이동열
			2025-07-24 터셋 분리 재	2025-10-14	2025-10-16	2025-10-14

층 진단 DID 할당

기 술 표 준
ENGINEERING STANDARD

표준번호	PAGE (SHT/SHTS)
ES95486-02	1/1

차수		EO NO.	변경내역	변경 PAGE	개정일자	작성자
23		EESP0612	1. Routine 상세 명세 1) OTA Ready와 UpdateEnable Condition의 Bootloader Optional 변경 2) App 업데이트 목적의 CPD의 Parameter 추가 3) OTA1.0 SwapActiveArea의 NRC24 조건 추가 4) Routine control의 Security access 필요내용 설명 보완 2. 진단사양서 3.1절의 Table에서 암전류(D0, D1) 세션을 삭제하고 DiagnosticSessionControl의 Subfunction에서도 삭제 또한 Safety 세션은 DiagnosticSessionControl에서만 명세하고 3.1절에서 삭제. 3. RequestDownload에 NRC24 조건 추가 4. Response format에 OEUK 13 파라메터 추가, OEUK의 권한은 ASK+Rollback임을 명세 5. Functional address(18DB33F1 / DoIP : E000 / E400) 수신 시 제어기에서 타제어기로 요청하는 진단요청 중단을 하도록 사양 개정 6. F1C0 External Bootloader state를 확인하기 위한 Data parameter 추가 정의 7. Fault 진단 기능 DID 할당, 이더넷 물리계층 진단 DID 할당 8. SDV 아키텍처의 엔진조건 정의 / 기타 엔진조건 변경		2025-10-16	고인준

차수		EO NO.	변경내역	변경 PAGE	개정일자	작성자
22		EESO1769	1. 세션 컨트롤 7DF 10 01 대신 7DF 수신시 진단 요청 메시지 전송 중단(2.8절) 2. 고전압 전원 사용 제어기의 전원조건 예외/제약 추가 (3.1.1.3절) 3. RequestRoutineResult(0x03) subfunction에서 RequestRoutineStatus 허용 (Table 8-1) 4. 하나의 제어기에서 진단통신이 논리적으로 구분되어있는 경우, 엔진조건 E2E 기준 일치 (3.1.1.3절) 5. IVD의 CRC32 삭제 SHA256 이상으로 변경(Annex C, Annex K) 6. Security Access 표 4-81 No9의 Att_cnt +1 조건 삭제, DID F1C0 이름 변경 (Table 4-81, Annex C)		2024-12-12	고인준
21		EESO0587	1. Security access 사양 상세 명세 (4.3절) 2. DID F1C0 추가(Annex. C) 3. Routine control RID 0301 추가 (8.1.5절) 4. SW Unit 상한 확대 : F1B1~F1BF, F1C1~F1CF (Annex. C) 5. CAN-TP DLC 상세 명세 (Table 2-28) 6. Authentication NRC54 삭제 (4.5.5절) 7. OBD법규 제어기 대상 ClearDiagnosticInformation 서비스의 엔진조건 추가(3.1절) 8. OTA Ready Parameter(Optional) 추가(8.1.5.1절) 9. SW플랫폼 통신 Log 확인 DID 추가 (Annex C) 10. 29bit OBD 법규 진단 ID 영역 추가 (2.4.1절) 11. 사양서 기타 오기/누락사항		2024-08-07	고인준

차수		EO NO.	변경내역	변경 PAGE	개정일자	작성자
20		EESN0821	1. 미슬립 검출 기능 DID 적용 : AnnexC 2. OBD on UDS 법규 반영 -1)ReadDTCInformation Service Subfunction 추가 : Table 6-13 -2)DTC format Identifier(SAE_J2012-DA_DTCFormat_04) : Table 6-14 -3) ClearDiagnosticInformation Group 변경 : Annex D Table D-1 -4)Legislative use DID 영역 반영 : Annex C 3. 사양서 오기 수정 -ReadMemoryByAddress : Default session 삭제 3.1절 - RXSWIN figure 수정 : Figure J-13, J-14 - Authentication 4.5.6절 lengthOfCertificateClient 오기 수정 - Authentication 4.5절 규격 설명 삭제 - Authentication NRC list Annex A에 추가 - Addressing 범위 오기 수정 2.4.1.2절 4. RequestFileTransfer 서비스 추가 9.5절 5. Routine control 8.1.4절 Implementation rule의 세션 보안레벨 추가 6. EraseMemory parameter 추가 Table 8-96		2023-11-30	고인준

차수		EO NO.	변경내역	변경 PAGE	개정일자	작성자
19		EESN0487	1) 8.1.4 Routine control 수정 : ES98765-02 리프로그래밍 사양 제정에 따라 기존 리프로그래밍 관련 routine control Old 처리 SwapActiveArea 의 엔진조건 보충설명, NRC F4 추가(다운그레이드 관련) CheckProgrammingDependencies NRC F4 추가(다운그레이드관련) 2) 8.1.5 FoD관련 implementation rule 삭제 및 리프로그래밍 관련 Implementation rule 신규 : ES98765-02 리프로그래밍 사양 제정에 따라 리프로그래밍 관련 routine control 신규 생성 3) 표8-3. Request message routineIdentifier definition 수정 : 신규 Routine 추가 4) Annex C의 DID추가 F1C1~F1CA: Unit이 없을 때 F1C1값의 NRC31 케이스 추가 ED90~ED99 : 미 슬립 DID추가 F1A2 : Application 버전 DID 추가 5) 10.1 본 리프로그래밍 사양은 Legacy이며 ES98765-02 참조 문구 추가 6) Annex A NRC F4, NRC F5 추가 7) Annex J.4 29bit 진단 ID의 경우 Local RXSWIN 응답 예 추가		2023-07-12	고인준
18		EESN0189	[A] 2.4.1 Addressing 29bit CAN ID (Ethdiag / DoIP) 분리 [B] 3.1 ReadMemoryByAddress 보안 레벨 변경 with security access [C] 4.3 Security access OEUK 추가 subfunction(13h/14h) [D] 4.5 Authentication($29) 서비스 추가 [E] 9.1.1 Exception 추가		2023-02-14	고인준
17		EESM0695	1. Annex.C (RXSWINEOLRESULT) DID F1EE 추가 2. ReadDTCInformation 의 reportSupportedDTC(0x0A) Subfunction을 Optional에서 Mandatory로 변경		2022-11-04	고인준

차수		EO NO.	변경내역	변경 PAGE	개정일자	작성자
16		EESM0320	1) 표 9.1 IMPORTANT 추가 (Encrypting method) 2) Annex J RXSWIN 사양 상세 업데이트 (Example) 3) 2.4.1 절 29bit Addressing 추가 4) 3.1.1 엔진 Running/Stop 예시 업데이트 5) Annex C 진단기 표출 문구 통일을 위한 Notation 추가		2022-07-15	고인준
15		EESM0166	1. External test equipment 신규 사양 추가 2.9절 2. Engine 조건 변경 (Communication control의 IGN on 삭제) 3.1절 3. Routine control : Copy active area to, EraseTargetArea 신규 Parameter 속성(No specifying) 추가 (8.1.4.4절, 8.1.4.5절) 4. Annex C. 다음의 Data들은 ASCII code로 기록(DID F18C~F195) 5. 진단기의 DID표시 순서 변경 Table C-4.1. The order of DID in the diagnostic tool and Notation		2022-03-14	고인준
14		EESL0710	a) 3.1절 표3-1 과 표 3-1.1 의 불일치 수정 b) Annex C, F1C1~F1CA의 DID 이름 변경 'Hash -> IVD'		2021-11-01	고인준
13		EESL0584	1. Annex. C 진단 Data identifier 변경 : 제어기 Hash DID추가 (F1C1~F1CA) DID F1A0 User optional로 변경 2. Routine control CheckProgrammingDependancy : Downgrade protection을 위한 NRC 10 설명 추가 3. Engine condition 관련 3.1절 사양 추가 : 사양 상세화, Communication control 엔진 조건 추가(GN7 부터 적용)		2021-09-07	고인준

차수		EO NO.	변경내역	변경 PAGE	개정일자	작성자
12		EESL0492	1. Annex.J RXSWIN 사양 -RXSWIN format has been modified : 년도 제거 -Central Gateway RXSWIN management 추가 -RXSWIN 수집 조회 분리를 위한 Routine control 추가 2. Routine control (CheckProgrammingDependency) 에 누락된 NRC 72 추가 3. Annex.C Data Transmission Functional Unit Data Parameter Definitions 의 DID 설명 개선 4. Chapter (2.5.2.1) STmin 설명 개선 (내용은 기존과 동일)		2021-07-20	고인준
11		EESL0177	Chapter 4.3. Security access login lock time - Requiremet Cvt has been changed from Optional to Mandatory - Lock time and Attempts has been changed. Chapter 8.1.4.2 EraseTargetArea - Additional parameter has been added.(Memory block) Chapter Annex. J - RXSWIN diagnostics requirement		2021-04-15	고인준
10		EESL0083	Communication control -이 서비스는 게이트웨이 라우팅에 영향을 주지 않는다.		2021-03-10	고인준
9		EESL0032	1. FoD RoutineControl 요구사항 추가 2. OTA Swap active area DelayTime parameter 추가 3. Security access 및 RoutineControl 관련 잘못 된 표현 수정		2021-01-20	고인준
8		EESK0507	버전 관련 DID 변경. OTA 관련 routine control parameter 추가		2020-07-01	고인준
7		EESJ1207	2.6.6절 요구사항 C 삭제 (ECU Reset 삭제)		2019-12-30	박필용 Pilyong Park
6		EESJ1055	1. 게이트웨이 인증 NRC 추가 (F0~F3) 2. OTA DID 수정 3. CANFD TP 사양 문구 오류 수정 4. $22 NRC13/31 ISO 개정 사양 반영 5. $2A 응답 메시지 ISO 개정 사양 반영		2019-11-29	박필용 Pilyong Park

차수		EO NO.	변경내역	변경 PAGE	개정일자	작성자
5		EESJ0547	1. OTA 요구사양 반영 - 진단 세션별 서비스 지원 목록 수정 - Multiple Concurrent Request 예외 조건 추가 - OTA 백그라운드 전송 후 Auto Reset 발생하지 않도록 문구 추가 - OTA용 RoutineControl 명세 추가 - OTA용 DID 추가 - 압축/암호화 방식 적용을 위한 dataFormat 정의 2. DiagnosticSessionControl 문구 명확화		2019-07-04	박필용 Pilyong Park
4		EESI0984	1. SecurityAccess(27 hex) service - Unlocked flash level 상태 추가 2. RequestDownload(34 hex) service - DataFormatIdentifier 압축방식 및 암호화 사양 추가 3. Annex D 5 - DTCStatusAvailabilityMask 정의 추가 4. InputOutputControlByIdentifier(2F hex) service - controlState definition 표 삭제		2018-11-22	박필용 Pilyong Park
3		EESI0650	1. Diagnostic CAN ID 신규 추가 제어기 및 변경사항 일괄 반영 2. 로트 추적 시스템 구축 강화 위한 Data ID(DID) 명확화 및 조건 변경 : F187(차량 제조사 Part Number) / F18B(ECU 제조 일자) / F18C(ECU Serial Number) / F1A1[신규](ECU Supplier 코드) 3. Extended Session에서 Non-Default Session으로 천이 시 Security Unlock 상태 유지 (ICU, ESU, CCU 대상) 4. 이더넷 제어기 D-CAN(500kbps) 이용한 일반 진단통신 멀티프레임 전송 시, Flow Control 내 STmin값 10ms 이상 설정 5. 진단기 P2_Client 값 증대 : [기존] 50ms → [변경] 80ms		2018-09-13	박필용 Pilyong Park
2		EESI0109	Annex. H - 신규 제어기 진단 CAN ID 추가 및 일부 제어기 명칭 변경		2018-03-08	정호진

차수		EO NO.	변경내역	변경 PAGE	개정일자	작성자
1		EESI0061	1. 신규 제어기 CAN ID 신규 반영 2. 이더넷 관련 내용 추가 : 범위, ISO 참고 문서, H/KMC 참고 문서,Protocol 정의 등 추가 3. 부정응답 메시지 구조 추가 4. 진단 서비스 엔진 조건 예외처리 문구 추가		2018-01-22	정호진
0		EESH0520	신규 UDS 통합사양 제정		2017-05-19	정호진

1. 목적

본 문서는 진단장비(Client)가 CAN 통신을 통하여 현대기아자동차의 모든 차량 내 제어기(Server)에 구현된 진단 기능들을 제어하기 위해 필요한 진단 서비스에 대한 요구사항을 정의한다. CAN-FD 적용 차종은 CAN / CAN-FD / 이더넷 제어기 상관없이 본 UDS 진단통신 통합사양서를 준수한다. 본 사양을 적용한 제어기는 ES95486-12 진단통신단품평가 사양서에 따라 평가하여야 한다.

NOTE) 이 사양서는 ISO 14229-1/2 과 ISO 15765-2/3 에 기초하여 작성되었다.

본 문서는 진단 외 CAN 혹은 CAN-FD 통신 메시지에 대한 정의는 포함하지 않는다.

또한, 본 문서는 CARB-Mode 테스트(배출가스 관련 진단 서비스들)를 별도로 포함하고 있지 않다. CARB-Mode 테스트에 대한 사항은, ISO 15031-5 (Road vehicles – Communication between vehicle and external equipment for emissions-related diagnostics – Part 5: Emission-related diagnostic services) 또는 SAE J1979 (E/E Diagnostic Test Modes – Equivalent to ISO/DIS 15031-5)와 같은 OBD 법규 관련 문서를 반드시 만족하여야 한다.

Service Identifier (hex)	Service Type
00 – 0F	Reserved (OBD service request)
10 – 3E	HMC/KMC service request
3F	Reserved
40 – 4F	Reserved (OBD service response)
50 – 7E	HMC/KMC service positive response
7F	Negative response service identifier
80 – BE	HMC/KMC service request
BF	Reserved
C0 – FE	HMC/KMC service positive response
FF	Reserved

표 1-1. Service Identifier (SI) Overview

국문 사양과 영문 사양 사이에 해석의 차이가 있는 경우, 영문 사양이 우선한다.

1.1 참조 문서

본 문서와 진단 통신 프로토콜에 대한 설명들은 다음의 표준문서들을 따르고 있다. 본 문서에서 기술되지 않은 구현 관련된 상세 사항들은 다음 문서들을 참조해야 한다.

1.1.1 ISO 문서

ISO 15765-2 — Road vehicles – Diagnostics Communication over CAN (DoCAN) – Part 2 : Transport protocol and network layer services : 2016

ISO 15765-3 — Road vehicles – Diagnostics on CAN – Part 3 : Implementation of unified diagnostic services

	(UDS on CAN) : 2004
ISO 14229-5	Road vehicles – Unified Diagnostic Services (UDS) – Part 5 : Unified diagnostic services on Internet Protocol Implementation (UDS on IP) : 2013
ISO 14229-3	Road vehicles – Unified Diagnostic Services (UDS) – Part 3 : Unified diagnostic services on CAN implementation (UDS on CAN) : 2012
ISO 14229-2	Road vehicles – Unified diagnostic services (UDS) – Part 2 : Session layer services: 2013
ISO 14229-1	Road vehicles – Unified diagnostic services (UDS) – Part 1 : Specification and requirements : 2013
ISO 15031-6	Road vehicles – Communication between vehicle and external equipment for emissions-related diagnostics – Part 6: Diagnostic trouble code definitions : 2015
ISO 11898-1	Road vehicles – Controller area network (CAN) – Part 1: Data link layer and physical signaling : 2015
ISO 11898-2	Road vehicles – Controller area network (CAN) – Part 2: High-speed medium access unit : 2016
ISO 13400-2	Road vehicles – Diagnostic communication over Internet Protocol (DoIP) Part 2 : Transport protocol and network layer services
ISO 13400-3	Road vehicles – Diagnostic communication over Internet Protocol (DoIP) Part 3 : Wired vehicle interface based on IEEE 802.3
ISO 13400-4	Road vehicles – Diagnostic communication over Internet Protocol (DoIP) Part 4 : Ethernet-based high-speed data link connector

1.1.2 SAE 문서

SAE J1962	Diagnostic Connector : 2016
SAE J2012	Diagnostic Trouble Code Definitions : 2016

1.1.3 HMC/KMC 문서

ES 95480-00	HMC/KMC Engineering Standard – High Speed Controller Area Network (CAN) Design Specification
ES 95480-02	HMC/KMC Engineering Standard –

	Controller Area Network – Flexible Data Rate (CAN-FD) Design Specification
ES 95489-01	HMC/KMC Engineering Standard – ECU Secure Diagnosis Functional Specification
ES 96595-05	HMC/KMC Engineering Standard – IP Middleware Specification
ES 95486-03	HMC/KMC Engineering Standard – Ethernet IP Middleware Diagnostic Communication Specification
ES 95486-12	HMC/KMC Engineering Standard – Diagnostic Communication Test Specification for Electronic Control Unit

1.1.4 문서 구조

본 문서는 통신 컨셉을 구현하기 위한 일반적인 사양들에 대해 정의하고 있다. 각 제어기별로 본 문서에 정의된 모든 항목을 준수할 필요는 없다.

각 제어기별 사양서는 아래 사항에 대한 내용을 포함해야 한다 :

- 본 문서에 선택사항으로 정의된 내용들에 대한 해당 제어기의 적용 현황
- 해당 제어기의 송수신 상세 정보 (예: 시그널 및 메시지)

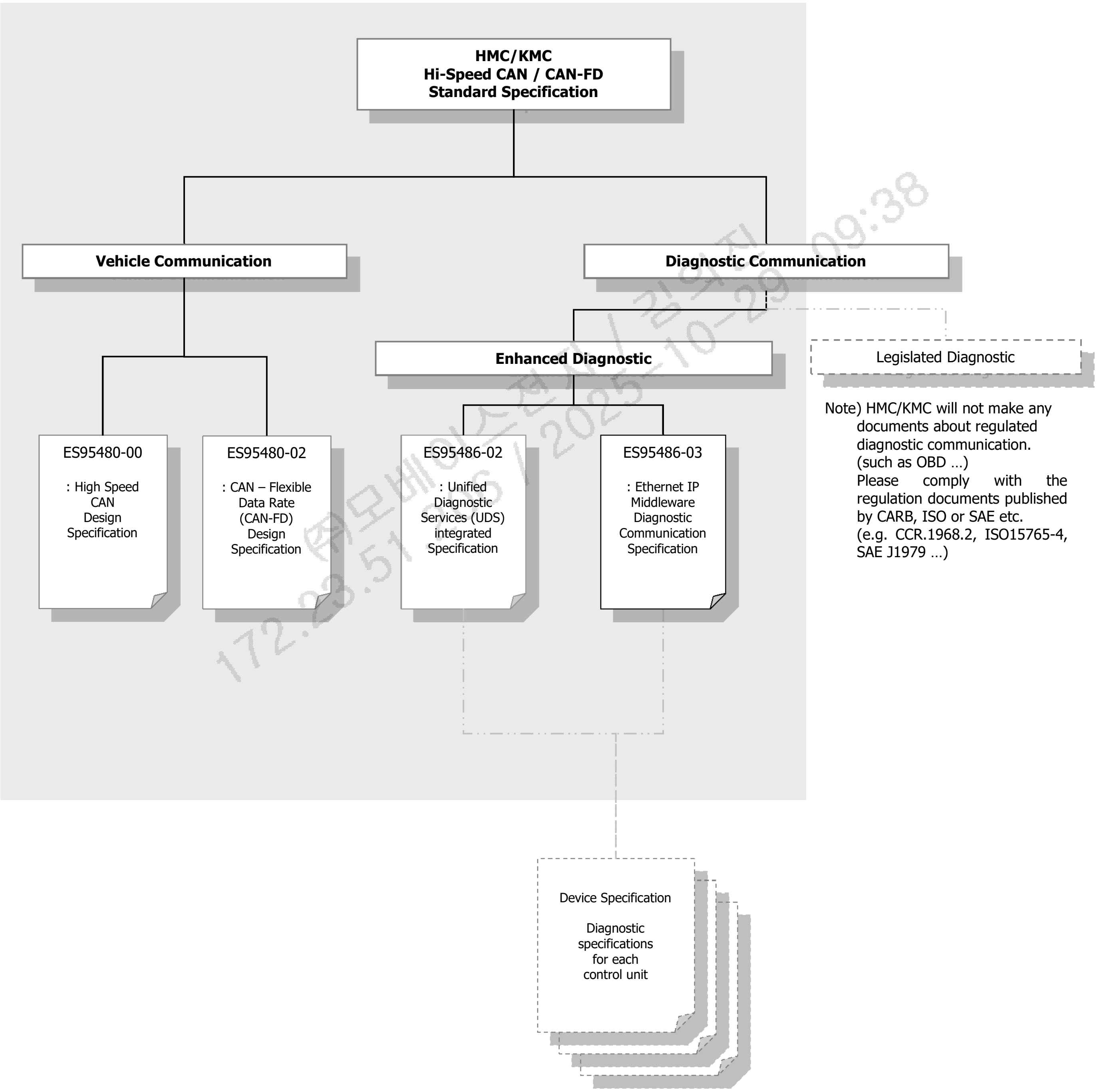


그림 1-1. HMC/KMC Standard Documents Structure

1.2 기호 정의

M : Mandatory [이 파라미터는 반드시 만족하여야 한다.]

U : User Optional [이 파라미터는 사용자의 의도에 따라 사용 여부를 선택할 수 있다.]

C : Conditional [이 파라미터는 서비스에 포함된 다른 파라미터에 따라 사용여부가 결정된다.]

CVT : Conventions

XXh : Hexadecimal number

XXd : Decimal number

XXb : Binary number

1.3 약어 정의

BS : Block Size

CAN : Controller Area Network

CAN-FD : Controller Area Network – Flexible Data Rate

CF : Consecutive Frame

DL(C) : Data Length (Code)

ECU : Electrical Control Unit (a generic term for any electrical control unit)

FC : Flow Control

FF_DL : First Frame Data Length

FS : Flow Status

ID : Identifier

N_Ar : Network layer timing parameter Ar (r = receiver of the message)

N_As : Network layer timing parameter As (s = sender of the message)

N_Br : Network layer timing parameter Br

N_Bs : Network layer timing parameter Bs

N_Cr	: Network layer timing parameter Cr
N_Cs	: Network layer timing parameter Cs
N_PCI	: network Protocol Control Information
N_PDU	: Network protocol data unit
N_USData	: Network layer Unacknowledged Segmented DATA transfer service name
SF_DL	: Single Frame Data Length
SN	: Sequence Number
STmin	: Separation Time Minimum

2. 프로토콜 정의

2.1 CAN 물리 계층 (CAN Physical Layer)

진단 CAN통신의 물리계층은 ES95480-00를 만족하여야 한다.

2.2 CAN-FD 물리 계층 (CAN-FD Physical Layer)

진단 CAN-FD통신의 물리계층은 ES95480-02를 만족하여야 한다. CAN-FD 통신라인에는 CAN-FD 프로토콜 메시지만 허용한다. (CAN-FD 제어기 진단통신은 CAN-FD 통신 (Tx, Rx 모두) 으로 수행한다.)

2.3 이더넷 물리 계층 (Ethernet Physical Layer)

진단 이더넷 통신의 물리계층은 ES96595-00를 만족하여야 한다.

2.4 네트워크 계층 (Network Layer)

외부 진단 장비와 차량 내 제어기들의 네트워크 계층은 ISO 15765-2를 만족하여야 한다.
추가적인 내용은 다음과 같다.

2.4.1 Addressing

진단 CAN 통신에서는 ISO 15765-2에서 정의된 Normal Addressing 구조와 11-bit CAN Identifier , 29-bit CAN Identifier를 사용한다.
- 11-bit CAN identifiers with normal addressing
- 29-bit CAN identifiers with normal addressing

진단 통신 Addressing 방식에는 Physical Addressing과 Functional Addressing이 있다.
- Physical Addressing (1 대 1 통신 방식) 은 네트워크 계층의 모든 메시지들에 대하여 지원되어야 한다.
- Functional addressing (1 대 다 통신 방식) 은 Single Frame 전송에서만 지원되어야 한다.
  ※ Functional addressing (07DF) Request에 대한 제어기 (Server) Response는 Physical Addressing 적용 (제어기 고유 Response ID) 되므로 정상적으로 Multi-Frame 전송해야 함.
- 29bit에는 일반(Non OBD) 제어기와 OBD 제어기의 addressing으로 구분된다. OBD (SAE J1979 준수) 제어기는 CAN identifiers 11bit 영역 또는 CAN identifiers 29bit (Non-OBD Controllers) 영역을 일반 진단 목적으로 사용할 수 있다. CAN identifiers 29bit - OBD Controller(SAE J1979)를 사용하는 제어기는 11bit OBD(7DF) 진단 요청에 대한 응답을 하여서는 안된다. 상세한 내용은 ISO 15765-4 “External test equipment initialization sequence.”를 참조한다.

2.4.1.1 CAN identifiers 11bit

- Physical addressing ID 범위 : 700-7FF, 580-5FF
- Functional addressing ID : 7DF

2.4.1.2 CAN identifiers 29bit

Non OBD Controller

- Physical addressing ID 범위 : 17FF0001~17FFFE00
- Functional addressing ID : 7DF
- Physical addressing ID 포맷 :
  17FF[TA 1byte][SA 1byte]

*TA : Target address
*SA : Source address

※ SA 할당 규칙
0x00 : 진단기
0x01~0x10 : Reserved
0x11~0xEF : 각 제어기 고유 값 할당
0xF0~0xFF : Reserved

예) 제어기의 Source Address : 0xCA
진단 Request ID : 0x17FFCA00
진단 Response ID : 0x17FF00CA

OBD Controller(SAE J1979)

- Physical addressing ID 범위 : 18DA####
- Functional addressing ID : 18DB33F1
- Physical addressing ID 포맷 :
  18DA[TA 1byte][SA 1byte]

※ SA 할당 규칙
0x00~0x10 : Reserved
0x11~0x32 : 각 제어기 고유 값 할당
0x33 : Reserved for functional group
0x34~0xEF : 각 제어기 고유 값 할당
0xF0~0xFF : Reserved (F1: Reserved for external test equipment)

2.4.1.3 CAN ID 2byte 변환

이더넷 진단 Logical address는 2byte이며 CAN ID를 2byte로 변환하여 Logical address로 사용한다. 또한 이더넷 진단 Logical address 외에도 local RXSWIN 응답 포맷 CAN ID 또한 2byte 변환 rule이 필요하다.

11bit CAN Identifier
11bit를 16bit로 확장하기 위해 MSB를 0으로 padding 한다.
이더넷 진단을 위한 Logical address 2byte 변환에는 Request ID가 사용된다
예)16 진수 : 0x700$_{(16)}$ -> 0x0700$_{(16)}$
2 진수 : 0b111 0000 0000$_{(2)}$ -> 0b0000 0111 0000 0000$_{(2)}$

29bit CAN Identifier

a) Ethdiag logical address

Request ID : 0x17FFCA00
Response ID : 0x17FF00CA
Request Logical address : CA00
Response Logical address : 00CA

Example)

CAN to	Ethdiag			
CAN ID	Payload type	REQ/RES	TargetAddress	SourceAddress
11bit REQ/RES : 700 / 708	8001	REQ	0x0700	–
		RES	0x0708	–
	FCBC	REQ	0x0700	0x1000
		RES	0x1000	0x0708
29bit REQ/RES : 17FFCA00 / 17FF00CA	8001	REQ	0xCA00	–
		RES	0x00CA	–
	FCBC	REQ	0xCA00	0x1000
		RES	0x1000	0x00CA

Ethdiag to				CAN
Payload type	REQ/RES	TargetAddress	SourceAddress	CAN ID
8001	REQ	0x07DF	–	0x7DF
	REQ	0x0700	–	0x700
	RES	0x0708	–	0x708
	REQ	0xCA00	–	0x17FFCA00
	RES	0x00CA	–	0x17FF00CA
FCBC	REQ	0xCA00	0x1000	–
	RES	0x1000	0x00CA	–

b) DoIP logical address

29bit 를 1byte 로 변환하기 위해 필요한 Bit 는 아래 2 진수를 참고한다
2 진수 : 1 01DD DDDD DDDD [TA] [SA]
*D 는 Don't care

최상위 3bit 와 제어기 Address(Source Address)의 조합으로 2bytes 변환한다
0(Fixed 1bit) + 101(MSB 3bit) + 0000(Reserved 4bit) + SA 1byte

예) Source address 가 0xCA 인 제어기의 경우
진단 Request : 0x17FFCA00
진단 Response : 0x17FF00CA
이더넷 진단을 위한 Logical address : 0x50CA

2.4.2 표준 진단 CAN ID

Annex. H 참조

2.4.3 Data Stream

2.4.3.1 Single Frame Transmission

Extended 또는 Mixed Addressing 의 경우 최대 6 바이트, Normal Addressing 의 경우 최대 7 바이트 데이터는, Single Frame 을 이용하여 메시지를 전송할 수 있다. (그림 2-1 참조)

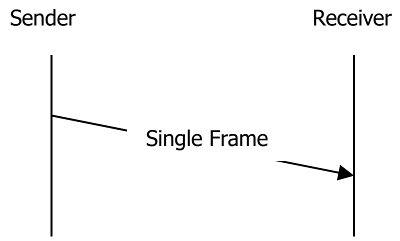


그림 2-1. Example of unsegmented message

2.4.3.2 Multi Frame Transmission

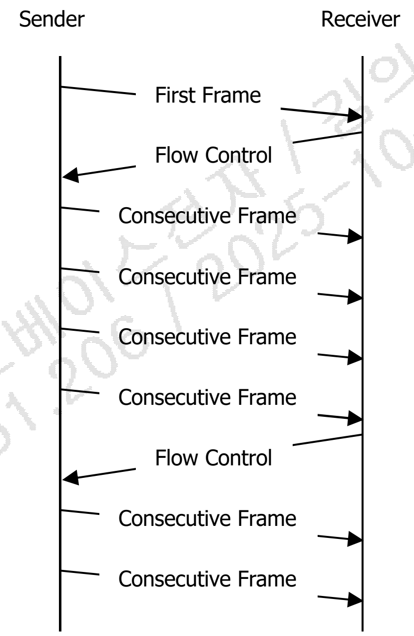


그림 2-2. Example of segmented message

Single Frame 으로 보낼 수 있는 데이터 이상을 송신하는 경우에는 데이터를 나누어 다중 메시지로 송신하게 된다. 대용량 데이터 수신은 다중 메시지의 수신과 수신된 데이터 바이트의 연속된 재조합으로 이루어진다. 이러한 다중 메시지는 First Frame 과 Consecutive Frame 으로 구성된다. (그림 2-2 참조)

다중 메시지 수신 시, 수신측은 Flow Control Protocol Data Unit 을 이용하여 수신측의 성능에 맞게 송신측이 데이터를 전송할 수 있도록 조정할 수 있다. 7 (또는 6) 이상의 데이터 바이트를 가지는 메시지는 다음과 같이 분리 되어 진다.

- Extended 또는 Mixed Addressing 에서 최초 5 데이터 바이트를, Normal Addressing 의 경우 최초 6 데이터 바이트를 포함하는 “First Frame Protocol Data Unit”, 그리고
- 각각 6 또는 7 데이터 바이트를 포함하는 하나 이상의 “Consecutive Frame Protocol Data Unit”. CF N_PDU 은 남아있는 데이터 바이트만을 포함하며, 따라서, 6 또는 7 데이터 바이트 길이보다 작다.

2.4.4 CAN 메시지 구조

DLC는 항상 8로 설정되며, CAN 프레임은 항상 8 바이트의 데이터를 가져야 한다. 따라서, 모든 프레임들은 비어있는 데이터 프레임을 Filler Byte로 채워야 한다.

- Request 메시지는 Filler Byte 55 hex로 채워진다.
- Response 메시지는 Filler Byte AA hex로 채워진다.

CAN 프레임의 8 바이트는 다음과 같이 정의 된다.

N_AI	N_PCI	N_Data
Address Information (Not used)	Protocol Control Information	Databytes for Application
1 – 8 bytes data of CAN Data Field		

표 2-3. Data bytes of CAN data field

Address Information(N_AI)은 Normal Addressing 모드 에서 필요하지 않으므로, Normal Addressing에서의 상기 8 데이터 바이트는 아래와 같이 정의 된다.

	Byte 1					Byte 2	Byte 3	Byte 4 ~ Byte 7	Byte 8
	N_PCItype				Bit 3-0				
	7	6	5	4					
Single Frame	0	0	0	0	SF_DL	DATA 1	DATA 2	...	DATA 7
First Frame	0	0	0	1	FF_DL		DATA 1	...	DATA 6
Consecutive Frame	0	0	1	0	SN	DATA 1	DATA 2	...	DATA 7
Flow Control	0	0	1	1	FS	BS	STmin	N/A	

표 2-4. Mapping of parameters and data into CAN frame – Normal Addressing

Hex	Description
0	Single Frame 미분할(unsegmented) 메시지의 경우, 네트워크 계층 프로토콜은 PCI byte 안에 메시지 길이값 만을 포함한 형태로 네트워크 프로토콜의 최적화된 구현을 제공한다. Single Frame (SF) 은 단일 CAN 프레임안에 들어갈 수 있는 메시지 전송을 지원하는 데 사용된다.
1	First Frame First Frame (FF) 은 단일 CAN 프레임안에 들어갈 수 없는 메시지, 즉 분할(segmented) 메시지를 전송하는 데에만 사용된다. 분할 메시지의 첫 번째 프레임은 FF 에 들어간다. FF 수신 시, 네트워크 계층 수신 측은 분할 메시지 조합을 시작한다.
2	Consecutive Frame 분할 데이터를 보낼 때, FF 뒤에 연속되는 프레임은 Consecutive Frame (CF)에 들어간다. CF 수신 시, 네트워크 계층 수신 측은 모든 메시지가 수신될 때까지 수신된 데이터 바이트를 재조합한다. 수신 측은 메시지의 최종 프레임이 에러 없이 수신 된 후, 조합된 메시지를 인접한 상위 프로토콜 계층으로 전달한다.

3	Flow Control Flow Control (FC)의 목적은 수신측에 전달되는 CF N_PDU 속도를 조절하는 것이다. 이 기능을 지원하기 위해 세가지 타입의 FC 프로토콜 제어 정보가 정의된다. 프로토콜 제어 정보 필드에 표현되는 이 타입은 이하 Flow Status (FS)라고 표현된다.

표 2-5. Definition of N_PCItype values

N_PCI	Description	
SF_DL	Single Frame Data Length	
	Value (HEX)	Description
	0	Reserved
	1 ~ 6	SF_DL 은 N_PCI byte #1 값의 Low Nibble 에 표현된다. 서비스 파라미터 <Length> 값이 할당된다.
	7	SF_DL = 7 은 Normal Addression 에서만 허용된다.
	8 ~ F	Invalid
FF_DL	First Frame Data Length	
	Value (HEX)	Description
	0 ~ 6	Invalid
	7	FF_DL = 7 은 Exteded 또는 Mixed Addressing 에서만 허용된다.
	8 ~ FFF	분할 메시지 길이는 12-bit 값 FF_DL 로 표현된다. 여기서 LSB 는 N_PCI byte #2 의 Bit 0 이고, MSB 는 N_PCI byte #1 의 Bit 3 이다. 분할 메시지 길이의 최대값은 4095-byte 이다. 서비스 파라미터 <Length> 값이 할당된다.
SN	Sequence Number	
	SN 은 N_PCI byte #1 값의 Low Nibble 에 표현된다. 값의 범위는 0 ~15 이다. - 모든 분할 메시지에 대해, SN 값은 0 부터 시작한다. - FF 의 SN 값은 0 에 해당한다. - FF 뒤에 바로 따라오는 첫번째 CF 의 SN 값은 1 이다. - SN 값은 분할 메시지가 전송되는 동안, 매 신규 CF 에 대해 1 씩 값이 증가한다. - SN 값이 15 가 되면, 그 다음 CF 에 대해 0 으로 돌아간다.	
FS	Flow Status	
	파라미터 Flow Status (FS)는 메시지 전송을 계속 진행할 수 있는지 여부를 나타낸다.	
	Value (HEX)	Description
	0	ContinueToSend (CTS) FC CTS 파라미터 값은 N_PCI byte#1의 Low Nibble에 0으로 표현되며, 송신자가 CF 전송을 재개하도록 한다. 이 값은 수신자가 BS 최대값 CF를 수신할 준비가 되어 있다는 것을 의미한다.
	1	Wait (WT) FC WT 파라미터 값은 N_PCI byte #1의 Low Nibble에 1로 표현되며, 전송자가 신규 FC N_PDU를 계속 기다리고, N_BS 타이머를 재시작하도록 한다.

	2	Overflow (OVFLW) FC OVFLW 파라미터 값은 N_PCI byte #1의 Low Nibble에 2로 표현되며, 전송자가 분할 메시지 전송을 중단하게 하고, N_USData가 파라미터 <N_Result>=N_BUFFER_OVFLW 값으로 서비스 콜을 확정하도록 한다. (ISO 15765-3 참조) N_PCI FS 파라미터 값은 FF N_PDU 다음에 오는 FC N_PDU에서만 전송되는 것이 허용되며, 수신된 FF N_PDU의 메시지 길이값 FF_DL이 수신자의 버퍼 사이즈를 초과하는 경우에만 사용될 수 있다.
BS	Block Size	
	BS 단위는 블록당 CF N_PDU 개수이다.	
	Value (HEX)	Description
	00	BS 파라미터 값 0은 분할 메시지를 전송하는 동안, 더 이상 FC 프레임이 전송되지 않는다는 것을 송신자에게 알리기 위해 사용된다. 송신 네트워크 계층 엔터티는 수신 네트워크 계층 엔터티로부터 더 이상의 FC 프레임을 위한 중단 없이 모든 남아있는 CF을 전송해야 한다.
	01 ~ FF	이 범위의 BS 파라미터 값은 수신 네트워크 엔터티로부터 중간에 FC 프레임 없이 수신할 수 있는 최대 CF 개수를 송신자에게 알리기 위해 사용된다.
STmin	Minimum Separation Time	
	STmin 파라미터 값은 CF 사이에 허용되는 최소 시간 간격을 정의한다.	
	Value (HEX)	Description
	00 ~ 7F	0 ms ~ 127 ms, 00 hex ~ 7F hex 범위의 STmin 단위는 ms 이다.
	F1 ~ F9	100 µs ~ 900 µs, F1 hex ~ F9 hex 범위의 STmin 단위는 100 µs 이다. 여기서, F1 hex 는 100 µs 이고, F9 hex 는 900 µs 이다.
	FA ~ FF	Reserved

표 2-6. Definition of N_PCI

2.4.5 CAN-FD 메시지 구조

DLC는 8 이상으로 설정되며, CANFD 프레임은 최대 64 바이트까지 사용 가능하다.
DLC는 아래 표와 같이 설정 가능하다. 단, 제어기에서 진단 응답 메시지 송신 시, DLC는 반드시 8로 고정해야 한다.
진단기가 CANFD 네트워크에 직결되는 경우, 진단기가 송신하는 Multiple Frame은 마지막 CF을 제외하고 DLC 64로 고정한다.

Data Length Code (DLC)				Number of data bytes (CAN_DL)
1	0	0	0	8
1	0	0	1	12
1	0	1	0	16
1	0	1	1	20
1	1	0	0	24
1	1	0	1	32
1	1	1	0	48
1	1	1	1	64

표 2-7. CAN_FD data length

위 테이블에 명기된 CAN_DL에 비어있는 데이터 프레임은 Filler Byte로 채워야 한다.

- Request 메시지는 Filler Byte 55 hex로 채워진다.
- Response 메시지는 Filler Byte AA hex로 채워진다.

CAN-FD 프레임은 다음과 같이 정의 된다.

N_AI	N_PCI	N_Data
Address Information (Not used)	Protocol Control Information	Databytes for Application
1 – 64 bytes data of CAN-FD Data Field		

표 2-8. Data bytes of CAN-FD data field

Address Information(N_AI)은 Normal Addressing 모드 에서 필요하지 않으므로, Normal Addressing에서 N_PCI 바이트는 아래와 같이 정의 된다.

N_PDU name	N_PCI bytes									
	Byte 1					Byte 2	Byte 3	Byte 4	Byte 5	Byte 6
	N_PCItype				Bit 3-0					
	7	6	5	4						
Single Frame (CAN_DL ≤ 8) (c)	0	0	0	0	SF_DL	DATA 1	DATA 2	…	…	…
Single Frame (CAN_DL > 8) (a)	0	0	0	0	0000	SF_DL	DATA 1	DATA 2	…	…
First Frame (FF_DL ≤ 4095) (d)	0	0	0	1	FF_DL		DATA 1	DATA 2	…	…
First Frame (FF) (FF_DL > 4095) (b)	0	0	0	1	0000	00	FF_DL			
Consecutive Frame	0	0	1	0	SN	DATA 1	DATA 2	…	…	…
Flow Control	0	0	1	1	FS	BS	STmin	N/A	N/A	N/A

Note 음영 처리된 부분은 PCI 정보를 위해 사용되지 않는다.

(a) CAN_DL 이 8 보다 큰 메시지의 경우는 Byte #1 의 lower nibble 을 0 으로 설정한다 (escape sequence). 이에 따라 SF_DL 값은 Byte #2 에 위치한다.

(b) 4095 바이트보다 큰 메시지의 경우 Byte #1 의 lower nibble 과 Byte #2 를 0 으로 설정한다 (escape sequence). 이에 따라 FF_DL 값은 이어지는 4 바이트에 위치한다. (Byte #3 은 MSB 그리고 Byte #6 은 LSB 로 정한다.)

(c) CAN-FD 제어기에서 진단 응답 메시지 송신 시, DLC 는 진단기와의 호환성을 위해 반드시 8 로 고정해야 한다.

(d) CAN-FD 제어기는 진단 메시지의 길이가 4095 를 넘지 못하며, 송신 시, First Frame Format 은 반드시 (d)를 따른다.

(*) CAN-FD 제어기 기준 진단 CAN-FD 메시지 수신 시, 위 Frame 모두 적용되며, 진단 CAN-FD 메시지 송신 시, 기존 Classical CAN 과 동일하게 적용된다. ((c), (d))

표 2-9. Mapping of parameters and data into CAN-FD frame – Normal Addressing

Hex	Description
0000_2 (0_{16})	Single Frame CAN_DL ≤ 8 인 미분할(unsegmented) 메시지의 경우, PCI Byte #1 의 lower nibble 로 message length 를 표시한다.CAN_DL > 8 미분할(unsegmented) 메시지의 경우는 PCI byte Byte #1 은 0000_2 로 셋팅하고, PCI Byte #2 로 message length 를 표시한다.Single Frame (SF) 은 단일 CAN 프레임안에 들어갈 수 있는 메시지 전송을 지원하는 데 사용된다.
0001_2 (1_{16})	First Frame First Frame (FF) 은 단일 CAN 프레임안에 들어갈 수 없는 메시지, 즉 분할(segmented) 메시지를 전송하는 데에만 사용된다. 분할 메시지의 첫 번째 프레임은 FF 에 들어간다. FF 수신 시, 네트워크 계층 수신 측은 분할 메시지 조합을 시작한다. 메시지의 길이가 4095 이상인 경우, First Frame(FF)의 첫번째 PCI byte 의 lower nibble 은 0000_2 로 설정하고 두번째 PCI Byte 0 으로 설정한다. 메시지의 길이는 세번째부터 여섯번째까지의 PCI Byte 로 나타낸다.
0010_2 (2_{16})	Consecutive Frame 분할 데이터를 보낼 때, FF 뒤에 연속되는 프레임은 Consecutive Frame (CF)에 들어간다. CF 수신 시, 네트워크 계층 수신 측은 모든 메시지가 수신될 때까지 수신된 데이터 바이트를 재조합한다. 수신 측은 메시지의 최종 프레임이 에러 없이 수신 된 후, 조합된 메시지를 인접한 상위 프로토콜 계층으로 전달한다.
0011_2 (3_{16})	Flow Control Flow Control (FC)의 목적은 수신측에 전달되는 CF N_PDU 속도를 조절하는 것이다. 이 기능을 지원하기 위해 세가지 타입의 FC 프로토콜 제어 정보가 정의된다. 프로토콜 제어 정보 필드에 표현되는 이 타입은 이하 Flow Status (FS)라고 표현된다..

표 2-10. Definition of N_PCItype values

N_PCI	Description	
SF_DL (CAN_DL ≤ 8)	Single Frame Data Length	
	Value (HEX)	Description
	0	Reserved
	1 ~ 6	SF_DL 은 N_PCI byte #1 값의 Low Nibble 에 표현된다. 서비스 파라미터 <Length> 값이 할당된다.
	7	SF_DL = 7 은 Normal Addressing 에서만 허용된다.
	8 ~ F	Invalid
SF_DL (CAN_DL > 8)	Single Frame Data Length	
	Value (HEX)	Description
	0 ~ 6	Invalid
	7	SF_DF = 7 은 Extended or Mixed Addressing 에서만 허용된다.
	8 ~ 61	SF_DL 은 N_PCI byte #2 값으로 표현된다. 이 값은 CAN FD 타입에서만 허용되며, 서비스 파라미터 <Length> 값이 할당된다.
	62	SF_DL = (CAN_DL −2)은 Normal Addressing 에서만 허용된다.
	Other values	Invalid
FF_DL	1. First Frame Data Length (If DLC = 8)	

<table>
<tr><td rowspan="11">(FF_DL ≤ 4095)</td><th>Value (HEX)</th><th>Description</th></tr>
<tr><td>0 ~ 6</td><td>Invalid</td></tr>
<tr><td>7</td><td>FF_DL = 7 는 Exteded 또는 Mixed Addressing 에서만 허용된다.</td></tr>
<tr><td>8 ~ FFF</td><td>분할 메시지 길이는 12-bit 값 FF_DL 로 표현된다. 여기서 LSB 는 N_PCI byte #2 의 Bit 0 이고, MSB 는 N_PCI byte #1 의 Bit 3 이다. 분할 메시지 길이의 최대값은 4095-byte 이다. 서비스 파라미터 <Length> 값이 할당된다.</td></tr>
<tr><td colspan="2">2. First Frame Data Length (If DLC ≠ 8)</td></tr>
<tr><th>Value (HEX)</th><th>Description</th></tr>
<tr><td>0 ~ 61</td><td>Invalid</td></tr>
<tr><td>62</td><td>FF_DL = 7 는 Exteded 또는 Mixed Addressing 에서만 허용된다.</td></tr>
<tr><td>63 ~ FFF</td><td>분할 메시지 길이는 12-bit 값 FF_DL 로 표현된다. 여기서 LSB 는 N_PCI byte #2 의 Bit 0 이고, MSB 는 N_PCI byte #1 의 Bit 3 이다. 분할 메시지 길이의 최대값은 4095-byte 이다. 서비스 파라미터 <Length> 값이 할당된다.</td></tr>
<tr><td rowspan="3">FF_DL
(FF_DL > 4095)</td><td colspan="2">First Frame Data Length</td></tr>
<tr><th>Value (HEX)</th><th>Description</th></tr>
<tr><td>FFF+1 ~ FFFF FFFF</td><td>분할 메시지 길이는 32-bit 값 FF_DL 로 표현된다. 여기서 LSB 는 N_PCI byte #6 의 Bit 0 이고, MSB 는 N_PCI byte #3 의 Bit 7 이다. 분할 메시지 길이의 최대값은 4 294 967 295 bytes 이다. 서비스 파라미터 <Length> 값이 할당된다.</td></tr>
<tr><td rowspan="2">SN</td><td colspan="2">Sequence Number</td></tr>
<tr><td colspan="2">SN 은 N_PCI byte #1 값의 Low Nibble 에 표현된다. 값의 범위는 0 ~15 이다.
- 모든 분할 메시지에 대해, SN 값은 0 부터 시작한다.
- FF 의 SN 값은 0 에 해당한다.
- FF 뒤에 바로 따라오는 첫번째 CF 의 SN 값은 1 이다.
- SN 값은 분할 메시지가 전송되는 동안, 매 신규 CF 에 대해 1 씩 값이 증가한다.
- SN 값이 15 가 되면, 그 다음 CF 에 대해 0 으로 돌아간다.</td></tr>
<tr><td rowspan="5">FS</td><td colspan="2">Flow Status</td></tr>
<tr><td colspan="2">파라미터 Flow Status (FS)는 메시지 전송을 계속 진행할 수 있는지 여부를 나타낸다.</td></tr>
<tr><th>Value (HEX)</th><th>Description</th></tr>
<tr><td>0</td><td>ContinueToSend (CTS)
FC CTS 파라미터 값은 N_PCI byte#1의 Low Nibble에 0으로 표현되며,
송신자가 CF 전송을 재개하도록 한다. 이 값은 수신자가 BS 최대값 CF를 수신할 준비가 되어 있다는 것을 의미한다.</td></tr>
<tr><td>1</td><td>Wait (WT)
FC WT 파라미터 값은 N_PCI byte #1의 Low Nibble에 1로 표현되며,
전송자가 신규 FC N_PDU를 계속 기다리고, N_BS 타이머를 재시작하도록 한다.</td></tr>
</table>

	2	Overflow (OVFLW) FC OVFLW 파라미터 값은 N_PCI byte #1의 Low Nibble에 2로 표현되며, 전송자가 분할 메시지 전송을 중단하게 하고, N_USData가 파라미터 <N_Result>=N_BUFFER_OVFLW 값으로 서비스 콜을 확정하도록 한다. (ISO 15765-3 참조) N_PCI FS 파라미터 값은 FF N_PDU 다음에 오는 FC N_PDU에서만 전송되는 것이 허용되며, 수신된 FF N_PDU의 메시지 길이값 FF_DL이 수신자의 버퍼 사이즈를 초과하는 경우에만 사용될 수 있다.
BS	Block Size	
	BS 단위는 블록당 CF N_PDU 개수이다.	
	Value (HEX)	Description
	00	BS 파라미터 값 0 은 분할 메시지를 전송하는 동안, 더 이상 FC 프레임이 전송되지 않는다는 것을 송신자에게 알리기 위해 사용된다. 송신 네트워크 계층 엔터티는 수신 네트워크 계층 엔터티로부터 더 이상의 FC 프레임을 위한 중단 없이 모든 남아있는 CF 을 전송해야 한다.
	01 ~ FF	이 범위의 BS 파라미터 값은 수신 네트워크 엔터티로부터 중간에 FC 프레임 없이 수신할 수 있는 최대 CF 개수를 송신자에게 알리기 위해 사용된다.
STmin	Minimum Separation Time	
	STmin 파라미터 값은 CF 사이에 허용되는 최소 시간 간격을 정의한다.	
	Value (HEX)	Description
	00 ~ 7F	0 ms ~ 127 ms, 00 hex ~ 7F hex 범위의 STmin 단위는 ms 이다.
	F1 ~ F9	100 µs ~ 900 µs, F1 hex ~ F9 hex 범위의 STmin 단위는 100 µs 이다. 여기서, F1 hex 는 100 µs 이고, F9 hex 는 900 µs 이다.
	FA ~ FF	Reserved

표 2-11. Definition of N_PCI

Addressing type	CAN_DL value						
	12	16	20	24	32	48	64
Normal	8 ≤ SF_DL ≤ 10	11 ≤ SF_DL ≤ 14	15 ≤ SF_DL ≤ 18	19 ≤ SF_DL ≤ 22	23 ≤ SF_DL ≤ 30	31 ≤ SF_DL ≤ 46	47 ≤ SF_DL ≤ 62
Mixed or ex-tended	7 ≤ SF_DL ≤ 9	10 ≤ SF_DL ≤ 13	14 ≤ SF_DL ≤ 17	18 ≤ SF_DL ≤ 21	22 ≤ SF_DL ≤ 29	30 ≤ SF_DL ≤ 45	46 ≤ SF_DL ≤ 61

표 2-28. Allowed SF_DL values for a given CAN_DL greater than 8

2.4.6 이더넷 메시지 구조

이더넷 메시지 구조는 적용하는 진단통신 프로토콜에 따라 ES95486-03 이더넷 IP 미들웨어 진단통신 사양서 또는 ES95486-05 DoIP 진단통신 사양서를 준수한다.

2.5 초기화와 타이밍

2.5.1 초기화

진단 CAN 통신에서는 별도의 초기화 과정이 필요하지 않다. 진단 통신은 임의의 진단 서비스 요청(functional 또는 physical request)이 있을 때 시작된다. Power ON 후, 제어기(Server)는 Default

Session에 위치하게 되며, Default Session 이외의 다른 진단 Session에 진입하지 않는 이상 Default Session은 항상 유지된다.

2.5.2 타이밍

진단 CAN 통신에서는 다음과 같은 두 가지의 타이밍 파라미터가 사용된다.

- Network layer timing parameters
- Application layer timing parameters

2.5.2.1 네트워크 계층 타이밍 파라미터

그림 2-3 은 네트워크 계층에서의 CAN 메시지의 흐름과 타이밍 파라미터를 나타내고 있다.

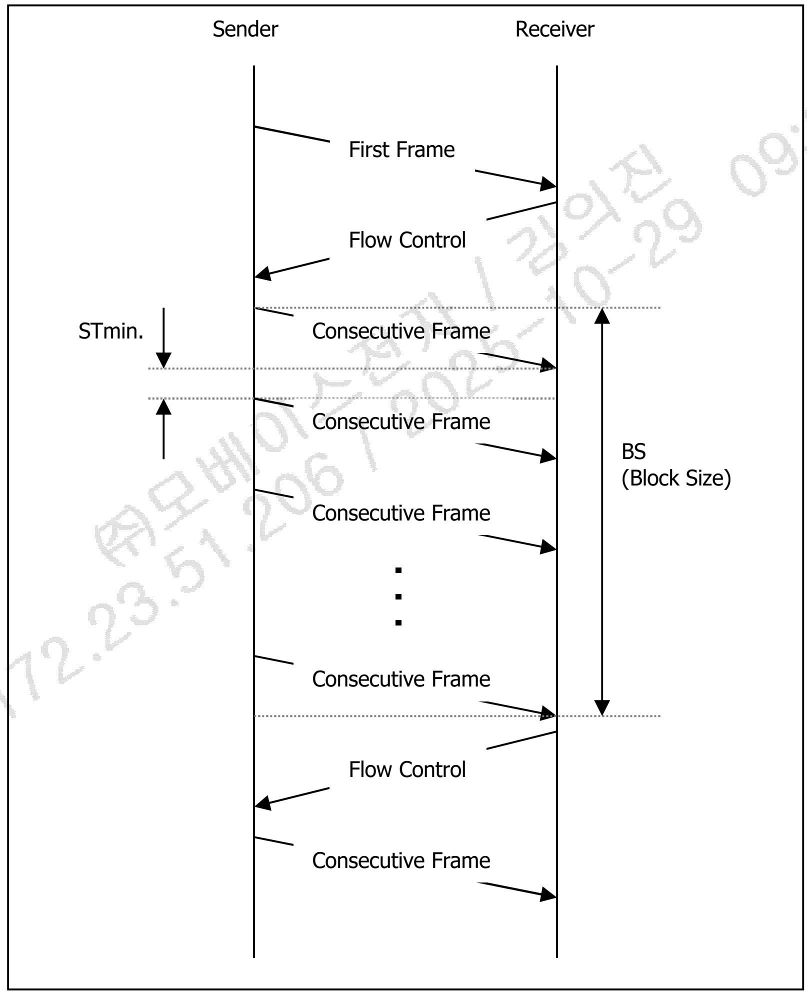


그림 2-3. Message frame flow diagram

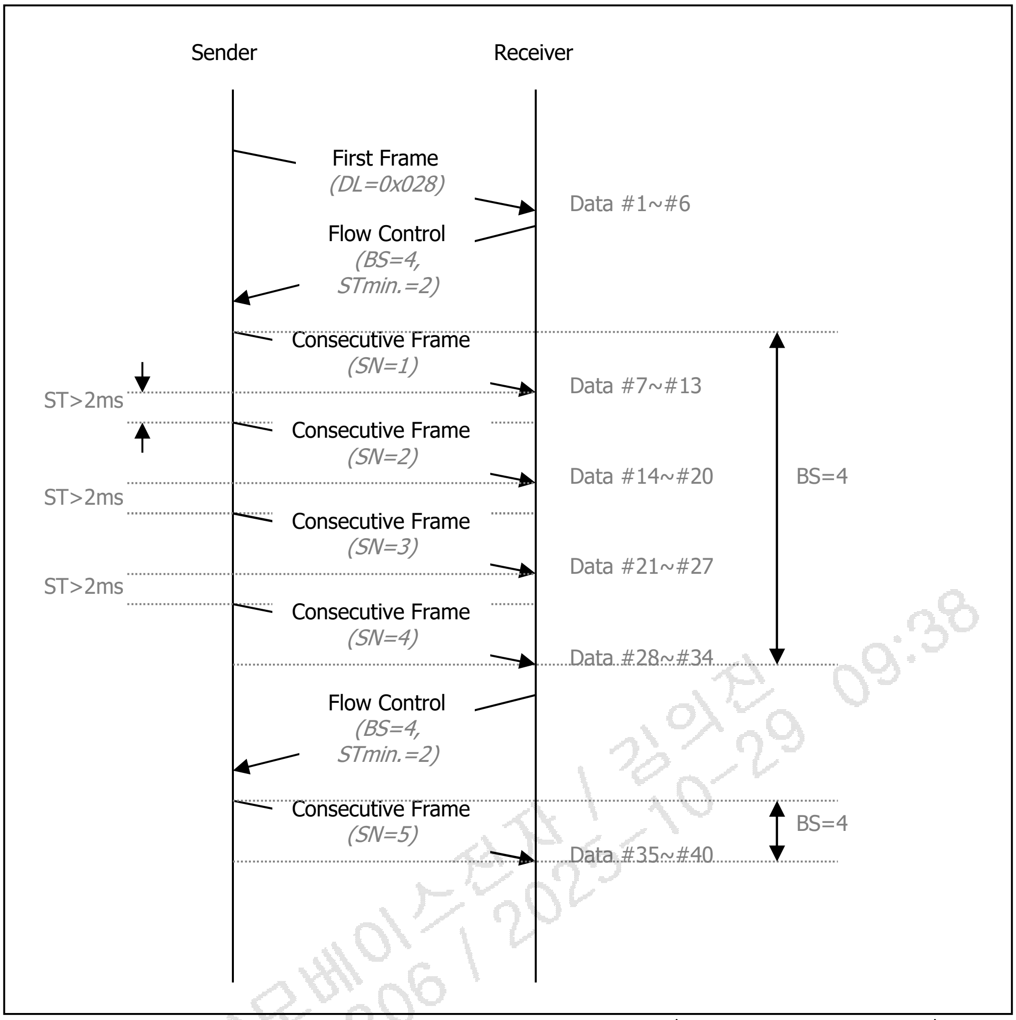


그림 2-4. Example of message frame flow (STmin = 2, BS = 4)

그림 2-3 에 표시된 각 타이밍 파라미터에 대한 설명은 아래 표와 같다.

[STmin]

Diagnostic type	Gateway	Network type	Description	Recommanded Value
리프로그램 (Boot Area or Programming session)	-	-	제어기 STmin 값 0 으로 설정된다. 이는 외부 진단 장비가 Consecutive Frame 을 최대한 빨리 보내야함을 의미한다.	0 ms
비 리프로그램	No Gateway / Client – Server 직결	-	제어기 (또는 외부 진단 장비) 의 FC(Flow Control) STmin 값을 0 ~ 8 ms 로 설정할 수 있다.	0 ~ 8 ms
	Gateway	HS CAN, CAN FD	제어기 (또는 외부 진단 장비) 의 FC(Flow Control) Stmin 값은 0.3 ~12ms 로 설정할 수 있다.	0.3ms ~ 12ms
	Gateway	Ethernet	이더넷 제어기의 FC(Flow Control) STmin 값은 최소 10ms 이상으로 한다.	10ms ~
※ ECU 가 진단기로부터 STmin 0.3ms 미만 값을 수신 받았을 경우 해당 ECU 는 최소 0.3ms 이상의 간격으로 Consecutive Frame 을 전송해야 한다.(e.g. OBD 제어기 STmin 0)				

※ 백그라운드 업데이트는 Communication control 서비스와 함께 프로그래밍 세션에서 수행되지 않으므로 리프로그램 type 범위에 해당하지 않는다

표 2-12a. Network layer parameters – STmin

※ 멀티프레임 전송하는 경우, ECU 와 진단 장비 사이에 Gateway 가 있고, Communication Control (DisableRxAndTx) 사용 불가 시, 제어기 응답과 상관없이 진단기에서 STmin 을 10~12ms 로 연장할 수 있다.

[BS]

Parameter	Description	Recommanded Value
BS	ECU (또는 외부 진단 장비)는 BSmax 값을 최대 8 로 설정할 수 있다. 외부 진단 장비 (또는 ECU)는 추가적인 Flow Control Frame 없이, 최대 8 개의 Consecutive Frame 을 전송할 수 있다.	0 ~ 8
	리프로그램 시 (ECU 가 Boot Area 에 있을 때), ECU 는 BS 값 0 으로 설정된다. 이는 추가적인 Flow Control 없이, ECU 가 가능한한 많은 Consecutive Frame 을 수신할수 있음을 의미한다. ※ 실제로, 외부 진단 장비로부터 전송되는 Consecutive Frame 의 개수는 RequestDownload 서비스 Positive Response Message 의 'maxNuberOfBlockLength' 값에 따라 설정된다.	0 (Infinite)

표 2-12b. Network layer parameters – BS

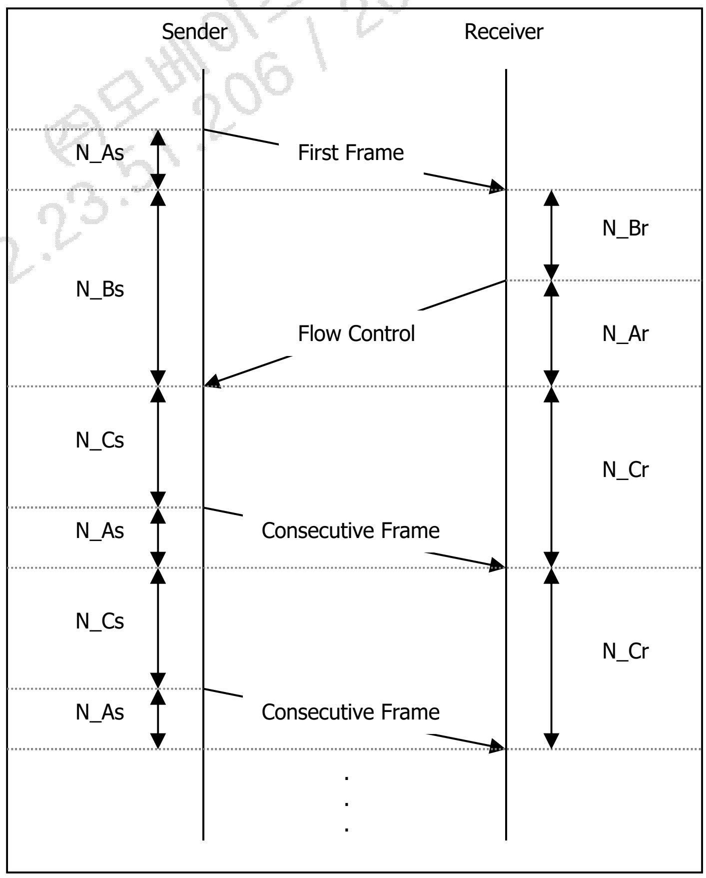


그림 2-5. Placement of network layer timing parameters

그림 2-5 에 표시된 각 타이밍 파라미터에 대한 설명은 아래 표와 같다.

Timing Parameter	Description	Timeout (ms)
N_As	송신자 측에서 CAN 프레임 전송에 소요되는 시간	1000
N_Ar	수신자 측에서 CAN 프레임 전송에 소요되는 시간	1000
N_Bs	다음 Flow Control 수신 까지 소요되는 시간	1000
N_Br	다음 Flow Control 전송 까지 소요되는 시간	N/A
N_Cs	다음 Consecutive Frame 전송 까지 소요되는 시간	N/A
N_Cr	다음 Consecutive Frame 수신 까지 소요되는 시간	1000

표 2-13. Network layer parameters

2.5.2.2 어플리케이션 계층 타이밍 파라미터

그림 2-6 은 ISO 14229-2 기준, 어플리케이션 계층에서의 진단 어플리케이션 메시지 흐름과 타이밍 파라미터를 나타낸다.

$P2_{CAN\_SERVER}$ / $S3_{SERVER}$ 와 각각의 설명들은 아래 표에 설명되어 있다. $P2_{CAN\_SERVER}$ 파라미터는 제어기(Server)의 성능과 연관되며, $S3_{SERVER}$ 파라미터는 진단 Session 의 Timeout 과 관련되어 있다.

Parameter	Description	Min (ms)	Max (ms)
$P2_{CAN\_SERVER}$	Performance requirement for the server to start with the response message after the reception of a request message.	0	50
$P2*_{CAN\_SERVER}$	Performance requirement for the server to start with the response message after the transmission of a negative response message with response code 78 hex.	0 1)	5000
$P2_{CAN\_CLIENT}$	Timeout for the client to wait after the successful transmission of a request message for the start of incoming response messages (First Frame of a multi-frame message or a SingleFrame message).	$P2_{CAN\_SERVER\_max}$ + $\Delta P2_{CAN}$ (80ms for Ethernet ECU)	N/A
$P2*_{CAN\_CLIENT}$	Enhanced timeout for the client to wait after the reception of a negative response message with response code 78 hex for the start of incoming response messages (First Frame of a multiframe message or a SingleFrame message).	$P2*_{CAN\_SERVER\_max}$ + $\Delta P2_{CAN\_rsp}$	N/A
$P3_{CAN\_CLIENT\_Phys}$	Minimum time for the client to wait after the successful transmission of a physically addressed request message with no response required before it can transmit the next physically addressed request message.	$P2_{CAN\_SERVER\_max}$ + $\Delta P2_{CAN}$	N/A
$P3_{CAN\_CLIENT\_Func}$	Minimum time for the client to wait after the successful transmission of a functionally addressed request message before it can transmit the next functionally addressed request message in case no response is required or the requested data is only supported by a subset of the functionally addressed servers.	$P2_{CAN\_SERVER\_max}$ + $\Delta P2_{CAN}$	N/A

1) During the enhanced response timing, the minimum time between the transmission of consecutive negative messages (each with negative response code 0x78 hex) shall be 0.3 * $P2*_{CAN\_SERVER\_max}$, in order to avoid flooding the data link with unnecessary negative response code 0x78 hex messages.

표 2-14. Application layer timing parameters

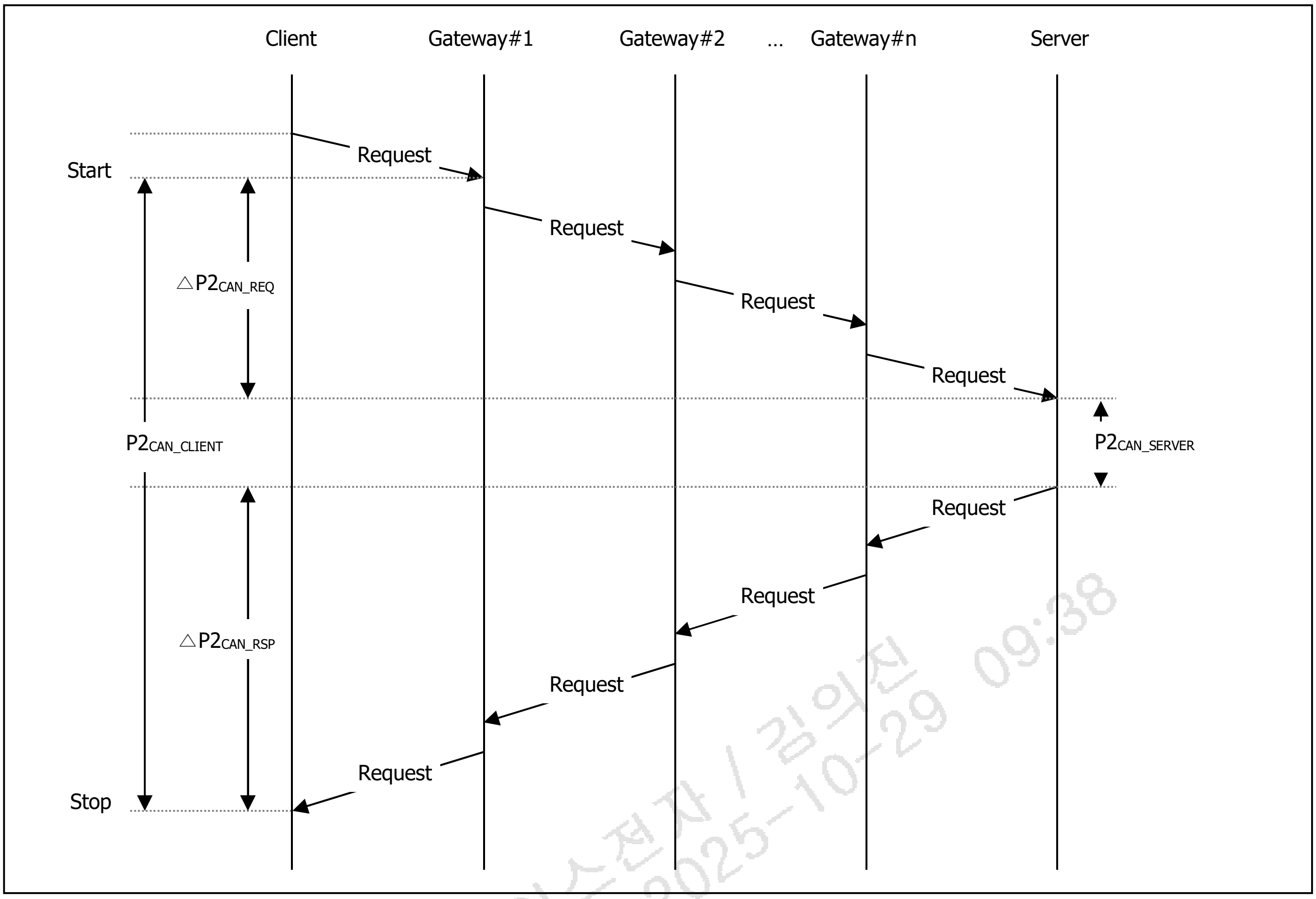


그림 2-6. Application layer message flow diagram

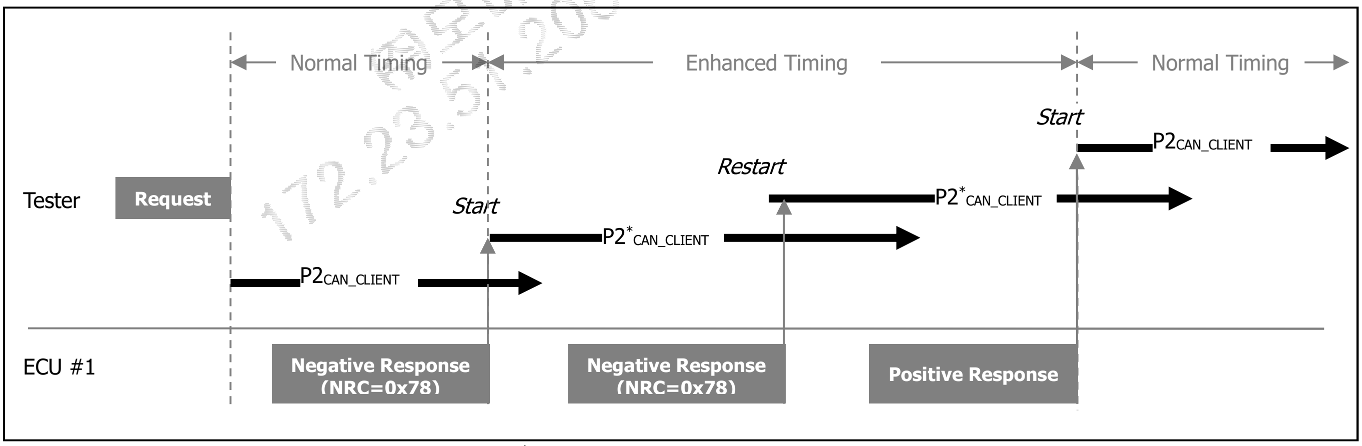


그림 2-7. $P2^{*}_{CAN\_CLIENT}$ timing parameter Definition

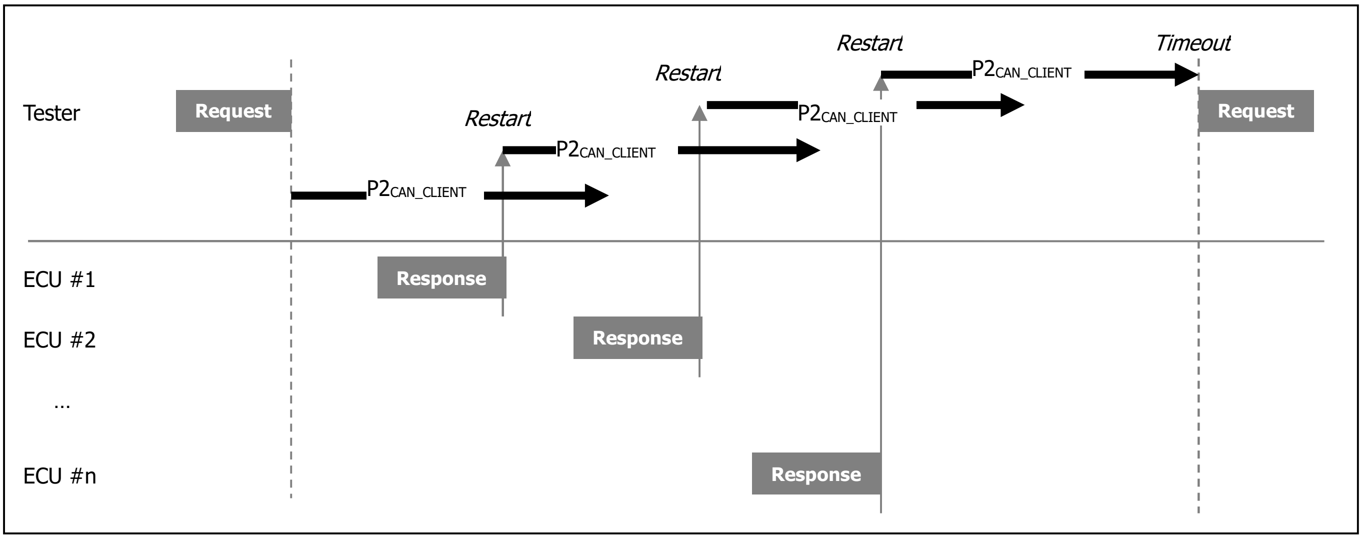


그림 2-8. $P2_{CAN\_CLIENT}$ timing parameter in Functional request

2.5.2.3 세션 레이어 타이밍 파라메터

Default Session 이외의 다른 진단 Session 이 시작되었을 때, 표 2-13 의 Timing 파라미터와 같은 Session 관리가 필요하다.

ISO15765-3 의 「6.3.5 Detailed timing parameter descriptions」를 참조하면, Application 과 Session 계층의 Timing 파라미터에 대한 보다 많은 정보를 얻을 수 있다.

Timing Parameter	Description	Type	Recommended Value	Timeout
$S3_{Client}$	Time between functionally addressed TesterPresent (3E hex) request messages transmitted by the client to keep a diagnostic session other than the defaultSession active in multiple servers (functional communication) or maximum time between physically transmitted request messages to a single server (physical communication). The $S3_{Client}$ timeout value includes the travel time travel of the message on the network (gateway delays, etc.)	Timer reload value	2000ms	4000ms
$S3_{Server}$	Time for the server to keep a diagnostic session other than the defaultSession active while not receiving any diagnostic request message. The tolerance of $S3_{Server}$ is -0ms, +200ms.	Timer reload value	N/A	5000ms

표 2-15. Session layer timing parameter definitions

Timing Parameter	Action	Physical and functional communication, using functionally addressed, periodically transmitted TesterPresent request message	Physical communication only, using a physically addressed, sequentially transmitted TesterPresent request message

$S3_{Client}$	Initial Start	The completion of the DiagnosticSession Control (10 hex) request message. ※ This is only true for if the session type is a non-default session.	The completion of the DiagnosticSession Control (10 hex) request message in case no response is required.
			The reception of the DiagnosticSession Control (10 hex) response message in case a response is required.
	Subsequent Start	The completion of the functionally addressed TesterPresent (3E hex) request message, which is transmitted each time the S3Client timer times out.	The completion of any request message in case no response is required.
			The reception of any response message in case a response is required.
			An error during the reception of a multi-frame response message.
$S3_{Server}$	Initial Start	The completion of the transmission of a DiagnosticSessionControl positive response message for a transition from the default session to a non-default session, in case a response message is required.	
		Successful completion of the requested action of the service DiagnosticSessionControl (10 hex) for a transition from the default session to a non-default session, in case no response message is required/allowed.	
	Subsequent Stop	The start of a multi-frame request message or the reception of any SingleFrame request message. If the defaultSession is active, the S3Server timer is disabled.	
	Subsequent Start	The completion of any response message that concludes a service execution (final response message) in case a response message is required/allowed to be transmitted (this includes positive and negative response messages). A negative response with response code 78 hex does not restart the S3Server timer.	
		The completion of the requested action (service conclusion) in case no response message (positive and negative) is required/allowed.	
		An error during the reception of a multi-frame request message.	

표 2-16. Session layer timing parameter definitions

2.5.2.4 Minimum time between client request messages

진단기로부터의 Request 메시지간 최소 송신 시간 간격은 제어기(Server)의 데이터 해석을 위한 Scheduling을 위하여 필요하다.

예를 들어, 제어기가 임의의 Scheduling rate (예, 10ms)로 진단 Request 메시지를 처리한다고 하면, 제어기의 요구사항을 만족시키기 위하여, 진단 서비스 데이터 해독을 위한 제어기 Scheduler의 시간은 $P2_{CAN\_Server}$ 보다 작아야 한다.

Request 메시지간 최소 시간 간격은 다음의 두 가지 Timing 파라미터에 따른다.

- $P3_{CAN\_Functional}$: Funtionally Addressed Request 메시지에 적용되는 Timing 파라미터로, Funtionally Addressed Request 메시지에 대하여 제어기가 Request 데이터를 지원하지 않을 경우, 응답이 필요하지 않으므로 정의되어야 한다.
- $P3_{CAN\_Physical}$: Physically Addressed Request 메시지에 적용되는 Timing 파라미터로, suppressPosRsp MsgIndicationBit가 TRUE로 설정되어 제어기가 Request 메시지에 대하여 응답하지 않을 경우

적용된다.

제어기(Server)의 응답 필요한 Physical 통신의 경우, 진단기(Client)는 Request 메시지가 제어기로부터 완전히 처리되었으므로, 마지막 Response 메시지에 대한 수신이 완전히 끝난 후 즉시 다음 Request 메시지를 송신할 수 있다.

Physically 또는 Functionally Addressed Request 메시지의 끝과 새로운 Physically 또는 Functionally Addressed Request 메시지의 시작 사이의 최소 $P3_{CAN\_Physical}$ and $P3_{CAN\_Functional}$ 시간은 진단기를 위해 정의되어 진다.

a) The value of $P3_{CAN\_Physical}$ will be identical to $P2_{CAN\_Server\_max}$ for the physically addressed server. The timing applies to any physically addressed request message in any diagnostic session (default and nondefault session) and in case no response is required by the server.

The $P3_{CAN\_Physical}$ timer is started in the client each time a physically addressed request message with no response required is successfully transmitted onto the bus, which is indicated via N_USData.con in the client. When the client wants to transmit a new physically addressed request message following a previous request that was completely handled, then this is only allowed in case the $P3_{CAN\_Physical}$ timer is no longer active at the time the client wants to transmit the physically addressed request message. In case $P3_{CAN\_Physical}$ would still be active at the point in time the client would like to transmit a new physically addressed request message, then the transmission shall be postponed until $P3_{CAN\_Physical}$ is timed out.

b) The value of $P3_{CAN\_Functional}$ will be the maximum (worst-case) value of all functionally addressed server's $P2_{CAN\_Server\_max}$ for any functionally addressed request message in any diagnostic session (default and non-default session).

The $P3_{CAN\_Functional}$ timer is started in the client each time a functionally addressed request message with response required or with no response required is successfully transmitted onto the bus, which is indicated via N_USData.con in the client. When the client wants to transmit a new functionally addressed request message following a previous request that was completely handled, then this is only allowed in case the $P3_{CAN\_Functional}$ timer is no longer active at the time the client wants to transmit the functionally addressed request message. In case $P3_{CAN\_Functional}$ would still be active at the point in time the client would like to transmit a new functionally addressed request message, then the transmission shall be postponed until $P3_{CAN\_Functional}$ is timed out.

NOTE) "Completely handled" means that either no response is received in case no response is required, all expected responses to a functionally addressed request are received in case the responding servers are known and responses are required or a P2CAN_Client timeout occurred in case the responding servers are not known and responses are required.

The requirement for the server is that it shall start with its response message within $P2_{CAN\_Server}$. This means that the diagnostic data interpretation rate of the server shall be less than $P2_{CAN\_Server}$.

2.6 어플리케이션 계층 (Application Layer)

2.6.1 Request Message sub-function parameter

Sub-function 파라미터는 아래 표와 같이 두 부분(Bit 레벨)으로 나뉘어 진다.

Bit position	Description
7	suppressPosRspMsgIndicationBit This bit indicates if a positive response message shall be suppressed by the server. '0' = FALSE, do not suppress a positive response message (a positive response message is required). '1' = TRUE, suppress response message (a positive response message shall not be sent; the server being addressed shall not send a positive response message). Independent of the suppressPosRspMsgIndicationBit, negateive respone messages are sent by the server(s) according to the restrictions specified 2.6.3~2.6.5.
6 – 0	sub-function parameter value The bits 0-6 of the sub-function parameter contain the sub-function parameter value of the service (00 – 7F hex). Each service utilizing the sub-function parameter byte, but only supporting the suppressPosRspMsgIndicationBit has to support the zeroSubFunction sub-function parameter value (00 hex).

표 2-17. Sub-function Parameter Structure

NOTE) When requestCorrectlyReceived-ResponsePending (NRC = 78 hex) response code is used, the server shall always send a final response (positive or negative) independent of the suppressPosRspMsgIndicationBit value.

2.6.2 Negative response service

Each diagnostic service has a negative response message according to Table 2-18. The Data Byte #1 is always the specific negative response service identifier. The Data Byte #2 shall be a copy of the service identifier value from the service request message that the negative response message corresponds to.

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	Negative Response SID	M	#7Fh	SIDNR
#2d	<Service Name> Request SID	M	#XXh	SIDR!
#3d	Negative Response Code	M	#XXh	NRC_

Table 2-18. Negative response message definition

2.6.3 General server response behaviour

Abbreviation	Description
SuppressPosRspMsgIndicationBit	TRUE ('1') = Server shall NOT send a positive response message FALSE ('0') = Server shall send a positive response message
PosRsp	Abbreviation for positive response message

NegRsp	Abbreviation for negative response message
NoRsp	Abbreviation for NOT sending a positive or negative response message
NRC	Abbreviation for negative response code
ALL	All of the requested data parameters (service without sub-function parameter) of the client request message are supported by the server
At Least 1	At least 1 data parameters (service without sub-function parameter) of the client request message must be supported by the server
NONE	None of the requested data parameters (service without sub-function parameter) of the client request message is supported by the server

표 2-19. Legend for this section

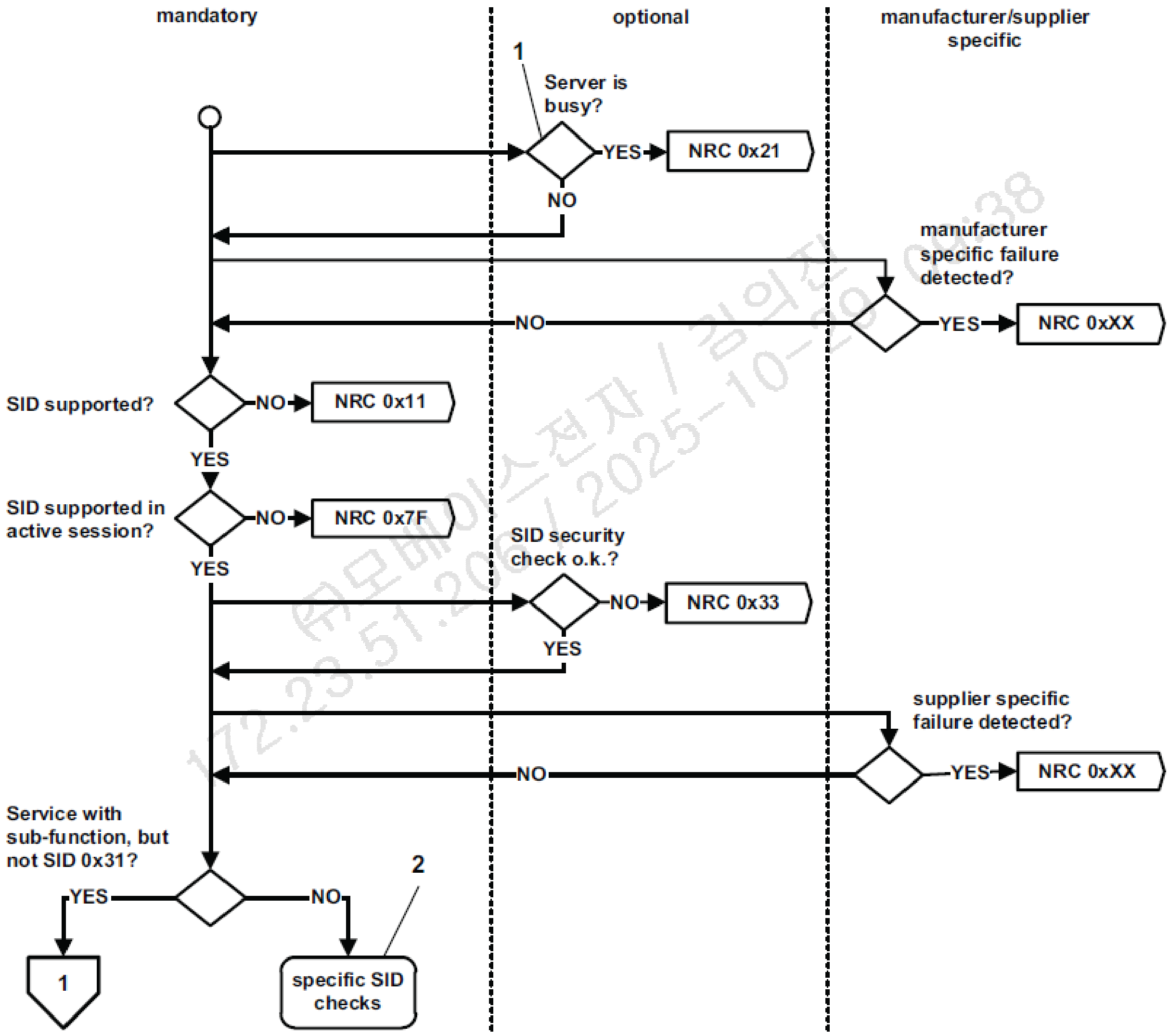


그림 2-9. General server response behavior

Key
1 Dianostic request can not be accepted because another diagnostic task is already and in progress by a different client.
2 refer to the response behavior (supported negateive response codes) of each service.

2.6.4 Request message with sub-function parameter and server response behaviour

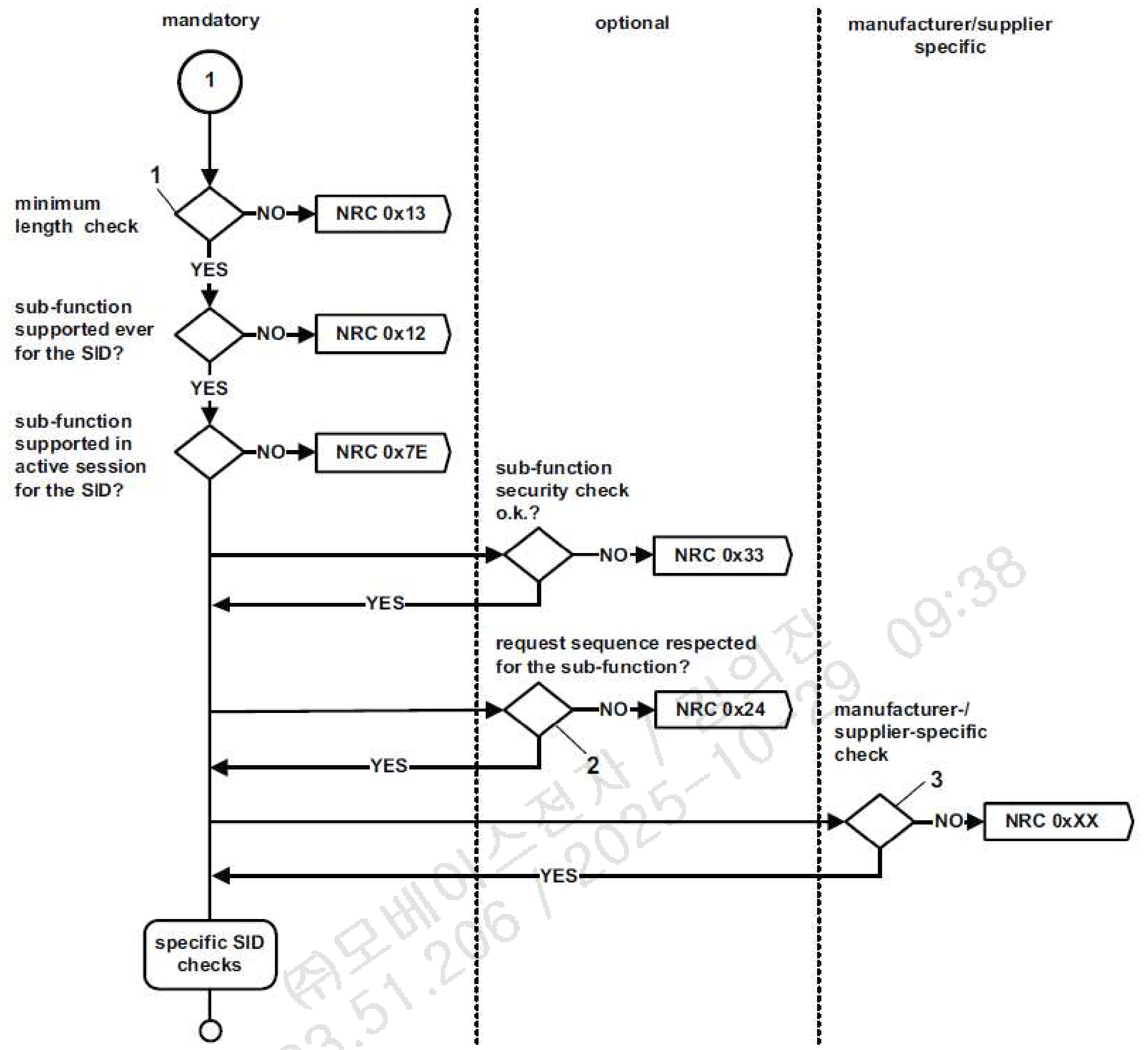


그림 2-10. General server response behavior for request messages with sub-function parameter

Key
1 at least 2 (SID+SubFunction Parameter).
2 if sub-function is subject to sequence check, e.g. LinkControl or SecurityAccess.
3 refer to the response behaviour (supported negative response codes) of each service.

2.6.4.1 Physically addressed client request message

이 장에 설명된 제어기의 Response 특성은 진단기로부터 Physically Addressed Request 메시지를 수신하였을 때, Sub-function 파라미터가 있는 각 서비스에 대한 내용이다.

Server Case #	Client Request Message		Server Capability			Server Response		Comments to Server Response
	Addressing Scheme	Sub-Function (SuppressPosRspMsgIndicationBit)	ServiceID Supported	Sub-Function Supported	Data Parameter Supported (only if applicable)	Message	NRC	
1	Physical	FALSE (bit=0)	YES	YES	At Least 1	PosRsp	-	Server sends positive response
2					At Least 1	NegRsp	NRC=0xXX	Server sends negative response because error occurred reading the data parameters of the request message
3					None		NRC= ROOR	Negative response with NRC 0x31
4			NO	-	-		NRC=SNS or SNSIAS	Negative response with NRC 0x11 or NRC 0x7F
5			YES	NO	-		NRC=SFNS or SFNSIAS	Negative response with NRC 0x12 or NRC 0x7E
6		TRUE (bit=1)	YES	YES	At Least 1	NoRsp	-	Server does NOT send a response
7					At Least 1	NegRsp	NRC=0xXX	Server send negative response because error occurred reading the data parameters of the request message
8					None		NRC= ROOR	Negative response with NRC 0x31
9			NO	-	-		NRC=SNS	Negative response with NRC 0x11
10			YES	NO	-		NRC=SFNS	Negative response with NRC 0x12

표 2-20. Physically addressed request message with sub-function parameter and server response behaviour

Description of server response cases on physically addressed client request messages with subFunction:

1) Server sends a positive response message because the service identifier and sub function parameter is supported of the client's request with indication for a response message,

2) Server sends a negative response message (e.g. IMLOIF: incorrectMessageLengthOrIncorrectFormat) because the service identifier and sub function parameter is supported of the client's request, but some other error appeared (e.g. wrong PDU length according to service identifier and sub function parameter in the request message) during processing of the sub function.

3) Server sends a negative response message with the negative response code ROOR (requestOutOfRange) because the service identifier and sub-function parameter are supported but none of the requested data-parameters are supported by the client's request message.

4) Server sends a negative response message with the negative response code SNS (serviceNotSupported) or SNSIAS (serviceNotSupportedInActiveSession) because the service identifier is not supported of the client's request with indication for a response message.

5) Server sends a negative response message with the negative response code SFNS (subfunctionNotSupported) or SFNSIAS (sub-FunctionNotSupportedInActiveSession) because the service identifier is supported and the sub-function parameter is not supported for the data-parameters (if applicable) of the client's request with indication for a response message.

6) Server sends no response message because the service identifier and sub-function parameter is supported of the client's request with indication for no response message.

NOTE If a negative response code RCRRP (requestCorrectlyReceivedResponsePending) is used, a final response shall be given independent of the suppressPosRspMsgIndicationBit value.

7) Same effect as in 2) (i.e., a negative response message is sent) because the suppressPosRspMsgIndicationBit is ignored for any negative response that needs to be sent upon a physically addressed request messages.

8) Same effect as in 3) (i.e., the negative response message is sent) because the suppressPosRspMsgIndicationBit is ignored for any negative response that needs to be sent upon receipt of a physically addressed request messages.

9) Same effect as in 4) (i.e., the negative response message is sent) because the suppressPosRspMsgIndicationBit is ignored for any negative response that needs to be sent upon a physically addressed request messages.

10) Same effect as in 5) (i.e., the negative response message is sent) because the suppressPosRspMsgIndicationBit is ignored for any negative response that needs to be sent upon a physically addressed request messages.

2.6.4.2 Functionally addressed client request message

이 장에 설명된 제어기의 Response 특성은 진단기로부터 Functionally Addressed Request 메시지를 수신하였을 때, Sub-function 파라미터가 있는 각 서비스에 대한 내용이다.

Server Case #	Client Request Message		Server Capability			Server Response		Comments to Server Response
	Addressing Scheme	Sub-Function (SuppressPosRspMsgIndicationBit)	ServiceID Supported	Sub-Function Supported	Data Parameter Supported (only if applicable)	Message	NRC	
1	Functional	FALSE (bit=0)	YES	YES	At Least 1	PosRsp	-	Server send positive response
2					At Least 1	NegRsp	NRC=0xXX	Server send negative response because error occurred reading the data parameters of the request message

3					None	NoRsp	–	Server does NOT send a response
4			NO	–	–		–	Server does NOT send a response
5			YES	NO	–		–	Server does NOT send a response
6					At Least 1	NoRsp	–	Server does NOT send a response
7		TRUE (bit=1)	YES	YES	At Least 1	NegRsp	NRC=0xXX	Server send negative response because error occurred reading the data parameters of the request message
8					None	NoRsp	–	Server does NOT send a response
9			NO	–	–		–	Server does NOT send a response
10			YES	NO	–		–	Server does NOT send a response

표 2-21. Functionally addressed request message with sub-function parameter and server response behaviour

Description of server response cases on functionally addressed client request messages with subFunction:

1) Server sends a positive response message because the service identifier and sub function parameter is supported of the client's request with indication for a response message,

2) Server sends a negative response message (e.g. IMLOIF: incorrectMessageLengthOrIncorrectFormat) because the service identifier and sub function parameter is supported of the client's request, but some other error appeared (e.g. wrong PDU length according to service identifier and sub function parameter in the request message) during processing of the sub function.

3) Server sends no response message because the negative response code ROOR (requestOutOfRange, which is identified by the server because the service identifier and sub function parameter are supported but a required data parameter is not supported of the client's request) is always suppressed in case of a functionally addressed request message. The suppressPosRspMsgIndicationBit does not matter in such case.

4) Server sends no response message because the negative response codes SNS (serviceNotSupported) and SNSIAS (serviceNotSupportedInActiveSession), which are identified by the server because the service identifier is not supported of the client's request, are always suppressed in case of a functionally addressed request message. The suppressPosRspMsgIndicationBit does not matter in such case.

5) Server sends no response message because the negative response codes SFNS (subfunctionNotSupported) and SFNSIAS (sub-functionNotSupportedInActiveSession), which are identified by the server because the service identifier is supported and the sub-function parameter is not supported for the data-parameters (if applicable) of the client's request, are always suppressed in case of a functionally addressed request. The suppressPosRspMsgIndicationBit does not matter in such case.

6) Server sends no response message because the service identifier and sub function parameter is supported of the client's request with indication for no response message.

NOTE) If a negative response code RCRRP (requestCorrectlyReceivedResponsePending) is used, a final response shall be given independent of the suppressPosRspMsgIndicationBit value.

7) Same effect as in 2) (i.e., a negative response message is sent) because the suppressPosRspMsgIndicationBit is ignored for any negative response. This is also true in case the request message is functionally addressed.

8) Same effect as in 3) (i.e., no response message is sent) because the negative response code ROOR (requestOutOfRange, which is identified by the server because the service identifier and sub-function parameter are supported but a required data-parameter is not supported of the client's request) is always suppressed in case of a functionally addressed request message. The suppressPosRspMsgIndicationBit does not matter in such case.

9) Same effect as in 4) (i.e., no response message is sent) because the negative response codes SNS (serviceNotSupported) and SNSIAS (serviceNotSupportedInActiveSession), which are identified by the server because the service identifier is not supported of the client's request, are always suppressed in case of a functionally addressed request message. The suppressPosRspMsgIndicationBit does not matter in such case.

10) Same effect as in 5) (i.e., no response message is sent) because the negative response codes SFNS (sub-functionNotSupported) and SFNSIAS (sub-functionNotSupportedInActiveSession), which are identified by the server because the service identifier is supported and the sub-function parameter is not supported of the client's request, are always suppressed in case of a functionally addressed request message. The suppressPosRspMsgIndicationBit does not matter in such case.

2.6.5 Request message without sub-function parameter and server response behaviour

2.6.5.1 Physically addressed client request message

이 장에 설명된 제어기의 Response 특성은 진단기로부터 Physically Addressed Request 메시지를 수신하였을 때, Sub-function 파라미터가 없고, Data Parameter 가 있는 각 서비스에 대한 내용이다.

Server Case #	Client Request Message	Server Capability		Server Response		Comments to Server Response
	Addressing Scheme	ServiceID Supported	Data Parameter Supported (only if applicable)	Message	NRC	
1	Physical	YES	ALL	PosRsp	-	Server send positive response
2			At Least 1		-	Server send positive response
3			At Least 1	NegRsp	NRC=0xXX	Server send negative response because error occurred reading the data parameters of the request message
4			None		NRC = ROOR	Negative response with NRC 0x31
5		NO	-		NRC=SNS or SNSIAS	Negative response with NRC 0x11 or NRC 0x7F

표 2-22. Physically addressed request message without sub-function parameter and server response behaviour

Description of server response cases on physically addressed client request messages without sub-function (data parameter follows service identifier):

1) Server sends a positive response message because the service identifier and all data parameters are supported of the client's request message,

2) Server sends a positive response message because the service identifier and a single data parameter is supported of the client's request message,

3) Server sends a negative response message (e.g., IMLOIF: incorrectMessageLengthOrIncorrectFormat)
because the service identifier is supported and at least one data-parameter is supported of the client's request message, but some other error occurred (e.g. wrong length of the request message) during processing of the service.

4) Server sends a negative response message with the negative response code ROOR (requestOutOfRange) because the service identifier is supported and none of the requested data parameters are supported of the client's request message,

5) Server sends a negative response message with the negative response code SNS (serviceNotSupported) or SNSIAS (serviceNotSupportedInActiveSession) because the service identifier is not supported of the client's request message.

2.6.5.2 Functionally addressed client request message

이 장에 설명된 제어기의 Response 특성은 진단기로부터 Functionally Addressed Request 메시지를 수신하였을 때, Sub-function 파라미터가 없고, Data Parameter가 있는 각 서비스에 대한 내용이다.

<table>
<tr><th rowspan="2">Server Case #</th><th>Client Request Message</th><th colspan="2">Server Capability</th><th colspan="2">Server Response</th><th rowspan="2">Comments to Server Response</th></tr>
<tr><th>Addressing Scheme</th><th>ServiceID Supported</th><th>Data Parameter Supported (only if applicable)</th><th>Message</th><th>NRC</th></tr>
<tr><td>1</td><td rowspan="5">Functional</td><td rowspan="4">YES</td><td>YES</td><td rowspan="2">PosRsp</td><td>-</td><td>Server send positive response</td></tr>
<tr><td>2</td><td>At Least 1</td><td>-</td><td>Server send positive response</td></tr>
<tr><td>3</td><td>At Least 1</td><td>NegRsp</td><td>NRC=xx</td><td>Server send negative response because error occurred reading the data parameters of the request message</td></tr>
<tr><td>4</td><td>None</td><td rowspan="2">NoRsp</td><td>-</td><td>Server does NOT send a response</td></tr>
<tr><td>5</td><td>NO</td><td>-</td><td>-</td><td>Server does NOT send a response</td></tr>
</table>

표 2-23. Functionally addressed request message without sub-function parameter and server response behaviour

Description of server response cases on functionally addressed client request messages without sub-function (data parameter follows service identifier):

1) Server sends a positive response message because the service identifier and single data parameter is supported of the client's request message,

2) Server sends a positive response message because the service identifier and at least one data parameter is supported of the client's request message,

3) Server sends a negative response message (e.g. IMLOIF: incorrectMessageLengthOrIncorrectFormat) because the service identifier is supported and at least one, more than one, or all data parameters are supported of the client's request message, but some other error occured (e.g. wrong length of the request message) during processing of the service,

4) Server sends no response message because the negative response code ROOR (request out of range; which would occur because the service identifier is supported, but none of the requested data parameters is supported of the client's request) is always suppressed in case of a functionally addressed request,

5) Server sends no response message because the negative response codes SNS (serviceNotSupported) and SNSIAS (serviceNotSupportedInActiveSession), which are identified by the server because the service identifier is not supported of the client's request, are always suppressed in case of a functionally addressed request.

2.6.6 Multiple Concurrent Request Messages with Physical and Functional Addressing

일반적인 제어기는 한 번에 단 하나의 Request 메시지를 처리할 수 있다. 이러한 법칙은 임의의 수신된 메시지(Addressing 모드가 Physical 이든 Functional 이든 상관없이)가 Request 메시지가 처리되는 중에는 제어기의 리소스를 점유하고 있음을 의미한다.

위의 일반적인 경우와 별도로 처리되는 예외에는 다음 두 가지가 있다.

a) The keep-alive logic is used by a client to keep a previously enabled session active in one or multiple servers. Keep-Alive-Logic is defined as the functionally addressed valid TesterPresent message with suppressPosRspMsgIndicationBit=true and has to be processed by a bypass logic. It is up to the server to make sure that this specific message can not "block" the server’s application layer and that an immediately following addressed message can be processed.

b) If a server supports one or more legislated diagnostic requests and one of these requests is received while a non legislated service (e.g., enhanced diagnostics) is active, then the active service shall be aborted, the default session shall be started and the legislated diagnostic service shall be processed. This requirement does not apply if the programming session is active.

2.7 에러 처리

Error handling for the application layer and session management to be fulfilled by the client and the server communicating with the client during physical and functional communication shall be in accordance with Tables 2-23, in respect of which it is assumed that the client and the server implement the application and session layer timing according to this part of ISO 14229-1/2/3.

2.7.1 네트워크 / 데이터링크 계층 에러 처리

Error Type			Handling
CAN	DLC	DLC value is smaller than eight (8).	The reception of a CAN frame shall be ignored by the network layer without any further action.
	SF_DL	SF_DL is equal to zero (0) or SF_DL is greater than seven (7).	The network layer shall ignore the received SF.
	FF_DL	FF_DL is greater than the available receiver buffer size.	The network layer shall abort the message reception and send FC with the parameter "Flow Status = Overflow".
		FF_DL is less than eight (8).	The network layer shall ignore the received FF and not transmit FC.
CAN -FD	DLC	표 2-7. CAN_FD data length 외의 DLC	The reception of a CAN-FD frame shall be ignored by the network layer without any further action.
		SF_DL 을 통해 예상된 값보다 작은 DLC	
	SF_DL	CAN_DL 이 8 보다 작거나 같고, SF_DL PCI byte #1 이 0 인 SF	Network Layer 는 수신한 SF 를 무시한다.
		CAN_DL 이 8 보다 크고, PCI byte #1 이 0 이 아닌 SF	
		Normal Addressing 모드에서 CAN_DL 이 8 보다 작거나 같으며 SF_DL 이 CAN_DL -1 보다 큰 SF	
		CAN_DL 이 8 보다 크고, SF_DL 이 유효하지 않은 SF (SF_DL 유효성은 표 2-11 참조)	
	FF_DL	FF_DL is less than eight (8).	Network Layer 는 수신한 FF 를 무시하고 FC 를 전송하지 않는다.
		CAN_DL 이 8 보다 크고 FF_DL_{min}(63)보다 작은 FF_DL 을 갖는 First Frame(FF)	
		FF_DL 4095 보다 작거나 같고 PCI byte #1 의 lower nibble 과 PCI byte #2 가 0 으로 설정된 First Frame(FF)	
		FF_DL 이 수신 가능 버퍼 크기보다 큰 First Frame(FF)	Network Layer 는 수신한 FF 를 에러로 간주하여 메시지 수신을 중단하고, FlowStatus 가 Overflow 인 FC 를 전송한다.

ST	Reserved ST parameter value.	The sending network entity shall use the longest ST value specified by this document (7F hex = 127 ms) instead of the value received from the receiving network entity for the duration of the ongoing segmented message transmission.
FS	Invalid (reserved) FS parameter	The message transmission shall be aborted.

표 2-24. Protocol Control Information Error Handling

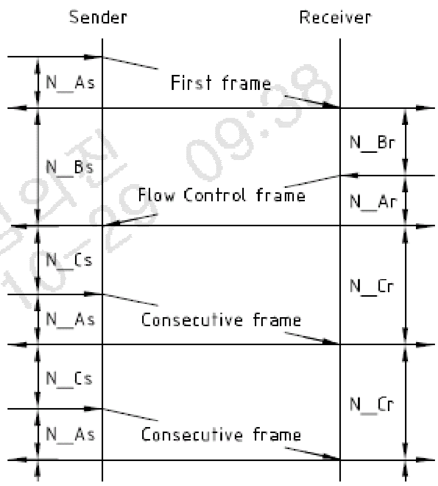
Error Type		Handling	Timing Parameter
N_As	Any message not transmitted in time on the sender side.	Abort message transmission.	
N_Ar	Any message not transmitted in time on the receiver side.	Abort message reception.	
N_Bs	Flow Control not received (lost, overwritten) on the sender side.	Abort message transmission.	
	Preceding First Frame or Consecutive Frame not received (lost, overwritten) on the sender side.		
N_Cr	Consecutive Frame not received (lost, overwritten) on the receiver side.	Abort message reception.	
	Preceding Flow Control not received (lost, overwritten) on the receiver side.		

표 2-25. Timeout Error Handling

2.7.2 어플리케이션 계층 에러 처리

Error Type		Client Handling	
		Physical Communication	Functional Communication
Request Transmission	Request message from network layer with a negative result value.	The client shall repeat the last request, after the time $P3_{CAN\_Phyiscal}$ following the error indication. Restart S3Client in the case of a physically addressed and sequentially transmitted TesterPresent (because S3Client has been stopped based on the request message transmission).	The client shall repeat the last request, after the time $P3_{CAN\_Functional}$ following the error indication.
$P2_{CAN\_CLIENT}$ $P2*_{CAN\_CLIENT}$	Timeout	The client shall repeat the last request. Restart $S3_{CLIENT}$ in the case of a physically addressed and sequentially transmitted TesterPresent (because $S3_{CLIENT}$ has been stopped based on the request message transmission).	Where the client does not know the number of servers responding, then this is the indication for the client that no further response messages are expected. No retry of the request message is required. The client shall completely receive all response messages that are in progress until it can continue with further requests. Where the client knows the number of responding servers, then this is the indication for the client that not all expected servers responded. The client shall repeat the request after it has completely received any response message that is in progress at the point in time the timeout occurs.
Response Reception	Response message from network layer with a negative result value.	The client shall repeat the last request. Restart $S3_{Client}$ in the case of a physically addressed and sequentially transmitted Tester Present (because $S3_{Client}$ has been stopped based on the request message transmission).	The client shall repeat the last request after it has completely received any response message that is in progress at the point in time the error has been indicated.
The client error handling defined shall be performed for a maximum of two (2) times, which means that the worst-case of service request transmissions is three (3).			

표 2-26. Client Error Handling

Error Type		Server Handling
Request Reception	Request message from network layer with a negative result value.	Restart S3Server timer (because it has been stopped based on the previously received FirstFrame indication). The server shall ignore the request.
$P2_{CAN\_CLIENT}$ $P2*_{CAN\_CLIENT}$	Timeout	N/A

Response Transmission	Response message from network layer with a negative result value.	Restart S3Server timer (because it has been stopped based on the previously received request message). The server shall *not* perform a retransmission of the response message.

표 2-27. Server Error Handling

2.8 외부 테스트 장비 초기화 절차

외부 테스트 장비는 본 절에서 정의된 초기화 절차를 준수해야 한다.

외부 테스트 장비 초기화 절차의 목적은 외부 테스트 장비와 차량 내부 특정 제어기 사이의 통신 충돌을 막기 위함이다. 여기서 차량 내부 특정 제어기는 타 제어기의 진단 정보를 수집하기 위해 진단 요청 메시지를 보내는 제어기이다.

만약 어떤 제어기가 고속 CAN 에 연결되어 있고, 외부 테스트 장비와 동일한 Request CAN ID 값을 갖고 항상 주기적으로 진단 요청 메시지를 전송한다면, 동일한 CAN 에 외부 테스트 장비가 연결되었을 때, 통신 충돌을 야기할 수 있다.

이와 같은 충돌을 막기 위하여, 외부 테스트 장비가 고속 CAN 에 연결되어 진단 통신을 시작할 때마다, 외부 테스트 장비는 먼저 초기화 메시지를 전송해야 한다. (functional addressed (7DF hex) DiagnosticSessionControl service (10 hex) with Default Session (81 hex, SPRMIB=True)). 어떤 진단 서비스 요청을 받았는지와 관계 없이 Functional addressing (CAN/Ethdiag : 7DF / 18DB33F1 hex, DOIP : E000 / E400 hex) 메시지를 수신한 모든 제어기는 진단 요청 메시지 전송을 중지해야 한다.

이 초기화 메시지는 외부 테스트 장비의 연결 알람 이외에는 목적이 없습니다. 제어기는 전원을 켤 때 항상 표준 진단 모드를 시작해야 합니다. 초기화 메시지가 수신된 후에도 동일한 진단 모드(표준 진단 모드)로 작동합니다.

2.9 외부 테스트 장비 요구사항

본 절은 HMC / KIA 의 외부 테스트 장비(e.g. EOL)의 동작을 통일하기 위한 목적이다.
본 요구사항은 ECU 에 기능에 영향을 미치지 않는다.

2.9.1 ECU Identification Data display rules

데이터 식별자 내용이 갖는 Data 크기는 동적 길이로 정의될 수 있다. 데이터 길이가 최대 크기를 초과하는 경우, 외부 테스트 장비는 동적 길이로 ASCII 데이터를 표시할 수 있다

Data Identifier 의 표시 순서는 표 C-4.1 진단 도구 및 Notation 의 DID 순서로 정의한다. 외부 테스트 장비는 DID 표시 순서를 준수해야 한다. 표 C-4.1 에 정의되지 않은 DID 는 본 사양에서 관리되지 않고 각 ECU 사양에서 정의되어야 한다.

2.9.2 Tester present

외부 테스트 장비는 Functional addressing 으로 Tester present 를 보내야 한다.
물론 Tester Present 는 Physical addressing 을 사용하여 전송할 수 있지만, 차량에서의 진단 시나리오는 다수의 ECU 를 한꺼번에 진단할 수 있다. 또한 진단 대상 ECU 는 항상 전환이 가능하다. 특별한 문제가 없는 경우 Tester Present 는 Functional addressing 으로 보내야한다.

3. 진단 서비스 구현

3.1 진단 서비스 목록

모든 제어기들은 아래 표에 따라 진단 서비스를 지원하여야 한다.

Diagnostic Service Name	Req. SID (hex)	Cvt.	Diagnostic Modes			Engine / Propulsion system Conditions
			Default Session	Program-ming Session	Extended Diagnostic Session	
			01h	02h	03h	
Diagnostic and Communication Management Functional Unit						
DiagnosticSessionControl	10	M	◆	◆	◆	no restriction, C2
ECUReset	11	M	◆	◆	◆	Engine stop, IGN on
SecurityAccess	27	M	X	◆	◆	no restriction
CommunicationControl	28	M	X	◆	◆	Engine stop
Authentication	29	C3	◆	◆	◆	no restriction
TesterPresent	3E	M	◆	◆	◆	no restriction
SecuredDataTransmission	84	U	X		◆	no restriction
ControlDTCSetting	85	M	X	◆	◆	no restriction
ResponseOnEvent	86	U	◆		◆	no restriction
LinkControl	87	U	X		◆	no restriction
Data Transmission Functional Unit						
ReadDataByIdentifier	22	M	◆		◆	no restriction
ReadMemoryByAddress	23	U	X		◇	no restriction
ReadScalingDataByIdentifier	24	U	◆		◆	no restriction
ReadDataByPeriodicIdentifier	2A	U	X		◆	no restriction
DynamicallyDefineDataIdentifier	2C	U	◆		◆	no restriction
WriteDataByIdentifier	2E	C1	X	◇	◇	Engine stop, IGN on
WriteMemoryByAddress	3D	U	X	◇	◇	Engine stop, IGN on

Stored Data Transmission Functional Unit						
ReadDTCInformation	19	M	◆		◆	no restriction
ClearDiagnosticInformation	14	M	◆		◆	no restriction, C4
Input/Output Control Functional Unit						
InputOutputControlByIdentifier	2F	U	X	X	◆	C5
Remote Activation Of Routine Functional Unit						
RoutineControl	31	C1	○	○	○	Engine stop, IGN on
Upload/Download Functional Unit						
RequestDownload	34	C1	X	◇	◇	Engine stop, IGN on
RequestUpload	35	U	X	◇	◇	Engine stop, IGN on
TransferData	36	C1	X	◇	◇	Engine stop, IGN on
RequestTransferExit	37	C1	X	◇	◇	Engine stop, IGN on
RequestFileTransfer	38	U	X	◇	◇	Engine stop, IGN on

◆ : Function possible in the selected diag. session without security access

◇ : Function possible in the selected diag. session with security access

○ : Specific sub-function possible in the selected diag.Mode (see each function descriptions)

X : Function impossible in selected diag. session.

(blank) : No restriction.

C1 : This service function shall be supported if the controller is reprogrammable.

C2 : When the server transitions from any other session (including default session) to Programming session, Engine / Propulsion system conditions is Engine stop, IGN on.

C3 : This service function shall be supported if the controller is the central gateway(CCU2).

C4 : Engine / Propulsion system condition is 'Engine Stop' only for OBD controllers. Refer to J1979, J1979-2/3

C5 : Engine / Propulsion system condition shall be handled in each controller diagnostic specification. The engine/ Propulsion system conditions could be different depending on the DID. Due to functional safety, restriction of the engine / propulsion system conditions could be exist.

※ Secured dataIdentifiers, secured memory areas and secured routines require a SecurityAccess service and therefore a non-default diagnostic session

※ ACU controllers can or can not support some diagnotic services at default session regardless of security levels. Details refer to the diagnostic specification of the controllers.

※ All ECU need to support all diagnostic services at both physical ID and functional ID.

※ "Engine stop" refers to the "Engine stop" state for internal combustion engine (ICE) vehicles, and the "not ready" status for vehicles with electric propulsion systems (applicable to HEV, PHEV, EREV, and EV). For electric propulsion vehicles, "IGN on" means "Power on" and also includes "Service mode." For vehicles with SDV power architecture (e.g., XP2), "IGN on" corresponds to Drive (S4), ON (S3), and Ready (S2) states. In addition, "Engine stop" includes S0, S1, S2, and S3 states. Therefore, both "Engine stop" and "IGN on" correspond to S2 and S3. Note, however, that in S0 and S1 states, diagnostic communication is generally not possible because the ECU power is not supplied.

※ In the boot software, 10/11/27/3E/31/34/36/37 hex services must be presented and other services are optionally presented.

※ Engine / Propulsion system Condition (Engine stop, IGN on) of some diagnostic services can be changed according to the ECU's specific situation (Exception handling). Refer to the each ECU diagnostic specification in detail.

- If OTA target ECUs support memory redundancy and differential data patch, Upload/Download Functional Unit(34/35/36/37) and RoutineControl(31) services can be used in the extended session during engine running or IGN off.

– According to OTA specification ECU can change session to programming session under IGN off regardless of C2 restriction.

– RoutineControl(31) services can be used in the extended session during engine running In the production process

※ OTA target ECUs need to support ECUReset(11) service and CommunicationControl(28) service during IGN off.

표 3-1. Diagnostic services overview

3.1.1 엔진 / 추진시스템 조건 요약

Diagnostic Service Name	Req. SID (hex)	Standard Engine / Propulsion system Condition	OTA Target ECU
DiagnosticSessionControl	10	No restriction, Engine stop, IGN on (Programming session)	No restriction, Engine stop (Programming session)
ECUReset	11	Engine stop, IGN on	Engine stop
WriteDataByIdentifier	2E	Engine stop, IGN on	Engine stop, IGN on
WriteMemoryByAddress	3D	Engine stop, IGN on	Engine stop, IGN on
InputOutputControlByIdentifier	2F	System specific	System specific
RoutineControl	31	Engine stop, IGN on	No restriction
RequestDownload	34	Engine stop, IGN on	No restriction
RequestUpload	35	Engine stop, IGN on	No restriction
TransferData	36	Engine stop, IGN on	No restriction
RequestTransferExit	37	Engine stop, IGN on	No restriction
RequestFileTransfer	38	Engine stop, IGN on	Engine stop, IGN on
CommunicationControl	28	Engine stop	Engine stop

표 3-1.1. Diagnostic engine condition overview

3.1.1.1 엔진 / 추진시스템 조건 예시

본 예시는 일반적인 예시로써, 실제 Signal 은 다를 수 있다. 따라서 네트워크 DB 를 참조해야한다.

Message	Signal	Vehicle Type

EMS_02_10ms	ENG_EngSta	Emission Engine
HCU_03_10ms	HCU_HevRdySta	Hybrid
VCU_01_10ms	VCU_EvDrvRdySta	EV
VehiclePowerState_200ms	VehiclePowerState	SDV
Vehicle Base API: PowerSystemBase.PowerState	(Refer to Vehicle API)	SDV

Engine Running/Stop : Engine running 이나 Drivable 을 제외한 나머지 값은 ENGINE STOP 이다.

EMS_02		HCU_03		VCU_01		VehiclePowerState	
Value Description	Engine / Propulsion system condition	Value Description	Engine / Propulsion system condition	Value Description	Engine / Propulsion system condition	Value Description	Engine / Propulsion system condition
0x00 Engine stop	STOP	0x00 HEV not ready	STOP	0x00 None	STOP	0x00 None(Default)	STOP
0x01 Cranking	STOP	**0x01 HEV Drivable**	**Running**	**0x01 EV Drivable**	**Running**	0x01 Standby(S1)	STOP
0x02 Stalled	STOP	0x02 Not Used	STOP	0x02 Not used	STOP	0x02 Ready(S2)	STOP
0x03 Running	**Running**	0x03 Error indicator	STOP	0x03 Error Indicator	STOP	0x03 ON(S3)	STOP
0x07 Fault	STOP					**0x04 Drive(S4)**	**Running**

Vehicle Base API: PowerSystemBase.PowerState
pub enum PowerState
{ off = 0, // STOP StandBy = 1, // STOP Ready = 2, // STOP On = 3, // STOP Driving = 4, // Running }

3.1.1.2 Signal 상태에 따른 엔진/추진시스템 조건

Signal status	Engine / Propulsion system Condition
Never received (After start up ECU)	Engine Off
Time out	The last received engine / propulsion system condition
Normal	The last received engine / propulsion system condition

- Never received (After start up ECU) : ECU has never been received engine / propulsion system signal.
- Time out : Cyclic signal has been time out.

3.1.1.3 제약사항과 예외사항

Bootloader 에서 엔진 / 추진 시스템 조건의 적용은 Optional 이다.

Bootloader(FBL)에서, 엔진/추진시스템 상태를 판단하기에는 어려움이 따른다.
그 이유는 FBL(Flash bootloader) 이 SW update 를 위한 최소한의 기능들을 제공하기 때문이다.

일부 진단 서비스의 엔진/추진시스템조건(Engine stop, IGN on)은 제어기의 특정 상황(예외) 에 따라 변경될 수 있다. 이는 각 제어기의 특화 진단사양서를 참조해야한다.
예) 만약 EOL 관련 어떤 기능이 Engine On 상태를 필요로 한다면 엔진/추진시스템 조건은 제어기의 특화 사양에 의해 변경할 수 있다.

전기차 전원 체계의 Power on 상태에서는 고전압 Main relay 가 on 이 된다. 고전압 전원 사용 제어기는 Programming session 에 진입을 하거나 Communication control 을 수행하게 되면 협조제어가 잠시 불가능 하게 된다. 때문에 Main relay on 상태에서도 협조제어가 반드시 필요한 고전압 전원 사용 제어기는 Programming session 또는 Communication control 요청을 수신했을 때 Negative response code 22 로 응답 할 수 있다.

단일 제어기가 2 개 이상의 진단 ID 와 진단 ID 별 UDS server 가 존재하여 논리적으로 구분되어 있는 경우, 특수한 사유가 없다면 각각의 UDS server 의 Engine 조건에 사용되는 Signal 의 유효성을 CRC 나 Alive Counter 등으로 판단할 때, 각 UDS 서버는 Signal 판단 기준을 일치시켜야 한다.
e.g.) MCU ID : 05D0 AP ID : 05D1
유효하지 않은 Engine signal 수신 시 MCU 는 Negative response code 22 로 응답하고, AP 는 Positive response 로 응답 할 수 있다. 이 처럼 각각에 대해 유효성을 다르게 판단하여 응답 결과에 차이가 발생할 수 있다. 때문에 각 UDS 서버는 signal 유효성 판단 기준을 일치시켜야 한다.

4. Diagnostic and Communication Control functional unit

4.1 DiagnosticSessionControl(10 hex) service

Diagnostic Session Control 서비스는 제어기(Server)의 진단 세션을 여러 가지로 변경하여 활성화 시키는데 사용된다.

제어기는 항상 하나의 진단 세션만이 활성화 되어야 한다. 제어기는 전원이 인가되었을 때, 항상 Default Session 에서 시작된다. 만약 Default Session 이외의 다른 진단 세션이 시작되지 않는다면, 제어기에 전원이 인가되어 있는 한 Default Session 이 유지된다.

만약 진단기(Client)가 활성화되어 있는 진단 세션을 재 요청하였을 경우, 제어기(Server)는 Positive Response 로 응답하며, 그림 4-1 의 세션 변경에 따른 제어기 내부 동작특성에 따라 동작되어야 한다.

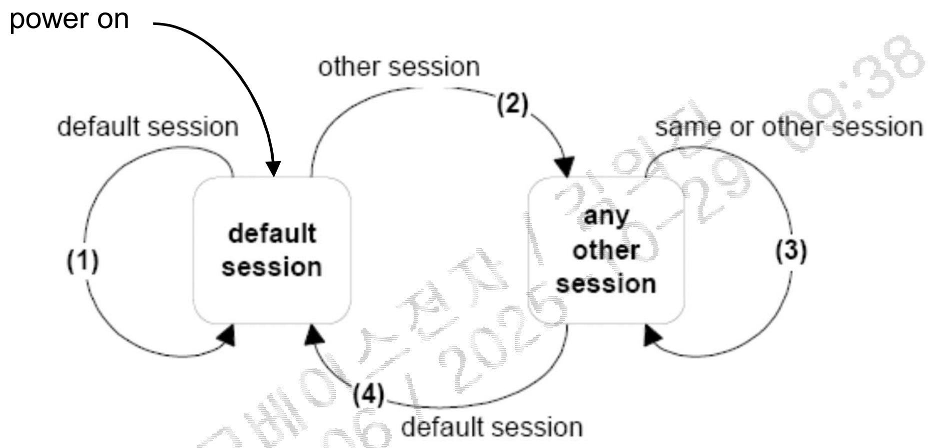


그림 4-1. Server diagnostic session state diagram

진단 세션 변경에 대한 설명은 아래와 같다.

1) 제어기가 defaultSession(standard diagnostic session)에 있고, 진단기가 defaultSession 을 요청하면, 제어기는 defaultSession 을 다시 초기화 해야 한다. 제어기는 활성화된 진단 모드에서 활성화된/초기화된/변경된 설정/제어를 리셋해야 한다. 이것은 비휘발성 메모리에 프로그램된 정보는 포함하지는 않는다.

2) defaultSession 에서 defaultSession 이외 다른 session 으로의 제어기의 진단 모드가 변경될 때, 제어기는 defaultSession 에서 ResponseOnEvent (86 hex) 서비스를 통하여 설정된 이벤트에 대해서만 Reset 해야 한다.

3) 제어기가 default session 이외의 session 에서 default session 이외의 세션(현재 활성화된 진단모드 포함)으로 진단 모드 천이시, 제어기는 진단 세션을 (재)초기화 하여야 한다. 즉, ResponseOnEvent (86 hex) 서비스에 의하여 설정된 각각의 이벤트들은 Reset 되어야만 하며 Security access 의 security level 은 lock 상태가 되어야한다.
Periodic Scheduler 에 대한 설정은 세션 변경과 상관없이 활성화 상태를 유지해야 한다.
CommunicationControl 및 ControlDTCSetting 서비스는 영향을 받지 않아야 한다. 즉, session 이 변경될 때 normal communication 이 disable 상태였다면, session 변경 후에도 제어기는 normal communication 의 disable 상태를 유지해야 한다.

4) 제어기가 default session 이외의 session 에서 default session 으로 진단 세션을 변경할 때, 제어기는 ResponseOnEvent (86 hex) 서비스에 의하여 설정된 각각의 이벤트들은 Reset 되어야만 하며 Security 설정은 초기화되어야 한다.
Periodic Scheduler 에 대한 설정은 중지되며, CommunicationControl 및 ControlDTCSetting 서비스는 리셋되어야 한다. 즉, session 이 변경될 때 normal communication 이 disable 상태였을지라도, session 변경 후에는 reenable 되어야 한다. 제어기는 활성화된 진단 모드에서 활성화된/초기화된/변경된 설정/제어를 리셋해야 한다. 이것은 비휘발성 메모리에 프로그램된 정보를 포함하지는 않는다.

4.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	DiagnosticSessionControl Request Service ID	M	#10h	DSC
#2d	Sub-function = [diagnosticSessionType]	M	#XXh	LEV_ DS_

표 4-1. Request Message

Hex	Description	Cvt	Mnemonic
00	Reserved	M	
01	defaultSession This diagnostic session enables the default diagnostic session in the server(s) and does not support any diagnostic application timeout handing provisions. (e.g. no Tester Present service is necessary to keep the session arctive). If any other session than the defaultSession has been active in the server and the defaultSession in once again started, then the following implementation rules shall be followed : - The server shall stop the current diagnostic session when it has sent the DiagnosticSessionControl positive response message and shall start the newly requested diagnostic session afterwards. - If the server has sent a DiagnosticSessionControl positive response message it shall have re-locked the server if the client unlocked it during the diagnostic session - If the server sends a negative response message with the DiagnositcSessionControl request service identifier the active session shall be continued.	M	DS

02	programmingSession This diagnosticSession enables all diagnostic services required to support the memory programming of a server. In case the server runs the programmingSession in the boot software, the programmingSession shall only be left via an ECUReset (11 hex) service initiated by the client, a DiagnosticSessionControl (10 hex) service with sessionType equal to defaultSession, or a session layer timeout in the server. In case the server runs in the boot software when it receives the DiagnosticSessionControl (10 hex) service with sessionType equal to defaultSession or a session layer timeout occurs and a valid application software is present for both cases then the server shall restart the application software. This document does not specify the various implementation methods of how to achieve the restart of the valid application software (e.g. a valid application software can be determined directly in the boot software, during the ECU startup phase when performing an ECU reset, etc.).	C	ECUPS
03	extendedDiagnosticSession This diagnosticSession can e.g. be used to enable all diagnostic services required to support the adjustment of functions like "Idle Speed, CO Value, etc." in the server's memory. It can also be used to enable diagnostic services, which are not specifically tied to the adjustment of functions.	M	ECUEDS
04	safetySystemDiagnosticSession This diagnosticSessin enables all diagnostic services required to support safety system related functions (e.g. airbag deployment).	U	SSDS
05~7F	Reserved	M	
C : This session shall be supported if the controller is reprogrammable.			

표 4-2. diagnosticSessionType definition

Whenever the service DSC (DiagnosticSessionControl) is called successfully (that includes also calling DSC with the currently active DiagnosticSesstionType), the securityAccess rights and the timing parameters will be initialized again. When the CGW(ICU, ESU, CCU) transitions from the extendedDiagnosticSession to the non-default session, the security right should not be initialized (remain unlock condition). (refer to ES95489-01 in detail.)

4.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	DiagnosticSessionControl Response Service ID	M	#50h	DSCPR
#2d	diagnosticSessionType	M	#XXh	DS_

	SessionParameterRecord = [	M		SPREC
#3d	$P2_{CAN\_SERVER\_MAX}$ (high byte)		#XXh	
#4d	$P2_{CAN\_SERVER\_MAX}$ (low byte)		#XXh	
#5d	$P2*_{CAN\_SERVER\_MAX}$ (high byte)		#XXh	
#6d	$P2*_{CAN\_SERVER\_MAX}$ (low byte)]		#XXh	
※ $P2_{CAN\_SERVER\_MAX}$: Range = 0~50ms (resolution = 1ms) ※ $P2*_{CAN\_SERVER\_MAX}$: Range = 0~5000ms (resolution = 10ms)				

표 4-3. Positive Response Message definition

4.1.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This code shall be returned if the criteria for the request DiagnosticSessionControl are not met.

표 4-4. Supported Negative Response Codes

4.1.4 Implementaion requirements

This service is used by the diagnostic tool to enable different types of diagnostic sessions in a server. In order to execute a diagnostic service the appropriate session has to be started first.

There shall be only one diagnostic session active at a time.

Default Session (01) shall be enabled automatically by the ECU if no diagnostic session has been requested at power up.

An ECU shall return to /Default Session (01) after timeout of another diagnostic session.

An ECU shall be capable of providing all diagnostic functionality defined for the default diagnostic session under normal operating conditions.

The ECU shall first send a DiagnosticSessionControl Positive Response (50 xx) message before the new session becomes active in the ECU.

A DiagnosticSessionControl Positive Response (50 xx) message shall be returned by an ECU if the diagnostic tool requests a session that is already running. If the client has requested a diagnostic session, which is already running, then the server shall send a positive response message and be have as clause 4.1 that describes the server internal behavior when transitioning between sessions.

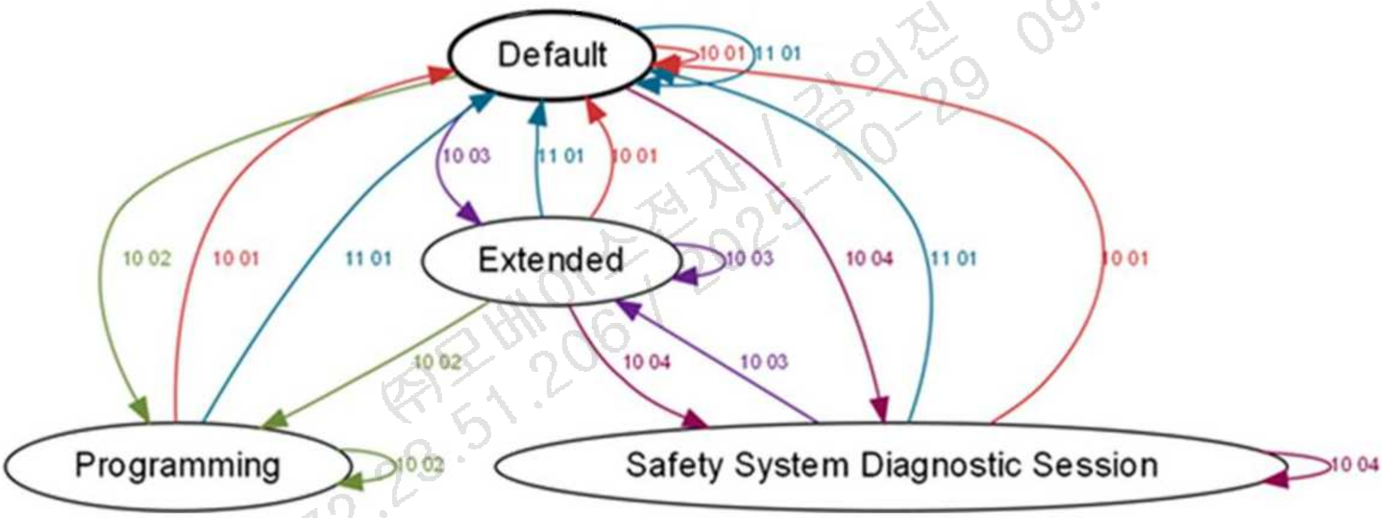
An ECU shall remain in its current diagnostic session if it is not able to switch into the re-quested diagnostic session.

The Tester Present (3E) service shall be used to keep the diagnostic sessions active by re-triggering S3server. Also any other service request keeps the diagnostic session alive.

A functional Tester Present (3E) request without response may be sent at any time, even regardless of any other service in progress.

When receiving or transmitting any diagnostic messages, including $3E service, the timer will reset. Deactivating a session leads to reinitialisation of all activities, started already.

The following state transitions shall be implemented in the ECU. (In case a session is not available in the ECU, the state transitions from and to the state are omitted):



4.2 ECUReset (11hex) service

ECU Reset 서비스는 진단기를 통하여 제어기를 Reset 시키는데 사용된다.

이 서비스는 ECUReset Request 메시지에 포함되어 있는 resetType 파라미터의 설정에 따라 제어기를 효과적으로 Reset 시킨다. 제어기는 ECUReset 서비스에 대한 Positive Response 메시지(필요 시)를 제어기가 Reset 을 수행하기 이전에 보내야 한다. 성공적인 제어기 Reset 후, 제어기는 Default Session 에서 활성화 되어야 한다.
OTA Target 제어기는 OTA 백그라운드 데이터 수신 완료 이후 자동으로 ECU Reset 되어서는 안된다.

4.2.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	EcuReset Request Service ID	M	#11h	ER
#2d	Sub-function = [resetType]	M	#XXh	LEV_ RT_

표 4-5. Request Message

Byte pos. In record	Description	Cvt	Mnemonic
00	Reserved	M	
01	hardReset This value identifies a “hard reset” condition which simulates the power-on / start-up sequence typically performed after a server has been previously disconnected from its power supply (i.e. battery). It might result in the re-initialization of both volatile memory and non-volatile memory location to predetermined values.	M	HR
02	keyOffOnReset This value identifies a condition similar to the driver turning the ignition key off and back on. This reset condition should simulate a key-off-on sequence (i.e. interrupting the switched power supply). The performed action is implementation specific and not defined by the standard. Typically the values of non-volatile memory locations are preserved; volatile memory will be initialized.	U	KOFFONR
03	softReset This value identifies a "soft reset" condition, which causes the server to immediately restart the application program if applicable. The performed action is implementation specific and not defined by the standard. A typical action is to restart the application without reinitializing of previously learned configuration data, adaptive factors and other long-term adjustments.	U	SR

04	enableRapidPowerShutDown This value requests the server to enable and perform a "rapid power shut down" function. The server shall execute the function immediately after "key/ignition" is switched off. While the server executes the power down function, it shall transition either directly or after a defined stand-by time to sleep mode. If the client requires a response message and the server is already prepared to execute the "rapid power shut down" function, the server shall send the positive response message prior to the start of the "rapid power shut down" function. The next occurrence of a "key on" or "ignition on" signal terminates the "rapid power shut down" function. The client shall not send any request messages other than the ECUReset with the sub-function disableRapidPowerShutDown in order to not disturb the rapid power shut down function. NOTE) This sub-function is only applicable to a server supporting a stand by-mode	U	ERPSD
05	disableRapidPowerShutDown This value requests the server to disable the previously enabled "rapid power shut down" function.	U	DRPSD
06~7F	Reserved	M	

표 4-6. Request message sub-function parameter definition

4.2.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	EcuReset Response Service ID	M	#51h	ERPR
#2d	resetType	M	#XXh	RT_
#3d	powerDownTime	C	#XXh	PDT
C : This parameter is present if the sub-function parameter is set to the enableRapidPowerShutDown value (04hex);				

표 4-7. diagnosticSessionControl Positive Response Message definition

Definition	Value	Description
powerDownTime This parameter indicates to the client the minimum time of the stand-by-sequence the server will remain in the power down sequence.	00 – FE hex	1 second per count (0-254 seconds powerDowntime)
	FF hex	Failure or time not available

표 4-8. powerDownTime parameter definition

4.2.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This code shall be returned if the criteria for the ECUReset request is not met.

표 4-9. Supported Negative Response Codes

4.3 SecurityAccess (27 hex) service

이 서비스의 목적은 보안/배기가스/안전 등의 이유로 접근이 제한된 데이터나 진단 서비스를 수행하기 위한 방법을 제공한다. 보안 컨셉으로는 Seed 와 Key 간의 관계를 이용한다.

이 서비스에 대한 실례는 아래와 같다.

- 진단기가 “Seed”를 요청
- 제어기가 “Seed”를 송신
- 진단기가 (수신한 “Seed”에 대한) “Key”를 송신
- 제어기가 “Key”에 대한 유효함과 제어기 자체 보안을 해제함을 응답

“Seed”는 0000 hex 또는 FFFF hex 가 아닌 Random 값으로 2 바이트 이상의 크기를 가져야 한다. “Key”는 “Seed”를 이용하여 Security Algorithm 으로 계산된 값으로, Security Algorithm 은 ASK 와 OEUK 를 적용한다. OEUK 는 ASK 가 수행 가능한 모든 권한을 갖고 있으며 추가로 Rollback 수행 권한도 갖고 있다.

3 회에 걸쳐 Security access 잘못된 "sendKey" 요청을 시도한 후, 제어기는 Security access 요청에 대해 180 초 동안 잠겨야한다. 본 요구사항은 Mandatory 사항이며 180 초 Delay Timer 가 동작하는 동안 테스터가 제어기에 Seed 를 요청하면 제어기는 부정응답 코드를 전송한다. “ requiredTimeDelayNotExpired ”.
관련 파라메터 설명은 표 4-80 을 참조한다.

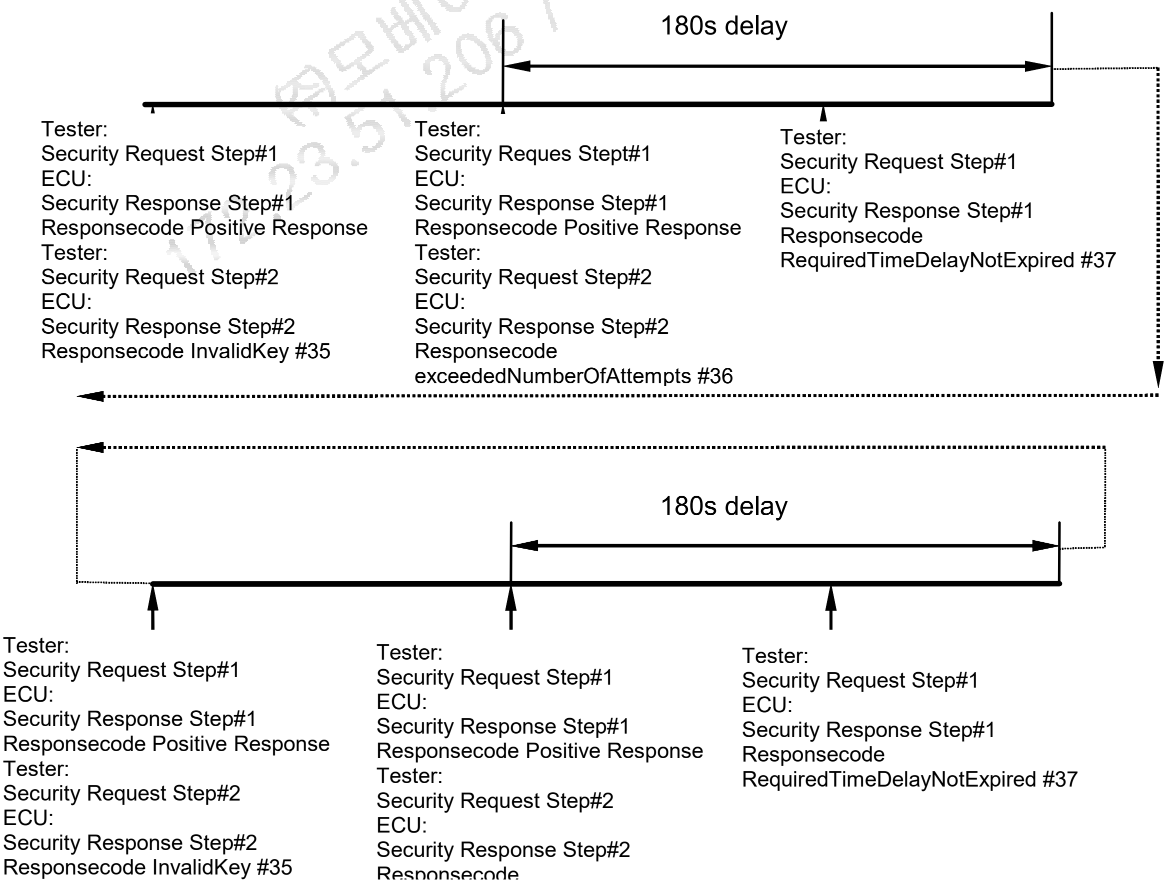


그림 4-2. Example of SecurityAccess service procedure (false attempt)

Name	Description	Value
Delay_Timer	This value represents the required minimum time between security access attempts. This timer shall started when Att_Cnt reaches Att_Cnt Limit(Att_Cnt == Att_Cnt_Limit). A server shall implement a single timer for all security levels(e.g. ASK, OEUK), which means that all security levels share a single timer. When memory initialization such as ECU reset occurs when the delay timer has not yet expired, the controller starts the timer again from 180 seconds. Therefore, the parameter may be managed as volatile. IMPORTANT : this value shall not be initialized by Diagnostic Session Control Service(0x10)	(volatile) 180sec
Att_Cnt_Limit	This value represents the number of false security access attempts before a delay time (Start_Delay) is inserted.	(constant) 3 times
Att_Cnt	this variable tracks the current value of the number of false access attempts. A separate counter is required for each individual security level. A server have same limits for each security level. Att_Cnt shall be implemented for a given SubFunction. If a valid key is received, the count is initialized to zero. And Delay_Timer expired then Att_cnt of security level which have reached Att_Cnt_Limit shall be initialized to zero. IMPORTANT : this value shall not be initialized by ECU reset or Diagnostic Session Control Service(0x10) NOTE : Counters can be managed separately for the boot loader and application.	(non-volatile)0~3 NOTE : This parameters can be used.

표 4-80 SecurityAccess parameters

SecurityAccess 의 상태 천이도는 그림 4-11 과 같고 상세 동작은 표 4-81 과 같이 정의된다.

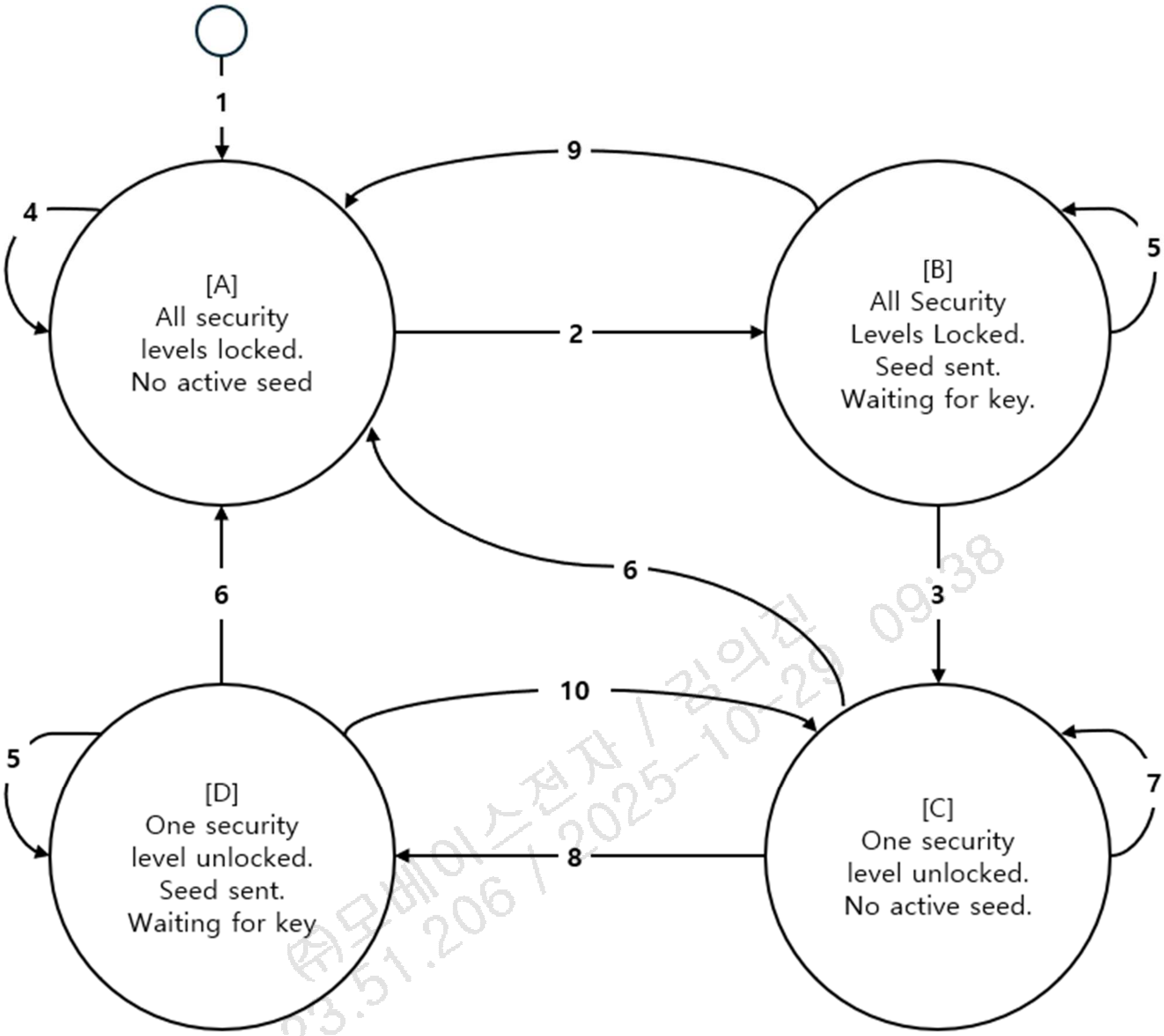


그림 4-11 SecurityAccess state chart

No	Operation		Condition	Action
1			Start/Restart of ECU Application(e.g. ECU reset, Power cycle, Key cycle, Sleep -> wake transition, etc)	Initialize Att_Cnt (Load Att_Cnt from NVM) If at least one of the Att_Cnt reaches Att_Cnt_Limit then Delay_Timer shall be started.
2		AND	SecurityAccess requestSeed received Message length OK Delay expired	Generate and store Seed for the requested securityAccessType(e.g. ASK, OEUK). Transmit SecurityAccess positive response on requestSeed request with Seed as active for the requested securityAccessType.
3		AND	SecurityAccess sendKey received SubFunction: yy == xx+1 Sendkey Subfunction must be of the expected securityAccessType (the	Att_Cnt = 0 for SubFunction xx Store Att_Cnt in non-volatile memory Unlock security level for SubFunction xx. Transmit SecurityAccess positive response on sendKey request.

			active stored seed is for the corresponding requestSeed securityAccessType)	
			Message length OK	
			Key OK	
4	OR	AND	SecurityAccess requestSeed received	Transmit negative response NRC13
			Message Length NOK	
			SecurityAccess sendKey received	Transmit negative response NRC 24
		AND	SecurityAccess requestSeed received	Transmit negative response NRC 37
			Message length OK	
			Delay NOT expired	
			SecurityAccess request results in a general negative response code (e.g. minimum length, SubFunction supported) according to the general negative response handling (see Annex A).	Transmit negative response code as defined in Annex A
			Delay Timer Expiration Occurs for SubFunction xx.	Att_Cnt = 0 for SubFunction xx. Store Att_Cnt in non-volatile memory
5		AND	SecurityAccess requestSeed received	Generate new seed and transmit SecurityAccess positive response with the new seed for the requested securityAccessType Save SubFunction: xx = securityAccess Type.
			Static_Seed == False NOTE : Only randomseed (Static_Seed == False) is allowed for HMC. For Static seed is described in ISO14229-1	
6			DiagnosticSessionControl accepted or session timeout occurs.	Start appropriate diagnostic session. Lock ECU.
7		AND	SecurityAccess requestSeed received Message Length NOK	Transmit negative response NRC13
			SecurityAccess sendKey received	Transmit negative response NRC24
		AND	SecurityAccess requestSeed received	Transmit negative response NRC37
			Message length OK	
			Delay NOT expired	
		AND	SecurityAccess requestSeed received	Transmit SecurityAccess positive response with zero seed.
			Requested level is unlocked	
			SecurityAccess request results in a general negative response code (e.g. minimum length, SubFunction supported)	Transmit negative response code as defined in Annex A

			according to the general negative response handling (see Annex A).	
			Delay Timer Expiration Occurs for SubFunction xx.	Att_Cnt = 0 for SubFunction xx Store Att_Cnt in non-volatile memory
8		AND	SecurityAccess requestSeed received Request level is NOT unlocked Message length OK Delay expired	Generate and store Seed for the requested securityAccessType Save SubFunction: xx = securityAccess Type. Transmit SecurityAccess positive response on requestSeed request with Seed as active for the requested securityAccessType.
9	OR	AND	SecurityAccess sendKey received SubFunction: yy == xx+1 Message length OK Key NOK (Att_Cnt+1) < Att_Cnt_Limit	Att_Cnt++ for SubFunction xx Store Att_Cnt in non-volatile memory Transmit negative response NRC 35.
		AND	SecurityAccess sendKey received SubFunction:yy==xx+1 Message length OK Key NOK (Att_Cnt+1) >= Att_Cnt_Limit	Att_Cnt = Att_Cnt_Limit for SubFunction xx. Start Delay_Timer. Store Att_Cnt in non-volatile memory. Transmit negative response NRC 36.
		AND	SecurityAccess sendKey received SubFunction:yy<>xx+1	Transmit negative response NRC 24
		AND	SecurityAccess sendKey received SubFunction:yy==x+1 Message length NOK	Transmit negative response NRC 13
			DiagnosticSessionControl accepted or session timeout occurs.	Start appropriate diagnostic session.
			SecurityAccess request results in a general negative response code (e.g. minimum length, SubFunction supported) according to the general negative response handling (see Annex A).	Transmit negative response code as defined in Annex A.
		AND	SecurityAccess requestSeed received Message Length NOK	Transmit negative response NRC13
10	OR	AND	SecurityAccess requestSeed received Requested level is unlocked	Transmit SecurityAccess positive response with zero seed
		AND	SecurityAccess sendKey received SubFunction: yy==xx+1	Att_Cnt = 0 forSubFunction xx. Store Att_Cnt in non-volatile memory.

				Message length OK Key OK	Lock currently unlocked security level. Unlock security level forSubFunction xx.
			AND	SecurityAccess sendKey received SubFunction: yy==xx+1 Message length OK Key NOK (Att_Cnt+1) < Att_Cnt_Limit SecurityAccess sendKey received	Att_Cnt++ for SubFunction xx. Store Att_Cnt in non-volatile memory. Transmit negative response NRC 35.
			AND	SecurityAccess sendKey received SubFunction:yy=xx+1 Message length OK Key NOK (Att_Cnt+1) >= Att_Cnt_Limit	Att_Cnt = Att_Cnt_Limit for SubFunction xx. Start Delay_Timer for SubFunction xx. Store Att_Cnt in non-volatile memory. Transmit negative response NRC 36.
			AND	SecurityAccess sendKey received SubFunction: yy <> xx+1	Transmit negative response NRC 24
			AND	SecurityAccess sendKey received SubFunction: yy == xx+1 Message length NOK	Transmit negative response NRC 13
				SecurityAccess request results in a general negative response code (e.g. minimum length, SubFunction supported) according to the general negative response handling (see Annex A).	Transmit negative response code as defined in Annex A.
			AND	SecurityAccess requestSeed received Message Length NOK	Transmit negative repsonse NRC 13

표 4-81 SecurityAccess State transition

Delay Timer 가 만료되면, Delay Timer 를 시작시킨 Att_Cnt(X)에 대해서만 값을 0 으로 초기화 시켜야 한다.
예)

Sequence	Att_Cnt(ASK)	Att_Cnt(OEUK)	DelayTimer	Action
1	0	0	–	–
2	1	0	–	ASK key NOK
3	2	0	–	ASK key NOK
4	2	1	–	OEUK key NOK
5	2	2	–	OEUK key NOK
6	3	2	Start 180sec Delay_Timer	ASK key NOK
7	0	2	Timer Expired	–

8	0	3	Start 180sec Delay_Timer	OEUK key NOK
9	0	0	Timer Expired	

표 4-82 Att_Cnt(X) Example

DelayTimer 는 Single timer 를 사용하며 DelayTimer 를 모든 레벨에서 공유한다.
즉 당사의 Security level 인 ASK 와 OEUK 를 적용 중인 제어기의 경우 ASK 의 Timer 와 OEUK 의 Timer 는 동일하며 특정 Security level 에 의해 DelayTimer 가 동작하고있는 경우 또 다른 레벨 또한 Timer 가 동작하고 있는 것으로 판단한다. 따라서 DelayTimer 가 만료되지 않았을때 Seed 요청이 있으면 Security level unlock 상태와 무관하게 NRC 37 로 대응한다.

NOTE : Att_Cnt 는 별도 관리하므로 DelayTimer 만료시 DelayTimer 를 시작시킨 Security Level 의 Att_Cnt 만 0 으로 초기화한다.

Boot SW 와(Programming session) Application SW 가 별도의 SW 로 구분이 되는 경우에 Att_Cnt(X) 값을 Boot SW 와 Application 에서 별도로 관리하는 것을 허용하지만, Boot SW 와 Application 에서 Att_Cnt(X)값을 공유하는 것을 강력히 권장한다.

제어기가 Security 를 지원하지만, 이미 보안이 해제된 상태라면, Security Access 서비스의 'request Seed' 메시지를 요청 받았을 때, 'Seed'값의 모든 바이트를 '0'으로 한 Positive Response 메시지로 응답한다. 진단기는 이를 통하여, 제어기의 보안이 이미 해제되었음을 확인한다.
'requestSeed' 서브함수 파라미터 값은 항상 홀수가 되어야 하며, 이에 상응하는 동일 레벨에 대한 'sendKey' 서브함수 파라미터 값은 'requestSeed' 서브함수 파라미터 값 + 1 값이 되어야 한다.

4.3.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	SecurityAccess Request Service ID	M	#27h	SA
#2d	Sub-function = [securityAccessType = requestSeedType#1 requestSeedType#2]	M	#XXh= [#01h/0Bh #11h/#13h]	LEV_ SAT_RSD1 SAT_RSD2
※ Type#1 : Normal Seed/Key Algorithm (size ≤ 5-byte) If a ECU supports a specific state for flash(Unlock flash state), 0B hex can be used. ※ Type#2 : Advanced Seed/Key (ASK) Algorithm (size > 5-byte) ※ ASK is an improved version of Normal Seed/Key Algorithm. Type#1 can be replaced with Type#2 to improve Security. ※ Type#3 : OEUK Algorithm - 13h				

표 4-10. Request Message definition – sub-function = #01h / #0Bh / #11h / #13h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	SecurityAccess Request Service ID	M	#27h	SA
#2d	Sub-function = [securityAccessType = sendCertificateandrequestSeed]	M	#41h	LEV_ SAT_SCRSD

#3d .. #nd	securityAccessDataRecord[] = [Certificate]	M	#XXh .. #XXh	SECACCDR
※ In case of application of Secure Access function in CGW, sub-function #41h must be used. (refer to ES95489-01)				

표 4-11. Request Message definition – sub-function = #41h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	SecurityAccess Request Service ID	M	#27h	SA
#2d	Sub-function = [securityAccessType = sendKey]	M	#XXh=[#02h/0Ch #12h/#14h #42h]	LEV_ SAT_SK
#3d .. #nd	sendKey[] = [key (high byte) .. key (low byte)]	M .. U	#XXh .. #XXh	SEKEY_ KEYHB .. KEYLB
The data size of key from the client if the securityAccessType parameter(=02/0C hex) shall grater than 2-byte / securityAccessType parameter(=12 hex) shall be 8-byte / securityAccessType parameter(= 14/42 hex) shall be 256-byte. ※ 256-byte Key is the value of “PKCS#1 Signature“ by “the tester’s private key“				

표 4-12. Request Message definition – sub-function = #02h / #0Ch / #12h / #14h / #42h

4.3.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	SecurityAccess Response Service ID	M	#67h	SAPR
#2d	securityAccessType	M	#XXh	SAT_
#3d .. #nd	securitySeed[] = [seed (high byte) .. seed (low byte)]	C .. C	#XXh .. #XXh	SECSEED_ SEEDHB .. SEEDLB
C : The presence of this parameter depends on the securityAccessType parameter. It is mandatory to be present if the securityAccessType parameter(=01/0B/11/13/41 hex) indicates that the client wants to retrieve the seed from the server. The data size of seed from the server if the securityAccessType parameter (=01/0B hex) shall greater than 2-byte. The data size of seed from the server if the securityAccessType parameter(=11/13/41 hex) shall be 8-byte.				

표 4-13. Positive Response Message definition

4.3.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong

22	conditionNotCorrect	M	This code shall be returned if the criteria for the request Security Access are not met.
24	requestSequenceError	M	Send if the 'sendKey' sub-function is received without first receiving a 'requestSeed' request message.
31	requestOutOfRange	M	This code shall be sent if the user optional securityAccessDataRecord contains invalid data.
35	invalidKey	M	Send if an expected 'sendKey' sub-function value is received and the value of the key does not match the server's internally stored/calculated key.
36	exceededNumberOfAttempts	M	Send if the delay timer is active due to exceeding the maximum number of allowed false access attempts.
37	requiredTimeDelayNotExpired	M	Send if the delay timer is active and request is transmitted.

표 4-14. Supported Negative Response Codes

4.3.4 Message Flow Example SecurityAccess – server is in a "locked" state

In the following example, it is assumed:

- the node has a physical request CANId of $7E0
- the node has a physical response CANId of $7E8
- AABB hex is a valid seed value for the target node
- CCDD hex is a valid key value for the target node

4.3.4.1 SecurityAccess – requestSeed

CAN ID	Client Request / Server Response	DLC	Data Byes	Comments
7E0	Physical Request	8	02 27 01 55 55 55 55 55	Single Frame
7E8	Response	8	04 67 01 AA BB AA AA AA	Single Frame

표 4-15 SecurityAccess(requestSeed) Message Example

4.3.4.2 SecurityAccess – sendKey

CAN ID	Client Request / Server Response	DLC	Data Byes	Comments
7E0	Physical Request	8	04 27 02 CC DD 55 55 55	Single Frame
7E8	Response	8	02 67 02 AA AA AA AA AA	Single Frame

표 4-16 SecurityAccess(sendKey) Message Example

4.3.5 Message Flow Example SecurityAccess – server is in a "unlocked" state

4.3.5.1 SecurityAccess – requestSeed

CAN ID	Client Request / Server Response	DLC	Data Byes	Comments
7E0	Physical Request	8	02 27 01 55 55 55 55 55	Single Frame
7E8	Response	8	04 67 01 00 00 AA AA AA	Single Frame

표 4-17 SecurityAccess(requestSeed) Message Example

4.4 CommunicationControl (28 hex) service

이 서비스는 제어기의 메시지 송수신을 ON/OFF 하는 기능을 가진다.
Note) 이 서비스는 게이트웨이 제어기의 라우팅 송/수신에 영향을 주지 않는다.
이 서비스는 제어기의 진단메시지 송/수신에 영향을 주지 않는다.

Communication Control (28hex) disableRxAndTx (03hex) request 메시지 이후 $S3_{server}$ 내 어떠한 명령도 수신하지 않을 경우 Default Session 으로 돌아가 일반 메시지 전송과 “Communication Time Out” 모니터링을 500ms 이내에 재개해야 한다.
단, 제어기별 통신 재개 시점이 상이할 수 있으니, 해당 제어기에 대한 CAN 관련 고장 진단은 통신 재개 이후에 실시한다. (통신 재개는 500ms 이내에 재개, 고장 진단은 500ms 이후에 실시 함)
ES95480-00 의 ‘Initialization Time Parameter’와 같이, Communication Control (28hex) enableRxAndTx (00hex) 서비스 후, 제어기는 Normal 메시지의 송신 및 Time Out 기능을 500ms 이내에 재개하여야 한다.

4.4.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	CommunicationControl Request Service ID	M	#28h	CC
#2d	Sub-function = [controlType]	M	#XXh	LEV_ CTRLTP
#3d	communicationType	M	#XXh	CTP
Communication Type : See annex B.1 for the coding of the communicationType data parameter in detail.				

표 4-18. Request Message definition

Hex	Description	Cvt	Mnemonic
00	enableRxAndTx This value indicates that the reception and transmission of messages shall be enabled for the specified communicationType.	M	ERXTX
01	enableRxAndDisableTx This value indicates that the reception of messages shall be enabled and transmission shall be disabled for the specified communication Type.	U	ERXDTX
02	diableRxAndEnableTx This value indicates that the reception of messages shall be disabled and transmission shall be enabled for the specified communication Type.	U	DRXETX
03	disableRxAndTx This value indicates that the reception and transmission of messages shall be disabled for the specified communicationType.	M	DRXTX
04~7F	Reserved	M	

표 4-19. Request message sub-function (controlType) parameter definition

HEX	Parameter Name	Cvt	Mnemonic

01	normalCommunicationMessages This value references all application-related communication (interapplication signal exchange between multiple in-vehicle servers).	M	NCM
02	networkManagementCommunicationMessages This value references all network management related communication.	M	NMCM
03	networkManagementCommunicationMessages and normalCommunicationMessages This value references all network management and application-related communication.	M	NMCMNCM

표 4-20. Request message communication Type definition

4.4.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	CommunicationControl Response Service ID	M	#68h	CCPR
#2d	controlType	M	#XXh	CTRLTP

표 4-21. Positive Response Message definition

4.4.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	Used when the server is condition that cannot disable/enable the requested communication type.
31	requestOutOfRange	M	The server shall use this response code, if it detects and error in the communicationType parameter.

표 4-22. Supported Negative Response Codes

4.4.4 Message Flow Example

4.4.4.1 Communication Control – EnableRxAndTx

CAN ID	Client Request / Server Response	DLC	Data Byes	Comments
7DF	Functioanl Request	8	03 28 00 03 55 55 55 55	Single Frame
7E8	Response	8	02 68 00 AA AA AA AA AA	Single Frame
7E9	Response	8	02 68 00 AA AA AA AA AA	Single Frame

표 4-23 CommunicationControl Message Example

4.4.4.2 Communication Control – DisableRxAndTx

CAN ID	Client Request / Server Response	DLC	Data Byes	Comments
7DF	Functional Request	8	03 28 03 03 55 55 55 55	Single Frame
7E8	Response	8	02 68 03 AA AA AA AA AA	Single Frame
7E9	Response	8	02 68 03 AA AA AA AA AA	Single Frame

표 4-24 CommunicationControl Message Example

4.5 Authentication (29 hex) service

HMC have conventional security methods with SecurityAccess($27), Nomal seed key(shall not be used), Advanced Seed key(ASK), and C-SAC(Certificate based security access control). Those security methods utilize securityAccessType of Security access($27) service. Conventional method might be replaced by Authentication($29) and it give some advantage of removing exception of SecurityAccess($27) for HMC specific. The reason why CCU / ICU / ESU SecurityAccess login shall not be initialized whenever DSC(Diatnostic Session Control) is called is that C-SAC shall not be initilize by functionally addressed DSC causes initialization of the SecurityAccess login. If login is initialized whenever DSC is called, it is hard to route diagnostic services to the target ECU because functionally addressed DSC can cause initialization. So this exception is defined for the Central Gateway. Authentication would be an option to overcome this exception.
To send diagnostic services to the target ECU, Client shall optain right through SecurityAccess(C-SAC). This is conventional method and it shall be changed to APCE method of the Authentication($29) service.

The purpose of this service is to provide a means for the client to prove its identity, allowing it to access data and/or diagnostic services, which have restricted access for, for example security, emissions, or safety reasons. Diagnostic services for downloading/uploading routines or data into a server and reading specific memory locations from a server are situations where authentication may be required. Improper routines or data downloaded into a server could potentially damage the electronics or other vehicle components or risk the vehicle's compliance to emission, safety, or security standards. On the other hand, data security might be violated when retrieving data from a server.

NOTE 1 The generation, distribution and storage of cryptographic material is out of scope for this document.

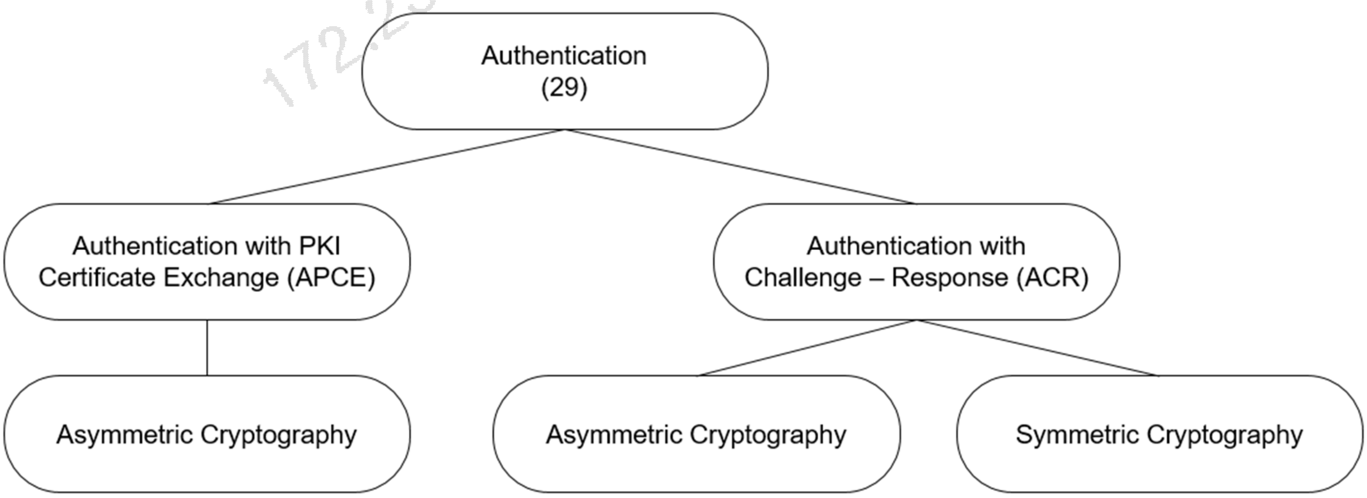


Figure 4-8 Overview over the Authentication service security concepts

NOTE 2 Figure 4-8 shows only the major options of authentication.

To identify the supported concept in the server, the client can send this service with the SubFunction parameter 'authenticationConfiguration'.

NOTE 3 Bidirectional authentication is not a scope for the HMC specification.
NOTE 4 Authentication with Challenge – Response (ACR) is not a scope for the HMC specification

4.5.1 Authentication with PKI Certificate Exchange (APCE)

The Authentication service is used for authentication, deAuthentication and explicit certificate transmission. The 'authenticationTask' SubFunction parameter identifies the corresponding task to process.

"deAuthenticate", this SubFunction parameter actively ends the authenticated state.

"verifyCertificateUnidirectional", this SubFunction parameter initiates the unidirectional Authentication procedure to only authenticate the client against the server.

"verifyCertificateBidirectional", this SubFunction parameter initiates the bidirectional Authentication procedure to authenticate the client against the server and the server against the client.

"proofOfOwnership", this SubFunction parameter is used to transmit the proof of ownership data to the client.

"transmitCertificate", this SubFunction parameter transfers a certificate independently or subsequent to a previous authentication.

Prerequisites:
Different sets of certificates (and corresponding private keys) shall be available at both sides (client and server):
In case of unidirectional authentication, the client needs a certificate client with its private key, which allows the client to identify itself as a legitimate client. Depending on the trust model of the public key infrastructure (PKI), the server may need a certificate of the certificate authority (CA) which issued and signed the certificate client.

In case of bidirectional authentication, the client needs a certificate client with its private key, which allows the client to identify itself as a legitimate client. Additionally, the server also needs a certificate server with its private key, which allows the server to identify itself as a legitimate server. Depending on the trust model of the public key infrastructure (PKI), the client and the server may need a certificate of the certificate authority (CA) which issued and signed the certificate client and certificate server.

The authentication process triggered by "verifyCertificateUnidirectional" or "verifyCertificateBidirectional" takes place in two steps as depicted in Figure 4-9.

4.5.1.1 Unidirectional authentication:

— Client creates challenge client if used in proof of ownership client (1).
It is recommended to create the challenge based on ISO/IEC 9798-3 (unilateral, two pass authentication) or as a security-related equivalent challenge to the one as per ISO/IEC 9798-3.

— Client sends its certificate client and, if generated in (1), its challenge client via SubFunction verifyCertificateUnidirectional (2).

— Server verifies the certificate client (3).

— Server creates a challenge server (4).

— If session key establishment based on the Ephemeral Diffie-Hellman key agreement is indicated by the client in (2), then the server generates an ephemeral private/public key pair to later derive a session key for further secure communication (5).
NOTE 1 Diffie-Hellman key agreement is necessary if algorithms are used in the certificates which are only usable for signature calculation, but are not usable for a key agreement protocol.

— Server sends the challenge server and, if generated in (5), its ephemeral public key (7).

— If session key establishment based on the Ephemeral Diffie-Hellman key agreement is indicated by the client in (2), then the client also generates an ephemeral private/public key pair to later derive a session key for further secure communication (9).
NOTE 2 Diffie-Hellman key agreement is necessary if algorithms are used in the certificates which are only usable for signature calculation, but are not usable for a key agreement protocol.

— Client calculates proof of ownership client by building an appropriate authentication token whose content to be signed include at least (parts of) the challenge server and, if generated in (9), its ephemeral public key (10).

It is recommended to build the authentication token based on ISO/IEC 9798-3 (unilateral, two pass authentication) or as a security-related equivalent authentication token to the one as per ISO/IEC 9798-3.
— Client sends the proof of ownership client and, if generated in (9), its ephemeral public key via SubFunction proofOfOwnership (11).

— Server verifies the proof of ownership client with the public key from the received certificate client (12).

— If session key establishment is indicated by the client in (2), then the server creates or derives and enables the session key(s) for further secure communication and sets up the session key info (13).

— The server grants access to diagnostic objects according to access rights (14).

— The server responds that the authentication was successful and, if present from (13), sends the session key info (15).

— If session key establishment is indicated by the client in (2), then the client extracts the session key(s) from the session key Info or derives the session key(s) for further secure communication (16).

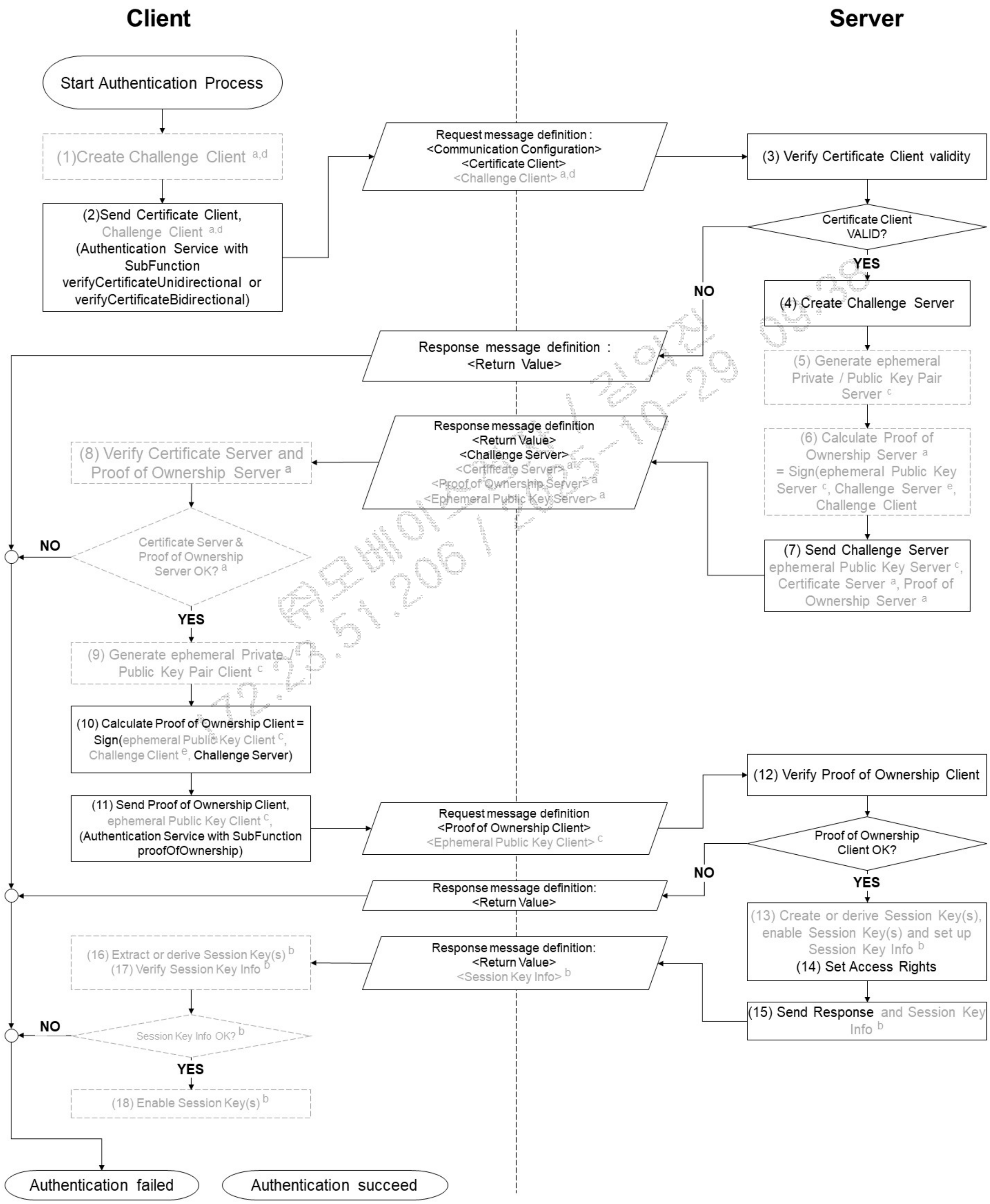
— If session key establishment is indicated by the client in (2), then the client verifies the session key info using the session key(s) (17).

NOTE 3 Step (17) ensures that the session key establishment is complete and valid.

— If session key establishment is indicated by the client in (2), then the client enables the session key(s) for further secure diagnostic communication (18).

NOTE 4 When using unidirectional authentication, the server does not authenticate against the client. Thus, the client cannot be sure that it is communicating with the correct server.

.



Key
[a] When using bidirectional authentication.
[b] Only when using secure diagnostic communication.
[c] Only when using Diffie-Hellman key agreement for secure diagnostic communication.
[d] If challenge client is used in Proof of Ownership client.
[e] Optionally include challenge server and challenge client in each Proof of Ownership server and client.

Figure 4-9 Authentication sequence with PKI Certificate Exchange (APCE)

4.5.2 Common requirements

The authentication is designed to secure applicable diagnostic sessions/functions/services. Therefore the following sequence of services shall be required:
- Authentication service,
- Any diagnostic service that is secured or restricted by authentication.

For authentication there is no direct relationship to diagnostic session or security level. Once authenticated, the server shall be in an authenticated state which it only shall leave if a security timeout occurs, a mileage offset limit is reached or the authenticated state is intentionally left by the “deAuthenticate” request. Applicable diagnostic services assigned to the corresponding authentication settings shall be accessible as long as the client is authenticated.

There are several possibilities to leave the authenticated state:
- Explicitly: the authentication request shall be called with the SubFunction parameter “deAuthenticate”.
- Implicitly using a timeout as fallback: a timer shall be installed. The timer shall start when transitioning to the authenticated state. If the timer times out, then the authenticated state shall be ended. The timeout can be set individually or can be bound to an existing timing parameter (e.g. session layer timing parameter definitions). At least every request message of the same diagnostic protocol received by the transport protocol layer shall keep the authenticated state active and reset the timeout period.

NOTE 1 The timer value is vehicle manufacturer specific.

It is recommended that the usage of the timeout is based on a reliable internal time.
- Implicitly using a mileage offset limit as fallback: a mileage monitoring shall be installed. The mileage monitoring shall start when transitioning to the authenticated state. If the mileage limit is reached, then the authenticated state shall be ended.

NOTE 2 The mileage offset limit is vehicle manufacturer specific.

It is recommended that the usage of the mileage monitoring is based on reliable mileage information.
The explicit exit condition shall be mandatory, the implementation of at least one of the implicit exit conditions shall be mandatory, both are optionally possible.

If a server supports authentication and is already in an authenticated state when an authentication request message is received from the same client, that server shall stay in the authenticated state until the authentication is again successfully completed. The server shall change its state to the new received authentication information.

Unsuccessful attempts to access an authenticated state shall not prevent other diagnostic communication. An authenticated state shall be linked to a certain diagnostic channel. Multiple clients can be handled on multiple channels with different authentication settings. The basis for this would be a multi-user server system.

Servers which provide security shall support the NRC 34 “authenticationRequired”, if a secured service is requested while the server is in a non-authenticated state.

As an option, session keys can be established and used to further secure communication between client and server.

This can be achieved, for example, using one of the following approaches:
- by creation of the session key on the server side and an encrypted transmission to the client, or
- by using an asymmetric key agreement protocol, or
- by derivation of existing pre-shared keys on both sides (in case of symmetric cryptography).

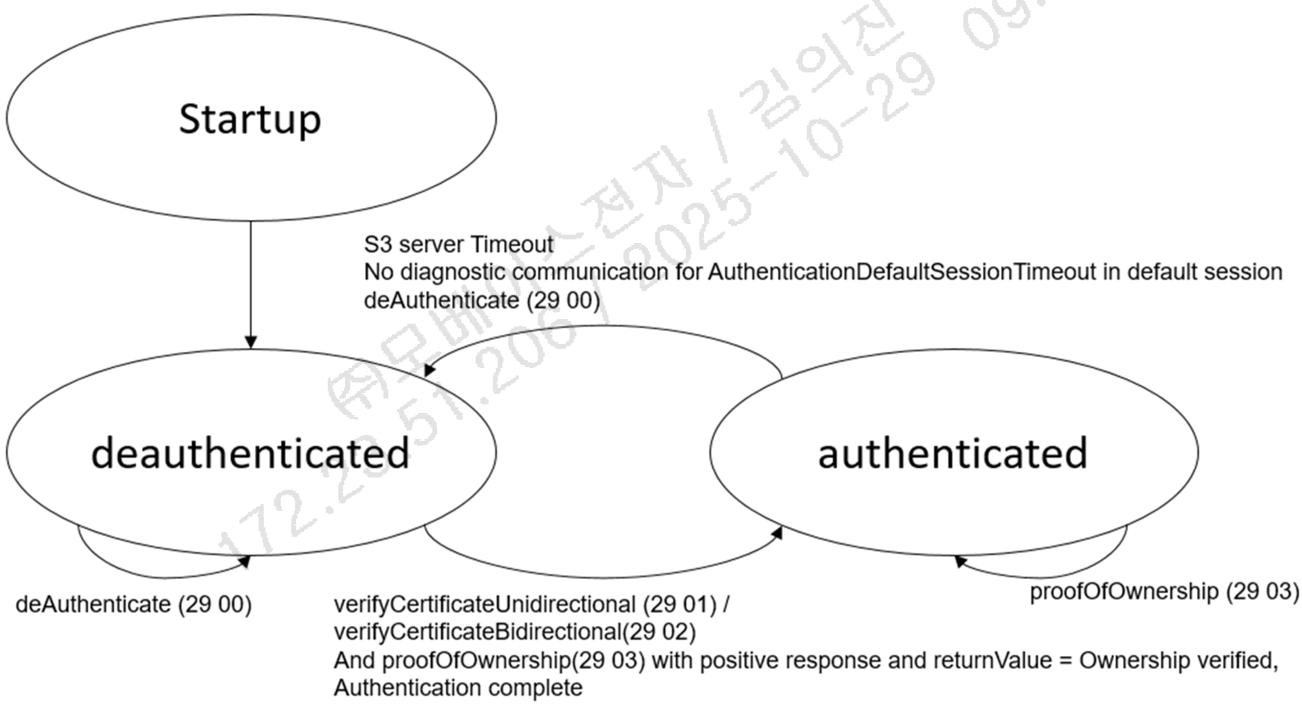


Figure 4-10 Authentication state

Timing Parameter	Description	Type	Recommended Value	Timeout
AuthenticationDefaultSessionTimeout	Time for the server to keep a authenticated state session in defaultSession while not receiving valid diagnostic request message. The tolerance is -0ms, +200ms.	Timer reload value	N/A	10000ms

Table4-60. Timing parameter definitions – AuthenticationDefulatsessionTimeout

Timer parameter	Action	Description

AuthenticationDefault session	Start	ECU is in default session with authenticated state.
	Reset	valid diagnostic request was received within 10000ms(-0ms/+200ms) after the last Server_T.data confirmed. Persisted authentication mode is deactivated.
	Stop	ECU is in non-default session. ECU is in deauthenticated state.

Table4-61. Timer of the AuthenticationDefaultsession

4.5.3 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	Authentication Request SID	M	#29h	ARS
#2d	Sub-function = [authenticationTask = deAuthenticate]	M	#00h	LEV_AT_DA

Table4-62. Request message - sub-function = #00h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	Authentication Request SID	M	#29h	ARS
#2d	Sub-function = [authenticationTask = verifyCertificateUnidirectional]	M	#01h	LEV_AT_VCU
#3d	communicationConfiguration[] =[noSecuredCommunication]	M	#00h	COCO
#4d #5d	lengthOfCertificateClient[] = [byte#1(MSB) byte#2(LSB)]	M	 #XXh #XXh	LOCECL
#6 … #m+5	certificateClient[] = [byte#1 … byte#m]	M	 #XXh … #XXh	CECL
#m+6 #m+7	lengthOfChallengeClient[] = [byte#1(MSB) byte#2(LSB)]	M	 #00h #00h	LOCHCL
#m+8 … #n+m+7	challengeClient[] = [byte#1 … byte#n	C	 #XXh … #XXh	CHCL
C : The presence of this parameter depends on the length information parameter of the lengthOfChallengeClient. If lengthOfChallengeClient is equal to 0x0000 then the parameter challengeClient will not be transmitted.				

Table4-63. Request message - sub-function = #01h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	Authentication Request SID	M	#29h	ARS
#2d	Sub-function = [authenticationTask = proofOfOwnership]	M	#03h	LEV_AT_POWN

#3d #4d	lengthOfProofOfOwnershipClient[] = [byte#1(MSB) byte#2(LSB)]	M	 #XXh #XXh	LPOWNCL
#5 … #m+4	certificateClient[] = [byte#1 … byte#m]	M	 #XXh … #XXh	POWNCL
#m+5 #m+6	lengthOfEphemeralPublicKeyClient[] = [byte#1(MSB) byte#2(LSB)]	M	 #00h #00h	LOEPKCL
#m+7 … #n+m+6	ephemeralPublicKeyClient[] = [byte#1 … byte#n	C	 #XXh … #XXh	EPKCL
C : The presence of this parameter depends on the length information parameter of the lengthOfEphemeralPublicKeyClient. If lengthOfEphemeralPublicKeyClient is equal to 0x0000 then the parameter ephemeralPublicKeyClient will not be transmitted.				

Table4-64. Request message – sub-function = #03h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	Authentication Request SID	M	#29h	ARS
#2d	Sub-function = [authenticationTask = authenticationConfiguration]	M	#08h	LEV_AT_AC

Table4-65. Request message – sub-function = #08h

Hex	Description	Cvt	Mnemonic
00	deAuthenticate Request to leave the authenticated state.	M	DA
01	verifyCertificateUnidirectional Initiate Authentication by verifying the Certificate	M	VCU
02	verifyCertificateBidirectional Initiate Authentication by verifying the Certificate and generating a Proof of Ownership from the server.	U	VCB
03	proofOfOwnership Verify the Proof of Ownership from the client.	M	POWN
04	transmitCertificate Verify Certificate and extract information from Certificate to handle it according to its contents.	U	TC
05	requestChallengeForAuthentication Initiate the Authentication process by requesting server to output a challenge.	U	RCFA
06	verifyProofOfOwnershipUnidirectional Request server to verify the POWN for unidirectional authentication.	U	VPOWNU

07	verifyProofOfOwnershipBidirectional Request server to verify the client side POWN and provide server-side POWN for bidirectional authentication.	U	VPOWNB
08	authenticationConfiguration Indicates the provided authentication configuration of the server.	M	AC
09~7F	Reserved	M	RESRVD

Table4-66. Request message sub-function parameter definition

Hex	Description	Cvt	Mnemonic
00	noSecuredCommunication No Secure communication for further communication	M	NSC
01~FF	Reserved	M	RESRVD

Table4-67. Request message communicationConfiguration parameter definition

4.5.4 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	Authentication Response SID	M	#69h	ARS
#2d	Sub-function = [authenticationTask = deAuthenticate]	M	#00h	LEV_AT_DA
#3d	returnValue [] = [authenticationReturnParameter]	M	#XXh	RV

Table4-68. Positive Response Message definition - sub-function = #00h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	Authentication Response SID	M	#69h	ARS
#2d	Sub-function = [authenticationTask = verifyCertificateUnidirectional]	M	#01h	LEV_AT_VCU
#3d	returnValue[] =[authenticationReturnParameter]	M	#XXh	RV
#4d #5d	lengthOfChallengeServer[] = [byte#1(MSB) byte#2(LSB)]	M	 #XXh #XXh	LOCHSE
 #6 … #m+5	challengeServer[] = [byte#1 … byte#m]	M	 #XXh … #XXh	CHSE
#m+6 #m+7	lengthOfEphemeralPublicKeyServer[] = [byte#1(MSB) byte#2(LSB)]	M	 #00h #00h	LOEPKSE
#m+8 … #n+m+7	ephemeralPublicKeyServer[] = [byte#1 … byte#n	C	 #XXh … #XXh	EPKSE

C : The presence of this parameter depends on the length information parameter of the lengthOfEphemeralPublicKeyServer. If lengthOfEphemeralPublicKeyServer is equal to 0x0000 then the parameter ephemeralPublicKeyServer shall not be transmitted.

Table4-69. Positive Response Message definition – sub-function = #01h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	**Authentication Response SID**	**M**	**#69h**	**ARS**
#2d	Sub-function = [authenticationTask = proofOfOwnership]	M	#03h	LEV_AT_POWN
#3d	returnValue [] =[authenticationReturnParameter]	M	#XXh	RV
 #4d #5d	lengthOfSessionKeyInfo [] = [byte#1(MSB) byte#2(LSB)]	M	 #00h #00h	LOSKI
 #6 … #m+5	sessionKeyInfo [] = [byte#1 … byte#m	C	 #XXh … #XXh	SKI
C : The presence of this parameter depends on the length information parameter of the lengthOfSessionKeyInfo. If lengthOfSessionKeyInfo is equal to 0x0000 then the parameter sessionKeyInfo shall not be transmitted.				

Table4-70. Positive Response Message definition – sub-function = #03h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	**Authentication Response SID**	**M**	**#69h**	**ARS**
#2d	Sub-function = [authenticationTask = authenticationConfiguration]	M	#08h	LEV_AT_AC
#3d	returnValue [] =[authenticationReturnParameter]	M	#XXh	RV

Table4-71. Positive Response Message definition – sub-function = #08h

Hex	Description	Cvt	Mnemonic
00	RequestAccepted Request was successful.	U	RA
01	GeneralReject Request was not successful	U	GR
02	AuthenticationConfiguration APCE Indicates the provided authentication configuration as Authentication with PKI Certificate Exchange (APCE).	M	ACAPCE
03	AuthenticationConfiguration ACR with asymmetric cryptography Indicates the provided authentication configuration as Authentication with Challenge-Response (ACR) and asymmetric cryptography.	U	ACACRAC
04	AuthenticationConfiguration ACR with symmetric cryptography Indicates the provided authentication configuration as Authentication with Challenge-Response(ACR) and symmetric cryptography.	U	ACACRSC
05 ~ 0F	Reserved	M	RESRVD

10	DeAuthentication successful DeAuthentication was successful, server is protected again.	M	DAS
11	CertificateVerified, OwnershipVerificationNecessary Certificate could be verified in first step, second step is pending	M	CVOVN
12	OwnershipVerified, AuthenticationComplete Proof of Ownership could be verified, Authentication is complete.	M	OVAC
13	CertificateVerified Certificate could be verified	U	CV
14 ~ 9F	Reserved	M	RESRVD
A0 ~ CF	HMCSpecific This range of values is reserved for vehicle manufacturer specific use.	U	HMCS
D0 ~ FE	SystemSupplierSpecific This range can be used while development time and shall not be used after production.	U	SSS
09 ~ 7F	Reserved	M	RESRVD

Table4-72. Positive Response Message – authenticationReturnParameter parameter definition

4.5.5 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionsNotCorrect	M	This NRC shall be returned if the criteria for the request Authentication are not met.
24	requestSequenceError	M	This NRC shall be returned in case of the following scenarios: – If the ‘proofOfOwnership’ SubFunction is received without first successfully processing either a ‘verifyCertificateUnidirectional’ or ‘verifyCertificateBidirectional’ request message, or – If either the ‘verifyProofOfOwnershipUnidirectional’ or ‘verifyProofOfOwnershipBidirectional’ SubFunction is received without first successfully processing a ‘requestChallengeForAuthentication’ request message.
50	Certificate verification failed – Invalid Time Period	M	Date and time of the server does not match the validity period of the Certificate.
51	Certificate verification failed – invalid Signature	M	Signature of the Certificate could not be verified.

56	Certificate verification failed – Invalid Scope	M	The scope of the Certificate does not match the contents of the server.
57	Certificate verification failed – Invalid Certificate (revoked)	M	Certificate received from client is invalid, because the server has revoked access for some reason.
58	Ownership verification failed	M	Delivered Ownership does not match the provided challenge or could not verified with the own private key
59	Challenge calculation failed	M	The challenge could not be calculated on the server side.
5A	Setting Access Rights failed	M	The server could not set the access rights.
5C	Configuration data usage failed	M	Configuration data usage failed.
5D	DeAuthentication failed	M	DeAuthentication was not successful, server could still be unprotected.
F0	CRLintegrityFailed	M	CRL integrity check (Verification of CRL Signature) is failed. The hash value of SHA_256 of Certificate authority's public key and the CRL Issuer Public Key Identifier value are not matched
F1	CRLvalidityPeriodFailed	M	CRL is expired
F2	RoleandRightofCertificateDenied	M	Role and Right of Certificate verification failed. The request service or the target from the external diagnostic tool are not matched with Certificate Holder Reference : Authorization

Table4-73. Supported Negative Response Codes

4.5.6 Message Flow Example – Unidirectional Authentication with PKI Certificate Exchange without session key establishment

In the following example, it is assumed:
NOTE : Assumption might be different from ES95489-01. Please refer to ES95489-01

- Step #1 : authentication configuration of the server is optional step
- Step #2 : No secure communication for further communication – communicationConfiguration : 00h
- Step #2 : Length of Certificate client : 600dec(0x258) [Certificate] + 501dec(0x1F5) [CRL]: 1101dec (0x44D)
- Step #2 : Parameter returnValue in the positive response indicating Certificate verified, Ownership verification necessary
- Length of the challenge Client : 0dec (0x0000)
- Length of Proof of Ownership : 256dec (0x100)

#Step #1 : RequestAuthentication Configuration

Data byte	Description	Byte Value	Mnemonic
#1	Authentication Request SID	29h	ARS
#2	authenticationTask = authenticationConfiguration	08h	LEV_AT_AC

Table4-74. Authentication Configuration request message flow example – step #1

Data byte	Description	Byte Value	Mnemonic
#1	Authentication Request SID	69h	ARS
#2	authenticationTask = authenticationConfiguration	08h	LEV_AT_AC
#3	returnValue = AuthenticationConfiguration APCE	02h	RV_ACAPCE

Table4-75. Authentication Configuration positive response message flow example – step #1

#Step #2 : Send Certificate Client

Data byte	Description	Byte Value	Mnemonic
#1	Authentication Request SID	29h	ARS
#2	authenticationTask = verifyCertificateUnidirectional	01h	LEV_AT_VCU
#3	communicationConfiguration = no secure communication	00h	COCO
#4 #5	lengthOfCertificateClient[Byte #1] lengthOfCertificateClient[Byte #2]	04h 4Dh	LOCECL
#6 … #1106	certificateClient[byte#1] … certificateClient[byte#1101]	XXh … XXh	CECL
#1107 #1108	lengthOfChallengeClient [byte#1] lengthOfChallengeClient [byte#2]	00h 00h	LOCHCL

Table4-76. Authentication request message flow example – step #2

Data byte	Description	Byte Value	Mnemonic
#1	Authentication Request SID	69h	ARS
#2	authenticationTask = verifyCertificateUnidirectional	01h	LEV_AT_VCU
#3	returnValue = Certificate verified, Ownership verification	11h	RV_CVOVN
#4 #5	lengthOfChallengeServer[Byte #1] lengthOfChallengeServer [Byte #2]	00h 08h	LOCHSE
#6 … #13	challengeServer[byte#1] … challengeServer [byte#8]	XXh … XXh	CHSE
#14 #15	lengthOfEphemeralPublicKeyServer [byte#1] lengthOfEphemeralPublicKeyServer [byte#2]	00h 00h	LOEPKSE

Table4-77. Authentication positive response message flow example – step #2

#Step #3 : Validate the Proof of Ownership

Data byte	Description	Byte Value	Mnemonic
#1	Authentication Request SID	29h	ARS
#2	authenticationTask = proofOfOwnership	03h	LEV_AT_POWN
#3 #4	lengthOfProofOfOwnershipClient[Byte #1] lengthOfProofOfOwnershipClient [Byte #2]	01h 00h	LPOWNCL
#5 … #260	proofOfOwnershipClient[byte#1] … proofOfOwnershipClient [byte#256]	XXh … XXh	POWNCL
#261 #262	lengthOfEphemeralPublicKeyClient [byte#1] lengthOfEphemeralPublicKeyClient [byte#2]	00h 00h	LOEPKCL

Table4-78. Authentication request message flow example – step #3

Data byte	Description	Byte Value	Mnemonic
#1	Authentication Request SID	69h	ARS
#2	authenticationTask = proofOfOwnership	03h	LEV_AT_POWN
#3	returnValue = Certificate verified, Authentication complete	12h	RV_CVOVN
#4 #5	lengthOfSessionKeyInfo[Byte #1] lengthOfSessionKeyInfo [Byte #2]	00h 00h	LOSKI

Table4-79. Authentication positive response message flow example – step #3

4.6 TesterPresent (3E hex) service

이 서비스는 진단기가 차량에 체결된 상태를 유지하고 있으며 이미 동작 중인 진단 서비스 또는 통신의 상태를 계속 유지하게 하는데 이용된다.

이 서비스는 하나의 제어기 또는 모든 제어기들이 Default Session 이외의 다른 Session 에서 해당 Session 을 유지하게 한다. 이는 Tester Present Request 메시지를 주기적으로 요청하여 다른 진단 서비스가 요청되지 않을 때 제어기가 Default Session 으로 자동으로 복귀되는 것을 막게 된다.

4.6.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	TesterPresent Request Service ID	M	#3Eh	TP
#2d	Sub-function = [zeroSubFunction]	M	#00h or #80h	LEV_ ZSUBF

표 4-28. Request Message

Hex	Description	Cvt	Mnemonic
00	zeroSubFunction This parameter value is used to indicate that no sub-function value beside the suppressPosRspMsgIndicationBit is supported by this service.	M	ZSUBF
01~7F	Reserved	M	

표 4-29. Request message sub-function parameter definition

4.6.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	TesterPresent Response Service ID	M	#7Eh	TPPR
#2d	zeroSubFunction	M	#00h	ZSUBF

표 4-30. Positive Response Message definition

4.6.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported.
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong

표 4-31. Supported Negative Response Codes

4.7 SecuredDataTransmission (84 hex) service

4.7.1 Security Sub-layer

The purpose of this service is to transmit data that is protected against attacks from third parties – which could endanger data security – according to ISO 15764.

The SecuredDataTransmission service is applicable if a client intends to use diagnostic services defined in this document in a secured mode. It may also be used a transmit external data, which confirm to some other application protocol, in a secured mode between a client and a server. A secured mode in this context means that the data transmitted is protected by cryptographic methods.

This section briefly describes the security sub-layer as defined in ISO 15764 (see ISO 15764 for details).
그림 4-3 illustrates the security sub-Layer as defined in ISO 15764. The security sub-layer has to be added in the server and client application for the purpose of performing diagnostic services in a secured mode.

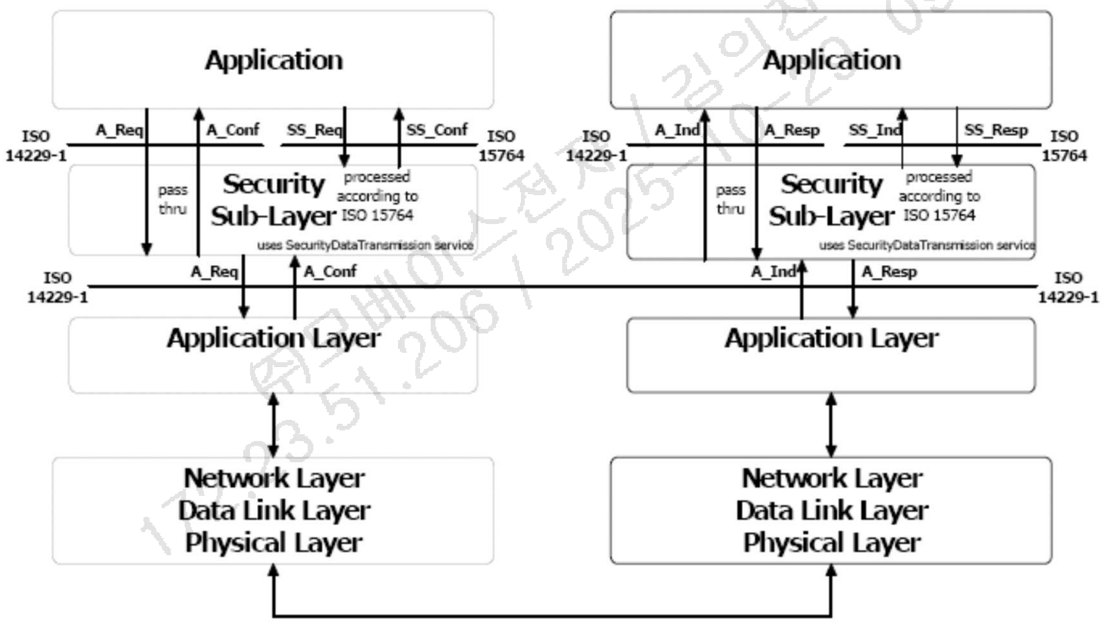


그림 4-3. Security Sub-Layer implementation

There are two (2) methods to perform diagnostic service data transfer between the client and server(s):

- Unsecured data transmission mode
  The application uses the diagnostic Services and Application Layer Service Primitives defined in this document to exchange data between a client and a server. The Security Sub-Layer performs a “Pass- Thru” of data between “Application” and “Application Layer” in the client and the server.

- Secured data transmission mode
  The application uses the diagnostic Services or external services and the Security Sub-Layer Service Primitives defined in ISO 15764 to exchange data between a client and a server. The security sub-layer uses the SecuredDataTransmission service for the transmission/reception of

the secured data. Secured links must be point-to-point communication. Therefore only physical addressing is allowed, which means that only one server is involved.

The interface of the security sub-layer to the application is according to the ISO/OSI model conventions and therefore provides the following four (4) security sub-layer (SS_) service primitives

- SS_SecuredMode.req : Security Sub-Layer Request
- SS_SecuredMode.ind : Security Sub-Layer Indication
- SS_SecuredMode.resp : Security Sub-Layer Response
- SS_SecuredMode.conf : Security Sub-Layer Confirmation

ISO 14229-1 defines both, confirmed and unconfirmed services. In a secured mode only confirmed services are allowed (suppressPosRspMsgIndicationBit = FALSE). Based on this requirement the following services are not allowed to be executed in a secured mode:

- ResponseOnEvent (86 hex)
- ReadDataByPeriodicIdentifier (2A hex)
- TesterPresent (3E hex)

The confirmed services (suppressPosRspMsgIndicationBit = FALSE) use the four (4) application layer service primitives request, indication, response and confirmation. Those are mapped onto the four (4) security sublayer service primitives and vice versa when executing a confirmed diagnostic service in a secured mode.
The task of the Security Sub-Layer when performing a diagnostic service in a secured mode is to encrypt data provided by the "Application", to decrypt data provided by the "Application Layer" and to add, check, and remove security specific data elements. The Security Sub-Layer uses the SecuredDataTransmission (84 hex) service of the application layer to transmit and receive the entire diagnostic message or message according to an external protocol (request and response), which shall be exchanged in a secured mode.

4.7.2 Security Sub-layer Access

The concept of accessing the security sub-layer for a secured service execution is similar to the application layer interface as described in this document. The security sub-layer makes use of the application layer service primitives.

The following describes the execution of confirmed diagnostic service in a secured mode:

- The client application uses the security sub-layer SecuredServiceExecution service request to perform a diagnostic service in a secured mode. The security sub-layer performs the required action to establish a link with the server(s), adds the specific security related parameters, encrypts the service data of the diagnostic service to be executed in a secured mode if needed and uses the application layer SecuredDataTransmission service request to transmit the secured data to the server.

- The server receives an application layer SecuredDataTransmission service indication, which is handled by the security sub-layer of the server. The security sub-layer of the server checks the security specific parameters, decrypts encrypted data and presents the data of the service to be executed in a secured mode to the application via the security sub-layer SecuredServiceExecution service indiciation. The application executes the service and uses the security sub-layer SecuredServiceExecution service response to respond to the service in a secured mode. The security sub-layer of the server adds the specific security related parameters, encrypts the response message data if needed and uses the application layer SecuredDataTransmission service response to transmit the response data to the client.

- The client receives an application layer SecuredDataTransmission service confirmation primitive, which is handled by the security sub-layer of the client. The security sub-layer of the client checks the security specific parameters, decrypts encrypted response data and presents the data via the security sub-layer SecuredServiceExecution confirmation to the application.

그림 11 graphically shows the interaction of the security sub-layer, the application layer, and the application when executing a confirmed diagnostic service in a secured mode.

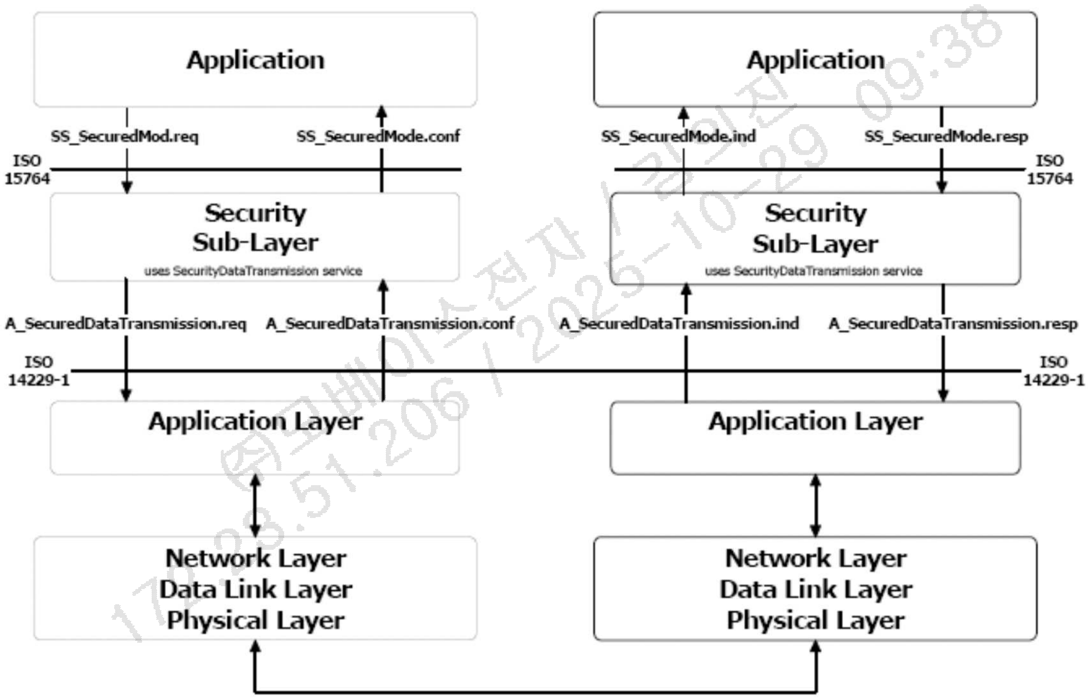


그림 4-4 Security sub-layer, application layer, and application interaction

4.7.3 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	SecuredDataTransmission Request Service ID	M	#84h	SDT
 #2d .. #nd	securityDataRequestRecord = [securityDataParameter#1 .. securityDataParameter#m]	 M .. M	 #XXh .. #XXh	SECDRQR_ SDP_ ,, SDP_

표 4-32. Request Message

4.7.4 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	SecuredDataTransmission Response Service ID	M	#C4h	SDTPR
 #2d .. #nd	securedDataResponseRecord[] = [securityDataParameter#1 .. securityDataParameter#m]	 M .. M	 #XXh .. #XXh	SECDRQR_ SDP_ .. SDP_

표 4-33. Positive Response Message definition

4.7.5 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The server shall use this response code, if the length of the request A_PDU is not correct.
38-4F	reservedByExtendedDataLinkSecurityDocument	M	This range of values is reserved by extended data link security

표 4-34. Supported Negative Response Codes

4.8 ControlDTCSetting (85 hex) service

Control DTC Setting 서비스는 진단기로부터 요청되며, 제어기가 고장코드의 저장을 중지 또는 재개하게 한다.

Control DTC Setting Request 메시지는 각각의 제어기 또는 제어기 그룹이 고장코드 설정을 중지하게 할 수 있다. 지정된 제어기가 고장코드의 설정을 중지할 수 없을 경우, Control DTC Setting Negative Response 메시지를 송신하여 서비스가 거부된 이유를 나타내게 된다.
고장코드 상태는 Control DTC Setting Request 가 sub-function 이 'ON'으로 요청되거나, 세션 계층의 Timeout 이 발생하였을 때(제어기가 Default Session 으로 전환되었을 때) 갱신된다.

진단기가 Clear Diagnostic Information (14 hex) 서비스를 요청하였을 때, Control DTC Setting 은 제어기가 고장코드 메모리를 Reset 하는 것을 막지 못한다. 또한, ECU Reset 서비스가 성공적으로 수행된다면, 고장코드의 저장은 다시 재개된다.

4.8.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ControlDTCSetting Service ID	M	#85h	CDTCS
#2d	Sub-function = [DTCSettingType]	M	#XXh	LEV_ DTCSTP_

표 4-35. Request Message

Hex	Description	Cvt	Mnemonic
00	Reserved	M	
01	On	M	ON
02	Off	M	OFF
03~7F	Reserved	M	

표 4-36. Request message sub-function parameter definition

4.8.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ControlDTCSetting Response Service ID	M	#C5h	CDTCSPR
#2d	DTCSettingType	M	#XXh	DTCSTP

표 4-37. Positive Response Message definition

4.8.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	Used when the server is condition that cannot perform the requested DTC control functionality.

표 4-38. Supported Negative Response Codes

4.9 ResponseOnEvent (86 hex) service

The ResponseOnEvent service requests a server to start or stop transmission of response on a specified event. This service provides the possibility to automatically execute a diagnostic service in case a specified event occurs in the server. The client specifies the event (including optional event parameters) and the service (including service parameters) to be executed in case the event occurs.

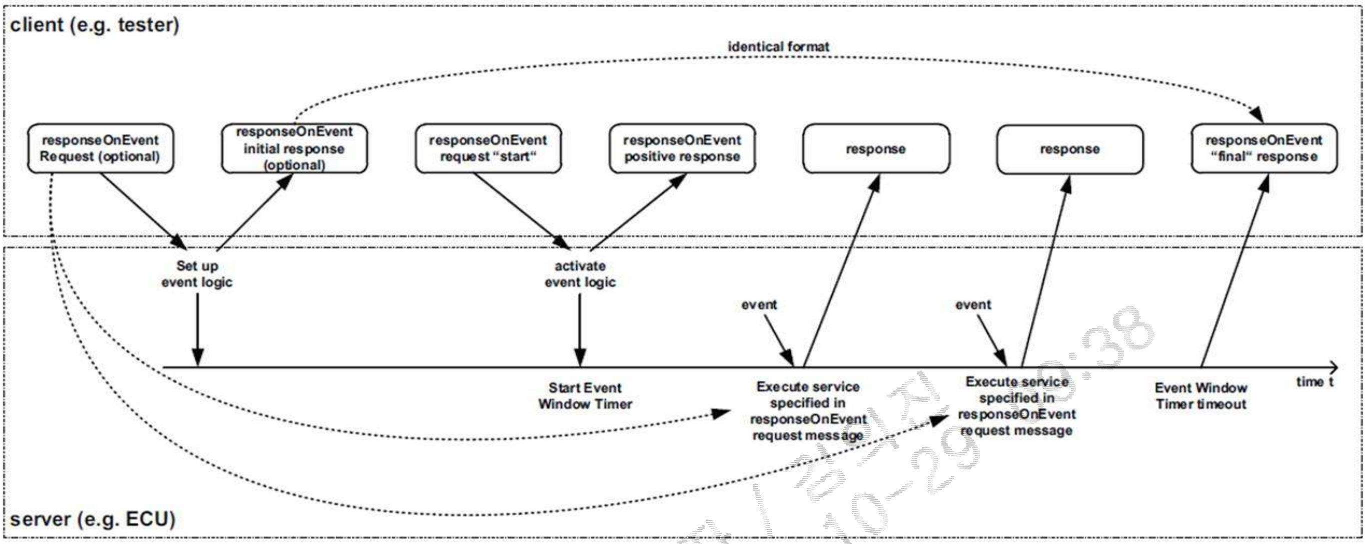


그림 4-5 ResponseOnEvent service – client and server behaviour

The server shall evaluate the sub-function and data content of the ResponseOnEvent request message at the time of the reception. This includes the following sub-function and parameters:

- eventType,
- eventWindowTime,
- eventTypeRecord (eventTypeParameter #1-#m).

In case of invalid data in the ResponseOnEvent request message a negative response with the negative response code 31 hex shall be sent. The serviceToRespondToRecord is not part of this evaluation. The serviceToRespondToRecord parameter will be evaluated when the specified event occurs, which triggers the execution of the service contained in the serviceToRespondToRecord. At the time the event occurs the serviceToRespondToRecord (diagnostic service request message) shall be executed. In case conditions are not correct a negative response message with the appropriate negative response code shall be sent. Multiple events shall be 86tilizin in the order of their occurrence.

The following requirements shall apply for this service when implemented on CAN.

8) Multiple ResponseOnEvent services may run concurrently with different requirements (different EventTypes, serviceToRespondTo-Records, …) to start and stop diagnostic services.

b) While the ResponseOnEvent service is active, the server shall be able to process concurrent diagnostic request and response messages accordingly. This should be accomplished with a (different) pair of serviceToRespondTo-request/response CAN identifiers. See 그림 4-2. If the same diagnostic request/response CAN identifiers are used for diagnostic communication and the serviceToRespondToresponses, the following restrictions shall apply.

1) The server shall ignore an incoming diagnostic request after an event has occurred and the serviceToRespondTo-response is in progress, until the serviceToRespondTo-response is completed.
2) After the client receives any response after sending a diagnostic request, the response shall be classified according to the possible serviceToRespondTo-responses and the expected diagnostic responses that have been sent.

3) If the response is a serviceToRespondTo-response (one of the possible responses set up with ResponseOnEvent-service), the client shall repeat the request after the serviceToRespondToresponse has been received completely.

4) If the response is ambiguous (i.e. the response could originate from the serviceToRespondTo initiated by an event or from the response to a diagnostic request), the client shall present the response both as a serviceToRespondTo-response and as the response to the diagnostic request. The client shall not repeat the request with the exception of NegativeResponseCode busyRepeatRequest (21 hex). (See the negative response code definitions in ISO 14229-1.)

8) The ResponseOnEvent service shall only be allowed to use those diagnostic services available in the active diagnostic session.

8) While the ResponseOnEvent service is active, any change in a diagnostic session shall terminate the current ResponseOnEvent service(s). For instance, if a ResponseOnEvent service has been set up during extendedDiagnosticSession, it shall terminate when the server switches to the defaultSession.

8) If a ResponseOnEvent (86 hex) service has been set up during defaultSession, then the following shall apply :

8) If Bit 6 of the eventType subfunction parameter is set to 0 (do not store event), then the event shall terminate when the server powers down The server shall not continue a ResponseOnEvent diagnostic service after a reset or power on (i.e. the ResponseOnEvent service is terminated).

2) If Bit 6 of the eventType subfunction parameter is set to 1 (store event), it shall resume sending serviceToRespondTo-responses according to the ResponseOnEvent-set up after a power cycle of the server.

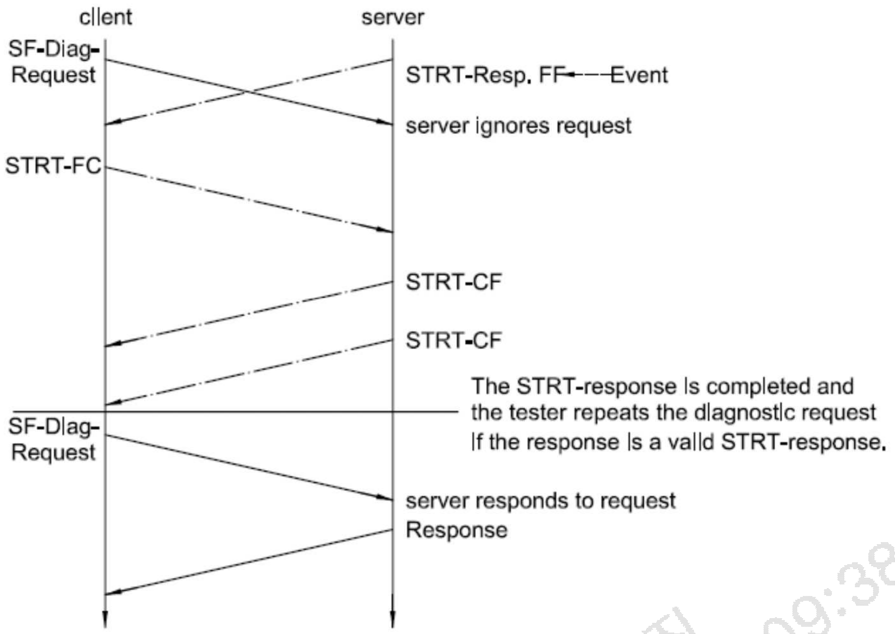


그림 4-6 Concurrent request when the event occurs

f) The subfunction parameter value responseRequired = "no" should only be used for the eventType = stopResponseOnEvent, startResponseOnEvent or clearResponseOnEvent The server shall always return a response to the event-triggered response when the specified event is detected.

g) The server shall return a final positive response to indicate the ResponseOnEvent (86 hex) service has reached the end of the finite event window, unless one of the following conditions apply:

8) if eventTypes do not setup ResponseOnEvent, such as stopResponseOnEvent, startResponseOnEvent, clearResponseOnEvent or reportActivatedEvents;

2) if the infinite event window was established
   - if the Service has been deactivated before the event window was closed,
   - Bit 6 of the eventType subfunction parameter is set to 0 (do not store) and the server powers down or resets.

h) When the specified event is detected, the server shall respond immediately with the appropriate serviceToRespondTo-response message. The immediate serviceToRespondTo-response message shall not disrupt any other diagnostic request or response transmission already in progress (i.e. the serviceToRespondTo-response shall be delayed until the current message transmission has been completed — see 그림 4-7).

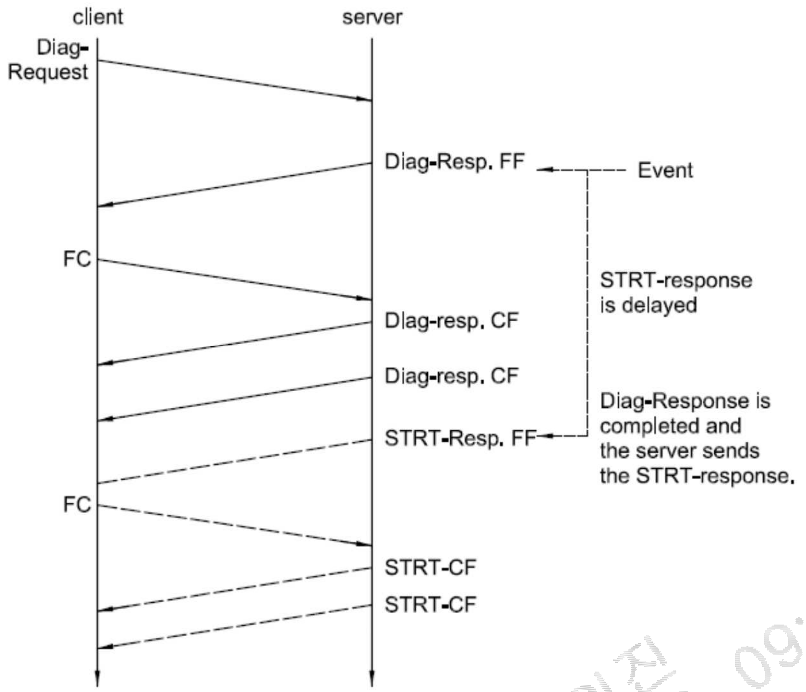


그림 4-7 Event occurrence during a message in progress

8) The ResponseOnEvent service shall only apply to transient events and conditions. The server shall return a response once per event occurrence. For a condition that is continuously sustained over a period of time, the response service shall be executed only one time at the initial occurrence. In case the eventType is defined so that serviceToRespondTo-responses could occur at a high frequency, then appropriate measures have to be taken in order to prevent back to back serviceToRespondTo-responses. A minimum separation time between serviceToRespondTo-responses could be part of the eventTypeRecord (vehicle-manufacturer-specific).

The following 표 defines the data parameters applicable for the implementation of this service on CAN.

Recommended services (ServiceToRespondTo)	RequestService Identifier (Sid)
ReadDataByIdentifier	22 hex
ReadDTCInformation	19 hex
RoutineControl	31 hex
InputOutputControlByIdentifier	2F hex

표 4-39. Recommended services to be used with the ResponseOnEvent service

4.9.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ResponseOnEvent Request Service ID	M	#86h	ROE
#2d	Sub-function = [eventType]	M	#XXh	LEV_ ETP
#3d	eventWindowTime	M	#XXh	EWT

#4d .. #(m-1)+4d	eventTypeRecord[] = [eventTypeParameter 1 .. eventTypeParameter m]	C1 .. C1	#XXh .. #XXh	ETR_ ETP1 .. ETPm
#n-(r-1)-1d #n-(r-1)d .. #nd	servieToRespondToRecord[] = [serviceId serviceParameter 1 .. serviceParameter r]	C2 C3 .. C3	#XXh #XXh .. #XXh	STRTR_ SI SP1 .. SPr

C1 : present if the eventType requires additional parameters to be specified for the event to respond to.
C2 : mandatory to be present if the sub-function parameter is not equal to reportActivateEvents, stopResponseOnEvent, startResponseOnEvent, ClearResponseOnEvent
C3 : present if the service request of the service to respond to requires additional service parameters
eventWindowTime : See annex B-2 for this parameter
eventTypeRecord : This parameter record contains additional parameters for the specified eventType.
See definition of each eventType(표 4-33).

표 4-40. Request Message definition

Bit 6 value	Description	Cvt	Mnemonic
0	doNotStoreEvent This value indicates that the event shall terminate when the server powers down and the server shall not continue a ResponseOnEvent diagnostic service after a reset or power on (i.e. the ResponseOnEvent service is terminated).	M	DNSE
1	storeEvent This value indicates that the event shall resume sending serviceToRespondTo-responses according to the ResponseOnEvent-set up after a power cycle of the server.	U	SE

표 4-41. Sub-function (eventType) bit 6 definition – StorageState

Hex (bit 5-0)	Description	Cvt	Mnemonic
00	stopResponseOnEvent This value is used to stop the server sending responses on event. The event logic that has been set up is not cleared but can be restarted with the startResponseOnEvent sub-function parameter. Length of eventTypeRecord: 0 byte	U	STPROE

01	onDTCStatusChange This value identifies the event as a new DTC detected matching the DTCStatusMask specified for this event. This eventType requires the specification of the DTCStatusMask in the request message (eventTypeParameter#1). Length of eventTypeRecord: 1 byte	U	ONDTCS
02	onTimeInterrupt This value identifies the event as a timer interrupt, but the timer and its values are not part of the ResponseOnEvent service. This eventType requires the specification of more details in the request message (eventTypeRecord). Length of eventTypeRecord: 1 byte	U	OTI
03	onChangeOfDataIdentifier This value identifies the event as a new internal data record identified by dataIdentifier. The data values are vehicle manufacturer specific. This eventType requires the specification of more details in the request message (eventTypeRecord). Length of eventTypeRecord: 3 bytes	U	OCOCID
04	reportActivatedEvents This value is used to indicate that in the positive response all events are reported that have been activated in the server with the ResponseOnEvent service (and are currently active). Length of eventTypeRecord: 0 bytes	U	RAE
05	startResponseOnEvent This value is used to indicate to the server to activate the event logic (including event window timer) that has been set up and start sending responses on event. Length of eventTypeRecord: 0 byte.	M	STRTROE
06	clearResponseOnEvent This value is used to clear the event logic that has been set up in the server (This also stops the server sending responses on event.) Length of eventTypeRecord: 0 byte.	U	CLRROE

07	onComparisonOfValues A defined alteration of a data value out of a specific record identified by a dataIdentifier which identifies a data value event. With this sub-function the user shall have the possibility to define an event at the occurrence of a specific result gathered from a defined measurement value comparison. A specific measurement value included in a data record assigned to a defined dataIdentifier is compared with a given comparison value. The specified operator defines the kind of comparison. The event occurs if the comparison result is positive. Length of eventTypeRecord: 10 Byte	U	OCOV
08~3F	Reserved	M	

표 4-42. Sub-function (eventType) bit5-0 definition

4.9.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ResponseOnEvent Response Service ID	M	#C6h	ROEPR
#2d	eventType	M	#XXh	ETP
#3d	numberOfIdentifiedEvents	M	#XXh	NOIE
#4d	eventWindowTime	M	#XXh	EWT
 #5d .. #(m-1)+5d	eventTypeRecord[] = [eventTypeParameter 1 .. eventTypeParameter m]	 C1 .. C1	 #XXh .. #XXh	ETR_ ETP1 .. ETPm
 #n-(r-1)-1d #n-(r-1)d .. #nd	servieToRespondToRecord[] = [serviceId serviceParameter 1 .. serviceParameter r]	 M C2 .. C2	 #XXh #XXh .. #XXh	STRTR_ SI SP1 .. SPr
C1: present if the eventType required additional parameters to be specified for the event to respond to. C2: present if the service request of the service to respond to required additional service parameters				

표 4-43. Positive Response Message definition for all sub-functions but reportActivatedEvents

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ResponseOnEvent Response Service ID	M	#C6h	ROEPR
#2d	eventType = reportActivatedEvents	M	#04h	ETP_RAE
#3d	numberOfIdentifiedEvents	M	#XXh	NOIE
#4d	eventTypeOfActivateEvent #1	C1	#XXh	EVOAE
#5d	eventWindowTime #1	C1	#XXh	EWT

#6d .. #(m-1)+6d	eventTypeRecord#1[] = [eventTypeParameter 1 .. eventTypeParameter m]	C2 .. C2	#XXh .. #XXh	ETR_ ETP1 .. ETPm
#p-(o-1)-1d #p-(o-1)d .. #pd	servieToRespondToRecord#1[] = [serviceId serviceParameter 1 .. serviceParameter o]	C3 C4 .. C4	#XXh #XXh .. #XXh	STRTR_ SI SP1 .. Spo
..	..	..	..	..
#n-(r-1)-4-(q-1)d	eventTypeOfActivateEvent #k	C1	#XXh	EVOAE
#n-(r-1)-3-(q-1)d	eventWindowTime #k	C1	#XXh	EWT
#n-(r-1)-2-(q-1)d .. #n-(r-1)-2d	eventTypeRecord#k[] = [eventTypeParameter 1 .. eventTypeParameter q]	C2 .. C2	#XXh .. #XXh	ETR_ ETP1 .. ETPq
#n-(r-1)-1d #n-(r-1)d .. #nd	servieToRespondToRecord#k[] = [serviceId serviceParameter 1 .. serviceParameter r]	C3 C4 .. C4	#XXh #XXh .. #XXh	STRTR_ SI SP1 .. SPr
C1: present if an active event is reported C2: present if the reported eventType of the active event (eventTypeOfActiveEvent) requires additional parameters to be specified for the event to respond to. C3: mandatory to be present when reporting an active event. C4: present if the report service request of the service to respond to requires additional service parameters.				

표 4-44. Positive Response Message definition, sub-function = reportActivatedEvents

4.9.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	Used when the server is condition that cannot perform the requested functionality.
31	requestOutOfRange	M	The server shall use this response code : 1. if it detects an error in the eventTypeRecord parameter 2. If the specified eventWindowTime is invalid.

표 4-45. Supported Negative Response Codes

4.9.4 Message flow example(s) ResponseOnEvent (Finite Event Window)

In the following example, it is assumed:

- the node has a physical request CANId of $7E0
- the node has a physical response CANId of $7E8
- storageStatus = doNotStoreEvent (0)
- suppressPosRspMsgIndicationBit = FALSE (0)

4.9.4.1 Step #1 : Set Up Event Logic

- eventWinowTime equal to 08 hex defines an event window of 80 seconds (eventWindowTime * 10 s)
- eventType = onDTCStatusChange (01 hex)
- eventTypeParameter = testFailed status (01 hex)
- serviceToRespondToRecord :
  serviceId = ReadDTCInformation (19 hex)
  sub-function = reportNumberOfDTCByStatusMask (01 hex)
  DTCStatusMask = testFailed status (01 hex)

CAN ID	Client Request / Server Response	DLC	Data Byes	Comments
7E0	Physical Request	8	07 86 01 08 01 19 01 01	Single Frame
7E8	Response	8	10 08 C6 01 00 08 01 19	First Frame
7E0	Physical Request	8	30 00 00 55 55 55 55 55	Flow Control
7E8	Response	8	21 01 01 AA AA AA AA AA	1st Consecutive Frame

표 4-46 ResponseOnEvent (FiniteEventWindow) Message Example – Set Up Event Logic

4.9.4.2 Step #2 : Activate Event Logic

- eventType = startResponseOnEvent (05 hex)
- DTCStatusAvailibilityMask = FF hex
- DTCCount = 4 (00 04 hex)

CAN ID	Client Request / Server Response	DLC	Data Byes	Comments
7E0	Physical Request	8	02 86 04 55 55 55 55 55	Single Frame
7E8	Response	8	04 C6 01 00 08 AA AA AA	Single Frame
7E8	Response	8	04 59 FF 00 04 AA AA AA	Single Frame
7E8	Response	8	04 59 FF 00 04 AA AA AA	Single Frame

표 4-47 ResponseOnEvent (FiniteEventWindow) Message Example – Activate Event Logic

4.9.4.3 Step #3-1 (during the active event window) : Report Currently Active Events

- eventType = reportActivatedEvents (04 hex)

CAN ID	Client Request / Server Response	DLC	Data Byes	Comments
7E0	Physical Request	8	03 86 05 08 55 55 55 55	Single Frame
7E8	Response	8	10 09 C6 04 02 01 08 01	First Frame
7E0	Physical Request	8	30 00 00 55 55 55 55 55	Flow Control
7E8	Response	8	21 19 01 01 AA AA AA AA	1st Consecutive Frame

표 4-48 ResponseOnEvent (FiniteEventWindow) Message Example –Report Currently Active Events

4.9.4.4 Step #3-2 (Event Window Time has expired) : Event Window Timer Timeout

CAN ID	Client Request / Server Response	DLC	Data Byes	Comments
7E8	Response	8	10 08 C6 01 02 08 01 19	First Frame
7E0	Physical Request	8	30 00 00 55 55 55 55 55	Flow Control
7E8	Response	8	21 01 01 AA AA AA AA AA	1st Consecutive Frame

표 4-49 ResponseOnEvent (FiniteEventWindow) Message Example –Event Window Timer Timeout

4.10 LinkControl(87 hex) service

The LinkControl service is used to control the communication link baudrate between the client and the server(s) for the exchange of diagnostic data. This service optionally applies to those data link layers, which allow for a baudrate transition during an active diagnostic session.

- Step #1 : The client verifies if the transition can be performed and informs the server(s) about the baudrate to be used. Each server has to respond positively (suppressPosRspMsgIndicationBit = FALSE) before the client performs step#2. This step actually does not perform the baudrate transition.

- Step #2: The client actually requests the transition of the baudrate. This step shall only be performed if it is verified that the baudrate transition can be performed (step #1 performed). In case of functional communication it is recommended that there shall not be any response from a server when the baudrate is transitioned (suppressPosRspMsgIndicationBit = TRUE), because one server might already have been transitioned to the new baudrate while others still need to transmit their response message(s) (baudrate mismatch avoidance).

Any baudrate transition shall occur as follows:

- suppressPosRspMsgIndicationBit = TRUE: after the successful transmission/reception of the client request message, which requests the baudrate transition.

- suppressPosRspMsgIndicationBit = FALSE: after the successful transmission/reception of the server positive response message, which confirms the successful reception of the request, which requests the baudrate transition.

This service is tied to a non-defaultSession. A session layer timer timeout will transition the server(s) back to its(their) normal speed of operation. The same applies in case an ECUReset service (11 hex) is performed. The transition into another non-defaultSession shall not influence the baudrate.

4.10.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	LinkControl Request Service ID	M	#87h	LC
#2d	Sub-function = [linkControlType]	M	#XXh	LEV_ LCTP_
#3d	baudrateIdentifier	C1	#XXh	BI_

표 4-50. Request Message definition
(linkControlType = verifyBaudrateTransitionWithFixedBaudrate)

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	LinkControl Request Service ID	M	#87h	LC
#2d	Sub-function = [linkControlType]	M	#XXh	LEV_ LCTP_

	linkBaudrateRecord[] = [			LBR_
#3d	baudrateHighByte	C	#XXh	BRHB
#4d	baudrateMiddleByte	C	#XXh	BRMB
#5d	baudrateLowByte]	C	#XXh	BRLB

표 4-51. Request Message definition
(linkControlType = verifyBaudrateTransitionWithSpecificBaudrate)

Hex	Description	Cvt	Mnemonic
00	Reserved	M	
01	verifyBaudrateTransitionWithFixedBaudrate This parameter is used to verify if a transition to a pre-defined baudrate, which is specified by the baudrateIdentifier data parameter can be performed.	C	VBTWFBR
02	verifyBaudrateTransitionWithSpecificBaudrate This parameter is used to verify if a transition to a specially defined baudrate, which is specified by the linkBaudrateRecord data parameter can be performed.	C	VBTWSBR
03	transitionBaudrate This sub-function parameter requests the server(s) to transition the baudrate to the one that was specified in the preceding verification message.	U	TB
04 ~ 7F	Reserved	M	
C : More than one identifier shall be supported.			

표 4-52. Request message sub-function (LinkControlType) parameter definition

Hex	Description	Cvt	Mnemonic
00	Reserved	M	
01	PC9600Baud This value specifies the standard PC baudrate of 9.6Kbaud.	U	PC9600
02	PC19200Baud This value specifies the standard PC baudrate of 19.2Kbaud.	U	PC19200
03	PC38400Baud This value specifies the standard PC baudrate of 38.4Kbaud.	U	PC38400
04	PC57600Baud This value specifies the standard PC baudrate of 57.6Kbaud.	U	PC57600
05	PC115200Baud This value specifies the standard PC baudrate of 115.2Kbaud.	U	PC115200
10	CAN125000Baud This value specifies the standard CAN baudrate of 123Kbaud.	U	CAN125000

11	CAN250000Baud This value specifies the standard CAN baudrate of 250Kbaud.	U	CAN250000
12	CAN500000Baud This value specifies the standard CAN baudrate of 500Kbaud.	U	CAN500000
13	CAN1000000Baud This value specifies the standard CAN baudrate of 1Mbaud.	U	CAN1000000
14~FF	Reserved	M	

표 4-53. Definition of baudrateIdentifier value

4.10.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	LinkControl Response Service ID	M	#C7h	LCPR
#2d	linkControlType	M	#XXh	LCTP

표 4-54. Positive Response Message definition

4.10.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This code shall be returned if the criteria for the requested LinkControl are not met.
24	requestSequenceError	M	This code shall be returned if the client requests the transition of the baudrate without a preceding verification step, which specifies the baudrate to trasition to.
31	requestOutOfRange	M	This code shall be returned if, 1. the requested fixed baudrate (baudrateIdenfitier) is invalid 2. the specific baudrate (linkBaudrateRecord) is invalid.

표 4-55. Supported Negative Response Codes

4.10.4 Message Flow Examples LinkControl

In the following example, it is assumed:

- the node has a physical request CANId of $7E0
- the node has a physical response CANId of $7E8

4.10.4.1 Example #1 – Transition baudrate to fixed baudrate

– PC baudrate = 115200 kBit/s (05 hex)

4.10.4.1.1 Step #1 : Verify if all criteria are met for a baudrate switch

– linkControlType = verifyBaudrateTransitionWithFixedBaudrate (01 hex)
– suppressPosRspMsgIndicationBit = FALSE (0)

CAN ID	Client Request / Server Response	DLC	Data Byes	Comments
7E0	Physical Request	8	03 87 01 05 55 55 55 55	Single Frame
7E8	Response	8	02 C7 01 AA AA AA AA AA	Single Frame

표 4-56 LinkControl Message Example – VeriftyBaudrateTransitionWithFixedBaudrate

4.10.4.1.2 Step #2 : Transition the baudrate

– linkControlType = transitionBaudrate (03 hex)
– suppressPosRspMsgIndicationBit = True (1)

CAN ID	Client Request / Server Response	DLC	Data Byes	Comments
7E0	Physical Request	8	02 87 83 55 55 55 55 55	Single Frame

표 4-57 LinkControl Message Example –TransitionBaudrateWithFixedBaudrate

4.10.4.2 Example #2 – Transition baudrate to specific baudrate

– PC baudrate = 150 kBit/s (02 49 F0 hex)

4.10.4.2.1 Step #1 : Verify if all criteria are met for a baudrate switch

– linkControlType = verifyBaudrateTransitionWithSpecificBaudrate (02 hex)
– suppressPosRspMsgIndicationBit = FALSE (0)

CAN ID	Client Request / Server Response	DLC	Data Byes	Comments
7E0	Physical Request	8	05 87 02 02 49 F0 55 55	Single Frame
7E8	Response	8	02 C7 02 AA AA AA AA AA	Single Frame

표 4-58 LinkControl Message Example – VeriftyBaudrateTransitionWithSpecificBaudrate

4.10.4.2.2 Step #2 : Transition the baudrate

– linkControlType = transitionBaudrate (03 hex)
– suppressPosRspMsgIndicationBit = True (1)

CAN ID	Client Request / Server Response	DLC	Data Byes	Comments
7E0	Physical Request	8	02 87 83 55 55 55 55 55	Single Frame

표 4-59 LinkControl Message Example – TransitionBaudrateWithSpecificBaudrate

5. Data Transition functional unit

5.1 ReadDataByIdentifier (22 hex) service

Read Data By Identifier 서비스는 진단기가 제어기로부터 한 개 또는 그 이상의 dataIdentifier 로 할당된 data record value 를 요청하는데 사용된다. Read Data By Identifier Request 메시지를 수신하면, 제어기는 dataIdentifier 파라미터로 지정된 항목의 값을 읽어서 그 값이 포함된 Read Data By Identifier Positive Response 로 응답하여야 한다.

Request 메시지는 동일한 dataIdentifier 를 여러 번 포함할 수 있다. 이 경우, 제어기는 각각의 dataIdentifier 를 별도로 처리하여, 요청된 횟수만큼 응답하여야 한다.

5.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDataByIdentifier Request Service ID	M	#22h	RDBI
 #2d #3d	dataIdentifier[]#1 = [byte#1 (MSB) byte#2]	 M M	 #XXh #XXh	DID_ HB LB
..	..	..	..	..
 #(n-1)d #nd	dataIdentifier[]#m = [byte#1 (MSB) byte#2]	 U U	 #XXh #XXh	DID_ HB LB
dataIdentifier : see Annex C.1				

표 5-1. Request Message

5.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDataByIdentifier Response Service ID	M	#62h	RDBIPR
 #2d #3d	dataIdentifier[] #1 = [byte#1 (MSB) byte#2]	 M M	 #XXh #XXh	DID_ HB LB
 #4d .. #(k-1)+4d	dataRecord[] #1 = [data#1 .. data#k]	 M .. U	 #XXh .. #XXh	DREC_ DATA_1 .. DATA_k
..	..	..	..	..
 #n-(o-1)-2d #n-(o-1)-1d	dataIdentifier[] #m = [byte#1 (MSB) byte#2]	 U U	 #XXh #XXh	DID_ HB LB
 #n-(o-1)d .. #nd	dataRecord[] #m = [data#1 .. data#k]	 U .. U	 #XXh .. #XXh	DREC_ DATA_1 .. DATA_o

표 5-2. Positive Response Message definition

5.1.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	This response code shall be sent if the length of the request message is invalid or the client exceeded the maximum number of dataIdentifiers allowed to be requested at a time.
22	conditionNotCorrect	U	This response code shall be sent if the operating condition of the server are not met to perform the required action.
31	requestOutOfRange	M	This code shall be sent if none of the requested dataIdentifier values are supported by the device; none of the requested dataIdentifiers are supported in the current session; the requested dynamicDefinedDataIdentifier has not been assigned yet;
33	securityAccessDenied	M	This code shall be sent if at least one of the dataIdentifier is secured and the server is not in and unlocked state.
14	responseTooLong	M	This NRC shall be if the total length of the response exceeds the limit of the underlying transport protocol (e.g., when multiple DIDs are requested in a single request)

표 5-3. Supported Negative Response Codes

5.2 ReadMemoryByAddress (23 hex) service

Read Memory By Address 서비스는 진단기가 시작 주소와 읽으려는 메모리의 크기 요청하여 제어기로부터 지정된 메모리의 데이터를 읽어오는 기능을 한다.

5.2.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadMemoryByAddress Request Service ID	M	#23h	RMBA
#2d	addressAndLengthFormatIdentifier	M	#XXh	ALFID
 #3d .. #(m-1)+3	memoryAddress[] = [byte#1 (MSB) .. byte#m]	 M .. C1	 #XXh .. #XXh	MA_ B1 .. Bm
 #n-(k-1)d .. #nd	memorySize[] = [byte#1 (MSB) .. byte#k]	 M .. C2	 #XXh .. #XXh	MS_ B1 .. Bk
addressAndLengthFormatIdentifier : This parameter is a one byte value with each nibble encoded separately. - bit 7-4 : Length(number of byte) of the memorySize parameter - bit 3-0 : Length(number of byte) of the memoryAddress parameter C1: The presence of this parameter depends on address length information parameter of the addressAndLengthFormatIdentifier. C2: The presence of this parameter depends on the memory size length information of the addressAndLengthFormatIdentifier.				

표 5-4. Request Message

5.2.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadMemoryByAddress Response Service ID	M	#63h	RMBAPR
 #2d .. #nd	dataRecord[] = [data#1 .. data#m]	 M .. U	 #XXh .. #XXh	DREC_ DATA_1 .. DATA_m

표 5-5. Positive Response Message definition

5.2.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This response code shall be sent if the operating conditions of the server are not met to perform the required action

31	requestOutOfRange	M	This response code shall be sent if ; 1. any memory address within the interval [$MA, ($MA+$MS-$1)] is invalid. 2. any memory address within the interval [$MA, ($MA+$MS-$1)] is restricted. 3. the memorySize parameter value in the request message is greater than the maximum value supported by the server. 4. the specified addressAndLengthFormatIdentifier is not valid.
33	SecurityAccessDenied	M	This code shall be sent if any memory address within the interval [$MA, ($MA+$MS-$1)] is secure and the server is locked.

표 5-6. Supported Negative Response Codes

5.3 ReadScalingDataByIdentifier (24 hex) service

Read Scaling Data By Identifier 서비스는 dataIdentifier 로 지정된 scaling data record information 을 제어기로부터 읽는 기능을 한다.

Read Scaling Data Identifier Request 를 수신 시, 제어기는 지정된 dataIdentifier 에 따른 Scaling 정보를 읽고, 이를 Read Scaling Data By Identifier Positive Response 로 응답해야 한다.

5.3.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadScalingDataByIdentifier Request Service ID	M	#24h	RSDBI
 #2d #3d	dataIdentifier[] = [byte#1 (MSB) byte#2]	 M M	 #XXh #XXh	DID_ HB LB
dataIdentifier : see Annex C.1				

표 5-7. Request Message

5.3.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadScalingDataByIdentifier Response Service ID	M	#64h	RSDBIPR
 #2d #3d	dataIdentifier[] = [byte#1 (MSB) byte#2]	 M M	 #XXh #XXh	DID_ HB LB
#4d	scalingByte#1	M	#XXh	SB_1
 #3d .. #(p-1)+5d	scalingByteExtension[]#1 = [scalingByteExtensionParameter#1 .. scalingByteExtensionParameter#p]	 C1 .. C1	 #XXh .. #XXh	SBE_ PAR1 .. PARp
#(n-r)d	scalingByte#k	C2	#XXh	SB_k
 #n-(r-1)d .. #nd	scalingByteExtension[]#1 = [scalingByteExtensionParameter#1 .. scalingByteExtensionParameter#r]	 C1 .. C1	 #XXh .. #XXh	SBE_ PAR1 .. PARr
C1 : The presence of this parameter depends on the scalingByte high nibble. It is mandatory to be present if the scalingByte high nibble is encoded as formula, unit/format, or bitMappedReported/WithOutMask. C2 : The presence of this parameter depends on whether the encoding of the scaling information requires more than one byte scalingByte : The parameter scalingByte consists of one byte (high and low nibble). The scalingByte high nibble defines the data type, which is used to represent the dataIdentifeir (DID). The scalingByte low nibble defines the number of bytes used to represent the parameter in a datastream. (see Annex C.2) scalingByteExtension : This parameter is used to provide additional information for scalingBytes with a high nibble encoded as formula, unit/format, or bitmappedReportedWithOutMask. (see Annex C.3)				

표 5-8. Positive Response Message definition

5.3.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	This response code shall be sent if the length of the request message is invalid.
22	conditionNotCorrect	M	This code shall be sent if the operating conditions of the server are not met to perform the required action.
31	requestOutOfRange	M	This return code shall be sent if ; 1. the requested dataIdentifier value is not supported by the device (physical addressing only) 2. the requested dataIdentifier value is supported by the device, but no scaling information is available for the specified dataIdentifier.
33	securityAccessDenied	M	This code shall be sent if the dataIdentifier is secured and the server is not in an unlocked state.

표 5-9. Supported Negative Response Codes

5.4 ReadDataByPeriodicIdentifier (2A hex) service

The ReadDataByPeriodicIdentifier service allows the client to request the periodic transmission of data record values from the server identified by on or more periodicDataIdentifiers.

The client request message contains one or more 1-byte periodicDataIdentifier values that identify data record(s) maintained by the server. The periodicDataIdentifier represents the low byte of a dataIdentifier out of the dataIdentifier range reserved for this service (0xF2XX, see table C-4 for allowed periodicDataIdentifier values), e.g. the periodicDataIdentifier 0xE3 used in this service is the identifier 0xF2E3.

The format and definition of the dataRecord shall be vehicle manufacturer specific, and may include analog input and output signals, digital input and output signals, internal data, and system status information if supported by the server.

Upon receiving a ReadDataByPeriodicIdentifier request other than stopSending the server shall check whether the conditions are correct to execute the service.

A periodicDataIdentifier shall only be supported with a single transmissionMode at a given time. A change to the schedule of a periodicDataIdentifier shall be performed on reception of a request message with the transmissionMode parameter set to a new schedule for the same periodicDataIdentifier. Multiple schedules for different periodicDataIdentifiers shall be supported upon vehicle manufacturer’s request.

IMPORTANT — If the conditions are correct then the server shall transmit a positive response message, including only the service identifier. The server shall never transmit a negative response message once it has accepted the initial request message by responding positively.

Following the initial positive response message the server shall access the data elements of the records specified by the periodicDataIdentifier parameter(s) and transmit their value in separate periodic data response messages for each periodicDataIdentifier containing the associated dataRecord parameters.

The separate periodic data response messages defined to transmit the periodicDataIdentifier data to the client following the initial positive response message shall include the periodicDataIdentifier and the data of the periodicDataIdentifier, but not the positive response service identifier. The mapping of the periodic response message onto certain data link layers is described in the appropriate implementation specifications of ISO 14229.

The documented periodic rate for a specific transmissionMode is defined as the time between any two consecutive response messages with the same periodicDataIdentifier, when only a single periodicDataIdentifier is scheduled. If multiple periodicDataIdentifiers are scheduled concurrently, the effective period between the same periodicDataIdentifier will vary based upon the following design parameters:
- the call rate of the periodic scheduler,
- the number of available protocol specific periodic data response message address information IDs allocated per scheduler call (e.g., CAN identifier on CAN)
- the number of periodicDataIdentifiers that can be defined in parallel to be transmitted

concurrently.

These parameter values will impact the extent of how much the effective period between the same periodicDataIdentifier will increase if multiple periodicDataIdentifers are transmitted concurrently. Therefore, all of the previously mentioned design parameters shall be specified by the vehicle manufacturer. Each time the periodic scheduler is called it shall determine if any periodicDataIdentifiers are ready to transmit.

Example, two distinct ECU implementations may both support a fast transmissionMode with a periodic rate of 10 ms and a single unique periodic data response message address information ID. If the first implementation calls the periodic scheduler every 10ms, the time between the same periodicDataIdentifier would increase to 20 ms when two periodicDataIdentifiers are scheduled and would increase to 40 ms when four periodicDataIdentifiers are scheduled. If the second implementation calls the periodic scheduler every 5 ms, the time between the same periodicDataIdentifier would remain at 10 ms when two periodicDataIdentifiers are scheduled and would increase to 20 ms when four periodicDataIdentifiers are scheduled.
Upon receiving a ReadDataByPeriodicIdentifier request including the transmissionMode stopSending the server shall either stop the periodic transmission of the periodicDataIdentifier(s) contained in the request message or stop the transmission of all periodicDataIdentifier if no specific one is specified in the request message. The response message to this transmissionMode only contains the service identifier.

The server may limit the number of periodicDataIdentifiers that can be simultaneously supported as agreed upon by the vehicle manufacturer and system supplier. Exceeding the maximum number of periodicDataIdentifier that can be simultaneously supported shall result in a single negative response and none of the periodicDataIdentifiers in that request shall be scheduled. Repetition of the same periodicDataIdentifier in a single request message is not allowed and the server shall ignore them all except one periodicDataIdentifer if the client breaks this rule.

5.4.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDataByPeriodicIdentifier Request Service ID	M	#2Ah	RDBPI
#2d	transmissionMode	M	#XXh	TM
#3d	periodicDataIdentifier[] #1	C	#XXh	PDID1
..	..	..	..	..
#m+2d	periodicDataIdentifier[] #m	U	#XXh	PDIDm
C: The first periodicDataIdentifier is mandatory to be present in the request message if the transmissionMode is equal to sendAtSlowRate, sendAtMediumRate, or sendAtFastRate. In case the transmissionMode is equal to stopSending there can either be no periodicDataIdentifier present in order to stop all scheduled periodicDataIdentifier or the client can explicitly specify one or more periodicDataIdentifier(s) to be stopped.				

표 5-10. Request Message

Hex	Description	Cvt	Mnemonic
00	Reserved	M	

01	sendAtSlowRate This parameter specifies that the server shall transmit the requested dataRecord information at a slow rate in response to the request message (where the # of responses to be sent = maximumNumberOfResponsesToSend). The repetition rate specified by the transmissionMode parameter slow is vehicle manufacturer specific, and predefined in the server.	C	SASR
02	sendAtMediumRate This parameter specifies that the server shall transmit the requested dataRecord information at a medium rate in response to the request message (where the # of responses to be sent = maximumNumberOfResponsesToSend). The repetition rate specified by the transmissionMode parameter medium is vehicle manufacturer specific, and pre-defined in the server.	C	SAMR
03	sendAtFastRate This parameter specifies that the server shall transmit the requested dataRecord information at a fast rate in response to the request message (where the # of responses to be sent = maximumNumberOfResponsesToSend). The repetition rate specified by the transmissionMode parameter fast is vehicle manufacturer specific, and predefined in the server.	C	SAFR
04	stopSending The server stops transmitting positive response messages send periodically/repeatedly. Note that maximumNumberOfResponsesToSend parameter should be set to 0x01 if transmissionMode = stopSending (otherwise, server operation could be undefined).	M	SS
05~FF	Reserved	M	
C : More than one identifier shall be supported.			

표 5-11. Data parameter definition – TransmissionMode

5.4.2 Positive response message

It has to be distinguished between the initial positive response, which indicates that the server accepts the service and subsequent positive response message, which include periodicDataIdentifier data. Table5-12/13 defines the initial positive response message to be transmitted by the server when it accepts the request.

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDataByPeriodicIdentifier Response Service ID	M	#6Ah	RDBPIPR

표 5-12. Positive Response Message definition

The data of a periodicDataIdentifier is transmitted periodically (with updated data) at a rate determined

by the transmissionMode parameter of the request.
After the initial positive response, for each supported periodicDataIdentifier in the request the server shall start sending a single periodic data response message as defined below.

Table 5-13 defines the periodic data response message data definition.

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	periodicDataIdentifier	M	#XXh	PDID
 #2d .. #k+2d	dataRecord[] = [data#1 .. data#k]	 M .. U	 #XXh .. #XXh	DREC_ DATA_1 .. DATA_k

표 5-13. Periodic data reponse message data definition

5.4.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	This NRC shall be sent if the length of the request message is invalid or the client exceeded the maximum number of periodicDataIdentifiers allowed to be requested at a time.
22	conditionNotCorrect	M	This NRC shall be sent if the operating conditions of the server are not met to perform the required action. E.g. this could occur if the client requests periodicDataIdentifiers with different transmissionModes and the server does not support multiple transmissionModes simultaneously.
31	requestOutOfRange	M	This NRC shall be sent if none of the requested periodicDataIdentifier values are supported by the device; none of the requested periodicDataIdentifiers are supported in the current session; The specified transmissionMode is not supported by the device; the requested dynamicDefinedDataIdentifier has not been assigned yet; the client exceeded the maximum number of periodicDataIdentifiers allowed to be scheduled concurrently
33	securityAccessDenied	M	This NRC shall be sent if at least one of the periodicDataIdentifier is secured and the server is not in an unlocked state.

표 5-14. Supported Negative Response Codes

5.5 DynamicallyDefinedDataIdentifier (2C hex) service

The DynamicallyDefinedDataIdentifier service allows the client to dynamically define in a server a data identifier that can be read via the ReadDataByIdentifier service at a later time. The intention of this service is to provide the client with the ability to group one or more data elements into a data superset that can be requested en masse via ReadDataByIdentifier or ReadDataByPeriodicIdentifier service. The data elements to be grouped together can either be referenced by ;

- a source data identifier, a position and size or,
- a memory address and a memory length, or,
- a combination of the two methods listed above using multiple requests to define the single data element. The dynamically defined dataIdentifier will then contain a concatenation of the data parameter definitions.

This service allows greater flexibility in handling ad hoc data needs of the diagnostic application that extend beyond the information that can be read via statically defined data identifiers, and can also be used to reduce bandwidth utilization by avoiding overhead penalty associated with frequent request/response transactions. The definition of the dynamically defined data identifier can either be done via a single request message or via multiple request messages. This allows for the definition of a single data element referencing source identifier(s) and memory addresses. The server has to concatenate the definitions for the single data element. A redefinition of a dynamically defined data identifier can be achieved by clearing the current definition and start over with the new definition.

Although this service does not prohibit such functionality, it is not recommended for the client to reference one dynamically defined data record from another, because deletion of the referenced record could create data consistency problems within the referencing record. This service also provides the ability to clear an existing dynamically defined data record. Requests to clear a data record shall be positively responded to if the specified data record identifier is within the range of valid dynamic data identifiers supported by the server (see ISO 14229-1.2 annex C.1 for more details).

The server shall maintain the dynamically defined data record until it is cleared or as specified by the vehicle manufacturer (e.g., deletion of dynamically defined data records upon session transition or upon power down of the server).

The server can implement data records in two different ways:

1) Composite data records containing multiple elemental data records which are not individually referenced.
2) Unique 2-byte identification “tag” or parameter identifier (PID) value for individual, elemental data records supported within the server (an example elemental data record, or PID, is engine speed or intake air temperature). This implementation of data records is a subset of a composite data record implementation, because it only references a single elemental data record instead of a data record including multiple elemental data records.

Both types of implementing data records are supported by the DynamicallyDefineDataIdentifier service to
define a dynamic data identifier.
1) Composite block of data: The position parameter has to reference the starting point in the

composite block of data and the size parameter has to reflect the length of data to be placed in the dynamically defined data identifier. The tester is responsible to not include only a portion of an elemental data record of the composite block of data in the dynamic data record.
2) 2-byte PID: The position parameter has to be set to one (1) and the size parameter has to reflect the length of the PID (length of the elemental data record). The tester is responsible to not include only a portion of the 2-byte PID value in the dynamic data record.

The ordering of the data within the dynamically defined data record shall be of the same order as it was specified in the client request message(s). Also, first position of the data specified in the client's request shall be oriented such that it occurs closest to the beginning of the dynamic data record, in accordance with the ordering requirement mentioned in the preceding sentence.

In addition to the definition of a dynamic data identifier via a logical reference (a record data identifier) this service provides the capability to define a dynamically defined data identifier via an absolute memory address and a memory length information. This mechanism of defining a dynamic data identifier is recommended to be used only during the development phase of a server.

5.5.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	DynamicallyDefinedDataIdentifier Request Service ID	M	#2Ch	DDDI
#2d	sub-function = [defineByIdentifier]	M	#01h	LEV_ DBID
 #3d #4d	dynamicallyDefinedDataIdentifier[] = [byte#1 (MSB) byte#2 (LSB)]	 M M	 #XXh #XXh	DDDDI_ HB LB
 #5d #6d	sourceDataIdentifier[] #1 = [byte#1 (MSB) byte#2 (LSB)]	 M M	 #XXh #XXh	SDI_ HB LB
#7d	positionInSourceDataRecord #1	M	#XXh	PISDR#1
#8d	memorySize #1	M	#XXh	MS#1
..	..	..	..	..
 #n-3d #n-2d	sourceDataIdentifier[] #m = [byte#1 (MSB) byte#2 (LSB)]	 U U	 #XXh #XXh	SDI_ HB LB
#n-1d	positionInSourceDataRecord #m	U	#XXh	PISDR#m
#nd	memorySize #m	U	#XXh	MS#m
positionInSourceDataRecord : This 1-byte parameter is used to specify the starting byte position of the except of the source data record to be included in the dynamic data record. A position of one(1) shall reference the first byte of the data record referenced by the sourceDataIdentifier				

표 5-15. Request message definition – sub-function = defineByIdentifier

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	DynamicallyDefinedDataIdentifier Request Service ID	M	#2Ch	DDDI

#2d	sub-function = [defineByMemoryAddress]	M	#02h	LEV_ DBMA
#3d #4d	dynamicallyDefinedDataIdentifier[] = [byte#1 (MSB) byte#2 (LSB)]	M M	#XXh #XXh	DDDDI_ HB LB
#5d	addressAndLengthFormatIdentifier	M1	#XXh	ALFID
#6d .. #(m-1)+6d	memoryAddress[] = [byte#1 (MSB) .. byte#m]	M .. C	#XXh .. #XXh	MA_ B1 .. Bm
#m+6d .. #m+6+(k-1)d	memorySize[] = [byte#1 (MSB) .. byte#k]	M .. C	#XXh .. #XXh	MS_ B1 .. Bk
..	..	..	..	..
#n-k-(m-1)d .. #n-kd	memoryAddress[] = [byte#1 (MSB) .. byte#m]	U .. U/C	#XXh .. #XXh	MA_ B1 .. Bm
#n-(k-1)d .. #nd	memorySize[] = [byte#1 (MSB) .. byte#k]	U .. U/C	#XXh .. #XXh	MS_ B1 .. Bk
M1 : The addressAndLengthFormatIdetifier parameter is only present once at the very beginning of the request message and defines the length of the address and length information for each memory location reference throughout the whole request message.				

표 5-16. Request message definition - sub-function = defineByMemoryAddress

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	DynamicallyDefinedDataIdentifier Request Service ID	M	#2Ch	DDDI
#2d	sub-function = [clearDynamicallyDefinedDataIdentifier]	M	#03h	LEV_ CDDDID
#3d #4d	dynamicallyDefinedDataIdentifier[] = [byte#1 (MSB) byte#2 (LSB)]	C C	#XXh #XXh	DDDDI_ HB LB
C : The present of this parameter requires the server to clear the dynamicallydefinedDataIdentifier included in byte#1 and byte#2. If the parameter is not present all dynamicallyDefinedDataIdentifier in the server shall be cleared.				

표 5-17. Request message definition - sub-function = clearDynamicallyDefinedDataIdentifier

Hex	Description	Cvt	Mnemonic
00	Reserved	M	
01	defineByIdentifier	U	DBID
02	defineByMemoryAddress	U	DBMA
03	clearDynamicallyDefineDataIdentifier	U	CDDDI

04~7F	Reserved	M	

표 5-18. Request message sub-function parameter definition

Parameter	Description	# of bytes	Resolution	Min value	Max value
$P2_{CAN\_SERVER\_MAX}$	Default $P2_{CAN\_SERVER\_MAX}$ timing supported by the server for the activated diagnostic session	2	1ms	0ms	65535ms
$P2*_{CAN\_SERVER\_MAX}$	Enhanced (NRC 78h) $P2_{CAN\_SERVER\_MAX}$ supported by the server for the activated diagnostic session.	2	10ms	0ms	655350ms

표 5-19. sessionParameterRecord definition

5.5.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	**DynamicallyDefineDataIdentifier Response Service ID**	**M**	**#6Ch**	**DDDIPR**
#2d	definitionType	M	#XXh	DM
 #3d #4d	dynamicallyDefinedDataIdentifier [] = [byte#1 (MSB) byte#2 (LSB)]	 C C	 #XXh #XXh	DDDDI_ HB LB

표 5-20. Positive Response Message definition

5.5.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	This response code shall sent if the sub-function parameter is not supported.
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This response code shall be sent if the operating conditions of the server are not met to perform the required action.
31	requestOutOfRange	M	This response code shall be sent if ; 1. Any data identifier (dynamicallyDefinedData Identifier or any sourceDataIdentifer) in the request message is not supported/invalid. 2. The positionSourceDataRecord was incorrect (less than 1, or greater than maximum allowed by server) 3. Any memory address in the request message is not supported in the server 4. The specified memorySize was invalid. 5. The amount of data to be packed into the dynamic data exceeds the maximum allowed by server. 6. The specified addressAndLengthFormat Identifier is not valid.

33	securityAccessDenied	M	This code shall be sent if; 1. any data identifier (dynamicallyDefinedData Identifier) in the request message is secured and the server is not in an unlocked state. 2. any memory address in the request message is secured and the server is not in unlocked state.

표 5-21. Supported Negative Response Codes

5.6 WriteDataByIdentifier (2E hex) service

Write Data By Identifier 서비스는 지정된 dataIdentifier 에 따른 제어기의 내부 메모리에 진단기로부터 요청된 정보를 쓰는 기능을 한다.
Dynamically defined dataIdentifier 는 이 서비스에서 사용될 수 없다.

5.6.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	WriteDataByIdentifier Request Service ID	M	#2Eh	WDBI
 #2d #3d	dataIdentifier = [byte#1 (MSB) byte#2]	 M M	 #XXh #XXh	DID_ HB LB
 #4d .. #m+3d	dataRecord = [data#1 .. data#m]	 M .. U	 #XXh .. #XXh	DREC_ DATA_1 .. DATA_m
dataIdentifier : see Annex C.1				

표 5-22. Request Message

5.6.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	WriteDataByIdentifier Response Service ID	M	#6Eh	WDBIPR
 #2d #3d	dataIdentifier = [byte#1 (MSB) byte#2]	 M M	 #XXh #XXh	DID_ HB LB

표 5-23. Positive Response Message definition

5.6.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This response code shall be sent if the operating condition of the server are not met to perform the required action.
31	requestOutOfRange	M	This response code shall be sent if 1. The dataIdentifier in the request message is not supported in the server or the dataIdentifier is supported for read only purpose (via ReadDataBy Identifier service). 2. Any data transmitted in the request message after the dataIdentifier is invalid (if applicable to the node).
33	securityAccessDenied	M	This code shall be sent if the dataIdentifier, which reference a specific address, is secured and the server is not in an unlocked state.

72	generalProgrammingFailure	M	This return code shall be sent if the server detects an error when writing to a memory location.

표 5-24. Supported Negative Response Codes

5.7 WriteMemoryByAddress (3D hex) service

Write Memory By Address 서비스는 제어기의 메모리 위치에 진단기의 요청 정보를 쓰는 기능을 한다. Write Memory By Address Request 메시지는 정의된 dataRecord 파라미터에 따라 지정된 제어기의 메모리의 시작 주소와 영역에 정보를 쓰게 한다.

5.7.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	WriteMemoryByAddress Request Service ID	M	#3Dh	WMBA
#2d	addressAndLengthFormatIdentifier	M	#XXh	ALFID
#3d .. #m+2d	memoryAddress[] = [byte#1 (MSB) .. byte#m]	M .. C1	#XXh .. #XXh	MA_ B1 .. Bm
#n-r-2-(k-1)d .. #n-r-2d	memorySize[] = [byte#1 (MSB) .. byte#k]	M .. C2	#XXh .. #XXh	MS_ B1 .. Bk
#n-(r-1)d .. #nd	dataRecord[] = [data#1 .. data#r]	M .. U	#XXh .. #XXh	DREC_ DATA_1 .. DATA_r
C1: The presence of this parameter depends on address length information parameter of the addressAndLengthFormatIdentifier. C2: The presence of this parameter depends on the memory size length information of the addressAndLengthFormatIdentifier.				

표 5-25. Request Message

5.7.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	WriteMemoryByAddress Response Service ID	M	#7Dh	WMBAPR
#2d	addressAndLengthFormatIdentifier	M	#XXh	ALFID
#3d .. #(m-1)+3	memoryAddress[] = [byte#1 (MSB) .. byte#m]	M .. C1	#XXh .. #XXh	MA_ B1 .. Bm
#n-(k-1)d .. #nd	memorySize[] = [byte#1 (MSB) .. byte#k]	M .. C2	#XXh .. #XXh	MS_ B1 .. Bk
C1: The presence of this parameter depends on address length information parameter of the addressAndLengthFormatIdentifier. C2: The presence of this parameter depends on the memory size length information of the addressAndLengthFormatIdentifier.				

표 5-26. Positive Response Message definition

5.7.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This response code shall be sent if the operating condition of the server are not met to perform the required action.
31	requestOutOfRange	M	1. Any memory address within the interval [$MA, ($MA + $MS –$1)] is invalid. 2. Any memory address within the interval [$MA, ($MA + $MS –$1)] is restricted. 3. The memorySize parameter value in the request message is greater than the maximum value supported by the server. 4. The specified addressAndLengthFormat Identifier is not valid.
33	securityAccessDenied	M	This code shall be sent if any memory address within the interval [$MA, ($MA + $MS –$1)] is secure and the server is locked.
72	generalProgrammingFailure	M	This return code shall be sent if the server detects an error when writing to a memory location.

표 5-27. Supported Negative Response Codes

6. Stored Data Transmission Functional Unit

<table>
<tr><th colspan="8">DTC High Byte</th><th colspan="8">DTC Middle Byte</th><th colspan="8">DTC Low Byte</th></tr>
<tr><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="2">00 : P
01 : C
10 : B
11 : U</td><td colspan="2">1st Char. Of Code</td><td colspan="4">2nd Character of Code</td><td colspan="4">3rd Character of Code</td><td colspan="4">4th Character of Code</td><td colspan="4">DTC Failure Category</td><td colspan="4">DTC Failure Sub Type</td></tr>
</table>

표 6-1. 3 Byte Diagnostic Trouble Code Structure

Existing ISO documents for defining DTCs such as ISO 15031-6 currently define DTCs for existing systems. If a standard DTC is already defined for a component / system and that DTC description already comprehends the DTC Failure Type information, then the standard DTC number can be used and the DTCFailureTypeByte shall be set to a value of 00 hex. A DTCFailureTypeByte value of 00 hex indicates that no additional information is contained in the DTCFailureTypeByte.

The following example shows the three (3) principle combinations of DTC and DTCFailureTypeByte.

- DTC which does not require any additional description included in the DTCFailureTypeByte (no DTC Failure Category name and no DTC Failure Sub Type)
  <e.g.> emissions-related DTC (012700 hex): P0127 Intake Air Temperature Too High,
- DTC which requires additional description included in the DTCFailureTypeByte (DTC Failure Category name and no DTC Failure Sub Type)
  <e.g.> DTC (803901): B0039-01 Second Row Right Frontal Stage 1 Deployment Control – General Electrical Failure
- DTC, which requires additional description included in the DTCFailureTypeByte (DTC Failure Category name and DTC Failure Sub Type)
  <e.g.> DTC (403123): C0031-23 Left Front Wheel Speed Sensor –General Signal Failure – Signal Stuck Low DTC Failure Category Definition

Refer to the ISO 15031-6 Annex. D “DTC Failure Category and Sub Type definition”, if you need more information.

6.1 ClearDiagnosticInformation (14 hex) service

ClearDiagnosticInformation 서비스는 한 개 또는 여러 개의 제어기의 진단 정보를 지우는데 사용된다. 제어기는 저장된 고장코드가 없더라도, Positive Response 로 응답하여야 한다. 제어기의 메모리에 여러 개의 복사된 고장코드 상태정보(예, RAM 과 EEPROM 에 각 각 저장)가 지원되더라도, 제어기는 복사본을 지워야 한다.

Long-term 메모리에 저장된 고장코드와 같은 부가적인 복사본들은 적절한 backup 전략에 따라 갱신될 것이다.

진단기의 Request 메시지는 한 개의 파라미터를 가진다. Group of DTC 파라미터는 진단기가 Powertrain, Body, Chassis 등의 고장코드 그룹 또는 개별 고장코드를 삭제하게 한다.

이 서비스는 다음과 같은 정보 등을 포함한 고장코드 정보를 삭제(또는 초기화) 한다.

- DTC status byte
- Captured DTC snapshot data
- Captured DTC extended data
- Other DTC related data such as first/most recent DTC, flags, counters, timers, etc. specific to DTC's

DTC Mirror 메모리 저장되어 선택적으로 사용 가능한 임의의 고장코드는 이 서비스에 영향을 받지 않을 수 있다.

Note) Mirror Memory : The DTC mirror memory is an additional optional error memory in the server that cannot be erased by the ClearDiagnosticInformation (14 hex) service. The DTC mirror memory mirrors the normal DTC memory and can be used for example if the normal error memory is erased. To clear Mirror Memory an according Routine Identifier shall be used.

6.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ClearDiagnosticInformation Request Service ID	M	#14h	CDTCI
	groupOfDTC[] = [			GODTC_
#2d	groupOfDTCHighByte	M	#XXh	HB
#3d	groupOfDTCMiddleByte	M	#XXh	MB
#4d	groupOfDTCLowByte]	M	#XXh	LB
groupOfDTC : see Annex D.1				

표 6-2. Request Message

6.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ClearDiagnosticInformation Response Service ID	M	#54h	CDTCIPR

표 6-3. Positive Response Message definition

6.1.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence

13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This response code shall be used if internal conditions within the server prevent the clearing of DTC related information stored in the server.
31	requestOutOfRange	M	The return code shall be sent if the specified groupOfDTC parameter is not supported.
72	generalProgrammingFailure	U	This NRC shall be returned if the server detects an error when writing to a memory location.

표 6-4. Supported Negative Response Codes

6.1.4 Implementaion rule

The ECU behavior shall be consistent: The DTCs shall be cleared before the positive respons e is sent. This does not require that the non volatile memory is cleared, but any memory cac he has to be.

With the parameter groupOfDTC it is also possible to clear only a specific group of DTCs (e. g. Powertrain) or a specific DTC.

A subsequent call to read out the fault memory shall not retrieve any DTC which has been se t before the last execution of ClearDiagnosticInformation.

Even if no DTC was stored, the ECU shall return with a positive response.

There shall be no sequence dependency to any other service. Even if the fault memory was not read, it may be cleared.

A DTC can just be 122tilizi by tester with service #14h.

6.2 ReadDTCInformation (19 hex) service

This service allows a client to read the status of server resident Diagnostic Trouble Code (DTC) information from any server, or group of servers within a vehicle. Unless otherwise stated, the server shall return both emission-related and non emission-related DTC information. This service allows the client to do the following :

- Retrieve the number of DTC's matching a client defined DTC status mask (at the point of the request),
- Retrieve the list of all DTC's matching a client defined DTC status mask,
- Retrieve DTCSnapshot data associated with a client defined DTC and status mask combination: DTC Snapshots are specific data records associated with a DTC, that are stored in the server's memory. The content of the DTC Snapshots is not defined by this standard, but typical usage of DTC Snapshots is to store data upon detection of a system malfunction. The DTC Snapshots will act as a snapshot of data values from the time of the system malfunction occurrence.
- Retrieve DTCExtendedData associated with a client defined DTC and status mask combination out of the DTC memory or the DTC mirror memory. DTC Extended Data consist of extended status information associated with a DTC. DTC Extended Data contains DTC parameter values, which have been identified t the time of the request. A typical use of DTC Extended Data is to store dynamic data associated with the DTC, e.g.
    - DTC occurrence counter,
    - current threshold values,
    - time of last occurrence (etc.).
    - fault validation counters (e.g. counts number of reported "test failed" and possible other counters if the validation is performed in several steps),
    - uncompleted test counters (e.g. counts numbers of driving cycles since the test was latest completed i.e. since the test reported "test passed" or "test failed"),
    - fault occurrence counters (e.g. counts number of driving cycles in which "testFailed" have been reported),
    - DTC aging counter (e.g. counts number of driving cycles since the fault was latest failed excluding the driving cycles in which the test haven't report "test pass" or the test report "test fail"),
    - specific counters for OBD (e.g. number of remaining driving cycles until the "check engine" lamp is switched off).
- Retrieve the number of DTC's matching a client defined severity mask (at the point of the request),
- Retrieve the list of DTC's matching a client defined severity mask record,
- Retrieve severity information for a client defined DTC,
- Retrieve the status of all DTC's supported by the server,
- Retrieve the first DTC failed by the server,
- Retrieve the most recently failed DTC within the server,
- Retrieve the first DTC confirmed by the server,
- Retrieve the most recently confirmed DTC within the server,
- Retrieve the list of DTC's out of the DTC mirror memory matching a client defined DTC status mask,
- Retrieve mirror memory DTCExtendedData record data for a client defined DTC mask and a client defined DTCExtendedData record number out of the DTC mirror memory,
- Retrieve the number of DTC's out of the DTC mirror memory matching a client defined DTC status mask,
- Retrieve the number of "only" emissions-related OBD DTC's matching a client defined DTC status

mask. Emissions-related OBD DTC's cause the malfunction indicator to be turned on/display a message in case such DTC is detected,
- Retrieve the status of "only" emissions-related OBD DTC's matching a client defined DTC status mask. Emissions-related OBD DTC's cause the malfunction indicator to be turned on/display a message in case such DTC is detected,
- Retrieve all current "prefailed" DTCs which have not yet been detected as "pending" or "confirmed",
- Retrieve all DTCs with "permanentDTC" status. These DTCs have been previously cleared by the clearDiagnosticInformation service but remain in the non-volatile memory of the server until the appropriate monitors for each DTC have successfully passed.

This service uses a sub-function to determine which type of diagnostic information the client is requesting.
Further details regarding each sub-function parameter are provided in the following sections.

This service makes use of the following terms:

- Enable Criteria: Server/vehicle manufacturer/system supplier specific criteria used to control when the server actually performs a particular internal diagnostic.
- Test Pass Criteria: Server/vehicle manufacturer/system supplier specific conditions, that define, whether a system being diagnosed is functioning properly within normal, acceptable operating ranges (e.g. no failures exist and the diagnosed system is classified as "OK").
- Test Failure Criteria: Server/vehicle manufacturer/system supplier specific failure conditions, that define, whether a system being diagnosed has failed the test.
- Confirmed Failure Criteria: Server/vehicle manufacturer/system supplier specific failure conditions that define whether the system being diagnosed is definitively problematic (confirmed), warranting storage of the DTC record in long term memory.
- Occurrence Counter: A counter maintained by certain servers that records the number of instances in which a given DTC test reported a unique occurrence of a test failure.
- Aging: A process whereby certain servers evaluate past results of each internal diagnostic to determine if a confirmed DTC can be cleared from long-term memory, e.g. in the event of a calibrated number of failure free cycles. (see ISO 14229-1.2 "10.3 ReadDTCInformation Service" for more details)

6.2.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [	M		LEV_
	reportNumberOfDTCByStatusMask		#01h	RNODTCBSM
	reportDTCByStatusMask		#02h	RDTCBSM
#3d	DTCStatusMask	M	#XXh	DTCSM
DTCStatusMask : see Annex D.2				

표 6-5. Request Message definition – sub-function = #01h/02h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [	M		LEV_
	reportDTCSnapshotIdentification		#03h	RDTCSSI

	reportDTCSnapshotRecordByDTCNumber]		#04h	RDTCSSBDTC
 #3d #4d #5d	DTCMaskRecord[] = [DTCHighByte DTCMiddleByte DTCLowByte]	 C C C	 #XXh #XXh #XXh	DTCMREC_ DTCHB DTCMB DTCLB
#6d	DTCSnapshotRecordNumber	C	#XXh	DTCSSRN
DTCSnapshotRecordNumber : DTCSnapshotRecordNumber is a 1-byte value indicating the number of the specific DTCSnapshot data record requested for a client defined DTCMaskRecord via the reportDTCSnapshotByDTCNumber / reportDTCSnapshotByRecordNumber sub-functions. For emissions-related servers (OBD compliant ECUs) the DTCSnapshot data record number 00 hex shall be the equivalent data record as specified in ISO 15031-5 service 02 hex frame number 00 hex. If the server supports multiple DTCSnapshot data records the range of 01 hex through FE hex shall be used. A value of FF hex requests the server to report all stored DTCSnapshot data records at once.				

표 6-6. Request Message definition – sub-function = #03/ 04h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [reportDTCSnapshotRecordByRecordNumber]	M	 #05h	LEV_ RDTCSSBRN
#3d	DTCSnapshotRecordNumber	M	#XXh	DTCSSRN

표 6-7. Request Message definition – sub-function = #05h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [reportDTCExtendedDataRecordByDTCNumber]	M	 #06h	LEV_ RDTCEDRBDN
 #3d #4d #5d	DTCMaskRecord[] = [DTCHighByte DTCMiddleByte DTCLowByte]	 M M M	 #XXh #XXh #XXh	DTCMREC_ DTCHB DTCMB DTCLB
#6d	DTCExtendedDataRecordNumber	M	#XXh	DTCEDRN
DTCExtendedDataRecordNumber : DTCExtendedDataRecordNumber is a 1-byte value indicating the number of the specific DTCExtendedData record requested for a client defined DTCMaskRecord via the reportDTCExtendedDataRecordByDTCNumber sub-function. For emissions-related servers (OBD compliant ECUs) the DTCExtendedDataRecordNumber 00 hex shall be reserved for future OBD use.				

표 6-8. Request Message definition – sub-function = #06h

Hex	Description	Cvt
00	Reserved (for future OBD use)	M
01~8F	Vehicle Manufacturer Specific Stored DTCExtendedData Records	(To be defined)
90~EF	Reserved (for Legislated OBD Stored DTCExtendedData Records)	M
F0~FD	Reserved (for future)	M
FE	Reserved (for All Legislated OBD Stored DTCExtendedData Records in a Single Response Message)	M
FF	All Stored DTCExtendedData Records in a Single Response Message	M

표 6-9. DTCExtendedDataRecordNumber definition

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-Function = [	M		LEV_
	reportSupportedDTC		#0Ah	RSUPDTC
	reportFirstTetFailedDTC		#0Bh	RFTFDTC
	reportFirstConfirmedDTC		#0Ch	RFCDTC
	reportMostRecentTestFailedDTC		#0Dh	RMRTFDTC
	reportMostRecentConfirmedDTC		#0Eh	RMRCDTC
	reportDTCFaultDetectionCounter		#14h	RDTCFDC
	reportDTCWithPermanentStatus]		#15h	RDTCWPS

표 6-10. Request Message definition – sub-function = #0A~15

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [			LEV_
	reportNumberOfDTCBySeverityMaskRecord	M	#07h	RNODTCBSMR
	reportDTCBySeverityMaskRecord]	M	#08h	RDTCBSMR
	DTCSeverityMaskRecord[] = [			DTCSVMREC_
#3d	DTCSeverityMask	M	#XXh	DTCSVM
#4d	DTCStatusMask]	M	#XXh	DTCSM
DTCSeverityMask, DTCSeverity : See Annex D.3				

표 6-11. Request Message definition – sub-function = #07 / 08h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [	M		LEV_
	reportSeverityInformationOfDTC]		#09h	RSIODTC
	DTCMaskRecord[] = [			DTCMREC_
#3d	DTCHighByte	M	#XXh	DTCHB
#4d	DTCMiddleByte	M	#XXh	DTCMB
#5d	DTCLowByte]	M	#XXh	DTCLB

표 6-12. Request Message definition – sub-function = #09h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [	M		LEV_
	reportSupportedDTCExtDataRecord]		#1Ah	RDTCEDI
#3d	DTCExtDataRecordNumber	M	#01~FDh	DTCEDRN

표 6-23. Request Message definition – sub-function = #1Ah

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [	M		LEV_
	reportWWHOBDDTCByMaskRecord]		#42h	ROBDDTCBMR

#3d	FunctionalGroupIdentifier	M	#XX	FGID
#4d #5d	DTCSeverityMaskRecord[] = [DTCStatusMask DTCSeverityMask]	M	 #XX #XX	DTCSVMREC_ DTCSM DTCSVM

표 6-24. Request Message definition - sub-function = #42h

Hex	Description	Cvt	Mnemonic
00~32	ISO/SAE reserved	M	ISOSAERESRVD
33	Emissions-system group	M	EMSYSGRP
34~CF	ISO/SAE reserved	M	ISOSAERESRVD
D0	Safety-system group	M	SAFESYSGRP
D1~DF	Legislative system group	M	LEGSYSGRP
E0~FD	ISO/SAE reserved	M	ISOSAERESRVD
FE	VOBD system	M	VOBDSYSGRP
FF	ISO/SAE reserved	M	ISOSAERESRVD

표 6-25. FunctionalGroupIdentifiers definition

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [reportWWHOBDDTCWithPermanentStatus]	M	 #55h	LEV_ RWWHOBDDTCWPS
#3d	FunctionalGroupIdentifier	M	#XXh	FGID

표 6-26. Request Message definition - sub-function = #55h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [reportDTCInformationByDTCReadinessGroupIdentifier]	M	 #56h	LEV_ RDTCBRGI
#3d	FunctionalGroupIdentifier	M	#XX	FGID
#4d	DTCReadinessGroupIdentifier	M	#XX	DTCRGI

표 6-27. Request Message definition - sub-function = #56h

Hex	Description	Cvt	Mnemonic
00	Reserved	M	
01	reportNumberOfDTCByStatusMask This parameter specifies that the server shall transmit to the client the number of DTC's matching a client defined status mask.	M	RNODTCBSM
02	reportDTCByStatusMask This parameter specifies that the server shall transmit to the client a list of DTC's and corresponding statuses matching a client defined status mask.	M	RDTCBSM

03	reportDTCSnapshotIdentification This parameter specifies that the server shall transmit to the client all DTCSnapshot data record identifications (DTC number(s) and DTCSnapshot record number(s)).	U	RDTCSSI
04	reportDTCSnapshotRecordByDTCNumber This parameter specifies that the server shall transmit to the client the DTCSnapshot record(s) associated with a client defined DTC number and DTCSnapshot record number (FF hex for all records).	U	RDTCSSBDTC
05	reportDTCSnapshotRecordByRecordNumber This parameter specifies that the server shall transmit to the client the DTCSnapshot record(s) associated with a client defined DTCSnapshot record number (FF hex for all records). Note that this sub-function parameter can only be supported if the DTCSnapshotRecordNumber is unique to the server (each record number exists only once in the server) and not unique to a DTC.	U	RDTCSSBRN
06	reportDTCExtendedDataRecordByDTCNumber This parameter specifies that the server shall transmit to the client the DTCExtendedData record(s) associated with a client defined DTC number and DTCExtendedData record number (FF hex for all records, FE hex for all OBD records).	U	RDTCCEDRBDN
07	reportNumberOfDTCBySeverityMaskRecord This parameter specifies that the server shall transmit to the client the number of DTC's matching a client defined severity mask record.	U	RNODTCBSMR
08	reportDTCBySeverityMaskRecord This parameter specifies that the server shall transmit to the client a list of DTC's and corresponding statuses matching a client defined severity mask record.	U	RDTCBSMR
09	reportSeverityInformationOfDTC This parameter specifies that the server shall transmit to the client the severity information of a specific DTC specified in the client request message.	U	RSIODTC
0A	reportSupportedDTC This parameter specifies that the server shall transmit to the client a list of all DTC's and corresponding statuses supported within the server.	M	RSUPDTC
0B	reportFirstTestFailedDTC This parameter specifies that the server shall transmit to the client the first failed DTC to be detected by the server since the last clear of diagnostic information. Note that the information reported via this sub-function parameter shall be independent of whether or not the DTC was confirmed or aged.	U	RFTFDTC

0C	reportFirstConfirmedDTC This parameter specifies that the server shall transmit to the client the first confirmed DTC to be detected by the server since the last clear of diagnostic information. The information reported via this sub-function parameter shall be independent of the aging process of confirmed DTC's (e.g. if a DTC ages such that its status is allowed to be reset, the first confirmed DTC record shall continue to be preserved by the server, regardless of any other DTC's that become confirmed afterwards).	U	DFCDTC
0D	reportMostRecentTestFailedDTC This parameter specifies that the server shall transmit to the client the most recent failed DTC to be detected by the server since the last clear of diagnostic information. Note that the information reported via this sub-function parameter shall be independent of whether or not the DTC was confirmed or aged.	U	RMRVDTC
0E	reportMostRecentConfirmedDTC This parameter specifies that the server shall transmit to the client the most recent confirmed DTC to be detected by the server since the last clear of diagnostic information. Note that the information reported via this sub-function parameter shall be independent of the aging process of confirmed DTC's (e.g. if a DTC ages such that its status is allowed to be reset, the first confirmed DTC record shall continue to be preserved by the server assuming no other DTC's become confirmed afterwards).	U	RMRCDTC
0F	reportMirrorMemoryDTCByStatusMask This parameter specifies that the server shall transmit to the client a list of DTC's out of the DTC mirror memory and corresponding statuses matching a client defined status mask.	U	RMMDTCBSM
10	reportMirrorMemoryDTCExtendedDataRecordByDTCNumber This parameter specifies that the server shall transmit to the client the DTCExtendedData record(s) – out of the DTC mirror memory – associated with a client defined DTC number and DTCExtendedData record number (FF hex for all records) DTC's.	U	RMMDEDRBDN
11	reportNumberOfMirrorMemoryDTCByStatusMask This parameter specifies that the server shall transmit to the client the number of DTC's out of mirror memory matching a client defined status mask.	U	RNOMMDTCBSM

12	reportNumberOfEmissionsRelatedOBDDTCByStatusMask This parameter specifies that the server shall transmit to the client the number of emissionsrelated OBD DTC's matching a client defined status mask. The number of OBD DTC's reported shall only be those which are required to be compatible with emissions-related legal requirements.	U	RNOOBDDTCBSM
13	reportEmissionsRelatedOBDDTCByStatusMask This parameter specifies that the server shall transmit to the client a list of emissions-related OBD DTC's and corresponding statuses matching a client defined status mask. The list of OBD DTC's reported shall only be those which are required to be compatible with emissions-related legal requirements.	U	ROBDDTCBSM
14~19	Reserved	U	RESRVD
1A	reportDTCExtendedDataRecordIdentification This parameter specifies that the server shall transmit to the client the DTCs which supports a DTCExtendedDataRecord.	U	
1B~41	ISOSAEReserved	M	ISOSAERESRVD
42	reportWWHOBDDTCByMaskRecord This parameter specifies that the server shall transmit to the client a list of WWH OBD DTCs and corresponding status and severity information matching a client defined status mask and severity mask record.	U	RWWHOBDDTCBMR
43~54	ISOSAEReserved	M	ISOSAERESRVD
55	reportWWHOBDDTCWithPermanentStatus This parameter specifies that the server shall transmit to the client a list of WWH OBD DTCs with "permanent DTC" status	U	RWWHOBDDTCWPS
56	reportDTCInformationByDTCReadinessGroupIdentifier This parameter specifies that the server shall transmit to the client a list of OBD DTCs which matches the DTCReadiness Group Identifier.	U	RDTCBRGI
57~7F	ISOSAEReserved	M	ISOSAERESRVD

표 6-13. Request message sub-function definition

6.2.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTCIPR
#2d	reportType = [reportNumberOfDTCByStatusMask reportNumberOfDTCBySeverityMaskRecord reportNumberOfMirrorMemoryDTCByStatusMask reportNumberOfEmissionRelatedOBDDTCByStatusMask]	M	 #01h #07h #11h #12h	LEV_ RNODTCBSM RNODTCBSMR RNOMMDTCBSM RNOOBDDTCBSM
#3d	DTCStatusAvailabilityMask	M	#XXh	DTCSAM

#4d	DTCFormatIdentifier = [SAE_J2012-DA_DTCFormat_04]	M	#04h	DTCFID_ SAE_J2012-DA DTC Format_04
 #5d #6d	DTCCount[] = [DTCCountHighByte DTCCountLowByte]	 M M	 #XXh #XXh	DTCC_ DTCCHB DTCCLB

표 6-14. Positive Response Message definition – sub-function = #01 / 07 / 11 / 12h

Hex	Description	Mnemonic
00	SAE_J2012-DA DTC Format_00	J2012-DADTCF00
01	ISO 14229-1 DTC Format	14229-1DTCF
02	SAE J1939-73 DTC Format	J1939-73DTCF
03	ISO 11992-4 DTC Format	11992-4DTCF
04	SAE_J2012-DA DTC Format_04	J2012-DADTCF04
05~FF	Reserved	

표 6-15. DTCFormat Identifier definition

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	**ReadDTCInformation Response Service ID**	**M**	**#59h**	**RDTCIPR**
#2d	reportType = [reportDTCByStatusMask reportSupportedDTCs reportFirstTestFailedDTC reportFirstConfirmedDTC reportMostRecentTestFailedDTC reportMostRecentConfirmedDTC reportDTCWithPermanentStatus]	M	 #02h #0Ah #0Bh #0Ch #0Dh #0Eh #15h	LEV_ RDTCBSM RSUPDTC RFTFDTC RFCDTC RMRTFDTC RMRCDTC RDTCWPS
#3d	DTCStatusAvailabilityMask	M	#XXh	DTCSAM
 #4d #5d #6d #7d #8d #9d #10d #11d .. #n-3d #n-2d #n-1d #nd	DTCAndStatusRecord[] = [DTCHighByte#1 DTCMiddleByte#1 DTCLowByte#1 statusOfDTC#1 DTCHighByte#2 DTCMiddleByte#2 DTCLowByte#2 statusOfDTC#2 .. DTCHighByte#m DTCMiddleByte#m DTCLowByte#m statusOfDTC#m	 C C C C C C C C .. C C C C	 #XXh #XXh #XXh #XXh #XXh #XXh #XXh #XXh .. #XXh #XXh #XXh #XXh	DTCASR_ DTCHB DTCMB DTCLB SODTC DTCHB DTCMB DTCLB SODTC .. DTCHB DTCMB DTCLB SODTC

표 6-16. Positive Response Message definition – sub-function = #02 / 0A~0E / 15

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	**ReadDTCInformation Response Service ID**	**M**	**#59h**	**RDTCIPR**

#2d	reportType = [reportDTCSnapshotIdentification]	M	 #03h	LEV_ RDTCSSI
 #3d #4d #5d	DTCRecord[] #1 = [DTCHighByte#1 DTCMiddleByte#1 DTCLowByte#1	 C C C	 #XXh #XXh #XXh	DTCASR_ DTCHB DTCMB DTCLB
#6d	DTCSnapshotRecordNumber #1	C	#XXh	DTCSSRN
..	..	..	..	..
 #9d #10d #11d	DTCRecord[] #m = [DTCHighByte#m DTCMiddleByte#m DTCLowByte#m]	 C C C	 #XXh #XXh #XXh	DTCASR_ DTCHB DTCMB DTCLB
#nd	DTCSnapshotRecordNumber #m	C	#XXh	DTCSSRN

표 6-17. Positive Response Message definition – sub-function = #03h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	**ReadDTCInformation Response Service ID**	**M**	**#59h**	**RDTCIPR**
#2d	reportType = [reportDTCSnapshotRecordByDTCNumber]	M	 #04h	LEV_ RDTCSSBDTC
 #3d #4d #5d #6d	DTCAndStatusRecord[] = [DTCHighByte DTCMiddleByte DTCLowByte statusOfDTC]	 M M M M	 #XXh #XXh #XXh #XXh	DTCASR_ DTCHB DTCMB DTCLB SODTC
#7d	DTCSnapshotRecordNumber #1	C	#XXh	DTCSSRN
#8d	DTCSnapshotRecordNumberOfIdentifiers #1	C	#XXh	DTCSSRNI
 #9d .. #9+k-1d #9+kd .. #9+k+(p-1)d .. #r-(m-1)-2d .. #r-(m-1)-1d #r-(m-1)d .. #rd	DTCSnapshotRecord[] #1 = [dataIdentifier#1 byte#1 (MSB) .. dataIdentifier#1 byte#k snapshotData#1 byte#1 .. snapshotData#1 byte#p .. dataIdentifier#w byte#1 (MSB) .. dataIdentifier#2 byte#k snapshotData#w byte#1 .. snapshotData#w byte#m]	 C .. C C .. C .. C .. C C .. C	 #XXh .. #XXh #XXh .. #XXh .. #XXh .. #XXh #XXh .. #XXh	DTCSSR_ DIDB11 .. DIDB1k SSD11 .. SSD1p .. DIDBw1 .. DIDBwk SSDw1 .. SSDwm
..	..	..	..	..
#t	DTCSnapshotRecordNumber#x	C	#XXh	DTCSSRN
#t+1d	DTCSnapshotRecordNumberOfIdentifiers #x	C	#XXh	DTCSSRNI

 #t+2d .. #t+2+k-1d #t+2+kd .. #t+2+k+(p-1)d .. #n-(u-1)-2d .. #n-(u-1)-1d #n-(u-1)d .. #nd	DTCSnapshotRecord[] #x = [dataIdentifier#1 byte#1 (MSB) .. dataIdentifier#1 byte#k snapshotData#1 byte#1 .. snapshotData#1 byte#p .. dataIdentifier#w byte#1 (MSB) .. dataIdentifier#2 byte#k snapshotData#w byte#1 .. snapshotData#w byte#u]	 C .. C C .. C .. C .. C C .. C	 #XXh .. #XXh #XXh .. #XXh .. #XXh .. #XXh #XXh .. #XXh	DTCSSR_ DIDB11 .. DIDB1k SSD11 .. SSD1p .. DIDBw1 .. DIDBwk SSDw1 .. SSDwu

표 6-18. Positive Response Message definition - sub-function = #04h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTCIPR
#2d	reportType = [reportDTCSnapshotRecordByRecordNumber]	M	#05h	LEV_ RDTCSSBRN
#3d	DTCSnapshotRecordNumber#1	M	#XXh	DTCEDRN
 #4d #5d #6d #7d	DTCAndStatusRecord[] #1 = [DTCHighByte DTCMiddleByte DTCLowByte statusOfDTC]	 M M M M	 #XXh #XXh #XXh #XXh	DTCASR_ DTCHB DTCMB DTCLB SODTC
#8d	DTCSnapshotRecordNumberOfIdentifiers #1	C	#XXh	DTCSSRNI
 #9d .. #9+k-1d #9+kd .. #9+k+(p-1)d .. #r-(m-1)-2d .. #r-(m-1)-1d #r-(m-1)d .. #rd	DTCSnapshotRecord[] #1 = [dataIdentifier#1 byte#1 (MSB) .. dataIdentifier#1 byte#k snapshotData#1 byte#1 .. snapshotData#1 byte#p .. dataIdentifier#w byte#1 (MSB) .. dataIdentifier#2 byte#k snapshotData#w byte#1 .. snapshotData#w byte#m]	 C .. C C .. C .. C .. C C .. C	 #XXh .. #XXh #XXh .. #XXh .. #XXh .. #XXh #XXh .. #XXh	DTCSSR_ DIDB11 .. DIDB1k SSD11 .. SSD1p .. DIDBw1 .. DIDBwk SSDw1 .. SSDwm
..	..	..	..	..

#t	DTCSnapshotRecordNumber#x	C	#XXh	DTCSSRN
 #t+1d #t+2d #t+3d #t+4d	DTCAndStatusRecord[] #x = [DTCHighByte DTCMiddleByte DTCLowByte statusOfDTC]	 M M M M	 #XXh #XXh #XXh #XXh	DTCASR_ DTCHB DTCMB DTCLB SODTC
#t+5d	DTCSnapshotRecordNumberOfIdentifiers #x	C	#XXh	DTCSSRNI
 #t+6d .. #t+6+k-1d #t+6+kd .. #t+6+k+(p-1)d .. ??? .. #n-(u-1)-1d #n-(u-1)d .. #nd	DTCSnapshotRecord[] #x = [dataIdentifier#1 byte#1 (MSB) .. dataIdentifier#1 byte#k snapshotData#1 byte#1 .. snapshotData#1 byte#p .. dataIdentifier#w byte#1 (MSB) .. dataIdentifier#2 byte#k snapshotData#w byte#1 .. snapshotData#w byte#u]	 C .. C C .. C .. C .. C C .. C	 #XXh .. #XXh #XXh .. #XXh .. #XXh .. #XXh #XXh .. #XXh	DTCSSR_ DIDB11 .. DIDB1k SSD11 .. SSD1p .. DIDBw1 .. DIDBwk SSDw1 .. SSDwu

표 6-19. Positive Response Message definition - sub-function = #05h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTCIPR
#2d	reportType = [reportDTCExtendedDataRecordByDTCNumber reportMirrorMemoryDTCExtendedDataReordByDTCNumber]	M	 #06h #10h	LEV_ RDTCEDRBDN RMMDEDRBDN
 #3d #4d #5d #6d	DTCAndStatusRecord[] = [DTCHighByte DTCMiddleByte DTCLowByte statusOfDTC]	 M M M M	 #XXh #XXh #XXh #XXh	DTCASR_ DTCHB DTCMB DTCLB SODTC
#7d	DTCExtendedDataRecordNumber#1	C	#XXh	DTCEDRN
 #8d .. #8+(p-1)d	DTCExtendedDataRecord[] #1 = [extendedData#1 byte#1 .. extendedData#1 byte#p]	 C .. C	 #XXh .. #XXh	DTCSSR_ EDD11 .. EDD1p
..	..	..	..	..
#td	DTCExtendedDataRecord#x	C	#XXh	DTCEDRN

#t+1d .. #t+1+(q-1)d	DTCExtendedDataRecord[] #x = [extendedData#1 byte#1 .. extendedData#1 byte#q]	C .. C	#XXh .. #XXh	DTCSSR_ EDDx1 .. EDDxq

표 6-20. Positive Response Message definition - sub-function = #06 / 10h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTCIPR
#2d	reportType = [reportDTCBySeverityMaskRecord reportSeverityInformationOfDTC]	M	 #08h #09h	LEV_ RDTCBSMR RSIODTC
#3d	DTCStatusAvailabilityMask	C	#XXh	DTCSAM
 #4d #5d #6d #7d #8d #9d .. #n-5d #n-4d #n-3d #n-2d #n-1d #nd	DTCAndSeverityRecord[] = [DTCSeverity #1 DTCFunctionalUnit #1 DTCHighByte #1 DTCMiddleByte #1 DTCLowByte #1 statusOfDTC #1 .. DTCSeverity #m DTCFunctionalUnit #m DTCHighByte #m DTCMiddleByte #m DTCLowByte #m statusOfDTC #m	 C C C C C C .. C C C C C C	 #XXh #XXh #XXh #XXh #XXh #XXh .. #XXh #XXh #XXh #XXh #XXh #XXh	DTCASR_ DTCS DTCFU DTCHB DTCMB DTCLB SODTC .. DTCS DTCFU DTCHB DTCMB DTCLB SODTC

표 6-21. Positive Response Message definition - sub-function = #08 / 09h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTCIPR
 #2d	reportType = [reportSupportedDTCExtDataRecord]	M	 #1Ah	LEV_ RDTCEDI
#3d	DTCStatusAvailabilityMask	M	#XXh	DTCSAM
#4d	DTCExtDataRecordNumber	C1	#01~FD	DTCEDRN
 #5d #6d #7d #8d	DTCAndStatusRecord[]#1 = [DTCHighByte #1 DTCMiddleByte #1 DTCLowByte #1 statusOfDTC #1]	 C1 C1 C1 C1	 #XXh #XXh #XXh #XXh	DTCASR_ DTCHB DTCMB DTCLB SODTC
..	..	..	..	..
 #n-3d #n-2d #n-1d #nd	DTCAndStatusRecord[]#m = [DTCHighByte #m DTCMiddleByte #m DTCLowByte #m statusOfDTC #m]	 C2 C2 C2 C2	 #XXh #XXh #XXh #XXh	DTCASR_ DTCHB DTCMB DTCLB SODTC

C1: This parameter shall be present if at least one DTC supports the DTCExtendedDataRecord
C2: This parameter record is only present if more than one DTC support the DTCExtendedDataRecord

표 6-28. Positive Response Message definition – sub-function = #1Ah

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTCIPR
#2d	reportType = [reportWWHOBDDTCByMaskRecord]	M	#42h	LEV_ RWWHOBDDTCBSMR
#3d	FunctionalGroupIdentifier	M	#XXh	FGID
#4d	DTCStatusAvailabilityMask	M	#XXh	DTCSAM
#5d	DTCSeverityAvailabilityMask	M	#XXh	DTCSVAM
#6d	DTCFormatIdentifier = [SAE_J2012-DA_DTCFormat_04]	M	#04h	DTCFID_ J2012-DADTCF04
	DTCAndSeverityRecord[] = [			DTCASR_
#7d	DTCSeverity#1	C1	#XXh	DTCS
#8d	DTCHighByte#1 (MSB)	C1	#XXh	DTCHB
#9d	DTCMiddleByte#1	C1	#XXh	DTCMB
#10d	DTCLowByte#1	C1	#XXh	DTCLB
#11d	statusOfDTC#1	C1	#XXh	SODTC
..	..	..	..	..
#n-4d	DTCSeverity#m	C2	#XXh	DTCS
#n-3d	DTCHighByte#m (MSB)	C2	#XXh	DTCHB
#n-2d	DTCMiddleByte#m	C2	#XXh	DTCMB
#n-1d	DTCLowByte#m	C2	#XXh	DTCLB
#nd	statusOfDTC#m]	C2	#XXh	SODTC

C1: This parameter is only present if DTC information is available to be reported.
C2: This parameter is only present if more than one DTC information is available to be reported.

표 6-29. Positive Response Message definition – sub-function = #42h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTCIPR
#2d	reportType = [reportWWHOBDDTCWithPermanentStatus]	M	#55h	LEV_ RWWHOBDDTCWPS
#3d	FunctionalGroupIdentifier	M	#XXh	FGID
#4d	DTCStatusAvailabilityMask	M	#XXh	DTCSAM
#5d	DTCFormatIdentifier = [SAE_J2012-DA_DTCFormat_04]	M	#04h	DTCFID_ J2012-DADTCF04

	DTCAndStatusRecord[] = [			DTCASR_
#6d	DTCHighByte#1 (MSB)	C1	#XXh	DTCHB
#7d	DTCMiddleByte#1	C1	#XXh	DTCMB
#8d	DTCLowByte#1	C1	#XXh	DTCLB
#9d	statusOfDTC#1	C1	#XXh	SODTC
..	..	..	..	..
#n-3d	DTCHighByte#m (MSB)	C2	#XXh	DTCHB
#n-2d	DTCMiddleByte#m	C2	#XXh	DTCMB
#n-1d	DTCLowByte#m	C2	#XXh	DTCLB
#nd	statusOfDTC#m]	C2	#XXh	SODTC
C1: This parameter is only present if DTC information is available to be reported. C2: This parameter is only present if more than one DTC information is available to be reported.				

표 6-30. Positive Response Message definition – sub-function = #55h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTCIPR
#2d	reportType = [reportDTCByReadinessGroupIdentifier]	M	#56h	LEV_ RWWHOBDDT CWPS
#3d	FunctionalGroupIdentifier	M	#XXh	FGID
#4d	DTCStatusAvailabilityMask	M	#XXh	DTCSAM
#5d	DTCFormatIdentifier = [SAE_J2012-DA_DTCFormat_04]	M	#04h	DTCFID_ J2012- DADTCF04
#6d	DTCReadinessGroupIdentifier	M	#XXh	RGI
	DTCAndStatusRecord[] = [			DTCASR_
#7d	DTCHighByte#1 (MSB)	C1	#XXh	DTCHB
#8d	DTCMiddleByte#1	C1	#XXh	DTCMB
#9d	DTCLowByte#1	C1	#XXh	DTCLB
#10d	statusOfDTC#1	C1	#XXh	SODTC
..	..	..	..	..
#n-3d	DTCHighByte#m (MSB)	C2	#XXh	DTCHB
#n-2d	DTCMiddleByte#m	C2	#XXh	DTCMB
#n-1d	DTCLowByte#m	C2	#XXh	DTCLB
#nd	statusOfDTC#m]	C2	#XXh	SODTC
C1: This parameter shall be present if at least one DTC matches the client defined Readiness Group Identifier. C2: This parameter record is only present if more than one DTC matches the client defined Readines Group Identifier				

표 6-31. Positive Response Message definition – sub-function = #56h

6.2.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	This code is returned if the requested sub-function is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong

31	requestOutOfRange	M	This code is returned if : 1. The client specified a DTCMaskRecord / DTCSeverityMaskRecord that was not recognized by the server. 2. The client specified an invalid DTCSnapshotRecordNumber / DTCExtendedDataRecordNumber.

표 6-22. Supported Negative Response Codes

7. InputOutput Control functional unit

7.1 InputOutputControlByIdentifier (2F hex) service

The InputOutputControlByIdentifier service is used by the client to substitute a value for an input signal, internal server function and/or control an output (actuator) of an electronic system.
The server shall send a positive response message to a request message with an inputOutputControlParameter of returnControlToECU even if the dataIdentifier is currently not under tester control.

7.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	InputOutputControlByIdentifer Request Service ID	M	#2Fh	IOCBI
 #2d #3d	dataIdentifier #1[] = [byte #1 (MSB) byte #2 (LSB)]	 M M	 #XXh #XXh	IOI_ B1 B2
 #4d	controlOptionRecord #1[] = [inputOutputControlParameter	 M	 #XXh	CSR_ IOCP
 #5d .. #5+(m-1)d	controlOptionRecord #2[] = [controlState#1 .. controlState#m	 C1 .. C1	 #XXh .. #XXh	CSR_ CS_ .. CS_
 #5+md .. #5+m+(r-1)d	controlEnableMaskRecord #1[] = [controlMask#1 .. controlMask#r]	 C2 .. C2	 #XXh .. #XXh	CEM_ CM_ .. CM_

C1 : The presence of this parameter depends on the dataIdentifier#1 and the inputOutputControl Parameter of controlOptionRecord#1. (see Table 7-2)
C2 : The presence of this parameter depends on the dataIdentifier#1.
dataIdentifier : see Annex C.1
controlEnableMaskRecord : see Annex C.1.1
The value of each bit shall determine whether the corresponding parameter in the dataIdentifier will be affected by the request. A bit value of '0' in the ControlEnableMask shall represent that the corresponding parameter is not affected by this request and a bit value of '1' shall represent that the corresponding parameter is affected by this request. The most significant bit of ControlMask#1 shall correspond to the first parameter in the ControlState starting at the most significant bit of ControlState#1, the second most significant bit of ControlMask#1 shall correspond to the second parameter in the ControlState, and continuing on in this fashion 139tilizing as many ControlMask bytes as necessary to mask all parameters. For example, the least significant bit of ControlMask#2 would correspond to the 16th parameter in the controlState. For bitmapped dataIdentifiers, unsupported bits shall also have a corresponding bit in the ControlEnableMask so that the position of the mask bit of every parameter in the ControlEnableMask shall exactly match the position of the corresponding parameter in the controlState.

표 7-1. Request Message

Hex	Description	Cvt	Mnemonic
00	returnControlToECU This value shall indicate to the server that the client does no longer have control about the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier. Number of controlState bytes in request: 0 Number of controlState bytes in pos. response: if the dataIdentifier consists of more than one parameter	U	RCTECU
01	resetToDefault This value shall indicate to the server that it is requested to reset the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier to its default state. Number of controlState bytes in request: 0 Number of controlState bytes in pos. response: depends on the dataIdentifier	U	RTD
02	freezeCurrentState This value shall indicate to the server that it is requested to freeze the current state of the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier. Number of controlState bytes in request: 0 Number of controlState bytes in pos. response: depends on the dataIdentifier	U	FCS
03	shortTermAdjustment This value shall indicate to the server that it is requested to adjust the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier in RAM to the value(s) included in the controlOption parameter(s). (e.g. set Idle Air Control Valve to a specific step number, set pulse width of valve to a specific value/duty cycle). Number of controlState bytes in request: depends on the dataIdentifier Number of controlState bytes in pos. response: depends on the dataIdentifier	U	STA

표 7-2. inputOutputControlParameter definition

7.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	InputOutputControlByIdentifier Response Service ID	M	#6Fh	IOCBIPR
 #2d #3d	dataIdentifier #1[] = [byte #1 (MSB) byte #2 (LSB)]	 M M	 #XXh #XXh	IOI_ B1 B2
 #4d	controlOptionRecord #1[] = [inputOutputControlParameter	 M	 #XXh	CSR_ IOCP

#5d .. #5+(m-1)d	controlOptionRecord #2[] = [controlState#1 .. controlState#m	C .. C	#XXh .. #XXh	CSR_ CS_ .. CS_

표 7-3. Positive Response Message definition

7.1.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This code shall be returned if the criteria for the request InputOutputControlByIdentifier are not met.
31	requestOutOfRange	M	This code shall be returned if : 1. the requested dataIdentifier value is not supported by the device. 2. the dataIdentifier uses the controlState#1 parameter as an inputOutputControlParameter and the value contained in this parameter is invalid. 3. the one or multiple of the applicable controlStates of the controlOptionRecord record are invalid. 4. the combination of bits enabling control in the ControlEnableMaskRecord is not supported by the device.
33	securityAccessDenied	M	This code shall be returned if a client sends a request with a valid secure dataIdentifier and the server's security feature is currently active.

표 7-4. Supported Negative Response Codes

8. Remote Activation of Routine functional unit

The following section describes two (2) different methods of implementation (Methods “A” and “B”).

- Method “A” ;
  - This method is based on the assumption that after a routine has been started by the client in the server;s memory the client shall be responsible to stop the routine.
  - The server routine shall be started in the server’s memory some time between the completion of the RoutineControl request message that starts the routine and the completion of the first response message (if “positive” based on the server’s conditions).
  - The server routine shall be stopped in the server’s memory some time after the completion of the StopRoutine request message and the completion of the first response message (if “positive” based on the server’s conditions).
  - The client may request routine results after the routine has been stopped.

- Method “B” ;
  - This method is based on the assumption that after a routine has been started by the client in the server’s memory that the server shall be responsible to stop the routine.
  - The server routine shall be started in the server’s memory some time between the completion of the RoutineControl request message that starts the routine and the completion of the first response message (if “positive” based on the server’s conditions).
  - The server routine shall be stopped any time as programmed or previously initialized in the server’s memory.

8.1 RoutineControl (31 hex) service

The RoutineControl service is used by the client to ;
- start a routine,
- stop a routine, and
- request routine results

A routine is referenced by a 2-byte routineIdentifier.
Each routine has security option. (e.g. routineIdentifier XXXX : Security is not required, routineIdentifier YYYY : ASK required)
Reprogram routine (eraseMemory, checkProgrammingDependencies, checkmemory, …, etc) has to be secured by respective ECU security option(ASK, C-SAC, NormalSeedKey and so on).
For other routines, If there is no special purpose, it is necessary to apply security options to routines. Please refer to each functional specification or ECU diagnostic specification.

8.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	 #XXh	LEV_ RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #XXh #XXh	RI_ B1 B2

#5d .. #nd	routineControlOptionRecord [] = [routineControlOption #1 .. routineControlOption #m]	C/U .. C/U	#XXh .. #XXh	RCEOR_ RCO_ .. RCO_
routineControlOptionRecord : This parameter record contains either - Routine entry option parameters, which optionally specify start conditions of the routine, or - Routine exit option parameters, which optionally specify stop conditions of the routine. or - requestRoutineResult option parameters.				

표 8-1. Request Message

Hex	Description	Cvt	Mnemonic
01	startRoutine	M	STR
02	stopRoutine	U	STPR
03	requestRoutineResults	U	RRR

표 8-2. Request message sub-function definition

Hex	Description	Cvt	Mnemonic
0000 - 00FF	Reserved (ISOSAE Reserved)	M	ISOSAERESRVD
0100 - 01FF	Reserved (Tachograph TestIds)	M	TACHORI_
0200	CheckMemory	M	CM
0201 - 020F	Reserved (vehicleManufacturerSpecific)	M	VMS_
0210	ReadActiveArea	U	RAA
0211	FinishUpdate / (OLD)CopyActiveareaTo	U	CAT
0212	EraseTargetArea	U	ETA
0213	SwapActiveArea	U	SAA
0214 - 021F	Reserved (vehicleManufacturerSpecific)	M	VMS_
0220	WriteDeltaPatch	U	WDP
0221 - 02FF	Reserved (vehicleManufacturerSpecific)	M	VMS_
0300	OtaReady	U	OR
0301	EnableUpdateCondition	U	EUC
0302 - DFFF	Reserved (vehicleManufacturerSpecific)	M	VMS_
0400	ReadFoDStatus	U	RFS
0410	PrepareFoDActivation	U	RFA
0411	ExecuteFoDActivation	U	EFA
0412	ConfirmFoDActivation	U	CFA
0420	PrepareFoDDeactivation	U	PFD
0421	ExecuteFoDDeactivation	U	EFD
0422	ConfirmFoDDeactivation	U	CFD
0430	FoDReady	U	FR
0440	DeleteFoDCertificates	U	DFC
0441 - DFFF	Reserved (vehicleManufacturerSpecific)	M	VMS_
E000 - E1FF	Reserved (OBDTestIds) IMPORTANT : Do not use other than legislative ECU	M	OBDRI_
E200 - EFFF	Reserved (ISOSAE Reserved)	M	ISOSAERESRVD

F000 – FEFF	Reserved (SystemSupplierSpecific)	M	SSS_
FF00	eraseMemory	U	EM_
FF01	checkProgrammingDependencies	U	CPD_
FF02 – FFFF	Reserved (ISOSAE Reserved)	M	ISOSAERESRVD

표 8-3. Request message routineIdentifier definition

8.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#XXh	RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #XXh #XXh	RI_ B1 B2
 #5d .. #nd	routineStatusRecord [] = [routineStatus #1 .. routineStatus #m]	 U .. U	 #XXh .. #XXh	RSR_ RS_ .. RS_

표 8-4. Positive Response Message definition

8.1.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	This code is returned if the requested sub-function is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This code shall be returned if the criteria for the request RoutineControl are not met.
24	requestSequenceError	M	This code shall be returned if the 'stopRoutine' or 'requestRoutineResults' sub-function is received without first receiving a 'startRoutine' for the requested routineIdentifier.
31	requestOutOfRange	M	The code shall be returned if ; 1. The server does not supported the requested routineIdentifier. 2. The user optional routineControlOptionRecord contains invalid data for the requested routineIdentifier.
33	seurityAccessDenied	M	This NRC shall be sent if a client sends a requested with a valid secure routineIdentifier and the server's security feature is currently active.
72	GeneralProgrammingFailure	M	This NRC shall be returned if the server detects an error when performing a routine, which accesses server internal memory. An example is when the routine erases or prgrams a certain memory location in the permanent memory device (e.g. Flash Memory) and the access to that memory location fails.

표 8-5. Supported Negative Response Codes

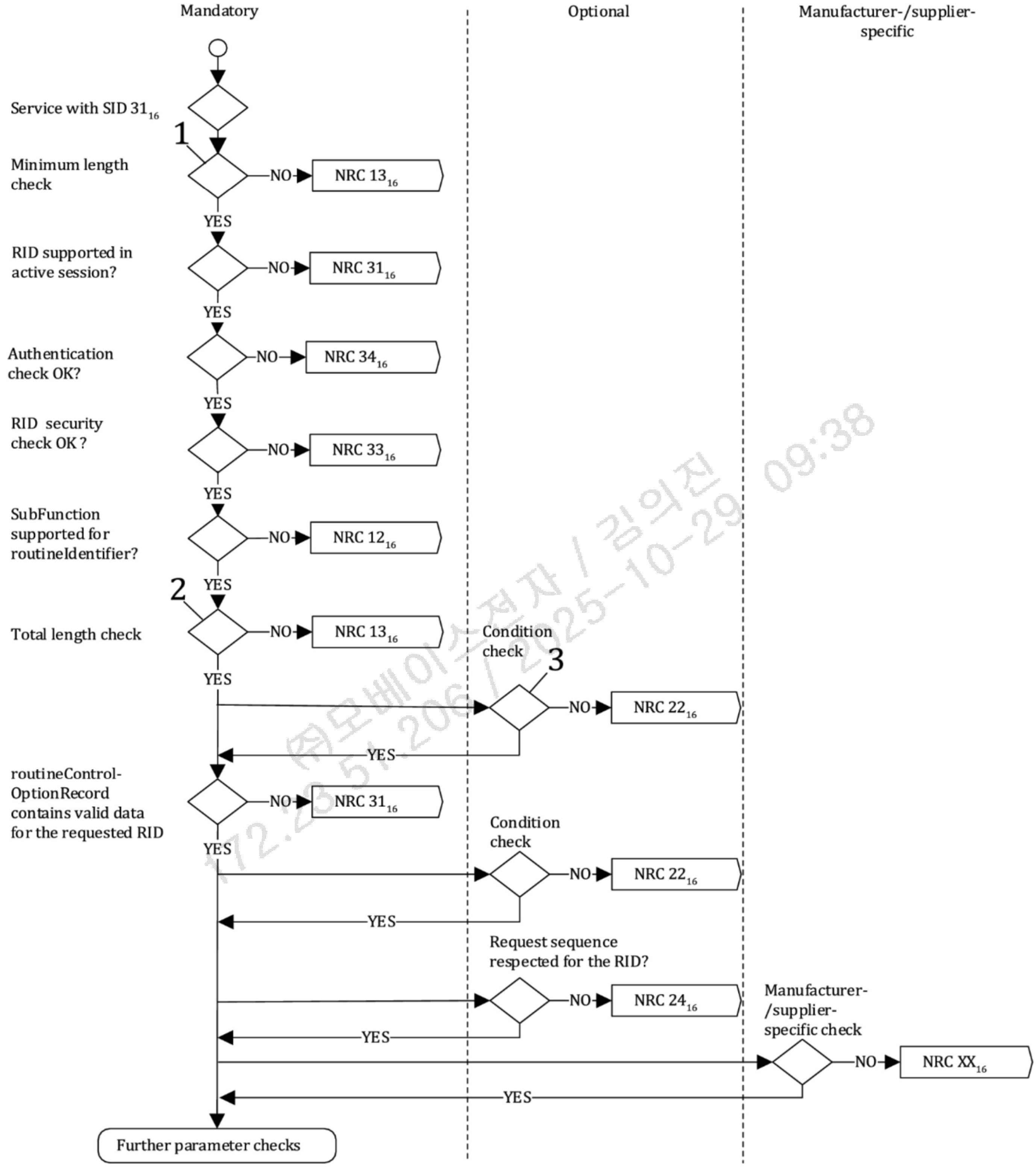


Figure8-1. NRC handling for RoutineControlService

Key

1 at least 4 (SI+SubFunction+RID Parameter)

2 1 byte SI + 1 byte SF + 2 byte RID + nth byte routineControlOptionRecord required for the specific RID

3 optional condition check independent from data content

8.1.4 Implementation rules for legacy update / OTA 1.0

8.1.4.1 OTA Ready

Session : Default and Non-default session (Bootloader All sessions including Programming(02) – Optional)
Security level : No

8.1.4.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #03h #00h	RI_ B1 B2

Table8-6. Request Message

8.1.4.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #03h #00h	RI_ B1 B2
#5d	routineControlOptionRecord [] = [routineStatus]	M	#XXh	RCOR_RS

Table8-7. Positive Response Message definition

Hex	Description	Cvt	Mnemonic
00	(Reserved)	M	
01	OK (Server can be reprogrammed immediately)	M	
02~FF	(Reserved)	M	

Table8-8. Routine status definition

8.1.4.1.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	
13	incorrectMessageLengthOrInvalidFormat	M	
31	requestOutOfRange	M	
33	securityAccessDenied	M	
78	requestCorrectlyReceived-ResponsePending	M	

Table8-9. Supported Negative Response Codes

8.1.4.2 (OLD)EraseTargetArea (ECUs without memory redundancy)

Session : Non-default session
Security level : Required (ASK / OEUK)

8.1.4.2.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #FFh #00h	RI_ B1 B2
 #5d #6d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	 M	 #XXh	 RCOR_SW UMA
#7d #8d	routineControlOptionRecord [] = [Memory block]	U	#XXh #XXh	RCOR_MB
The SWUnitMemoryArea represents the memory area to be erased. Memory block would be used as ECU specific memory index.				

Table8-10. Request Message

Hex	Description	Cvt	Mnemonic
F1B0	ECUSoftwareUNIT's entire area	M	ECUSUNIT
F1B1	ECUSoftwareUNIT1 area	M	ECUSUNIT1
F1B2	ECUSoftwareUNIT2 area	C	ECUSUNIT2
F1B3	ECUSoftwareUNIT3 area	C	ECUSUNIT3
F1B4	ECUSoftwareUNIT4 area	C	ECUSUNIT4
F1B5	ECUSoftwareUNIT5 area	C	ECUSUNIT5
F1B6	ECUSoftwareUNIT6 area	C	ECUSUNIT6
F1B7	ECUSoftwareUNIT7 area	C	ECUSUNIT7
F1B8	ECUSoftwareUNIT8 area	C	ECUSUNIT8
F1B9	ECUSoftwareUNIT9 area	C	ECUSUNIT9
F1BA	ECUSoftwareUNIT10 area	C	ECUSUNIT10
C: More than 1 SWUnit is defined.			

Table8-11. SWUnitMemoryArea definition

8.1.4.2.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_

#3d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #FFh #00h	RI_ B1 B2
 #5d #6d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	 M	 #XXh	 RCOR_SW UMA

Table8-12. Positive Response Message definition

Hex	Description	Cvt	Mnemonic
F1B0	ECUSoftwareUNIT's entire area	M	ECUSUNIT
F1B1	ECUSoftwareUNIT1 area	M	ECUSUNIT1
F1B2	ECUSoftwareUNIT2 area	C	ECUSUNIT2
F1B3	ECUSoftwareUNIT3 area	C	ECUSUNIT3
F1B4	ECUSoftwareUNIT4 area	C	ECUSUNIT4
F1B5	ECUSoftwareUNIT5 area	C	ECUSUNIT5
F1B6	ECUSoftwareUNIT6 area	C	ECUSUNIT6
F1B7	ECUSoftwareUNIT7 area	C	ECUSUNIT7
F1B8	ECUSoftwareUNIT8 area	C	ECUSUNIT8
F1B9	ECUSoftwareUNIT9 area	C	ECUSUNIT9
F1BA	ECUSoftwareUNIT10 area	C	ECUSUNIT10
C: More than 1 SWUnit is defined.			

Table8-13. SWUnitMemoryArea definition

8.1.4.2.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	
13	incorrectMessageLengthOrInvalidFormat	M	
31	requestOutOfRange	M	
33	securityAccessDenied	M	
78	requestCorrectlyReceived-ResponsePending	M	

Table8-14. Supported Negative Response Codes

8.1.4.3 (OLD)ReadActiveArea (ECU with memory redundancy)

Session : Non-default session
Security level : Required (ASK / OEUK)

8.1.4.3.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #10h	RI_ B1 B2
#5d #6d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	C	#F1h #BXh	RCOR_SW UMA
C : ECU supports OTA and More than 1 SWUnit is defined.				

Table8-15. Request Message

Hex	Description	Cvt	Mnemonic
F1B0	ECUSoftwareUNIT's entire area	M	ECUSUNIT
F1B1	ECUSoftwareUNIT1 area	M	ECUSUNIT1
F1B2	ECUSoftwareUNIT2 area	C	ECUSUNIT2
F1B3	ECUSoftwareUNIT3 area	C	ECUSUNIT3
F1B4	ECUSoftwareUNIT4 area	C	ECUSUNIT4
F1B5	ECUSoftwareUNIT5 area	C	ECUSUNIT5
F1B6	ECUSoftwareUNIT6 area	C	ECUSUNIT6
F1B7	ECUSoftwareUNIT7 area	C	ECUSUNIT7
F1B8	ECUSoftwareUNIT8 area	C	ECUSUNIT8
F1B9	ECUSoftwareUNIT9 area	C	ECUSUNIT9
F1BA	ECUSoftwareUNIT10 area	C	ECUSUNIT10
C: More than 1 SWUnit is defined.			

Table8-16. SWUnitMemoryArea definition

8.1.4.3.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #10h	RI_ B1 B2
#5d	routineControlOptionRecord [] = [MemoryArea]	M	#XXh	RCOR_MA

Table8-17. Positive Response Message definition

Hex	Description	Cvt	Mnemonic
0A	The current boot area is A	M	
0B	The current boot area is B	M	

Table8-18. MemoryArea definition

8.1.4.3.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	
13	incorrectMessageLengthOrInvalidFormat	M	
31	requestOutOfRange	M	
33	securityAccessDenied	M	
78	requestCorrectlyReceived-ResponsePending	M	

Table8-19. Supported Negative Response Codes

8.1.4.4 (OLD)CopyActiveareaTo (ECUs with memory redundancy)

Session : Non-default session
Security level : Required (ASK / OEUK)

8.1.4.4.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #11h	RI_ B1 B2
#5d	routineControlOptionRecord [] = [MemoryArea]	M	#XXh	RCOR_MA
 #6d #7d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	 M	 #XXh #XXh	RCOR_SW UMA
The MemoryArea represents the target memory area to write after copying data. The SWUnitMemoryArea represents the memory area of the S/W unit to be designated as write.				

Table8-20. Request Message

Hex	Description	Cvt	Mnemonic
00	No specifying	M	
0A	Copy active area B to area A	M	
0B	Copy active area A to area B	M	

Table8-21. MemoryArea definition

Hex	Description	Cvt	Mnemonic
F1B0	ECUSoftwareUNIT's entire area	M	ECUSUNIT
F1B1	ECUSoftwareUNIT1 area	M	ECUSUNIT1
F1B2	ECUSoftwareUNIT2 area	C	ECUSUNIT2
F1B3	ECUSoftwareUNIT3 area	C	ECUSUNIT3
F1B4	ECUSoftwareUNIT4 area	C	ECUSUNIT4
F1B5	ECUSoftwareUNIT5 area	C	ECUSUNIT5
F1B6	ECUSoftwareUNIT6 area	C	ECUSUNIT6
F1B7	ECUSoftwareUNIT7 area	C	ECUSUNIT7
F1B8	ECUSoftwareUNIT8 area	C	ECUSUNIT8
F1B9	ECUSoftwareUNIT9 area	C	ECUSUNIT9
F1BA	ECUSoftwareUNIT10 area	C	ECUSUNIT10
C: More than 1 SWUnit is defined.			

Table8-22. SWUnitMemoryArea definition

8.1.4.4.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #11h	RI_ B1 B2
 #5d	routineControlOptionRecord [] = [MemoryArea]	 M	 #XXh	 RCOR_MA
 #6d #7d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	 M	 #XXh	 RCOR_SW UMA

Table8-23. Positive Response Message definition

Hex	Description	Cvt	Mnemonic
0A	Copy active area B to area A	M	
0B	Copy active area A to area B	M	

Table8-24. MemoryArea definition

Hex	Description	Cvt	Mnemonic
F1B0	ECUSoftwareUNIT's entire area	M	ECUSUNIT
F1B1	ECUSoftwareUNIT1 area	M	ECUSUNIT1
F1B2	ECUSoftwareUNIT2 area	C	ECUSUNIT2
F1B3	ECUSoftwareUNIT3 area	C	ECUSUNIT3
F1B4	ECUSoftwareUNIT4 area	C	ECUSUNIT4
F1B5	ECUSoftwareUNIT5 area	C	ECUSUNIT5
F1B6	ECUSoftwareUNIT6 area	C	ECUSUNIT6
F1B7	ECUSoftwareUNIT7 area	C	ECUSUNIT7
F1B8	ECUSoftwareUNIT8 area	C	ECUSUNIT8
F1B9	ECUSoftwareUNIT9 area	C	ECUSUNIT9
F1BA	ECUSoftwareUNIT10 area	C	ECUSUNIT10
C: More than 1 SWUnit is defined			

Table8-25. S/W Unit MemoryArea definition

8.1.4.4.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	
13	incorrectMessageLengthOrInvalidFormat	M	
31	requestOutOfRange	M	
33	securityAccessDenied	M	

78	requestCorrectlyReceived-ResponsePending	M	

Table8-26. Supported Negative Response Codes

8.1.4.5 (OLD)EraseTargetArea (ECU with memory redundancy)

Session : Non-default session
Security level : Required (ASK / OEUK)

8.1.4.5.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #12h	RI_ B1 B2
#5d	routineControlOptionRecord [] = [MemoryArea]	M	#XXh	RCOR_MA
 #6d #7d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	 M	 #XXh #XXh	RCOR_SW UMA
The MemoryArea represents the memory area to be designated as erase.				

Table8-27. Request Message

Hex	Description	Cvt	Mnemonic
00	No specifying	M	
0A	Erase area A	M	
0B	Erase area B	M	

Table8-28. MemoryArea definition

Hex	Description	Cvt	Mnemonic
F1B0	ECUSoftwareUNIT's entire area	M	ECUSUNIT
F1B1	ECUSoftwareUNIT1 area	M	ECUSUNIT1
F1B2	ECUSoftwareUNIT2 area	C	ECUSUNIT2
F1B3	ECUSoftwareUNIT3 area	C	ECUSUNIT3
F1B4	ECUSoftwareUNIT4 area	C	ECUSUNIT4
F1B5	ECUSoftwareUNIT5 area	C	ECUSUNIT5
F1B6	ECUSoftwareUNIT6 area	C	ECUSUNIT6
F1B7	ECUSoftwareUNIT7 area	C	ECUSUNIT7
F1B8	ECUSoftwareUNIT8 area	C	ECUSUNIT8
F1B9	ECUSoftwareUNIT9 area	C	ECUSUNIT9
F1BA	ECUSoftwareUNIT10 area	C	ECUSUNIT10
C: More than 1 SWUnit is defined			

Table8-29. SWUnitMemoryArea definition

8.1.4.5.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #12h	RI_ B1 B2
 #5d	routineControlOptionRecord [] = [MemoryArea]	 M	 #XXh	 RCOR_MA

Table8-30. Positive Response Message definition

Hex	Description	Cvt	Mnemonic
0A	Erase area A	M	
0B	Erase area B	M	

Table8-31. MemoryArea definition

Hex	Description	Cvt	Mnemonic
F1B0	ECUSoftwareUNIT's entire area	M	ECUSUNIT
F1B1	ECUSoftwareUNIT1 area	M	ECUSUNIT1
F1B2	ECUSoftwareUNIT2 area	C	ECUSUNIT2
F1B3	ECUSoftwareUNIT3 area	C	ECUSUNIT3
F1B4	ECUSoftwareUNIT4 area	C	ECUSUNIT4
F1B5	ECUSoftwareUNIT5 area	C	ECUSUNIT5
F1B6	ECUSoftwareUNIT6 area	C	ECUSUNIT6
F1B7	ECUSoftwareUNIT7 area	C	ECUSUNIT7
F1B8	ECUSoftwareUNIT8 area	C	ECUSUNIT8
F1B9	ECUSoftwareUNIT9 area	C	ECUSUNIT9
F1BA	ECUSoftwareUNIT10 area	C	ECUSUNIT10
C: More than 1 SWUnit is defined.			

Table8-32. SWUnitMemoryArea definition

8.1.4.5.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	
13	incorrectMessageLengthOrInvalidFormat	M	
31	requestOutOfRange	M	
33	securityAccessDenied	M	
78	requestCorrectlyReceived-ResponsePending	M	

Table8-33. Supported Negative Response Codes

8.1.4.6 (OLD)SwapActiveArea (ECU with memory redundancy)

This routine can cause system reset. System reset should be performed according to the ECU reset service. For example, the Ethdiag TCP socket shall be closed(FIN or RST) after a positive response is transmitted and the Server shall perform a reset thereafter. And the engine conditions of the ECU reset service must be complied with.. See also OTA requirement(ES98765-01) indetailed.

Session : Non-default session
Security level : Required (ASK / OEUK)

8.1.4.6.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_
#3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	M M	#02h #13h	RI_ B1 B2
#5d	routineControlOptionRecord [] = [MemoryArea]	U	#XXh	RCOR_MA
#6d #7d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	C	#F1h #BXh	RCOR_SWUMA
The MemoryArea represents the memory area specified as the boot(operation) area for ECU with memory redundancy. C: If MemoryArea which is placed in byte field #5d is used, this field would be available.				

Table8-34. Request Message

Hex	Description	Cvt	Mnemonic
00	No specifying	U	
0A	Specify area A as boot(operation) area	M	
0B	Specify area B as boot(operation) area	M	
'No specifying' would be used if ECU can decide swap area on their own.			

Table8-35. MemoryArea definition

Hex	Description	Cvt	Mnemonic
F1B0	ECUSoftwareUNIT's entire area	M	ECUSUNIT
F1B1	ECUSoftwareUNIT1 area	M	ECUSUNIT1
F1B2	ECUSoftwareUNIT2 area	C	ECUSUNIT2
F1B3	ECUSoftwareUNIT3 area	C	ECUSUNIT3
F1B4	ECUSoftwareUNIT4 area	C	ECUSUNIT4
F1B5	ECUSoftwareUNIT5 area	C	ECUSUNIT5
F1B6	ECUSoftwareUNIT6 area	C	ECUSUNIT6
F1B7	ECUSoftwareUNIT7 area	C	ECUSUNIT7
F1B8	ECUSoftwareUNIT8 area	C	ECUSUNIT8
F1B9	ECUSoftwareUNIT9 area	C	ECUSUNIT9

F1BA	ECUSoftwareUNIT10 area	C	ECUSUNIT10
C: More than 1 SWUnit is defined.			

Table8-36. SWUnitMemoryArea definition

8.1.4.6.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #13h	RI_ B1 B2
 #5d	routineControlOptionRecord [] = [MemoryArea]	 U	 #XXh	 RCOR_MA
#6d #7d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	C	#F1h #BXh	RCOR_SW UMA
#8d #9d	routineControlOptionRecord [] = [DelayTime byte#1(MSB) DelayTime byte#2]	U	#XXh #XXh	RCOR_DT
The MemoryArea represents the memory area specified as the boot(operation) area for ECU with memory redundancy. Unit of DelayTime is Second. C: If MemoryArea which is placed in byte field #5d is used, this field would be available.				

Table8-37. Positive Response Message definition

Hex	Description	Cvt	Mnemonic
0000	(Reserved)	M	
0001 – 012C	Time Second (1 ~ 300 sec)	M	
012D – FFFF	(Reserved)	M	

Table8-38. DelayTime definition

8.1.4.6.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	Refer to Annex A
13	incorrectMessageLengthOrInvalidFormat	M	Refer to Annex A
22	conditionsNotCorrect	M	ECU Specific Conditions are not met
24	requestSequenceError	M	CheckProgramming dependencies were not successfully performed before the Swap Active Area request
31	requestOutOfRange	M	Refer to Annex A
33	securityAccessDenied	M	Refer to Annex A
78	requestCorrectlyReceived-ResponsePending	M	Refer to Annex A

F4	LowerVersionDownloadAttempt	M	This response code indicates that the requested action will not be taken because the downloaded software is a lower version than the existing version. e.g.) ECU downgrade attemtps IMPORTANT) The memory swap ECU supporting SwapActiveArea should determine the downgrade in SwapActiveArea instead of the CheckProgrammingdependencies step.

Table8-39. Supported Negative Response Codes

8.1.4.7 (OLD)WriteDeltaPatch (ECU supporting differential data patch)

Session : Non-default session
Security level : Required (ASK / OEUK)

8.1.4.7.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #20h	RI_ B1 B2
#5d	routineControlOptionRecord [] = [MemoryArea]	M	#XXh	RCOR_MA
 #6d #7d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	M	#XXh	RCOR_SW UMA
The MemoryArea represents the memory area specified as the boot(operation) area for ECU with memory redundancy.				

Table8-40. Request Message

Hex	Description	Cvt	Mnemonic
00	Write differential data patch (ECU without memory redundancy)	M	
0A	Write differential data patch to area A (ECU with memory redundancy)	M	
0B	Write differential data patch to area B (ECU with memory redundancy)	M	

Table8-41. MemoryArea definition

Hex	Description	Cvt	Mnemonic
F1B0	ECUSoftwareUNIT's entire area	M	ECUSUNIT
F1B1	ECUSoftwareUNIT1 area	M	ECUSUNIT1
F1B2	ECUSoftwareUNIT2 area	C	ECUSUNIT2
F1B3	ECUSoftwareUNIT3 area	C	ECUSUNIT3
F1B4	ECUSoftwareUNIT4 area	C	ECUSUNIT4
F1B5	ECUSoftwareUNIT5 area	C	ECUSUNIT5
F1B6	ECUSoftwareUNIT6 area	C	ECUSUNIT6
F1B7	ECUSoftwareUNIT7 area	C	ECUSUNIT7
F1B8	ECUSoftwareUNIT8 area	C	ECUSUNIT8
F1B9	ECUSoftwareUNIT9 area	C	ECUSUNIT9
F1BA	ECUSoftwareUNIT10 area	C	ECUSUNIT10
C: More than 1 SWUnit is defined.			

Table8-42. SWUnitMemoryArea definition

8.1.4.7.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #20h	RI_ B1 B2
 #5d	routineControlOptionRecord [] = [MemoryArea]	 M	 #XXh	 RCOR_MA

Table8-43. Positive Response Message definition

Hex	Description	Cvt	Mnemonic
00	Write differential data data (ECU without memory redundancy)	M	
0A	Write differential data patch to area A (ECU with memory redundancy)	M	
0B	Write differential data patch to area B (ECU with memory redundancy)	M	

Table8-44. MemoryArea definition

Hex	Description	Cvt	Mnemonic
F1B0	ECUSoftwareUNIT's entire area	M	ECUSUNIT
F1B1	ECUSoftwareUNIT1 area	M	ECUSUNIT1
F1B2	ECUSoftwareUNIT2 area	C	ECUSUNIT2
F1B3	ECUSoftwareUNIT3 area	C	ECUSUNIT3
F1B4	ECUSoftwareUNIT4 area	C	ECUSUNIT4
F1B5	ECUSoftwareUNIT5 area	C	ECUSUNIT5
F1B6	ECUSoftwareUNIT6 area	C	ECUSUNIT6
F1B7	ECUSoftwareUNIT7 area	C	ECUSUNIT7
F1B8	ECUSoftwareUNIT8 area	C	ECUSUNIT8
F1B9	ECUSoftwareUNIT9 area	C	ECUSUNIT9
F1BA	ECUSoftwareUNIT10 area	C	ECUSUNIT10
C: More than 1 SWUnit is defined.			

Table8-45. SWUnitMemoryArea definition

8.1.4.7.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	
13	incorrectMessageLengthOrInvalidFormat	M	
31	requestOutOfRange	M	
33	securityAccessDenied	M	

78	requestCorrectlyReceived-ResponsePending	M	

Table8-46. Supported Negative Response Codes

8.1.4.8 (OLD)CheckProgrammingDependancy(Not only OTA but also reprogrammable ECU)

Session : Non-default session
Security level : Required (ASK / OEUK)

8.1.4.8.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_
#3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	M M	#FFh #01h	RI_ B1 B2
#5d	routineControlOptionRecord [] = [MemoryArea]	C	#XXh	RCOR_MA
#6d #7d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	C	#F1h #BXh	RCOR_SW UMA
The MemoryArea represents the memory area specified as the boot(operation) area for ECU with memory redundancy. C : ECU supports OTA and More than 1 SWUnit is defined.				

Table8-47. Request Message

Hex	Description	Cvt	Mnemonic
00	No specifying	U	
0A	Action request for area A	M	
0B	Action request for area B	M	
'No specifying' would be used if ECU can recognize MemoryArea on their own.			

Table8-48. MemoryArea definition

Hex	Description	Cvt	Mnemonic
F1B0	ECUSoftwareUNIT's entire area	M	ECUSUNIT
F1B1	ECUSoftwareUNIT1 area	M	ECUSUNIT1
F1B2	ECUSoftwareUNIT2 area	C	ECUSUNIT2
F1B3	ECUSoftwareUNIT3 area	C	ECUSUNIT3
F1B4	ECUSoftwareUNIT4 area	C	ECUSUNIT4
F1B5	ECUSoftwareUNIT5 area	C	ECUSUNIT5
F1B6	ECUSoftwareUNIT6 area	C	ECUSUNIT6
F1B7	ECUSoftwareUNIT7 area	C	ECUSUNIT7
F1B8	ECUSoftwareUNIT8 area	C	ECUSUNIT8
F1B9	ECUSoftwareUNIT9 area	C	ECUSUNIT9

F1BA	ECUSoftwareUNIT10 area	C	ECUSUNIT10
C: More than 1 SWUnit is defined.			

Table8-49. SWUnitMemoryArea definition

8.1.4.8.2 Positive response message

The positive response message has to be sent when the examination proves normal operation of the ECU is successful.

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #FFh #01h	RI_ B1 B2
 #5d	routineControlOptionRecord [] = [MemoryArea]	 C	 #XXh	 RCOR_MA
#6d #7d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	C	#F1h #BXh	RCOR_SWUMA

Table8-50. Positive Response Message definition

8.1.4.8.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	This response code indicates that the requested action has been rejected by the server. The generalReject response code shall only be implemented in the server if none of the negative response codes defined in this document meet the needs of the implementation.
13	incorrectMessageLengthOrInvalidFormat	M	Refer to Annex A
24	requestSequenceError	M	The requested action will not be taken because the server expects a different sequence of request messages or message as sent by the client. This may occur when sequence sensitive requests are issued in the wrong order. e.g.) expected reprogramming requests are not issued.
31	requestOutOfRange	M	Refer to Annex A
33	securityAccessDenied	M	Refer to Annex A

72	generalProgrammingFailure	M	This NRC shall be returned if the server detects an error when performing a routine, which accesses server internal memory. An example is when the routine erases or prgrams a certain memory location in the permanent memory device (e.g. Flash Memory) and the access to that memory location fails. e.g.) SW integrity fails
78	requestCorrectlyReceived-ResponsePending	M	Refer to Annex A
F4	LowerVersionDownloadAttempt	M	This response code indicates that the requested action will not be taken because the downloaded software is a lower version than the existing version. e.g.) ECU downgrade attemtps IMPORTANT) The memory swap ECU supporting SwapActiveArea should determine the downgrade in SwapActiveArea instead of the CheckProgrammingdependencies step.

Table8-51. Supported Negative Response Codes

8.1.4.9 (OLD)FinishUpdate

Note : The Identifier(0211) is identical with CopyActiveAreaTo which is legacy routine identifier. But request/response message format is different from CopyActiveAreaTo

When transmitting a FinishUpdate through functional addressing, copyActiveAreaTo with a different total length of the routine should not operate.

Session : Default(01), Extended(03)

Security level : No

8.1.4.9.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #11h	RI_ B1 B2

Table8-113. Request Message

8.1.4.9.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_

 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #11h	RI_ B1 B2

Table8-114. Positive Response Message definition

8.1.4.9.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	Refer to Annex A
13	incorrectMessageLengthOrInvalidFormat	M	Refer to Annex A
31	requestOutOfRange	M	Refer to Annex A

Table8-115. Supported Negative Response Codes

8.1.5 Implementation rules (regarding ES98765-02)

8.1.5.1 OTA Ready

Session : Default and Non-default session (Bootloader All sessions including Programming(02) – Optional)
Security level : No

8.1.5.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	 M	 #01h	LEV_ RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #03h #00h	RI_ B1 B2
#5d	routineControlOptionRecord [] = [InitParam]	C	#01h	RCOR_IP
C : The only adaptive platform that includes UCM can support this parameter. The others shall not be used. UCM ECU shall support both format. One is 31 01 03 00, the other is 31 01 03 00 01				

Table8-91. Request Message

8.1.5.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #03h #00h	RI_ B1 B2
 #5d	routineControlOptionRecord [] = [routineStatus]	 M	 #XXh	 RCOR_RS

Table8-92. Positive Response Message definition

Hex	Description	Cvt	Mnemonic
00	(Reserved)	M	
01	OK (Server can be reprogrammed immediately)	M	
02~FF	(Reserved)	M	

Table8-93. Routine status definition

8.1.5.1.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	Unable to reprogram. Negative response during pending. If operation is required for more than 3 minutes while IG is off. IG on during Pending.
13	incorrectMessageLengthOrInvalidFormat	M	Message format or Length error
31	requestOutOfRange	M	Routine identifier out of range. Routine identifier is not supported in active session
78	requestCorrectlyReceived-ResponsePending	M	Essential functions that occur only in special driving and use environments (less than 5 out of 100 drives) are in operation.

Table8-94. Supported Negative Response Codes

8.1.5.2 EnableUpdateCondition

Session : Default and Non-default session (Bootloader All sessions including Programming(02) – Optional)

Security level : No

8.1.5.2.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_
	routineIdentifier [] = [			RI_
#3d	byte #1 (MSB)	M	#03h	B1
#4d	byte #2]	M	#01h	B2

Table8-110. Request Message

8.1.5.2.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_
	routineIdentifier [] = [			RI_
#3d	byte #1 (MSB)	M	#03h	B1
#4d	byte #2]	M	#01h	B2

Table8-111. Positive Response Message definition

8.1.5.2.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	Refer to Annex. A
13	incorrectMessageLengthOrInvalidFormat	M	Message format or Length error
31	requestOutOfRange	M	Routine identifier out of range. Routine identifier is not supported in active session

Table8-112. Supported Negative Response Codes

8.1.5.3 EraseMemory

Session : Programming(02) – Mandaroty, Extended(03) – Conditional
※Conditional : Background Transfer & Memory redundancy ECU
Security level : Required (ASK / OEUK)

8.1.5.3.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #FFh #00h	RI_ B1 B2
 #5d #6d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	 M	 #XXh #XXh	 RCOR_SW UMA
The SWUnitMemoryArea represents the memory area to be erased.				

Table8-95. Request Message

Hex	Description	Cvt	Mnemonic
F1B0	ECUSoftwareUNIT's entire area	M	ECUSUNIT
F1B1	ECUSoftwareUNIT1 area	M	ECUSUNIT1
F1B2	ECUSoftwareUNIT2 area	C1	ECUSUNIT2
F1B3	ECUSoftwareUNIT3 area	C1	ECUSUNIT3
F1B4	ECUSoftwareUNIT4 area	C1	ECUSUNIT4
F1B5	ECUSoftwareUNIT5 area	C1	ECUSUNIT5
F1B6	ECUSoftwareUNIT6 area	C1	ECUSUNIT6
F1B7	ECUSoftwareUNIT7 area	C1	ECUSUNIT7
F1B8	ECUSoftwareUNIT8 area	C1	ECUSUNIT8
F1B9	ECUSoftwareUNIT9 area	C1	ECUSUNIT9
F1BA	ECUSoftwareUNIT10 area	C1	ECUSUNIT10
FFFF	initialize Adaptive AUTOSAR UCM	C2	IAASARU

C1: More than 1 SWUnit is defined. C2 :In case of Adaptive Autosar UCM

Table8-96. SWUnitMemoryArea definition

8.1.5.3.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_
#3d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #FFh #00h	RI_ B1 B2
 #5d #6d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	 M	 #XXh #XXh	 RCOR_SW UMA
#7d	routineStatus [] = [ErasedPartitionArea #1]	M	#XXh	RCOR_EPA

Table8-97. Positive Response Message definition

Hex	Description	Cvt	Mnemonic
F1B0	ECUSoftwareUNIT's entire area	M	ECUSUNIT
F1B1	ECUSoftwareUNIT1 area	M	ECUSUNIT1
F1B2	ECUSoftwareUNIT2 area	C	ECUSUNIT2
F1B3	ECUSoftwareUNIT3 area	C	ECUSUNIT3
F1B4	ECUSoftwareUNIT4 area	C	ECUSUNIT4
F1B5	ECUSoftwareUNIT5 area	C	ECUSUNIT5
F1B6	ECUSoftwareUNIT6 area	C	ECUSUNIT6
F1B7	ECUSoftwareUNIT7 area	C	ECUSUNIT7
F1B8	ECUSoftwareUNIT8 area	C	ECUSUNIT8
F1B9	ECUSoftwareUNIT9 area	C	ECUSUNIT9
F1BA	ECUSoftwareUNIT10 area	C	ECUSUNIT10
C: More than 1 SWUnit is defined.			

Table8-98. SWUnitMemoryArea definition

Hex	Description	Cvt	Mnemonic
00	Erased (if not memory redundant ECU or MMU ECU)	M	ER
0A	Erased area A	M	ERA
0B	Erased area B	M	ERB

Table8-99. ErasedPartitionArea definition

8.1.5.3.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	Refer to Annex A

13	incorrectMessageLengthOrInvalidFormat	M	Refer to Annex A
31	requestOutOfRange	M	Refer to Annex A
33	securityAccessDenied	M	Refer to Annex A

Table8-100. Supported Negative Response Codes

8.1.5.4 CheckMemory

Session : Programming(02) – Mandaroty, Extended(03) – Conditional
※Conditional : Background Transfer & Memory redundancy ECU
Security level : Required (ASK / OEUK)

8.1.5.4.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #00h	RI_ B1 B2
The SWUnitMemoryArea represents the memory area to be erased.				

Table8-101. Request Message

8.1.5.4.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #00h	RI_ B1 B2

Table8-102. Positive Response Message definition

8.1.5.4.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	Refer to Annex A
13	incorrectMessageLengthOrInvalidFormat	M	Refer to Annex A

24	requestSequenceError	M	The requested action will not be taken because the server expects a different sequence of request messages or message as sent by the client. This may occur when sequence sensitive requests are issued in the wrong order. e.g.) expected reprogramming requests are not issued.
31	requestOutOfRange	M	Refer to Annex A
33	securityAccessDenied	M	Refer to Annex A
72	generalProgrammingFailure	M	This NRC shall be returned if the server detects an error when performing a routine, which accesses server internal memory. An example is when the routine erases or prgrams a certain memory location in the permanent memory device (e.g. Flash Memory) and the access to that memory location fails. e.g.) SW integrity fails

Table8-103. Supported Negative Response Codes

8.1.5.5 CheckProgrammingDependancies
Session : Programming(02) – Mandaroty, Extended(03) – Optional
Security level : Required (ASK / OEUK)

8.1.5.5.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #FFh #01h	RI_ B1 B2
#5d	routineControlOptionRecord [] = [UpdateTarget]	C	#01h	RCOR_UT
C: The only adaptive platform that includes UCM can support this parameter. The others should not use this parameter. App update: 01h ECU that uses this parameter shall support both format. One is 31 01 FF 01, the other is 31 01 FF 01 01 can be used.				

Table8-104. Request Message

8.1.5.5.2 Positive response message

The positive response message has to be sent when the examination proves normal operation of the ECU is successful.

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #FFh #01h	RI_ B1 B2
#5d	routineStatusRecord [] = [UpdateTarget]	C	#01h	RS_UT
C: The only adaptive platform that includes UCM can support this parameter. The others should not use this parameter. App update: 01h ECU that uses this parameter shall support both format. One is 31 01 FF 01, the other is 31 01 FF 01 01 can be used.				

Table8-105. Positive Response Message definition

8.1.5.5.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	Refer to Annex A
13	incorrectMessageLengthOrInvalidFormat	M	Refer to Annex A

22	conditionNotCorrect	U	The functional safety related ECU shall transmit this negative response code when the value of the 'CCU_update_enable' signal is not equal to the 'enable'.See also ES98765-02
24	requestSequenceError	M	The requested action will not be taken because the server expects a different sequence of request messages or message as sent by the client. This may occur when sequence sensitive requests are issued in the wrong order. e.g.) expected reprogramming requests are not issued.
31	requestOutOfRange	M	Refer to Annex A
33	securityAccessDenied	M	Refer to Annex A
72	generalProgrammingFailure	M	This NRC shall be returned if the server detects an error when performing a routine, which accesses server internal memory. An example is when the routine erases or prgrams a certain memory location in the permanent memory device (e.g. Flash Memory) and the access to that memory location fails.
F4	LowerVersionDownloadAttempt	M	This response code indicates that the requested action will not be taken because the downloaded software is a lower version than the existing version. e.g.) ECU downgrade attemtps

Table8-106. Supported Negative Response Codes

8.1.5.6 FinishUpdate

Note : The Identifier(0211) is identical with CopyActiveAreaTo which is legacy routine identifier. But request/response message format is different from CopyActiveAreaTo

When transmitting a FinishUpdate through functional addressing, copyActiveAreaTo with a different total length of the routine should not operate.

Session : Default(01), Extended(03)

Security level : No

8.1.5.6.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_ RCTP_

 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #11h	RI_ B1 B2

Table8-107. Request Message

8.1.5.6.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#01h	RCTP_
 #3d #4d	routineIdentifier [] = [byte #1 (MSB) byte #2]	 M M	 #02h #11h	RI_ B1 B2

Table8-108. Positive Response Message definition

8.1.5.6.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurence
10	generalReject	M	Refer to Annex A
13	incorrectMessageLengthOrInvalidFormat	M	Refer to Annex A
31	requestOutOfRange	M	Refer to Annex A

Table8-109. Supported Negative Response Codes

9. Upload Download functional unit

9.1 RequestDownload (34 hex) service

The RequestDownload service is used by the client to initiate a data transfer from the client to the server (download). After the server has received the requestDownload request message the server shall take all necessary actions to receive data before it sends a positive response message.

9.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestDownload Request Service ID	M	#34h	RD
#2d	dataFormatIdentifier	M	#XXh	DFI_
#3d	addressAndLengthFormatIdentifier	M	#XXh	ALFID
 #4d .. #(m-1)+4d	memoryAddress[] = [byte#1 (MSB) .. byte#m]	 M .. C	 #XXh .. #XXh	MA_ B1 .. Bm
 #n-(k-1)d .. #nd	memorySize[] = [byte#1 (MSB) .. byte#k]	 M .. C	 #XXh .. #XXh	MS_ B1 .. Bk

dataFormatIdentifier : This parameter is a one byte value with each nibble encoded separately. The high nibble specifies the "compressionMethod, and the low nibble specifies the "encryptingMethod". The value 00 hex specifies that no compressionMethod nor encryptingMethod is used. Value other than 00 hex are vehicle manufacturer specific. IMPORTANT : Since secure flash is mandatory and its algorithm in use is one (AES-128 CTR), HMC/KIA doesn't use encryptingMethod of dataFormatIdentifier. ECU shall decrypt data (e.g. Secure flash 2.0 method) regardless of the encryptingMethod which is given from parameter of dataFormatIdentifier. Exception : If an ECU needs to download data without decrypting for development, 'No encrypting method'(0x0) can be used. E.g.) 0x0 : No encrypting, 0x1~F : Secure flash encrypting method.

표 9-1. Request Message definition

9.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestDownload Response Service ID	M	#74h	RDPR
#2d	lengthFormatIdentifier	M	#XXh	LFID
#3d .. #nd	maxNumberOfBlockLength [byte#1 (MSB) .. byte#m]	M .. M	#XXh .. #XXh	MNROB_ B1 .. Bm
maxNumberOfBlockLength : This parameter is used by the requestDownload positive response message to inform the client how many data bytes (maxNumberOfBlockLength) shall be included in each TransferData request message from the client. This length reflects the complete message length, including the service identifier and the data parameters present in the TransferData request message. This parameter allows the client to adapt to the receive buffer size of the server before it starts transferring data to the server.				

표 9-2. Positive Response Message definition

9.1.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This return code shall be sent if a server receives a request for this service while in the process of receiving a download of a software or calibration module. This could occur if there is a data size mismatch between the server and the client during the download of a module.
24	requestSequenceError	M	The server shall return this response code in the event that the requested memory area includes a reprogramming zone, and the 'Erase Memory' operation for the ruquested memory area has not been successfully completed prior to this request.

31	requestOutOfRange	M	This return code shall be sent if ; 1. The specified dataFormatIdentifier is not valid. 2. The specified addressAndLengthFormatIdentifeir is not valid. 3. The specified memoryAddress / memorySize is not valid.
33	securityAccessDenied	M	This return code shall be sent if the server is secure (for server's that support the SecurityAccess service) when a request for this service has been received.
70	uploadDownloadNotAccepted	M	This response code indicates that an attempt to download to a server's memory cannot be accomplished due to some fault conditions.

표 9-3. Supported Negative Response Codes

9.1.4 Implementation rules

ECU 내의 physical address 주소를 보통 address 로 사용하지만 mapping 된 주소도 사용 가능하다. 만약 메모리의 주소가 물리적인 주소와 일치하지 않는다면 진단 사양서 내에 정의된 mapping 방식을 준수하도록 한다.

dataFormIdentifier 는 2 개의 nibble 로 구성되는데, 왼쪽편은 압축 방식에 대한 정보를 포함하고, 오른쪽은 암호화 관련 정보이다. 만약 압축 및 암호화 미 적용 시에는 모두 0 으로 설정한다.

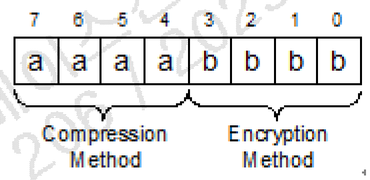


그림 9-1. Structure of dataFormatIdentifier

적어도 00 이란 dataFormIdentifier 는 지원해야 한다.

HEX	Description	Cvt	Mnemonic
0	No Compression Method	M	
1~9	Reserved (System Supplier Specific Range)		
A	LZMA	U	
B~F	Reserved (Manufacturer Specific Range)		

표 9-4. Compression Method

HEX	Description	Cvt	Mnemonic
0	No Encription Method	M	
1~9	Reserved (System Supplier Specific Range)		
A~F	Reserved (Manufacturer Specific Range)		

표 9-5. Encrption Method

addressAndLengthFormatIdentifier 은 nibble code 로 나누어 진 한 바이트의 값으로 구성된다.

- Bit 7 – 4 : memory size 파라미터의 길이
- Bit 3 – 0 : memory address 파라미터의 주소

memoryAddress 파라미터는 server 메모리의 시작 주소로 데이터가 write 되어 질 곳이다.

memorySize 파라미터는 TransferData 서비스 중 전송된 총 데이터 양과 메모리 크기를 비교하기 위하여 사용된다.

LengthFormatIdentifier 는 nibble code 로 나누어 진 한 바이트의 값으로 구성된다.

- Bit 7 – 4 : maxNumberofBlockLength 파라미터의 길이
- Bit 3 – 0 : 0 hex 값으로 채워진 예약 영역

해당 파라미터들의 포맷은 요청 메시지 안의 addressAndLengthFormatIdentifier 의 포맷과 호환되는데, lower nibble 영역이 모두 0 으로 설정된 것은 제외한다.

maxnumberOfBlockLength 파라미터는 requestDownload 의 긍정 응답에 사용되어 지며 client 의 데이터 request message 에 얼마만큼의 데이터가 들어있는지를 나타낸다. 해당 길이는 완성된 메시지의 길이로 Transfer Data request message 의 service identifier 와 data parameter 를 포함한다. 이 파라미터는 client 에서 server 의 수신 버퍼 크기를 채택할 수 있도록 한다.

9.2 RequestUpload (35 hex) service

The RequestUpload service is used by the client to initiate a data transfer from the server to the client (upload). After the server has received the requestUpload request message the server shall take all necessary actions to send data before it sends a positive response message.

9.2.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestUpload Request Service ID	M	#35h	RU
#2d	dataFormatIdentifier	M	#00h	DFI_
#3d	addressAndLengthFormatIdentifier	M	#XXh	ALFID
#4d .. #(m-1)+4d	memoryAddress[] = [byte#1 (MSB) .. byte#m]	M .. C	#XXh .. #XXh	MA_ B1 .. Bm
#n-(k-1)d .. #nd	memorySize[] = [byte#1 (MSB) .. byte#k]	M .. C	#XXh .. #XXh	MS_ B1 .. Bk

표 9-6. Request Message definition

9.2.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestUpload Response Service ID	M	#75h	RUPR
#2d	lengthFormatIdentifier	M	#XXh	LFID
#3d .. #nd	maxNumberOfBlockLength [byte#1 (MSB) .. byte#m]	M .. M	#XXh .. #XXh	MNROB_ B1 .. Bm

표 9-7. Positive Response Message definition

9.2.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This return code shall be sent if the criteria for the requestUpload are not met. This could occure if a server receives a request for this service while a requestUpload is already active, but not yet completed.

31	requestOutOfRange	M	This return code shall be sent if ; 1. The specified dataFormatIdentifier is not valid. 2. The specified addressAndLengthFormatIdentifeir is not valid. 3. The specified memoryAddress / memorySize is not valid.
33	securityAccessDenied	M	This return code shall be sent if the server is secure (for server's that support the SecurityAccess service) when a request for this service has been received.
70	uploadDownloadNotAccepted	M	This response code indicates that an attempt to upload to a server's memory cannot be accomplished due to some fault conditions.

표 9-8. Supported Negative Response Codes

9.3 TransferData (36 hex) service

The TransferData service is used by the client to transfer data either from the client to the server (download) or from the server to the client (upload).

The data transfer direction is defined by the preceding RequestDownload or RequestUpload service. If the client initiated a RequestDownload the data to be downloaded is included in the parameter(s) transferRequestParameter in the TransferData request message(s). If the client initiated a RequestUpload the data to be uploaded is included in the parameter(s) transferResponseParameter in the TransferData response message(s).

The TransferData service request includes a blockSequenceCounter to allow for an improved error handling in case a TransferData service fails during a sequence of multiple TransferData requests. The blockSequenceCounter of the server shall be initialized to one (1) when receiving a RequestDownload (34 hex) or RequestUpload (35 hex) request message. This means that the first TransferData (36 hex) request message following the RequestDownload (34 hex) or RequestUpload (35 hex) request message starts with a blockSequenceCounter of one (1).

Example use cases:

a) If a TransferData request to download data is correctly received and processed in the server but the positive response message does not reach the client then the client would determine an application layer timeout and would repeat the same request (including the same blockSequenceCounter). The server would receive the repeated TransferData request and could determine based on the included blockSequenceCounter that this TransferData request is repeated. The server would send the positive response message immediately without writing the data once again into its memory.

b) If the TransferData request to download data is not received correctly in the server then the server would not send a positive response message. The client would determine an application layer timeout and would repeat the same request (including the same blockSequenceCounter). The server would receive the repeated TransferData request and could determine based on the included blockSequenceCounter that this is a new TransferData. The server would process the service and would send the positive response message.

c) If a TransferData request to upload data is correctly received and processed in the server but the positive response message does not reach the client then the client would determine an application layer timeout and would repeat the same request (including the same blockSequenceCounter). The server would receive the repeated TransferData request and could determine based on the included blockSequenceCounter that this TransferData request is repeated. The server would send the positive response message immediately accessing the previously provided data once again in its memory.

d) If the TransferData request to upload data is not received correctly in the server then the server would not send a positive response message. The client would determine an application layer timeout and would repeat the same request (including the same blockSequenceCounter). The server would receive the repeated TransferData request and could determine based on the included blockSequenceCounter that this is a new TransferData. The server would process the service and would send the positive response message.

9.3.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	TransferData Request Service ID	M	#36h	TD
#2d	blockSequenceCounter	M	#XXh	BSC
 #3d .. #nd	transferRequestParameterRecord[] = [transferRequestParameter#1 .. transferRequestParameter#m]	 C .. U	 #XXh .. #XXh	TRPR_ TRTP_ .. TRTP_
C = Conditional : this parameter is mandatory if a download is in progress.				
blockSequenceCounter The blockSequenceCounter parameter value starts at 01 hex with the first TransferData request that follows the RequestDownload (34 hex) or RequestUpload (35 hex) service. Its value is incremented by 1 for each subsequent TransferData request. At the value of FF hex the blockSequenceCounter rolls over and starts at 00 hex with the next TransferData request message.				

표 9-9. Request Message definition

9.3.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	TransferData Response Service ID	M	#76h	TDPR
#2d	blockSequenceCounter	M	#XXh	BSC
 #3d .. #nd	transferResponseParameterRecord[] = [transferResponseParameter#1 .. transferResponseParameter#m]	 C .. U	 #XXh .. #XXh	TREPR_ TREP_ .. TREP_
C = Conditional : this parameter is mandatory if an upload is in progress.				

표 9-10. Positive Response Message definition

9.3.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	icorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong (e.g., message length does not meet requirements of maxNumberOfBlockLength parameter returned in the positive response to requestDownload).

24	requestSequenceError	M	The server shall use this response code 1. If the RequestDownload or RequestUpload service is not active when a request for this service is received. 2. If the RequestDownload or RequestUpload service is active, but the server has already received all data as determined by the memorySize parameter in the active RequestDownlod or RequestUpload service. NOTE) The repetition of a TransferData request message with a blockSequenceCounter equal to the one included in the previous TransferData request message shall be accepted by the server.
31	requestOutOfRange	M	This return code shall be sent if the transferRequestParameterRecord contains additional control parameters (e.g. additional address information) and this control information is invalid.
71	transferDataSuspended	M	This return code shall be sent if 1. The response code indicates that a data transfer operation was halted due to some fault. 2. The download module length does not meet the requirements of the memorySize parameter sent in the request message of the requestDownload service.
72	generalProgrammingFailure	M	This return code shall be sent if the server detects an error when erasing or programming a memory location in the permanent memory device (e.g. Flash Memory) during the download of data.
73	wrongBlockSequenceCounter	M	This return code shall be sent if the server detects an error in the sequence of the blockSequenceCounter. NOTE) The repetition of a TransferData request message with a blockSequenceCounter equal to the one included in the previous TransferData request message shall be accepted by the server.
92	voltageTooHigh	M	This return code shall be sent as applicable if the voltage measured at the primary power pin of the server is out of the acceptable range for downloading data into the server's permanent memory (e.g. Flash Memory).
93	voltageTooLow	M	

표 9-11. Supported Negative Response Codes

9.3.4 Implementation rules

The BlockSequenceCounter allows the diagnostic server to detect and handle transmission errors. It shall start at 1. Its value is incremented by 1 for each subsequent request of this service. When the block number 255 (0xFF) is transmitted, it shall continue with block number 0.

9.4 RequestTransferExit (37 hex) service

This service is used by the client to terminate a data transfer between client and server (upload or download).

9.4.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestTransferExit Request Service ID	M	#37h	RTE
 #2d .. #nd	transferRequestParameterRecord[] = [transferRequestParameter#1 .. trans ferRequestParameter#m]	 U .. U	 #XXh .. #XXh	TRPR_ TRTP_ .. TRTP_
transferReqeustParameterRecord - This parameter record contains parameter(s), which are required by the server to support the transfer of data. Format and length of this parameter(s) are vehicle manufacturer specific.				

표 9-12. Request Message

9.4.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestTransferExit Response Service ID	M	#77h	RTEPR
 #2d .. #nd	transferResponseParameterRecord[] = [transferResponseParameter#1 .. trans ferResponseParameter#m]	 U .. U	 #XXh .. #XXh	TREPR_ TREP_ .. TREP_
transferResponseParameterRecord - This parameter shall contain parameter(s) which are required by the client to support the transfer of data. Format and length of this parameter(s) are vehicle manufacturer specific.				

표 9-13. Positive Response Message definition

9.4.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
24	requestSequenceError	M	This return code shall be sent if ; 1. The programming process is not completed when a request for this service is received. 2. The RequestDownload or ReqeustUpload service is not active.
31	requestOutOfRange	U	This NRC shall be returned if the transferRequestParameterRecord contains invalid data.

72	GeneralProgrammingFailure	U	This NRC shall be returned if the server detects an error when finalizing the data transfer between the client and server (e.g., via an integrity check).

표 9-14. Supported Negative Response Codes

9.5 RequestFileTransfer (38 hex) service

본 서비스는 파일 데이터를 전송하기 위한 초기화 작업에 사용되며 필요한 유즈케이스가 있는 제어기에 한해 사용한다. 서비스는 기존의 Request download(34)와 Request upload(35)의 대안 솔루션으로써 제어기에 파일시스템이나 데이터 저장공간이 있는 경우 본 서비스를 활용할 수 있다. 본 서비스는 파일시스템 검색에 활용될 수 있으며 파일의 추가(download to ECU), 삭제, 교체, 읽기(upload to client), 디렉터리 읽기 등에 사용 될 수 있다.

IMPORTANT : 본 서비스는 제어기의 일부 허용된 영역만 접근 가능하도록 제한해야한다.

9.5.1 **Request message**

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestFileTransfer Request SID	M	#38h	RFT
#2d	modeOfOperation	M	#01~06h	MOOP
 #3d #4d	filePathAndNameLength[] = [byte#1 (MSB) byte#2 (LSB)]	 M	 #XXh #XXh	 FPL_B1 FPL_B2
 #5d .. #5+n-1d	filePathAndName[] = [byte#1 (MSB) .. byte#n]	 M .. C1	 #XXh .. #XXh	 FP_B1 .. FP_Bn
#5+nd	dataFormatIdentifier	C2	#00h	DFI_
#5+n+1d	fileSizeParameterLength	C2	#XXh	FSL
 #5+n+2d .. #5+n+2+k-1d	fileSizeUnCcompressed[] = [byte#1 (MSB) .. byte#k]	 C2 .. C2,3	 #XXh .. #XXh	 FSUC_B1 .. FSUC_Bk
 #5+n+2+kd .. #5+n+1+2kd	fileSizeCompressed[] = [byte#1 (MSB) .. byte#k]	 C2 .. C2,3	 #XXh .. #XXh	 FSUC_B1 .. FSUC_Bk
C1 : The length (number of bytes) of this message parameter is defined by the filePathAndNameLength parameter. C2 : The presence of these parameters depends on the modeOfOperation parameter. C3 : The length (number of bytes) of this message parameter is defined by the fileSizeParameterLength				

표 9-15. Request message definition

Parameter Name Definition
modeOfOperation This data-parameter specifies the type of operation to be applied to the file or directory indicated in the filePathAndName parameter. The values of the data-parameter shall be defined as specified in Annex G.
filePathAndNameLength Specifies the length in byte for the parameter filePath.
filePathAndName Specifies the file system location of the server where the file which shall be added, deleted, replaced or read from depending on the parameter modeOfOperation parameter. In addition this parameter includes the file name of the file which shall be added, deleted, replaced or read as part of the file path. If the modeOfOperation parameter equals 05 (ReadDir), this parameter indicates the directory to be read. Each byte of this parameter shall be encoded in ASCII format.
dataFormatIdentifier This data-parameter is a one Byte value with each nibble encoded separately. The high nibble specifies the "compressionMethod, and the low nibble specifies the "encryptingMethod". The value 00 specifies that neither compressionMethod nor encryptingMethod is used. Values other than 00 are vehicle manufacturer specific. If the modeOfOperation parameter equals to 02 (DeleteFile) and 05 (ReadDir) this parameter shall not be included in the request message.
fileSizeParameterLength Specifies the length in bytes for both parameters fileSizeUncompressed and fileSizeCompressed. If the modeOfOperation parameter equals to 02 (DeleteFile), 04 (ReadFile) or 05 (ReadDir) this parameter shall not be included in the request message.
fileSizeUncompressed Specifies the size of the uncompressed file in bytes. If the modeOfOperation parameter equals 02 (DeleteFile), 04 (ReadFile) or 05 (ReadDir) this parameter shall not be included in the request message.
fileSizeCompressed Specifies the size of the compressed file in bytes. If an uncompressed file is transferred all bytes of this parameter shall be set to the size information used in the parameter fileSizeUncompressed. If the modeOfOperation parameter equals to 02 (DeleteFile), 04 (ReadFile) or 05 (ReadDir) this parameter shall not be included in the request message.

표 9-16. Request parameter definition

Hex	Description	Cvt	Mnemonic
00	ISO/SAE reserved This value is reserved by this document for future definition.	M	ISOSAERESRVD
01	AddFile This value shall be used to add the file (download) defined in the filePathAndName parameter	U	ADDFILE
02	DeleteFile This value shall be used to delete the file defined in the filePathAndName parameter.	U	DELFILE

03	ReplaceFile This value shall be used to replace the file (download) defined in the filePathAndName parameter. If the file is not stored at the location the file shall be added.	U	REPLFILE
04	ReadFile This value shall be used to read the file (upload) at the location defined by the filePathAndName parameter.	U	RDFILE
05	ReadDir This value shall be used to read the directory defined in the filePathAndName parameter. This value implies that the request does not include a fileName.	U	RDDIR
06	ResumeFile This value shall be used to resume downloading the file defined in the filePathAndName parameter at the returned filePosition indicator. The file specified in the filePathAndName must already exist in the ECU's file system.	U	RSFILE
07~FF	ISO/SAE reserved This value is reserved by this document for future definition.	M	ISOSAERESRVD

표 9-17. modeOfOperation definition

9.5.2 **Positive response message**

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestFileTransfer Response SID	M	#78h	RRFT
#2d	modeOfOperation	M	#01~06h	MOOP
#3d	lengthFormatIdentifier	C1	#XXh	LFID
 #4d .. #4+(m-1)d	maxNumberOfBlockLength[] = [byte#1 (MSB) .. byte#m]	 C1,2 .. C1,2	 #XXh .. #XXh	MNROB B1 .. Bm
#4+md	dataFormatIdentifier	C1	#00h	DFI_
 #4+m+1d #4+m+2d	fileSizeOrDirInfoParameterLength [byte#1 (MSB) byte#2 (LSB)]	 C1 C1	#XXh	 FSDIL_B1 FSDIL_B2
 #4+m+3d .. #4+m+3+k-1d	fileSizeUncompressedOrDirInfoLength [] = [byte#1 (MSB) .. byte#k]	 C1,3 .. C1,3	 #XXh .. #XXh	 FSUDIL_B1 .. FSUDIL_Bk

#4+m+3+kd .. #4+m+3+2k-1d	fileSizeCompressed[] = [byte#1 (MSB) .. byte#k]	C1,3 .. C1,3	#XXh .. #XXh	FSC_B1 .. FSC_Bk
#4+m+4+2k .. #4+m+11+2k	filePosition [] = [byte#1 (MSB) .. byte#8 (LSB)]	C1 .. C1	#XXh .. #XXh	FPOS_B1 .. FPOS_B8
C1: The presence of these parameters depends on the modeOfOperation parameter. C2: The length (number of bytes) of this message parameter is defined by the lengthFormatIdentifier parameter. C3: The length (number of bytes) of this message parameter is defined by the fileSizeOrDirInfoParameterLength parameter.				

표 9-18. Positive response message definition

Parameter Name Definition
modeOfOperation This is parameter echoes the value of the request
lengthFormatIdentifier Specifies the length (number of bytes) of the maxNumberOfBlockLength parameter. If the modeOfOperation parameter equals to 02 (DeleteFile) this parameter shall not be included in the response message.
maxNumberOfBlockLength This parameter is used by the requestFileTransfer positive response message to inform the client how many data bytes (maxNumberOfBlockLength) to include in each TransferData request message from the client or how many data bytes the server will include in a TransferData positive response when uploading data. This length reflects the complete message length, including the service identifier and the data parameters present in the TransferData request message or positive response message. This parameter allows either the client to adapt to the receive buffer size of the server before it starts transferring data to the server or to indicate how many data bytes will be included in each TransferData positive response in the event that data is uploaded. A server is required to accept transferData requests that are equal in length to its reported maxNumberOfBlockLength. It is server specific what transferData request lengths less than maxNumberOfBlockLength are accepted (if any). NOTE The last transferData request within a given block can be required to be less than maxNumberOfBlockLength. It is not allowed for a server to write additional data bytes (i.e. pad bytes) not contained within the transferData message (either in a compressed or uncompressed format), as this would affect the memory address of where the subsequent transferData request data would be written. If the modeOfOperation parameter equals to 02 (DeleteFile) this parameter shall be not be included in the response message.

dataFormatIdentifier This is parameter echoes the value of the request. If the modeOfOperation parameter equals to 02 (DeleteFile) this parameter shall not be included in the response message.) If the modeOfOperation parameter equals to 05 (ReadDir) the value of this parameter shall be equal to 00.
fileSizeOrDirInfoParameterLength Specifies the length in bytes for both parameters fileSizeUncompressedOrDirInfoLength and fileSizeCompressed. If the modeOfOperation parameter equals to 01 (AddFile), 02 (DeleteFile), 03 (ReplaceFile) or 06 (ResumeFile) this parameter shall not be included in the response message.
fileSizeUncompressedOrDirInfoLength Specifies the size of the uncompressed file to be uploaded or the length of the directory information to be read in bytes. If the modeOfOperation parameter equals to 01 (AddFile), 02 (DeleteFile), 03 (ReplaceFile) or 06 (ResumeFile), this parameter shall not be included in the response message.
fileSizeCompressed Specifies the size of the compressed file in bytes. If the modeOfOperation parameter equals to 01 (AddFile), 02 (DeleteFile, 03 (ReplaceFile)), 05 (ReadDir) or 06 (ResumeFile) this parameter shall not be included in the response message.
filePosition Specifies the byte position within the file at which the Tester will resume downloading after an initial download is suspended. A download is suspended when the ECU stops receiving TransferData requests and does not receive the RequestTransferExit request before returning to the defaultSession. The filePosition is relative to the compressed size or uncompressed size, depending if the file was originally sent compressed or uncompressed during the initiating ModeOfOperation = AddFile or ReplaceFile. If the modeOfOperation parameter equals to 01 (AddFile), 02 (DeleteFile), 03 (ReplaceFile), 04 (ReadFile), or 05 (ReadDir) this parameter shall not be included in the request.

표 9-19. Response parameter definition

9.5.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionsNotCorrect	M	This NRC shall be returned if a server receives a request for this service while in the process of downloading or uploading data or other conditions to be able to execute this service are not met.
24	requestSequenceError	M	This NRC shall be returned when modeOfOperation is 0616 (ResumeFile) and the requested file has already been completely transferred and therefore is not in a state to be resumed.

31	requestOutOfRange	M	This NRC shall be returned if: – The specified dataFormatIdentifier is not valid. – The specified modeOfOperation is not valid. – The specified fileSizeParameterLength is not valid. – The specified filePathAndNameLength is not valid. – The specified fileSizeUncompressed is not valid. – The specified fileSizeCompressed is not valid. –The specified filePathAndName is not valid.
33	securityAccessDenied	M	This NRC shall be returned if the server is secure (for server's that support the SecurityAccess service) when a request for this service has been received.
70	uploadDownloadNotAccepted	M	This NRC indicates that an attempt to download to a server's memory cannot be accomplished due to some fault conditions.

표 9-20. Supported Negative Response Codes

10. Non-volatile server memory programming process

10.1 Requirements for Flash Reprogramming

NOTE : This specification describes Legacy reprogramming.
Refer to the ES98765-02 specification for reprogramming standards.

10.1.1 General Requirement

Identification information1), which would be compared with tool's one during the programming process or have to be always available, should be located in a non-erasable area. It will enable reprogramming process even if the server has incomplete set of application software and application data caused by failure of Flash Reprogramming (erasing identification information).

10.1.2 Boot Area

1) All programmable servers that support programming of the application software shall contain boot software in a boot memory region.

2) The boot memory shall be protected against inadvertent erasure.

3) The boot software can be protected via hardware or software.

4) Servers that support boot software shall continue to execute out of the boot until a complete set of application software and/or application data is programmed.

5) The server shall be able to recover and be reprogrammed if any of the following error conditions occur during the programming process:
① Loss of supplied power connection
② Loss of the ground connection
③ Disruption of CAN communication
④ Over- or under-voltage conditions

6) No programmable server operating out of boot memory shall transmit a non-diagnostic communication message or unsolicited diagnostic message.

7) All servers operation out of boot memory shall be able to receive diagnostic messages.

8) A server shall be capable of using the same diagnostic CAN IDs for the duration of a programming event.

9) The Boot software shall support 'Extended Diagnostic Mode' with the 'disable normal message transmission', 'ECU reset', 'Tester Present' and 'Start Diagnostic Session' services to enable to restart reprogramming procedure when a sever exits the programming process unsuccessfully.

10.1.3 Server reprogramming requirements

10.1.3.1 Requirements for all servers to support programming

The DiagnosticSessionControl (10 hex) service, with sessionType equal to extendedDiagnosticSession followed by a CommunicationControl (28 hex) service that disables non-diagnostic messages and a ControlDTCSetting (85 hex) service that disables setting of DTCs, shall be used by the server to recognize the disabling of normal communication and to ensure that a server does not set DTCs while another server is being programmed. The extendedDiagnosticSession shall be kept active by the client for the duration of the programming event.

10.1.3.2 Hardware requirements for programmable servers to support programming

All servers that are programmable shall be able to interface with the programming tools used by development, the assembly plant and by service via the appropriate pins of the vehicle diagnostic connector.

The only power required at the vehicle diagnostic connector for programming shall be vehicle battery power.

Any server that is properly installed in the vehicle and is programmable shall be able to be programmed via the vehicle diagnostic connector. Removal of the server from the vehicle in order to perform programming shall not be required.

10.2 Non-volatile server memory programming sequence

10.2.1 Programming Phase #1

Service Name	SID (hex)	Sub-function Name	Sub-function ID (hex)	Identifiers	Request Type	Remark
ReadDataByIdentifier	22	VehicleManufacturerECUSoftwareVersionNumberDataIdentifier	F189	-	Physical	

표 10-1. Programming Phase #1 Sequence

10.2.1.1 ReadDataByIdentifier – ECUSoftwareVersion

The tester will send request message to compare the software of ECU in the vehicle with the software in the tester. If the software in the vehicle is later version than the software in the tester or the same version, the programming sequence will be suspended by the tester.

10.2.2 Programming Phase #2

※ 리프로그래밍 시, 아래 서비스 절차대로 수행해야 한다.

Service Name	SID (hex)	Sub-function Name	Sub-function ID (hex)	Identifiers	Request Type	Remark
DiagnosticSessionControl	10	Default Session	81	-	Functional	Initialization Step
DiagnosticSessionControl	10	Extended Session	83	-	Functional	Pre-Programming Step ※ ICU 에 C-SAC 기능 적용 시, ES95489-01 sequence 준수
ControlDTCSetting	85	OFF	82	-	Functional	
CommunicationControl	28	disableRxAndTx	83	Communication Type = 03 hex	Functional	
DiagnosticSessionControl	10	Programmjng Session	02	-	Physical	-
SecurityAccess	27	Request Seed	01	-	Physical	-
SecurityAccess	27	Send Key	02	-	Physical	-
RequestDownload	34	Data Format Identifier		-	Physical	Download the Boot Loader Software (Optional)
TransferData	36	Block Sequence Counter		-	Physical	
RequestTransferExit	37	-	-	-	Physical	
RoutineControl	31	Start Routine	01	routineIdentifier = checkProgramming Dependencies (#FF01)	Physical	
RoutineControl	31	Start Routine	01	routineIdentifier = eraseMemory	Physical	Download the Application Software
RequestDownload	34	-	-	Data Format Identifier	Physical	
TransferData	36	-	-	Block Sequence Counter	Physical	
RequestTransferExit	37	-	-	-	Physical	
RoutineControl	31	Start Routine	01	routineIdentifier = checkProgramming Dependencies (#FF01)	Physical	
ECUReset	11	Hard Reset	01	-	Physical	-
CommunicationControl	28	enableRxAndTx	80	Communication Type = 03 hex	Functional	within 15ms afte ECUReset Pos. Rsp.

표 10-2. Programming Phase #2 Sequence

10.2.2.1 DiagnosticSessionControl – extended diagnostic session

In order to be able to disable the normal communication between the servers and the setting of DTCs, it is required to start a non-defaultSession in each server where normal communication and DTCs shall be disabled. This is achieved via a DiagnosticSessionControl (10 hex) service with sessionType equal to extendedDiagnosticSession. The request shall be transmitted functionally addressed to all servers. Response messages are not required in this step.

10.2.2.2 ControlDTCSetting – DTC setting type = off

The client disables the setting of DTCs in each server using the ControlDTCSetting (85 hex) service with DTCSettingType equal to "off". The request shall be transmitted functionally addressed to all servers. Response messages are not required in this step.

10.2.2.3 CommunicationControl – disable non-diagnostic communication

The client disables the transmission and reception of non-diagnostic messages using the CommunicationControl (28 hex) service. The request shall be transmitted functionally addressed to all servers. Response messages are not required in this step.

10.2.2.4 DiagnosticSessionControl – programming session

The programming event is started in the server(s) via a physically addressed request of the DiagnosticSessionControl (10 hex) service with sessionType equal to programmingSession. When the server receives the request, it shall allocate all necessary resources required for programming. It is implementation specific whether the server starts executing out of boot memory.

10.2.2.5 SecurityAccess – read seed / send key

A programming event should be secured. The SecurityAccess (27 hex) service shall be mandatory for all systems.

10.2.2.6 RoutineControl – erase memory

The memory of the server shall be erased in order to allow an application software/data download. This is achieved via a routine, using the RoutineControl (31 hex) service to execute the erase routine.

10.2.2.7 Request Download / TransferData / RequestTransferExit

Each download of a contiguous block of application software/data to a non-volatile server memory location (either a complete application software/data module or part of a software/data module) shall always follow the general data transfer method using the following service sequence:

- RequestDownload (34 hex)
- TransferData (36 hex)
- RequestTransferExit (37 hex)

10.2.2.8 RoutineControl – check reprogramming dependencies

Once all application software/data blocks/modules are completely downloaded, the client shall verify if the download has been performed successfully by initiating a routine in the server using the RoutineControl (31 hex) service. This routine either triggers the server to check the reprogramming dependencies and to perform all necessary action to proof that the download and programming into non-volatile memory was successful, or it requests the server to calculate the checksum and submit the checksum to the client via a RoutineControl positive response message (requestRoutineResults) for a comparison with a checksum contained in the client.

10.2.2.9 ECUReset – hard reset

The client transmits an ECUReset (11 hex) service request message onto the CAN network with resetType equal to hardReset. This shall be done physically addressed. The reception of the ECUReset (11 hex) request message causes the server(s) to perform a reset and to start the defaultSession.

10.2.2.10 CommunicationControl – enable non-diagnostic communication

The client enables the transmission and reception of non-diagnostic messages using the CommunicationControl (28 hex) service. This service must be requested within 15ms after ECU Reset positive response to prevent Time-Out Errors. When the Gateway is reprogramming by itself, the client transmits the CommunicationControl service in 400ms ~ 500ms after ECU Reset positive response. The request shall be transmitted functionally addressed to all servers. Response messages are not required in this step.

10.2.3 Programming Phase #3

Service Name	SID (hex)	Sub-function Name	Sub-function ID (hex)	Identifiers	Request Type	Remark
ReadDataByIdentifier	22	VehicleManufacturerECU SoftwareVersionNumber DataIdentifier	F189	-	Physical	

표 10-3. Programming Phase #3 Sequence

10.2.3.1 ReadDataByIdentifier – ECUSoftwareVersion

After the completion of reprogramming, the tester will send request message ReadDataByIdentifier to the ECU. If the ECU send a positive response message, it shows that the reprogramming procedure is successfully completed and the ECU has complete set of updated application software and application data.

10.3 Non-volatile server memory programming message flow examples

10.3.1 General Information

The following presents the message for a non-volatile server memory-programming event of a single server.

Assume that,
- Two ECUs (07E0/07E8 & 07E1/07E9) and Tester are in CAN or CAN-FD network
- Transfer of four (4) modules, where each module has a length of 511 bytes (data bytes 02 hex through FE hex)
- The network layer buffer size of the server that is re-programmed is 255 bytes (reported in the RequestDownload positive response message)

10.3.2 Programming Phase #1

Assume that,
- BS = 4, STmin = 2 (in Default Session)
- Vehicle Manufacturer ECU Software Version Number = 123456789 (ASCII)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	03	22	F1	89	55	55	55	55	Single Frame
07E8	Response	8	10	0D	62	F1	89	31	32	33	First Frame
07E0	Request	8	30	04	02	55	55	55	55	55	Flow Control
07E8	Response	8	21	34	35	36	37	38	39	AA	Consecutive Frame #1

표 10-4. ReadDataByIdentifier (VehicleManufacturerECUSoftwareVersionNumber)

10.3.3 Programming Phase #2

Assume that,
- BS = 0, STmin = 0 (in ProgrammingSession)
- seed of the server (2 bytes) : 3657 hex
- key of the server (2 bytes) : C9A9 hex

CAN ID	Request / Response	DLC	Data Bytes								Comments
07DF	Func. Request	8	02	10	83	55	55	55	55	55	Single Frame

표 10-5. DiagnosticSessionControl (extendedSession)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07DF	Func. Request	8	02	85	82	55	55	55	55	55	Single Frame

표 10-6. ControlDTCSetting (off)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07DF	Func. Request	8	03	28	83	03	55	55	55	55	Single Frame

표 10-7. CommunicationControl (disableRxAndTx)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	02	10	02	55	55	55	55	55	Single Frame
07E8	Response	8	02	50	02	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-8. DiagnosticSessionControl (programmingSession)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	02	27	01	55	55	55	55	55	Single Frame
07E8	Response	8	02	67	01	36	57	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-9. SecurityAccess (requestSeed)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	02	27	02	C9	A9	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	02	67	02	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-10. SecurityAccess (sendKey)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	04	31	01	FF	00	55	55	55	Single Frame
07E8	Response	8	03	7F	31	78	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	03	7F	31	78	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	04	71	01	FF	00	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-11. RoutineControl (eraseMemory)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	09	34	00	33	00	1B	67	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E0	Phys. Request	8	21	00	01	FF	55	55	55	55	Consecutive Frame #1
07E8	Response	8	04	74	20	00	FF	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-12. RequestDownload – Module #1

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	01	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control

07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
...	...	...	...	...	...	...	...	...	...	...	...
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	01	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-13. TransferData – Module #1 (block #1)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	02	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
...	...	...	...	...	...	...	...	...	...	...	...
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	02	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-14. TransferData – Module #1 (block #2)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	07	36	03	02	03	04	05	06	Single Frame
07E8	Response	8	02	76	03	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-15. TransferData – Module #1 (block #3)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	01	37	55	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	01	77	AA	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-16. RequestTransferExit – Module #1

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	09	34	00	33	00	1B	67	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E0	Phys. Request	8	21	00	01	FF	55	55	55	55	Consecutive Frame #1
07E8	Response	8	04	74	20	00	FF	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-17. RequestDownload – Module #2

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	01	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
…	…	…	…	…	…	…	…	…	…	…	…
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	01	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-18. TransferData – Module #2 (block #1)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	02	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
…	…	…	…	…	…	…	…	…	…	…	…
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	02	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-19. TransferData – Module #2 (block #2)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	07	36	01	02	03	04	05	06	Single Frame
07E8	Response	8	02	76	03	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-20. TransferData – Module #2 (block #3)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	01	37	55	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	01	77	AA	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-21. RequestTransferExit – Module #2

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	09	34	00	33	00	1B	67	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control

07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E0	Phys. Request	8	21	00	01	FF	55	55	55	55	Consecutive Frame #1
07E8	Response	8	04	74	20	00	FF	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-22. RequestDownload – Module #3

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	01	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
...	...	...	...	...	...	...	...	...	...	...	...
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	01	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-23. TransferData – Module #3 (block #1)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	02	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
...	...	...	...	...	...	...	...	...	...	...	...
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	02	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-24. TransferData – Module #3 (block #2)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	07	36	01	02	03	04	05	06	Single Frame
07E8	Response	8	02	76	03	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-25. TransferData – Module #3 (block #3)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	01	37	55	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	01	77	AA	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-26. RequestTransferExit – Module #3

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	09	34	00	33	00	1B	67	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E0	Phys. Request	8	21	00	01	FF	55	55	55	55	Consecutive Frame #1
07E8	Response	8	04	74	20	00	FF	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-27. RequestDownload – Module #4

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	01	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
…	…	…	…	…	…	…	…	…	…	…	…
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	01	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-28. TransferData – Module #4 (block #1)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	02	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
…	…	…	…	…	…	…	…	…	…	…	…
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	02	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-29. TransferData – Module #4 (block #2)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	07	36	01	02	03	04	05	06	Single Frame
07E8	Response	8	02	76	03	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-30. TransferData – Module #4 (block #3)

CAN ID	Request / Response	DLC	Data Bytes								Comments

07E0	Phys. Request	8	01	37	55	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	01	77	AA	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-31. RequestTransferExit – Module #4

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	04	31	01	FF	01	55	55	55	Single Frame
07E8	Response	8	03	7F	31	78	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	03	7F	31	78	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	04	71	01	FF	01	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

표 10-32. RoutineControl (checkProgrammingDependencies)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07DF	Phys. Request	8	02	11	01	55	55	55	55	55	Single Frame
07E8	Response	8	02	51	01	AA	AA	AA	AA	AA	Single Frame

표 10-33. ECUReset (hardReset)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07DF	Func. Request	8	03	28	80	03	55	55	55	55	Single Frame

표 10-34. CommnunicationControl (enableRxAndTx)

10.3.4 Programming Phase #3

Assume that,

- BS = 4, STmin = 2 (in Default Session)
- Vehicle Manufacturer ECU Software Version Number = 12345678A (ASCII)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	03	22	F1	89	55	55	55	55	Single Frame
07E8	Response	8	10	0D	62	F1	89	31	32	33	First Frame
07E0	Request	8	30	04	02	55	55	55	55	55	Flow Control
07E8	Response	8	21	34	35	36	37	38	41	AA	Consecutive Frame #1

표 10-35. ReadDataByIdentifier (VehicleManufacturerECUSoftwareVersionNumber)

10.4 Servers status in programming procedure

ECU #1 is the server actually being programmed.

ECU #2 is the server which is not actually programmed (programmable or non-programmable).
ECU #3 is the server which is participating in a CAN or CAN-FD network but the server itself is diagnosed over another vehicle bus (e.g. KWP2000 …) or does not support diagnostic.

Tester			ECU #1	ECU #2	ECU #3
Service Name	SID (hex)		DiagnosticSession / CommunicationControl setting / DTCSetting		
ReadDataByIdentifier	22	Phys.	defaultSession / default (enableRxAndTx) / default (ON)		– / default (enableRxAndTx) / –
DiagnosticSessionControl – extendedSession	10	Func.	*extendedSession* / default (enableRxAndTx) / default (ON)		
ControlDTCSetting	85	Func.	extendedSession / default (enableRxAndTx) / *OFF*		
CommunicationControl – disableRxAndTx	28	Func.	extendedSession / *disableRxAndTx* / OFF		– / EnableRxAndDisableTx / –
DiagnosticSessionControl – programmingSession	10	Phys.	*programmingSession* / disableRxAndTx / OFF		
SecurityAccess	27	Phys.			
SecurityAccess	27	Phys.			
RequestDownload	34	Phys.			
TransferData	36	Phys.			
RequestTransferExit	37	Phys.			
RoutineControl	31	Phys.			
RoutineControl	31	Phys.			
RequestDownload	34	Phys.			
TransferData	36	Phys.			
RequestTransferExit	37	Phys.			
RoutineControl	31	Phys.			
ECUReset – hardReset	11	Phys.	*defaultSession* / *default (enableRxAndTx)* / *default (ON)*		
CommunicationControl – enableRxAndTx	28	Func.		extendedSession / *enableRxAndTx* / OFF	– / enableRxAndTx / –
ReadDataByIdentifier	22	Phys.			

Annex. A Negative Response Codes

When an ECU detects any error condition on communication or have internal problem, it cannot respond or react to the request normally. In this case the ECU should respond to tester with negative response code described in table A-1.

*) These codes are general negative response codes. These NRCs are in general supported by each diagnostic service and not listed in the list of applicable response codes of the diagnostic services. The occurrence depends on the system configuration (Hex value – 10, 11, 33, 78, 7F, 21, 7E (with a sub-function))

Hex value	responseCode	Mnemonic
10	generalReject *) This response code indicates that the requested action has been rejected by the server. The generalReject response code shall only be implemented in the server if none of the negative response codes defined in this document meet the needs of the implementation.	GR
11	serviceNotSupported *) This response code indicates that the requested action will not be taken because the server does not support the requested service. The server shall send this response code in case the client has sent a request message with a service identifier, which is either unknown or not supported by the server. Therefore this negative response code is not shown in the list of negative response codes to be supported for a diagnostic service, because this negative response code is not applicable for supported services.	SNS
12	subFunctionNotSupported This response code indicates that the requested action will not be taken because the server does not support the service specific parameters of the request message. The server shall send this response code in case the client has sent a request message with a known and supported service identifier but with "sub function“ which is either unknown or not supported.	SFNS
13	incorrectMessageLengthOrInvalidFormat This response code indicates that the requested action will not be taken because the length of the received request message does not match the prescribed length for the specified service or the format of the paramters do not match the prescribed format for the specified service.	IMLOIF
14	responseTooLong This response code shall be reported by the server if the response to be generated exceeds the maximum number of bytes available by the underlying network layer. – EXAMPLE This problem may occur when several DIDs at a time are requested and the combination of all DIDs in the response exceeds the limit of the underlying transport protocol.	RTL

21	busyRepeatRequest This response code indicates that the server is temporarily too busy to perform the requested operation. In this circumstance the client shall perform repetition of the "identical request message" or "another request message". The repetition of the request shall be delayed by a time specified in the respective implementation documents. Example: In a multi-client environment the diagnostic request of one client might be blocked temporarily by a NRC $21 while a different client finishes a diagnostic task. NOTE) If the server is able to perform the diagnostic task but needs additional time to finish the task and prepare the response, the NRC $78 shall be used instead of NRC $21. This response code is in general supported by each diagnostic service, as not otherwise stated in the data link specific implementation document, therefore it is not listed in the list of applicable response codes of the diagnostic services.	BRR
22	conditionNotCorrect This response code indicates that the requested action will not be taken because the server prerequisite conditions are not met.	CNC
24	requestSequenceError This response code indicates that the requested action will not be taken because the server expects a different sequence of request messages or message as sent by the client. This may occur when sequence sensitive requests are issued in the wrong order. EXAMPLE A successful SecurityAccess service specifies a sequence of requestSeed and sendKey as susb-fuctions in the request messages. If the sequence is sent different by the client the server shall send a negative response message with the negative response code 24 hex – . requestSequenceError.	RSE
31	requestOutOfRange This response code indicates that the requested action will not be taken because the server has detected that the request message contains a parameter which attempts to substitute a value beyond its range of authority (e.g. attempting to substitute a data byte of 111 when the data is only defined to 100), or which attempts to access a dataIdentifier/routineIdentifer that is not supported or not supported in active session. This response code shall be implemented for all services, which allow the client to read data, write data or adjust functions by data in the server.	ROOR
33	securityAccessDenied *) This response code indicates that the requested action will not be taken because the server's security strategy has not been satisfied by the client. The server shall send this response code if one of the following cases occur:	SAD

	– the test conditions of the server are not met, – the required message sequence e.g. DiagnosticSessionControl, securityAccess is not met, – the client has sent a request message which requires an unlocked server. Beside the mandatory use of this negative response code as specified in the applicable services within this standard, this negative response code can also be used for any case where security is required and is not yet granted to perform the required service.	
35	invalidKey This response code indicates that the server has not given security access because the key sent by the client did not match with the key in the server's memory. This counts as an attempt to gain security. The server shall remain locked and increment ist internal securityAccessFailed counter.	IK
36	exceedNumberOfAttempts This response code indicates that the requested action will not be taken because the client has unsuccessfully attempted to gain security access more times than the server's security strategy will allow.	ENOA
37	requiredTimeDelayNotExpired This response code indicates that the requested action will not be taken because the client's latest attempt to gain security access was initiated before the server's required timeout period had elapsed.	RTDNE
50	Certificate verification failed – Invalid Time Period Date and time of the server does not match the validity period of the Certificate.	CVFITP
51	Certificate verification failed – Invalid Signature Signature of the Certificate could not be verified.	CVFIS
54	Certificate verification failed – Invalid Format Certificate could not be evaluated because the format requirement has not been met.	CVFIF
56	Certificate verification failed – Invalid Scope The scope of the Certificate does not match the contents of the server.	CVFIS
57	Certificate verification failed – Invalid Certificate (revoked) Certificate received from client is invalid, because the server has revoked access for some reason.	CVFIC
58	Ownership verification failed Delivered Ownership does not match the provided challenge or could not verified with the own private key.	OVF

59	Challenge calculation failed The challenge could not be calculated on the server side.	CCF
5A	Setting Access Rights failed The server could not set the access rights.	SARF
5C	Configuration data usage failed The server could not work with the provided configuration data.	CDUF
5D	DeAuthentication failed DeAuthentication was not successful, server could still be unprotected.	DAF
70	uploadDownloadNotAccepted This response code indicates that an attempt to upload/download to a server's memory cannot be accomplished due to some fault conditions.	UDNA
71	transferDataSuspended This response code indicates that a data transfer operation was halted due to some fault. The active transferData sequence shall be aborted.	TDS
72	generalProgrammingFailure This response code indicates that the server detected an error when erasing or programming a memory location in the permanent memory device (e.g. Flash Memory).	GPF
73	wrongBlockSequenceCounter This response code indicates that the server detected an error in the sequence of blockSequenceCounter values. Note that the repetition of a TransferData request message with a blockSequenceCounter equal to the one included in the previous TransferData request message shall be accepted by the server.	WBSC
78	requestCorrectlyReceived-ResponsePending *) This response code indicates that the request message was received correctly, and that all parameters in the request message were valid, but the action to be performed is not yet completed and the server is not yet ready to receive another request. As soon as the requested service has been completed, the server shall send a positive response message or negative response message with a response code different from this. The negative response message with this response code may be repeated by the server until the requested service is completed and the final response message is sent. This response code might impact the application layer timing parameter values. The detailed specification shall be included in the data link specific implementation document. This response code shall only be used in a negative response message if the server will not be able to receive further request messages from the client while completing the requested diagnostic service. When this response code is used, the server shall always send a final response (positive or negative)	RCRRP

	independent of the suppressPosRspMsgIndicationBit value. A typical example where this response code may be used is when the client has sent a request message, which includes data to be programmed or erased in flash memory of the server. If the programming/erasing routine (usually executed out of RAM) is not able to support serial communication while writing to the flash memory the server shall send a negative response message with this response code. This response code is in general supported by each diagnostic service, as not otherwise stated in the data link specific implementation document, therefore it is not listed in the list of applicable response codes of the diagnostic services.	
7E	subFunctionNotSupportedInActiveSession This response code indicates that the requested action will not be taken because the server does not support the requested sub-function in the session currently active. Within the programmingSession negative response code SFNS (subFunctionNotSupported) may optionally be reported instead of negative response code SNFSIAS (subFunctionNotSupportedInActiveSession). This response code shall only be used when the requested sub-function is known to be supported in another session, otherwise response code SFNS (subFunctionNotSupported) shall be used. This response code shall be supported by each diagnostic service with a sub-function parameter, if not otherwise stated in the data link specific implementation document, therefore it is not listed in the list of applicable response codes of the diagnostic services.	SFNSIAS
7F	serviceNotSupportedInActiveSession *) This response code indicates that the requested action will not be taken because the server does not support the requested service in the session currently active. This response code shall only be used when the requested service is known to be supported in another session, otherwise response code SNS (serviceNotSupported) shall be used. This response code is in general supported by each diagnostic service, as not otherwise stated in the data link specific implementation document, therefore it is not listed in the list of applicable response codes of the diagnostic services.	SNSIAS
81	rpmTooHigh This response code indicates that the requested action will not be taken because the server prerequisite condition for RPM is not met (current RPM is above a pre-programmed maximum threshold).	RPMTH
82	rpmTooLow This response code indicates that the requested action will not be taken because the server prerequisite condition for RPM is not met (current RPM is below a pre-programmed minimum threshold).	RPMTL

83	engineIsRunning This is required for those actuator tests which cannot be actuated while the Engine is running. This is different from RPM too high negative response, and needs to be allowed.	EIR
84	engineIsNotRunning This is required for those actuator tests which cannot be actuated unless the Engine is running. This is different from RPM too low negative response, and needs to be allowed.	EINR
85	engineRunTimeTooLow This response code indicates that the requested action will not be taken because the server prerequisite condition for engine run time is not met (current engine run time is below a preprogrammed limit).	ERTTL
86	temperatureTooHigh This response code indicates that the requested action will not be taken because the server prerequisite condition for temperature is not met (current temperature is above a preprogrammed maximum threshold).	TEMPTH
87	temperatureTooLow This response code indicates that the requested action will not be taken because the server prerequisite condition for temperature is not met (current temperature is below a preprogrammed minimum threshold).	TEMPTL
88	vehicleSpeedTooHigh This response code indicates that the requested action will not be taken because the server prerequisite condition for vehicle speed is not met (current VS is above a pre-programmed maximum threshold).	VSTH
89	vehicleSpeedTooLow This response code indicates that the requested action will not be taken because the server prerequisite condition for vehicle speed is not met (current VS is below a pre-programmed minimum threshold).	VSTL
8A	throttle/PedalTooHigh This response code indicates that the requested action will not be taken because the server prerequisite condition for throttle/pedal position is not met (current TP/APP is above a preprogrammed maximum threshold).	TPTH
8B	throttle/PedalTooLow This response code indicates that the requested action will not be taken because the server prerequisite condition for throttle/pedal position is not met (current TP/APP is below a preprogrammed minimum threshold).	TPTL
8C	transmissionRangeNotInNeutral This response code indicates that the requested action will not be taken because the server prerequisite condition for being in neutral is not met (current transmission range is not in neutral).	TRNIN
8D	transmissionRangeNotInGear	TRNIG

	This response code indicates that the requested action will not be taken because the server prerequisite condition for being in gear is not met (current transmission range is not in gear).	
8F	brakeSwitch(es)NotClosed (Brake Pedal not pressed or not applied) For saftey reasons, this is required for certain tests before it begins, and must be maintained for the entire duration of the test.	BSNC
90	shiftLeverNotInPark For saftey reasons, this is required for certain tests before it begins, and must be maintained for the entire duration of the test.	SLNIP
91	torqueConverterClutchLocked This response code indicates that the requested action will not be taken because the server prerequisite condition for torque converter clutch is not met (current TCC status above a preprogrammed limit or locked).	TCCL
92	voltageTooHigh This response code indicates that the requested action will not be taken because the server prerequisite condition for voltage at the primary pin of the server (ECU) is not met (current voltage is above a pre-programmed maximum threshold).	VTH
93	voltageTooLow This response code indicates that the requested action will not be taken because the server prerequisite condition for voltage at the primary pin of the server (ECU) is not met (current voltage is below a pre-programmed maximum threshold).	VTL
F0	CRLIntegrityFailed CRL integrity check (Verification of CRL Signature) is failed. The hash value of SHA-256 of Certificate authority's public key and the CRL Issuer Public Key Identifier value are not matched	CRLF
F1	CRLvalidityPeriodFailed The current time is not included between CRL effective date and CRL expiration date	CRLPF
F2	RoleandRightofCertificateDenied Role and Right of Certificate verification failed. The request service or the target from the external diagnostic tool are not matched with Certificate Holder Reference : Authorization	RRCD
F3	securityAccessDeniedfromCGW This response code indicates that the requested action will not be taken from the CGW because the CGW's security strategy (Secure Access) has not been satisfied by the client.	SADCGW
F4	LowerVersionDownloadAttempt This response code indicates that the requested action will not be taken because the downloaded software is lower version than the existing version.	LVDA

F5	ServiceNotSupportedInActiveConnection This response code indicates that the requested action will not be taken because the service is not allowed in active connection(CAN or Ethernet).	SNSIAC

표 A-1. Definition of responseCode values

Annex. B Diagnostic and Communication Management Functional Unit Data Parameter Definitions

B.1 CommnuicationType Parameter Definition

The communicationType is a 1-byte value. The bit-encoded low nibble of this byte represents the communicationTypes, which can be controlled via the CommunicationControl (28 hex) service.
For example, a communicationType with a bit c
ombination (Bits 1-0) of "11b" is valid and disables both "normalCommunicationMessages" and "networkManagementCommunicationMessages" messages.
The high nibble of the communicationType 1-byte value defines which of the subnets connected to the receiving node shall be disabled/enabled when an appropriate CommunicationControl service is received.

Encoding of Bit	Value	Description	Cvt	Mnemonic
0 - 1	0	Reserved	M	
	1	NormalCommunicationMessages This value references all application-related communication (inter-application signal exchange between multiple in-vehicle servers).	M	NCM
	2	NetworkManagementCommunicationMessages This value references all network management related communication.	M	NWMCM
	3	NetworkManagementCommnunicationMessage and NormalCommunicationMessages This value references all network management and application-related communication.	M	
2 - 3	0~3	Reserved		
4 - 7	0	Disable/Enable specified communicationType (see encoding of bit 0-1) in the receiving node and all connected subnets. This only disables the node's communication into the subnets but not the communication of other nodes on the subnet (receiving node is not responsible to disable communication in each node of the subnet).	M	
	1~E	Disable/Enable specific subnet identified by subnet number.	U	
	F	Disable/Enable network which request is received on (Receiving node (server))	U	

표 B-1. Definition of CommunicationType and SubnetNumber Byte

B.2 EventWindowTime Parameter Definition

HEX	Description	Cvt	Mnemonic
00 - 01	Reserved	M	

02	InfiniteTimeToResponse This value specifies that the event window shall stay active for an infinite amount of time. (e.g. open window until power off)	U	ITTR
03 – 7F	Reserved (vehicle manufacture specific range)		
80 – FF	Reserved	M	

표 B-2. Definition of EventWindowTime Parameter

B.3 BaudrateIdentifier Parameter Definition

HEX	Description	Cvt	Mnemonic
00	Reserved	M	
01	PC9600Baud This value specifies the standard PC baudrate of 9.6 KBaud.	U	PC9600
02	PC19200Baud This value specifies the standard PC baudrate of 19.2 KBaud.	U	PC19200
03	PC38400Baud This value specifies the standard PC baudrate of 38.4 KBaud.	U	PC38400
04	PC57600Baud This value specifies the standard PC baudrate of 57.6 KBaud.	U	PC57600
05	PC115200Baud This value specifies the standard PC baudrate of 115.2 KBaud.	U	PC115200
06 – 0F	Reserved	M	
10	CAN125000Baud This value specifies the standard CAN baudrate of 125 KBaud.	U	CAN125000
11	CAN250000Baud This value specifies the standard CAN baudrate of 250 KBaud.	U	CAN250000
12	CAN500000Baud This value specifies the standard CAN baudrate of 500 KBaud.	U	CAN500000
13	CAN1000000Baud This value specifies the standard CAN baudrate of 1 MBaud.	U	CAN1000000
14 – FF	Reserved		

표 B-3. Definition of BaudrateIdentifier Parameter

Annex. C Data Transmission Functional Unit Data Parameter Definitions

C.1 dataIdentifier parameter definitions

The parameter dataIdentifier (DID) is to identify a server specific local data record. This parameter shall be available in the server's memory. The dataIdentifier value shall either exist in fixed memory or temporarily stored in RAM if defined dynamically by the service dynamicallyDefineDataIdentifier.

The example of DataIdentifier Parameter Table is described in following table.

Example#1]

Data Identifier (HEX)	Byte	PID	Description Text	Unit	HEX Range Phy. Range	Conversion
0xZZZZ	#1 ~ #4	0x00	Supported PID : 0x01~0x20 Response : 4 Byte (A,B,C,D) Byte #1 : PID 0x01-0x08, Byte #2 : PID 0x09-0x10, Byte #3 : PID 0x11-0x18, Byte #4 : PID 0x19-0x20			
	…	…	…	…	…	…
	#	0xX1	Intake Manifold Absolute Pressure	kPa	00 ~ FF 0 ~ 255	X*1
	#	0xX2	Short Term Fuel Trim	%	80 ~ 7F -100 ~ 99.22	(singed) X*200/255
	#	0xX3	Intake Air Temperature	℃	00 ~ FF -40 ~ 215	X*1-40
	#	0xX4	Location of Oxygen Sensor	Bit 0 = 1 : Bank1 / Sensor1 Bit 1 = 1 : Bank1 / Sensor2 … Bit 7 = 1 : Bank2 / Sensor4		
	#	0xX5	OBD requirements to which vehicle is designed	01 = OBD II 02 = OBD … 0C~FF = Reserved		
	#	0xX6	Bit 0 : Variant Coding	0 = MT 1 = AT		
		0xX7	Bit 2 : Drive Position	0 = R 1 = D		
		0xX8	Bit 3 : Aircon Switch	0 = OFF 1 = ON		
		…	…	…	…	…
		0xXC	Bit 7 : Reserved	-	-	-
	…	…	…	…	…	…

The definition of Supported PID Byte (byte #0~byte#3) is as below.

Byte \ Bit	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Byte #0	PID 0x01	PID 0x02	PID 0x03	PID 0x04	PID 0x05	PID 0x06	PID 0x07	PID 0x08

Byte #1	PID 0x09	PID 0x0A	PID 0x0B	PID 0x0C	PID 0x0D	PID 0x0E	PID 0x0F	PID 0x10
Byte #2	PID 0x11	PID 0x12	PID 0x13	PID 0x14	PID 0x15	PID 0x16	PID 0x17	PID 0x18
Byte #3	PID 0x19	PID 0x1A	PID 0x1B	PID 0x1C	PID 0x1D	PID 0x1E	PID 0x1F	PID 0x20
*. Each bit would be set to '0' (if the assigned PID is not supported) or '1' (if the assigned PID is supported).								

표 C-1. Example#1 of Record DataIdentifier Parameter Table

Example#2]

Data Identifier (HEX)	Byte	Description Text	Data Size	Data Format
0xZZZZ	#1~#17	VIN (Vehicle Identification Number)	17	ASCII

표 C-2. Example#2 of Identification Option DataIdentifier Parameter Table

Example#3]

Data Identifier (HEX)	Byte#	Control Enable Mask Record	Description Text	Input Output Control Parameter	Control State	Control Type	Test Condition
0xAAAA	-	-	Desired Idle Adjustment	00, 01, 02, 03	(Hex) * 10 [r/min]	Until Stop Request	IG ON & Eng. Running
0xBBBB	-	-	MIL	00, 01, 02, 03	0x00 = OFF 0xFF = ON	Until Stop Request	IG ON & Eng. Stop
0xCCCC	-	-	Fuel Pump Relay	00, 01, 02, 03	0x00 = Stop 0xFF = Start	For 3 sec	IG ON & Eng. Stop
0xDDDD	#1	Bit 7	IAC Pintle Position	00, 01, 02, 03	X [count]	Until Stop Request	IG ON & Eng. Running
	#2 #3	Bit 6	RPM (High Byte) RPM (Low Byte)		X [U/min]		
	#4 [bit7-4]	Bit 5	Pedal Position A		X*8 [%]		
	#4 [bit3-0]	Bit 4	Pedal Position B		X*8 [%]		
	#5	Bit 3	EGR Duty Cycle		X*100/255 [%]		
...	...	...	...	...	...	...	...

표 C-3. Example#3 of Input Output DataIdentifier Parameter Table

Hex	Description	Cvt	Mnemonic
0100-A5FF	Reserved for HMCKIA Specific - Record Data Identifiers	U	
A600-A7FF	Reserved for Legislative Use - IMPORTANT : Do not use other than legislative ECU	M	
A800-ACFF	Reserved for HMCKIA Specific - Record Data Identifiers	U	
AD00-AFFF	Reserved for Legislative Use - IMPORTANT : Do not use other than legislative ECU	M	
B000-B1FF	Reserved for HMCKIA Specific - Record Data Identifiers	U	
B200-BFFF	Reserved for Legislative Use - IMPORTANT : Do not use other than legislative ECU	M	
C000-C2FF	Reserved for HMCKIA Specific - Record Data Identifiers	U	
C300-CEFF	Reserved for Legislative Use - IMPORTANT : Do not use other than legislative ECU	M	
CF00-EFFF	Reserved for HMCKIA Specific - Record Data Identifiers	U	

C0DE	EOLCodingNumber Notation – EOL Coding Number : This value shall be used to reference the vehicle variant coding i.e)EOL Coding Record data size shall be 4 or 8bytes(extended)	U	EOLCN
DE00-DE50	EthernetPhyDiagnosticRecordData These values are available for EthernetPHYDiagnosticsData, used to provide the health information/statistics of Ethernet PHY. Further details e.g. Ethernet PHY diagnostic features, protocols, and procedures, shall be referred to the relevant specification	U	EPDRD
ED90	NonSleepDataCollectedEachEcuDataIdentifier This value shall be used to reference non-sleep log data collecting from each ECU. Each ECU shall response the data to CCU by RDBI(ReadDataByIdentifier) when the ECU can't enter to the sleep mode abnormally and CCU sends diagnostic request as ED90 DID. CCU shall collect the data from the ECU and Tester can get the all collected data by another diagnostic RDBI from CCU. Each ECU shall response non-sleep log data in Max 100 Bytes. Specific data information responded shall be managed in each ECU's specification. For example, following data can be included in the data. - Network Message and PNC bit info related to sleep and wakeup - HW Input / Output pin info related to wakeup. - internal specific data and flag value in ECU to cause non-sleep. - Each application session info to hold entering sleep mode	M	NSDCEEDID
ED91-ED95	Reserved for NonSleepDataCollectedEachEcuDataIdentifier ED91-ED95 (Reserved for future non sleep data requirement)	M	RNSDCEEDID
ED96-ED99	VehicleLoggerEcuLoggingDataIdentifier This value shall be used to reference ECU internal logging information to be collected for vehicle logger. Vehicle logger in CCU references data in each ECU periodically at predefined interval, and the ECUs shall respond with their own internal logging information. Each response shall contain less than 1KB data, unless the exception is agreed with CCU.	U	VLELDID
EDC0	ECUPlatformLoggingData 1 – Network Mode Log Data This value shall be used to reference ECU internal logging information from the SW platform. This feature collect information of ComM and CanNm module in BSW such as ComM mode and also the state and transition value of CanNm. This functionality is provided by SW Platform itself and does not including any security and personal data at all.	U	ECUPLD1
EDC1	ECUPlatformLoggingData 2 – Wakeup/Sleep Log Data This value shall be used to reference ECU internal logging information from the SW platform. This feature collect information during the Sleep and Wakeup processes run on SW platform such as ComM Mode and NvM running state at requesting sleep and Wakeup delay at requesting wakeup. This functionality is provided by SW Platform itself and does not including any security and personal data at all.	U	ECUPLD2
EFF0-EFDF	FaultDiagnosticRecordData Please refer to Requirements document for this DIDs	U	FDRD
F010-F0FF	Reserved for HMCKIA Specific – Input Output Data Identifiers	U	
F100-F17F	Reserved for HMCKIA Specific – Identification Options Data Identifiers	U	
F180	BootSoftwareIdentificationDataIdentifier This value shall be used to reference the vehicle manufacturer specific ECU boot software identification record. The first data byte of the record data shall be the numberOfModules that are reported. Following the numberOfModules the boot software identification(s) are reported. The format of the boot software identification structure shall be ECU specific and defined by the vehicle manufacturer.	U	BSIDID

F181	ApplicationSoftwareIdentificationDataIdentifier This value shall be used to reference the vehicle manufacturer specific ECU application software number(s). The first data byte of the record data shall be the numberOfModules that are reported. Following the numberOfModules the application software identification(s) are reported. The format of the application software identification structure shall be ECU specific and defined by the vehicle manufacturer.	U	ASIDID
F182	ApplicationDataIdentificationDataIdentifier This value shall be used to reference the vehicle manufacturer specific ECU application data identification record. The first data byte of the record data shall be the numberOfModules that are reported. Following the numberOfModules the application data identification(s) are reported. The format of the application data identification structure shall be ECU specific and defined by the vehicle manufacturer.	U	ADIDID
F183	BootSoftwareFingerprintDataIdentifier This value shall be used to reference the vehicle manufacturer specific ECU boot software fingerprint identification record. Record data content and format shall be ECU specific and defined by the vehicle manufacturer.	U	BSFPDID
F184	ApplicationSoftwareFingerprintDataIdentifier This value shall be used to reference the vehicle manufacturer specific ECU application software fingerprint identification record. Record data content and format shall be ECU specific and defined by the vehicle manufacturer.	U	ASFPDID
F185	ApplicationDataFingerprintDataIdentifier This value shall be used to reference the vehicle manufacturer specific ECU application data fingerprint identification record. Record data content and format shall be ECU specific and defined by the vehicle manufacturer.	U	ADFPDID
F186	ActiveDiagnosticSessionDataIdentifier This value shall be used to report the active diagnostic session in the server. The values are defined by the diagnosticSessionType subfunction parameter in the DiagnosticSessionControl service.	U	ADSDID
F187	VehicleManufacturerSparePartNumberDataIdentifier Notation – Vehicle Part Number : or – Vehicle Part Number(Location) : *Location : e.g) FL,FR,RL,RR This value shall be used to reference the vehicle manufacturer spare part number.. Record data format shall be ASCII and size shall be 10 ~ 15 bytes. Color code could be max. 15 bytes. (e.g. 959102T250 → 0x 39 35 39 31 30 32 54 32 35 30)	M	VMSPNDID
F188	VehicleManufacturerECUSoftwareNumberDataIdentifier This value shall be used to reference the vehicle manufacturer ECU (server) software number. Record data content and format shall be server specific and defined by the vehicle manufacturer.	U	VMECUSNDID
F189	VehicleManufacturerECUSoftwareVersionNumberDataIdentifier This value shall be used to reference the vehicle manufacturer ECU (server) software version number. Record data content and format shall be server specific and defined by the vehicle manufacturer.	U	VMECUSVNDID
F18A	SystemSupplierIdentifierDataIdentifier This value shall be used to reference the system supplier name and address information. Record data content and format shall be server specific and defined by the system supplier.	U	SSIDDID
F18B	ECUManufacturingDateDataIdentifier Notation – Supplier H/W manufacturing Date : Or – Supplier H/W manufacturing Date(Location) : *Location : e.g.) FL, FR, RL,RR This value shall be used to reference the ECU (server) manufacturing date. Record data format shall be HEX and size shall be 4 bytes. The content shall be ordered as Year, Month, Day.(e.g. 2018.03.06 → 0x20180306)	M	ECUMDDID

F18C	ECUSerialNumberDataIdentifier Notation – Serial Number : Or – Serial Number(Nocation) : *Location : e.g.) FL,FR,RL,RR This value shall be used to reference the ECU (server) serial number. Data shall be defined as ASCII code.	M	ECUSNDID
F18D	SupportedFunctionalUnitsDataIdentifier This value shall be used to request the functional units implemented in a server.	U	SFUDID
F18E	VehicleManufacturerKitAssemblyPartNumberDataIdentifier This value shall be used to reference the vehicle manufacturer order number for a kit (assembled parts bought as a whole for production e.g. cockpit), when the spare part number designates only the server (e.g. for aftersales). The record data content and format shall be server specific and defined by the vehicle manufacturer.	U	VMKAPNDID
F18F	RXSWINDataIdentifier Notation – RXSWIN Data: RXSWIN is information that combines local RXSWIN information into one RXSWIN. Refer to Annex J	C2	
F190	VINDataIdentifier This value shall be used to reference the VIN number. VIN data shall be defined as ASCII code	U	VINDID
F191	VehicleManufacturerECUHardwareNumberDataIdentifier Notation – Vehicle H/W Version : This value is HMCKIA defined ECU hardware version number shown on the drwaing of each system. H/W version shall be defined as ASCII code.	M	VMECUHNDID
F192	SystemSupplierECUHardwareNumberDataIdentifier This value shall be used to reference the system supplier specific ECU (server) hardware number. This value shall be defined as ASCII code.	U	SSECUHWNDID
F193	SystemSupplierECUHardwareVersionNumberDataIdentifier Notation – Supplier H/W Version : Or – Supplier H/W Version(Location) : *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the system supplier specific ECU (server) hardware version number. This version shall be defined as ASCII code.	M	SSECUHWVNDID
F194	SystemSupplierECUSoftwareNumberDataIdentifier This value shall be used to reference the system supplier specific ECU (server) software number. This value shall be defined as ASCII code.	U	SSECUSWNDID
F195	SystemSupplierECUSoftwareVersionNumberDataIdentifier Notation – Supplier S/W Version : This value shall be used to reference the system supplier specific ECU (server) software version number. This value shall be defined as ASCII code.	U	SSECUSWVNDID
F196	ExhaustRegulationOrTypeApprovalNumberDataIdentifier This value shall be used to reference the exhaust regulation or type approval number (valid for those systems which require type approval). Record data content and format shall be server specific and defined by the vehicle manufacturer.	U	EROTANDID
F197	SystemNameOrEngineTypeDataIdentifier Notation – System Name : This value shall be used to reference the system name or engine type. Record data content and format shall be server specific and defined by the vehicle manufacturer. Data content format shall be ASCII e.g.) RR_C_RDR -> 8byte ASCII (0x82 0x82 0x95 0x67 0x95 0x82 0x68 0x82)	M	SNOETDID

F198	RepairShopCodeOrTesterSerialNumberDataIdentifier This value shall be used to reference the repair shop code or tester (client) serial number (e.g., to indicate the most recent service client used re-program server memory). Record data content and format shall be server specific and defined by the vehicle manufacturer.	U	RSCOTSNDID
F199	ProgrammingDateDataIdentifier This value shall be used to reference the date when the server was last programmed. Record data content and format shall be unsigned numeric, ASCII or BCD, and shall be ordered as Year, Month, Day.	U	PDDID
F19A	CalibrationRepairShopCodeOrCalibrationEquipmentSerialNumberDataIdentifier This value shall be used to reference the repair shop code or client serial number (e.g., to indicate the most recent service client used re-calibrate the server). Record data content and format shall be server specific and defined by the vehicle manufacturer.	U	CRSCOCESNDID
F19B	CalibrationDateDataIdentifier This value shall be used to reference the date when the server was last calibrated. Record data content and format shall be unsigned numeric, ASCII or BCD, and shall be ordered as Year, Month, Day.	U	CDDID
F19C	CalibrationEquipmentSoftwareNumberDataIdentifier This value shall be used to reference software version within the client used to calibrate the server. Record data content and format shall be server specific and defined by the vehicle manufacturer.	U	CESWNDID
F19D	ECUInstallationDateDataIdentifier This value shall be used to reference the date when the ECU (server) was installed in the vehicle. Record data content and format shall be either unsigned numeric, ASCII or BCD, and shall be ordered as Year, Month, Day.	U	EIDDID
F19E	ODXFileDataIdentifier This value shall be used to reference the ODX (Open Diagnostic Data Exchange) file of the server to be used to interprete and scale the server data.	U	ODXFDID
F19F	EntityDataIdentifier This value shall be used to reference the entity data identifier as defined in ISO 15764 for a secured data transmission.	U	EDID
F1A0	SoftwareVersionfor HMCKIA VehicleManufactureDataIdentifier Notation – Vehicle S/W Version Or – Vehicle S/W Version(Location) : *Location : e.g.) FL,FR,RL,RR This value is HMCKIA defined ECU software version number. This version is used to assist the unit version or to represent all SW units. S/W version shall be defined as ASCII code. Dataidentifier parameter. ※ In case of a ECU has sub ECUs, 4 * n bytes would be used. ('n' is the number of sub ECUs) In this case the order of ECU version shall be fixed.	U	SVHVMDID
F1A1	ECUSupplierCodeDataIdentifier Notation – System Supplier Code : Or – System Supplier Code(Location) : *Location : e.g.) FL,FR,RL,RR This value shall be used to reference the ECU (server) Supplier Code. Record data format shall be ASCII and size shall be 4 bytes. (e.g. DELPHI CORPORATION : AB9H → 0x41 42 39 48)	M	ECUSCD

F1A2	ECUApplicationVersionDataIdentifier This value shall be used to reference the Application version. following format shall be used. {App ID (ASCII 5bytes) + Length(Hex 1byte) + Version(ASCII 'n' bytes)} *'m' The number of iteration 'm' depends on the number of App installed in the ECU. The length of the Version is depends on application specific.	U	ECUAVD
F1A3	ECUApplicationIVDHashDataIdentifier This value shall be used to reference the Application IVD Hash. following format shall be used. {App ID (ASCII 5bytes) + Length(Hex 1byte) + IVD(32bytes)}	C3	ECUAIVDHD
F1A4-F1AF	Reserved for HMCKIA Specific – Identification Option		
F1B0	ECUSoftwareUNITnumberDataIdentifier Nontation – The number of S/W Unit : This Identifier shall be used to reference unique system specific SW version. This value shall be used to reference the number of system specific S/W Unit in the ECU(server). The number of S/W Unit must consist of within 1byte and data type is unit8. If there is only one S/W unit, it is expressed as 1(dec). (Increase the value by 1 according to the number of S/W units.), and It can be expressed up to 15(dec). DID starts from F1B1 and be increased by 1. e.g) The number of SW unit is 3 then only three DIDs are allowed to use. (F1B1, F1B2, F1B3)	M	ECUSUNDID
F1B1	ECUSoftwareUNIT1VersionDataIdentifier Notation –S/W Unit1 Version : Or – S/W Unit1 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 1 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	M	ECUSU1VDID
F1B2	ECUSoftwareUNIT2VersionDataIdentifier Notation –S/W Unit2 Version : Or – S/W Unit2 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 2 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU2VDID
F1B3	ECUSoftwareUNIT3VersionDataIdentifier Notation –S/W Unit3 Version : Or – S/W Unit3 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 3 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU3VDID
F1B4	ECUSoftwareUNIT4VersionDataIdentifier Notation –S/W Unit4 Version : Or – S/W Unit4 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 4 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU4VDID

F1B5	ECUSoftwareUNIT5VersionDataIdentifier Notation –S/W Unit5 Version : Or – S/W Unit5 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 5 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU5VDID
F1B6	ECUSoftwareUNIT6VersionDataIdentifier Notation –S/W Unit6 Version : Or – S/W Unit6 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 6 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU6VDID
F1B7	ECUSoftwareUNIT7VersionDataIdentifier Notation –S/W Unit7 Version : Or – S/W Unit7 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 7 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU7VDID
F1B8	ECUSoftwareUNIT8VersionDataIdentifier Notation –S/W Unit8 Version : Or – S/W Unit8 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 8 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU8VDID
F1B9	ECUSoftwareUNIT9VersionDataIdentifier Notation –S/W Unit9 Version : Or – S/W Unit9 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 9 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU9VDID
F1BA	ECUSoftwareUNIT10VersionDataIdentifier Notation –S/W Unit10 Version : Or – S/W Unit10 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 10 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU10VDID

F1BB	ECUSoftwareUNIT11VersionDataIdentifier Notation –S/W Unit11 Version : Or – S/W Unit11 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 11 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU11VDID
F1BC	ECUSoftwareUNIT12VersionDataIdentifier Notation –S/W Unit12 Version : Or – S/W Unit12 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 12 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU12VDID
F1BD	ECUSoftwareUNIT13VersionDataIdentifier Notation –S/W Unit13 Version : Or – S/W Unit13 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 13 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU13VDID
F1BE	ECUSoftwareUNIT14VersionDataIdentifier Notation –S/W Unit14 Version : Or – S/W Unit14 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 14 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU14VDID
F1BF	ECUSoftwareUNIT15VersionDataIdentifier Notation –S/W Unit15 Version : Or – S/W Unit15 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 15 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU15VDID

<table>
<tr><td rowspan="6">F1C0</td><td colspan="4">ECUSecurityInformationDataIdentifier
This value shall be used to reference the security status of the ECU, such as Secure boot, Secure Debug, and HSM configuration status.</td><td rowspan="6">M</td><td rowspan="6">ECUSSDID</td></tr>
<tr><td>Byte #</td><td>Description</td><td>Conversion rule</td><td>Data Size</td></tr>
<tr><td>#0</td><td>HSM Configuration Lock state
This value shall be used to check HSM configuration Lock state of ECU.</td><td>(bit 7..0)
0x00 = Disabled
0x01 = Enabled
0x02 ~ 0xFE = Reserved
0xFF = N/A</td><td>1byte</td></tr>
<tr><td>#1</td><td>Secure Boot State
This value shall be used to check Secure Boot state of ECU.</td><td>(bit 7..0)
0x00 = Disabled
0x01 = Enabled
0x02 ~ 0xFE = Reserved
0xFF = N/A</td><td>1byte</td></tr>
<tr><td>#2</td><td>Secure Debug State
This value shall be used to check Secure Debug state of ECU.</td><td>(bit 7..0)
0x00 = Disabled
0x01 = Enabled
0x02 = DebugProtectionTempStop
0x03 = Debug Disable
0x04 ~ 0xFE = Reserved
0xFF = N/A</td><td>1byte</td></tr>
<tr><td>#3</td><td>External bootloader state
This value shall be used to check External bootloader state of ECU.</td><td>(bit 7..0)
0x00 = Unsupported
0x01 = Disabled
0x02 = Enabled
0xFF = N/A</td><td>1byte</td></tr>
<tr><td>F1C1</td><td colspan="4">ECUSoftwareUNIT1IVDDataIdentifier
Notation – S/W Unit1 IVD Data :
This value shall be used to reference the IVD value of SW unit 1. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request.
If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31.
*IVD : Integrity Validation Data
Data type : HEX
Data length : SHA256 (32bytes)</td><td>M</td><td>ECUSU1HDID</td></tr>
</table>

F1C2	ECUSoftwareUNIT2IVDDataIdentifier Notation – S/W Unit2 IVD Data : This value shall be used to reference the IVD value of SW unit 2. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU2HDID
F1C3	ECUSoftwareUNIT3IVDDataIdentifier Notation – S/W Unit3 IVD Data : This value shall be used to reference the IVD value of SW unit 3. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU3HDID
F1C4	ECUSoftwareUNIT4IVDDataIdentifier Notation – S/W Unit4 IVD Data : This value shall be used to reference the IVD value of SW unit 4. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU4HDID
F1C5	ECUSoftwareUNIT5IVDDataIdentifier Notation – S/W Unit5 IVD Data : This value shall be used to reference the IVD value of SW unit 5. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU5HDID

F1C6	ECUSoftwareUNIT6IVDDataIdentifier Notation – S/W Unit6 IVD Data : This value shall be used to reference the IVD value of SW unit 6. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU6HDID
F1C7	ECUSoftwareUNIT7IVDDataIdentifier Notation – S/W Unit7 IVD Data : This value shall be used to reference the IVD value of SW unit 7. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU7HDID
F1C8	ECUSoftwareUNIT8IVDDataIdentifier Notation – S/W Unit8 IVD Data : This value shall be used to reference the IVD value of SW unit 8. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU8HDID
F1C9	ECUSoftwareUNIT9IVDDataIdentifier Notation – S/W Unit9 IVD Data : This value shall be used to reference the IVD value of SW unit 9. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU9HDID

F1CA	ECUSoftwareUNIT10IVDDataIdentifier Notation – S/W Unit10 IVD Data : This value shall be used to reference the IVD value of SW unit 10. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU10HDID
F1CB	ECUSoftwareUNIT11IVDDataIdentifier Notation – S/W Unit11 IVD Data : This value shall be used to reference the IVD value of SW unit 11. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU11HDID
F1CC	ECUSoftwareUNIT12IVDDataIdentifier Notation – S/W Unit12 IVD Data : This value shall be used to reference the IVD value of SW unit 12. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU12HDID
F1CD	ECUSoftwareUNIT13IVDDataIdentifier Notation – S/W Unit13 IVD Data : This value shall be used to reference the IVD value of SW unit 13. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU13HDID

F1CE	ECUSoftwareUNIT14IVDDataIdentifier Notation – S/W Unit14 IVD Data : This value shall be used to reference the IVD value of SW unit 14. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU14HDID
F1CF	ECUSoftwareUNIT15IVDDataIdentifier Notation – S/W Unit15 IVD Data : This value shall be used to reference the IVD value of SW unit 15. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU15HDID
F1D0-F1ED	Reserved for HMCKIA Specific – Identification Option	U	
F1EE	RXSWINEOLResultIdentifier This value shall be used to reference the result of Routine control 0x04F0 This value shall be maintained until new result has been stored. Result shall be maintained even if driving cycle has been changed. Data Type : Hex Data Length : 1Byte Volatile : Non-volatile data R/W : Read only Data definition(text table) : 0x00 : No result (Routine 0x04F0 has never been done before) 0x01 : RXSWIN Done (Routine 0x04F0 has been done) 0x02 : RXSWIN Aborted (Routine 0x04F0 has been aborted)	C2	RXSWINEOLRES
F1EF	LocalRXSWINDataIdentifier Notation – Local RXSWIN Data : RXSWIN information which is in a ECU. Refer to Annex J	M	LRXSWINDID
F1F0-F1FF	Reserved for System Specific – Identification Option	U	
F200-F2FF	Reserved – Periodic Data Identifier	U	
F300-F3FF	Reserved – Dynamically Defined Data Identifier	U	
F400-F8FF	Reserved for OBD – IMPORTANT : Do not use other than legislative ECU	M	
F900-F9FF	TachographDataIdentifier	U	TACHODID
FA00-FA0F	Airbag Deployment Data Identifier	U	ADDID
FA10-FA18	EDR Data Identifier	U	EDRDID
FA19-FAFF	Safety System Data Identifier	U	SSDID
FB00-FCFF	Reserved for Legislative Use – IMPORTANT : Do not use other than legislative ECU	M	
FD00-FEFF	Reserved for System Specific	M	
FF02-FFFF	Reserved for ISOSAE	M	

C1 : More than 1 SW unit is available on the system.
C2 : Central Gateway ECU
C3 : If Controller supports DID F1A2, This DID shall be supported.

표 C-4. DataIdentifier Parameter Table

Generic DID shall be shown as following order to the diagnostic tester.

No	DID	Notation of Diagnostic tool
1	F187	Vehicle Part Number
2	F1B0	The number of S/W Unit
3	F1B1~F1BA	S/W Unit(*N) Version : (*N is the numbering of SW unit)
4	F197	System Name :
5	F193	Supplier H/W Version
6	F18B	Supplier H/W Manufacturing Date :
7	F1A1	System Supplier Code :
8	C0DE	EOL Coding Number :
9	F18C	Serial Number :

표 C-4.1. The order of DID in the diagnostic tool and Notation

DID	Usage
F1A0	S/W version of HMCKIA
F187	Vehicle Part Number
F1B0	The number of SWUnits
F1B1	SW Unit1 version
F1B2	SW Unit2 version
F1B3	SW Unit3 version
F1B4	SW Unit4 version
F1B5	SW Unit5 version
F1B6	SW Unit6 version
F1B7	SW Unit7 version
F1B8	SW Unit8 version
F1B9	SW Unit9 version
F1BA	SW Unit10 version

표 C-4.2. List of SUMS related DID

C.1.1 ControlEnableMaskRecord

Byte#	Bit Evaluation		
	Bit	PID # (HEX)	Conversion
1	Bit 7	01	0 = doesn't affected by the request, 1 = affected by the request
	Bit 6	02	
	…	…	
	Bit 1	07	
	Bit 0	08	
2	Bit 7	09	
	Bit 6	0A	
	…	…	
	Bit 1	0E	
	Bit 0	0F	
…	…	…	
N	Bit 7	(N-1)*8+1	
	Bit 6	(N-1)*8+2	
	…	…	
	Bit 1	(N-1)*8+7	
	Bit 0	(N-1)*8+8	

표 C-5. Supported PID Check Byte Definitions

C.1.2 Supported PID Check Byte Definitions

Not all PIDs are applicable or supported by all ECUs or systems. PID $00 is a bit-encoded PID that indicates, for each ECU, which PIDs that ECU supports.

If there are more than 32 PIDs in each LID, 32*n-th PID (0x00, 0x20, 0x40, 0x60 …) will be a supported PID byte. The definition of Supported PID Byte is as below.

Requested PID (HEX)	Bit Evaluation		
	Bit	PID # (HEX)	Conversion
00	1st Byte, Bit 7	01	0 = not supported 1 = supported
	1st Byte, Bit 6	02	
	…	…	
	4th Byte, Bit 1	1F	
	4th Byte, Bit 0	20	
20	1st Byte, Bit 7	21	
	1st Byte, Bit 6	22	
	…	…	
	4th Byte, Bit 1	3F	
	4th Byte, Bit 0	40	
…	…	…	
C0	1st Byte, Bit 7	C1	
	1st Byte, Bit 6	C2	
	…	…	
	4th Byte, Bit 1	DF	
	4th Byte, Bit 0	E0	
E0	1st Byte, Bit 7	E1	
	1st Byte, Bit 6	E2	
	…	…	
	4th Byte, Bit 1	FF	
	4th Byte, Bit 0	Reserved Always zero(0) = not supported	

표 C-6. Supported PID Check Byte Definitions

C.2 ScalingByte Parameter Definitions

The parameter scalingByte (SBYT) consists of one byte (high and low nibble). The scalingByte high nibble defines the data type, which is used to represent the dataIdentifier (DID). The scalingByte low nibble defines the number of bytes used to represent the parameter in a datastream.

Encoding of High Nibble (HEX)	Description of Data Type	Cvt	Mnemonic
0	unSignedNumeric (1 to 4 bytes) This encoding uses a common binary weighting scheme to represent a value by mean of discrete incremental steps. One byte affords 256 steps; two bytes yields 65536 steps, etc.	U	USN
1	signedNumeric (1 to 4 bytes) This encoding uses a two's complement binary weighting scheme to represent a value by mean of discrete incremental steps. One byte affords 256 steps; two bytes yields 65536 steps, etc.	U	SN
2	bitMappedReportedWithOutMask Bit mapped encoding uses individual bits or small groups of bits to represent status. For every bit which represents status, a corresponding mask bit is required as part of the parameter definition. The mask indicates the validity of the bit for particular applications. This type of bit mapped parameter does not contain additional bytes to report the validity mask.	U	BMRWOM
3	bitMappedReportedWithMask Bit mapped encoding uses individual bits or small groups of bits to represent status. For every bit which represents status, a corresponding mask bit is required as part of the parameter definition. The mask indicates the validity of the bit for particular applications. This type of bit mapped parameter contains one validity mask byte for each status byte representing data.	U	BMRWM
4	BinaryCodedDecimal Conventional Binary Coded Decimal encoding is used to represent two numeric digits per byte. The upper nibble is used to represent the most significant digit (0 - 9), and the lower nibble the least significant digit (0 -9).	U	BCD
5	stateEncodedVariable (1 byte) This encoding uses a binary weighting scheme to represent up to 256 distinct states. An example is a parameter, which represents the status of the Ignition Switch. Codes "00", "01", "02" and "03" may indicate ignition off, locked, run, and start, respectively. The representation is always limited to one (1) byte.	U	SEV
6	ASCII (1 to 15 bytes for each scalingByte) Conventional ASCII encoding is used to represent up to 128 standard characters with the MSB = logic 0. An additional 128 custom characters may be represented with the MSB = logic 1.	U	ASCII

7	signedFloatingPoint Floating point encoding is used for data that needs to be represented in floating point or scientific notation. Standard IEEE formats shall be used according to ANSI/IEEE Std 754 – 1985.	U	SFP
8	packet Packets contain multiple data values, usually related, each with unique scaling. Scaling information is not included for the individual values. Refer to section C.3.1 scalingByteExtension for scalingByte high nibble of bitMappedReportedWithOutMask.	U	P
9	formula A formula is used to calculate a value from the raw data. Formula Identifiers are specified in the table definig the formulaIdentifier encoding. Refer to section C.3.2 scalingByteExtension for scalingByte high nibble of formula.	U	F
A	unit/format The units and formats are used to present the data in a more user-friendly format. Unit and Format Identifiers are specified in the Table defining the formulaIdentifier encoding. NOTE) If combined units and/or formats are used, e.g. mV, then one scalingByte (and scalingData) for each unit/format shall be included in the readScalingDataByIdentifier positive response. Refer to section C.3.3 scalingByteExtension for scalingByte high nibble of unit / format.	U	U
B	stateAndConnectionType (1 byte) This encoding is used especially for input and output signals. The information encoded in the data byte specifies the high level physical layout, electrical levels and functional state. It is recommended to use this option for digital input and output parameters. Refer to section C.3.4 scalingByteExtension for scalingByte high nibble of stateAndConnectionType.	U	SACT
C – F	Reserved		

표 C-7. ScalingByte (High Nibble) Parameter Definitions

Encoding of High Nibble (HEX)	Description of Data Type	Cvt	Mnemonic

0 – F	numberOfBytesOfParameter This range of values specifies the number of data bytes in a data stream referenced by a parameter identifier. The length of a parameter is defined by the scaling byte(s), which is always preceded by a parameter identifier (one or multiple bytes). If multiple scaling bytes follow a parameter identifier the length of the data referenced by the parameter identifier is the summation of the content of the low nibbles in the scaling bytes. e.g. VIN is identified by a single byte parameter identifier and followed by two scaling bytes. The length is calculated up to 17 data bytes. The content of the two low nibbles may have any combination of values that add up to 17 data bytes. Note) For the scalingByte with high nibble encoded as formula or unit/format this value is $0.	U	NROBOP

표 C-8. ScalingByte (Low Nibble) Parameter Definitions

C.3 ScalingByteExtension Parameter Definitions

C.3.1 ScalingByteExtension for ScalingByte high nibble of BitMappedReportedWithOutMask

The parameter scalingByteExtension (SBYE) is only supported for scalingByte parameters with the high nibble encoded as formula, unit/format, or bitMappedReportedWithOutMask.
A scalingByte with high nibble encoded as bitMappedReportedWithOutMask shall be followed by scalingByteExtension bytes representing the validity mask for the bit mapped dataIdentifier. Each byte shall indicate which bits of the corresponding dataIdentifier byte are supported for the current application.

ScalingByteExtension Byte	Description	Cvt
#1	dataIdentifier dataRecord#1 validity mask	M
…	…	C1
#p	dataIdentifier dataRecord#p validity mask	C1
C1 : The presence of this parameter depends on the size of the dataIdentifier the information is being requested for. The validity mask shall have as many bytes as the dataIdentifier has dataRecords.		

표 C-9. ScalingByteExtension Bytes for BitMappedReportedWithOutMask

C.3.2 ScalingByteExtension for ScalingByte high nibble of Formula

The parameter scalingByteExtension (SBYE) is only supported for scalingByte parameters with the high nibble encoded as formula, unit/format, or bitMappedReportedWithOutMask.
A scalingByte with high nibble encoded as formula shall be followed by scalingByteExtension bytes defining the formula. The scalingByteExtension consists a of one byte formulaIdentifier and constants as described in Table below.

ScalingByteExtension Byte	Description	Cvt
#1	formulaIdentifier (refer to Table defining the formulaIdentifier encoding for details)	M
#2	C0 high byte	M
#3	C0 low byte	M
#4	C1 high byte	U
#5	C1 low byte	U
…	…	U
#2n+2	Cn high byte	U
#2n+3	Cn low byte	U

표 C-10. ScalingByteExtension Bytes for Formula

FormulaIdentifier (HEX)	Description	Cvt

00	y = C0 * x + C1	U
01	y = C0 * (x + C1)	U
02	y = C0 / (x + C1) + C2	U
03	y = x / C0 + C1	U
04	y = (x + C0) / C1	U
05	y = (x + C0) / C1 + C2	U
06	y = C0 * x	U
07	y = x / C0	U
08	y = x + C0	U
09	y = x * C0 / C1	U
0A – 7F	Reserved	M
80 – FF	Reserved (vehicle manufacturer specific range)	U

표 C-11. FormulaIdentifier Encoding

Formulas are defined using variables (y, x, etc.) and constants (C0, C1, C2, etc.). The variable y is the calculated value. The other variables, in consecutive order, are part of the data stream referenced by a dataIdentifier. Each constant is expressed as a two byte real number defined in Table C.8 — Two byte real number format. The two byte real numbers (C = M * 10E) contain a 12 bit signed (2's complement) mantissa (M) and a 4 bit signed (2's complement) exponent (E). The mantissa can hold values within the range –2048 to +2047, and the exponent can scale the number by 10–8 to 107. The exponent is encoded in the high nibble of the high byte of the two byte real number. The mantissa is encoded in the low nibble of the high byte and the complete low byte of the two byte real number.

High Byte								Low Byte							
High Nibble				Low Nibble				High Nibble				Low Nibble			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Exponent				Mantissa											

표 C-12. Two Byte Real Number Format

C.3.3 ScalingByteExtension for ScalingByte high nibble of Unit / Format

The parameter scalingByteExtension (SBYE) is only supported for scalingByte parameters with the high nibble encoded as formula, unit/format, or bitMappedReportedWithOutMask.

A scalingByte with high nibble encoded as unit / format shall be followed by a single scalingByteExtension byte defining the unit / format. The one byte scalingByteExtension is defined in Table C.9 — Unit / format scalingByteExtension encoding. If combined units and/or formats are used, e.g. mV, then one scalingByte (and scalingByteExtension) shall be included for each unit / format.

ScalingByteExtension Byte #1	Name	Symbol	Description	Cvt
00	No unit, no prefix			U
01	Meter	m	Length	U
02	Foot	ft	Length	U

03	Inch	in	Length	U
04	Yard	yd	Length	U
05	mile (English)	mi	length	U
06	Gram	g	mass	U
07	ton (metric)	t	mass	U
08	Second	s	time	U
09	Minute	min	time	U
0A	Hour	h	time	U
0B	Day	d	time	U
0C	year	y	time	U
0D	ampere	A	current	U
0E	volt	V	voltage	U
0F	coulomb	C	electric charge	U
10	ohm	W	resistance	U
11	farad	F	capacitance	U
12	henry	H	inductance	U
13	siemens	S	electric conductance	U
14	weber	Wb	magnetic flux	U
15	tesla	T	magnetic flux density	U
16	kelvin	K	thermodynamic temperature	U
17	Celsius	°C	thermodynamic temperature	U
18	Fahrenheit	°F	thermodynamic temperature	U
19	candela	cd	luminous intensity	U
1A	radian	rad	plane angle	U
1B	degree	°	plane angle	U
1C	hertz	Hz	frequency	U
1D	joule	J	energy	U
1E	Newton	N	force	U
1F	kilopond	kp	force	U
20	pound force	lbf	force	U
21	watt	W	power	U
22	horse power (metric)	hk	power	U
23	horse power (UK and US)	hp	power	U
24	Pascal	Pa	pressure	U
25	bar	bar	pressure	U
26	atmosphere	atm	pressure	U
27	pound force per square	psi	pressure	U

28	becqerel	Bq	radioactivity	U
29	lumen	lm	light flux	U
2A	lux	lx	illuminance	U
2B	liter	l	volume	U
2C	gallon (British)	–	volume	U
2D	gallon (US liq)	–	volume	U
2E	cubic inch	cu in	volume	U
2F	meter per second	m/s	speed	U
30	kilometer per hour	km/h	speed	U
31	mile per hour	mph	speed	U
32	revolutions per second	rps	angular velocity	U
33	revolutions per minute	rpm	angular velocity	U
34	counts	–	–	U
35	percent	%	–	U
36	milligram per stroke	mg/stroke	mass per engine stroke	U
37	meter per square second	m/s2	acceleration	U
38	Newton meter	Nm	moment (e.g. torsion moment)	U
39	liter per minute	l/min	flow	U
3A	Watt per square meter	W/m2	Intensity	U
3B	Bar per second	bar/s	Pressure change	U
3C	Radians per second	rad/s	Angular velocity	U
3D	Radians per square	rad/s2	Angular acceleration	U
3E	Kilogram per square	kg/m2	–	U
3F	–	–	Reserved	M
40	exa (prefix)	E	1018	U
41	peta (prefix)	P	1015	U
42	tera (prefix)	T	1012	U
43	giga (prefix)	G	109	U
44	mega (prefix)	M	106	U
45	kilo (prefix)	k	103	U
46	hecto (prefix)	h	102	U
47	deca (prefix)	da	10	U
48	deci (prefix)	d	10-1	U
49	centi (prefix)	c	10-2	U
4A	milli (prefix)	m	10-3	U

4B	micro (prefix)	m	10-6	U
4C	nano (prefix)	n	10-9	U
4D	pico (prefix)	p	10-12	U
4E	femto (prefix)	f	10-15	U
4F	atto (prefix)	a	10-18	U
50	Date1	-	Year-Month-Day	U
51	Date2	-	Day/Month/Year	U
52	Date3	-	Month/Day/Year	U
53	week	W	calendar week	U
54	Time1	-	UTC Hour/Minute/Second	U
55	Time2	-	Hour/Minute/Second	U
56	DateAndTime1	-	Second/Minute/Hour/Day/Month/Year	U
57	DateAndTime2	-	Second/Minute/Hour/Day/Month/Year/Local minute offset/Local hour offset	U
58	DateAndTime3	-	Second/Minute/Hour/Month/Day/Year	U
59	DateAndTime4	-	Second/Minute/Hour/Month/Day/Year/Local minute offset/Local hour offset	U
B0	degree per second	deg/s	degree/second	U
5A-FF	-	-	Reserved	M

표 C-13. Unit / Format ScalingByteExtension Encoding

C.3.4 ScalingByteExtension for ScalingByte high nibble of StateAndConnectionType

A scalingByte with high nibble encoded as stateAndConnectionType shall be followed by a single scalingByteExtension byte defining the stateAndConnectionType. The one byte scalingByteExtension is defined in Table C.9 — Encoding of scalingByte High Nibble of stateAndConnectionType. The stateAndConnectionType encoding is used specially for input and output signals. Encoded in the scalingByteExtension data byte is information about the physical layout, electrical levels and functional state.

Encoding of Bit	Value	Used with Input Signal	Used with Output Signal
0 – 2	0	State : Not Active	State : Not Activated
	1	State : Active, Function 1	State : Active, Function 1
	2	State : Error Detected	State : Plausibility Error Detected
	3	State : Not Available	State : Not Available
	4	State : Active, Function 2 (only in combination with 3 states)	State : Active, Function 2 (only in combination with 3 states)

	5 – 7	Reserved	Reserved
3 – 4	0	Signal at Low Level (Ground)	Signal at Low Level (Ground)
	1	Signal at Middle Level (Between Ground Level and +)	Signal at Middle Level (Between Ground Level and +)
	2	Signal at High Level (+)	Signal at High Level (+)
	3	Reserved	Reserved
5	0	Input Signal	Not Defined
	1	Not Defined	Output Signal
6 – 7	0	Internal Signal or via CAN not exclusively available in ECU connector	Internal Signal or via CAN not exclusively available in ECU connector
	1	Pull-down Resistor input type (2 states)	Low side switch (2 states)
	2	Pull-up Resistor input type (2 states)	High side switch (2 states)
	3	Pull-up and Pull-down resistor input type (3 state)	Low side and High side switch (3 state)

표 C-14. Encoding of scalingByte High Nibble of stateAndConnectionType

HEX	Description	Cvt	Mnemonic
00	Reserved		
01	sendAtSlowRate This parameter specifies that the server shall transmit the requested dataRecord information at a slow rate in response to the request message (where the # of responses to be sent = maximumNumberOfResponsesToSend). The repetition rate specified by the transmissionMode parameter slow is vehicle manufacturer specific, and pre-defined in the server.	U	SASR
02	sendAtMediumRate This parameter specifies that the server shall transmit the requested dataRecord information at a medium rate in response to the request message (where the # of responses to be sent = maximumNumberOfResponsesToSend). The repetition rate specified by the transmissionMode parameter medium is vehicle manufacturer specific, and pre-defined in the server.	U	SAMR
03	sendAtFastRate This parameter specifies that the server shall transmit the requested dataRecord information at a fast rate in response to the request message (where the # of responses to be sent = maximumNumberOfResponsesToSend). The repetition rate specified by the transmissionMode parameter fast is vehicle manufacturer specific, and pre-defined in the server.	U	SAFR
04	stopSending The server stops transmitting positive response messages send periodically/repeatedly. Note that maximumNumberOfResponsesToSend parameter should be set to 01 hex if transmissionMode = stopSending (otherwise, server operation could be undefined).	C	SS
05 – FF	Reserved		
C : stopSending shall be supported if sendAtSlowRate, sendAtMediumRate, and/or sendAtFastRate are supported.			

표 C-15. Encoding of scalingByte High Nibble of stateAndConnectionType

Annex. D Stored Data Transmission Functional Unit Data Parameter Definitions

D.1 GroupOfDTC Parameter Definition

Table D-1. Definition of GroupOfDTC and range of DTC numbers provides group of DTC definitions.

HEX	Description	Cvt	Mnemonic
000000 – 0000FF	Reserved for future legislative requirements	M	RFLU
0001FF	Powertrain Group	U	PG
0001XX – 3FFFXX	Powertrain DTC's (except 0001FF)	U	PDTC_
4000FF	Chassis Group	U	CG
4001XX – 7FFFXX	Chassis DTC's	U	CDTC_
8000FF	Body Group	U	BG
8001XX – BFFFXX	Body DTC's	U	BDTC_
C000FF	Network Communication Gruop	U	NCG
C001XX – FFFEXX	Network Communication DTC's	U	NCDTC_
FFFF33	Emissions-system group	C	ESG
FFFFD0	Safety-system group	U	SSG
FFFFFF	All Groups (All DTC's)	M	AG
C : This parameter selects the emission-related systems.			

표 D-1. Definition of GroupOfDTC and range of DTC numbers

D.2 DTCStatusMask and StatusOfDTC Bit Definitions

D.2.1 Convention

This section defines the mapping of the DTCStatusMask / statusOfDTC parameters used with the ReadDTCInformation service. Every server shall adhere to the convention for storing bit-packed DTC status information as defined in table below. Actual usage of the bit-fields shall be defined in the implementation standards.

The following is a list of definitions used for the description of the DTC status bit definitions.

- Test : A test is an on-board diagnostic software algorithm that determines the malfunction status of a component or system. Some tests run only once during an operation cycle. Other tests can run every program loop, as often as every few milliseconds. The result of a test represents a completely matured / qualified failure condition. That means a test which needs a failing condition over a specific time or evaluation of additional plausibility checks before a component is considered to be failing will return a“Failed” condition after all maturation criteria have been fulfilled. Each test is associated with a unique DTC representing the root failure and detectable fault symptom.

- Complete : Complete is an indication that a test was able to determine whether a malfunction exists or does not exist for the current operation cycle (complete does not imply failed).

- Test results : While a test runs or after it has completed it may indicate one the following results to the internal failure handler :

  – PreFailed : This status may be used by tests in ECUs to indicate that the test is currently maturing a failure condition. One use case for this information is in manufacturing to speed up failure detection for optimised workflow while maintaining fault tolerance in the field.

  – Failed : This status is available after a monitor has run to its completion and indicates a completely matured failing condition.

  – Passed : This status is available after a monitor has run to its completion and indicates that the system or component is not failing.

- Failure : A failure is the inability of a component or system to meet its intended function. A failure has occurred when fault conditions have been detected for a sufficient period of time to warrant storage of a pending (for emission related components only) or confirmed DTC implying that a test returned a “Failed” result. The terms "failure" and "malfunction" are interchangeable.

- Monitor : A monitor consists of one or more tests used to determine the proper functioning of a component or system.

- Monitoring cycle : A monitoring cycle is the time in which a monitor runs to its completeness. This is a manufacturer defined set of conditions during which the tests of a monitor can run. A monitoring cycle may be executed several times during an operation cycle.

- Operation Cycle : An operation cycle defines the start and end conditions for monitors to run.

During an operation cycle several monitoring cycles may have passed (regardless of their test results). An ECU may support several operation cycles. For body and chassis ECU's it is up to the manufacturer to define an operation cycle (e.g. time between powering up and powering down the ECU or between ignition on and ignition off). For powertrain ECU's, there are additional criteria defining an operation cycle. Emissionsrelated powertrain ECU's use an engine-running or engine-off time period to define an operation cycle which is referred to as driving cycle.

NOTE) For emissions-related monitors the criteria for the beginning and the end of an operation cycle are defined by legislation.

- Pending : The pending status of a failure is defined as a Test Result having reported a “Failed” for this failure during the current and/or the last completed operation cycle. Once the test has reported a “Passed” condition for a complete operation cycle of this failure the pending status is reset.

- Driving cycle : A specific type of operation cycle used for emissions-related ECU's. Refer to “OperationCycle” for further details. In emissions-related ECU's only one operation cycle shall be supported, which is identical to the driving cycle as defined by legislation.

D.2.2 StatusOfDTC Bit Definition

Bit	Description	Default Value	Cvt	Mnemonic
0	TestFailed '0' = most recent result from DTC test indicated no failure detected '1' = most recent result from DTC test indicated a matured failing result.	0	M	TF
1	TestFailedThisOperationCycle '0' = testFailed: result has not been reported during the current operation cycle or after a call was made to ClearDiagnosticInformation during the current operation cycle. '1' = testFailed: result was reported at least once during the current operation cycle.	0	C1	TFTOC
2	PendingDTC '0' = This bit shall be set to 0 after completing an operation cycle during which the test completed and a malfunction was not detected or upon a call to the ClearDiagnosticInformation service. '1' = This bit shall be set to 1 and latched if a malfunction is detected during the current operation cycle.	0	U	PDTC
3	ConfirmedDTC '0' = DTC has never been confirmed since the last call to ClearDiagnosticInformation or after the aging criteria have been satisfied for the DTC (or DTC has been erased due to fault memory overflow). '1' = DTC confirmed at least once since the last call to ClearDiagnosticInformation and aging criteria have not yet been satisfied.	0	M	CDTC
4	TestNotCompletedSinceLastClear '0' = DTC test has returned either a passed or failed test result at least one time since the last time diagnostic information was cleared. '1' = DTC test has not run to completion since the last time diagnostic information was cleared.	1	C2	TNCSLC
5	TestFailedSinceLastClear '0' = DTC test has not indicated a failed result since the last time diagnostic information was cleared. '1' = DTC test returned a failed result at least once since the last time diagnostic information was cleared.	0	C2	TFSLC
6	TestNotCompleteThisOperationCycle '0' = DTC test has returned either a passed or testFailedThisOperationCycle = '1' result during the current drive cycle (or since the last time diagnostic information was cleared during the current operation cycle). '1' = DTC test has not run to completion this operation cycle (or since the last time diagnostic information was cleared this operation cycle).	1	U	TNCTOC
7	WarningIndicatorRequested '0' = Server is not requesting warningIndicator to be active '1' = Server is requesting warningIndicator to be active	0	M	WIR

C1 : Bit 1 (TestFailedThisOperationCycle) is Mandatory if Bit 2 (PendingDTC) is supported. Bit 1 is User Optional
if Bit 2 is not supported.
C2 : Bit 4 (TestNotCompletedSinceLastClear) and Bit 5 (TestFailedSinceLastClear) shall always be supported together.
*. DTCStatusMask will be determined by setting each bit of StatusOfDTC '0 = not supported' or '1 = supported'.

표 D-2. DTC Status Bit Definition Overview

DTC Case	Status Of Bit (하위 4Bit)				DTC Status Mask (request) (※ 결과값이 1 이상일 경우 해당 DTC 응답)		진단기 표출	미표출 사유
	Bit3	Bit2	Bit1	Bit0	09 (EOL HADS, A/S GDS)	0D (EOL U-Robot, A/S GDS)		
	Confirmed	Pending	TestFailedThisOoperationCycle	TestFailed				
#1	0	0	X	0	0 (미응답)	0 (미응답)	-	DTC 발생 및 Masking X
#2	0	0	X	1	1	1	-	발생 불가 1)
#3	0	1	X	0	0 (미응답)	1	임시 고장	
#4	0	1	X	1	1	2	임시 고장	
#5	1	0	X	0	1	1	과거 고장	
#6	1	0	X	1	2	2	-	발생 불가 1)
#7	1	1	X	0	1	2	과거 고장	
#8	1	1	X	1	2	3	현재 고장	

1) Pending DTC 지원 시, Test Failed Bit 가 set 되면 Pending DTC 도 set 되어야 함.

표 D-3. DTC Status Definition and Display of tester (ECU supported Pending DTC)

DTC Case	Status Of Bit		DTC Status Mask (request) (※ 결과값이 1 이상일 경우 해당 DTC 응답)	진단기 표출	미표출 사유
	Bit3	Bit0	08 (EOL, A/S GDS)		
	Confirmed	TestFailed			
#1	0	0	0 (미응답)	-	DTC 발생 및 Masking X
#2	0	1	0 (미응답)	-	발생 불가 2) 및 Masking X
#3	1	0	1	과거 고장	
#4	1	1	1	현재 고장	

2) Pending DTC 미지원 시, Test Failed Bit 가 set 되면 Confirmed DTC 도 set 되어야 함. (Confirmation threshold : 1)

표 D-4. DTC Status Definition and Display of tester (ECU not supported Pending DTC)

<table>
<tr><th>Bit</th><th colspan="2">Description</th><th>Cvt : Emission</th><th>Cvt : Non-Emission</th><th>Mnemonic</th></tr>
<tr><td rowspan="8">0</td><td colspan="2">TestFailed</td><td>U</td><td>U</td><td>TF</td></tr>
<tr><td colspan="5">This bit shall indicate the result of the most recently performed test. A logical ‘1’ shall indicate that the last test failed meaning that the failure is completely matured. Reset to logical '0' if the result of the most recently performed test returns a “pass” result meaning that all de-mature criteria have been fulfilled. Additional reset conditions may be defined by the vehicle manufacturer / implementation</td></tr>
<tr><th colspan="2">Bit state after a successful ClearDiagnosticInformation service</th><td colspan="3">Logical ‘0’</td></tr>
<tr><td colspan="5">Reset to logical '0' if a call has been made to ClearDiagnosticInformation.</td></tr>
<tr><th colspan="2">Bit state definition</th><th colspan="3">Test Equipment Display Text</th></tr>
<tr><td colspan="2">'0' = most recent result from DTC test indicated no failure detected (or test has not completed this operation cycle).</td><td colspan="3">DTC test is not failed at time of request</td></tr>
<tr><td colspan="2">'1' = most recent result from DTC test indicated a matured failing result.</td><td colspan="3">DTC test failed at time of request</td></tr>
</table>

표 D-5. DTC Status Bit 0 TestFailed Definitions

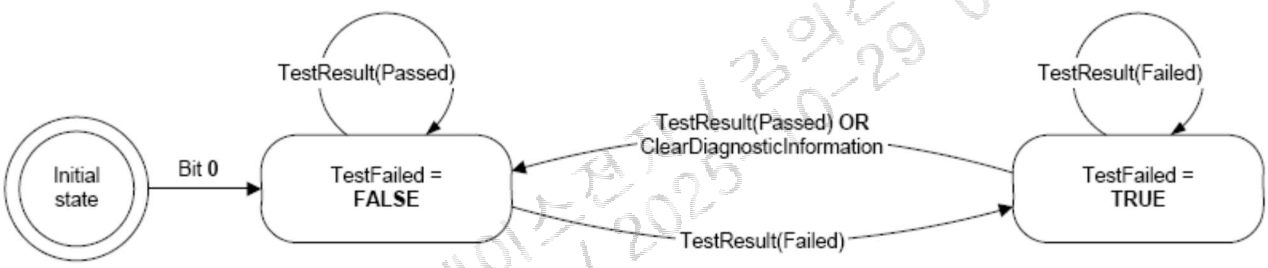


그림 D-1. DTC Status Bit 0 TestFailed Logic

<table>
<tr><th>Bit</th><th colspan="2">Description</th><th>Cvt : Emission</th><th>Cvt : Non-Emission</th><th>Mnemonic</th></tr>
<tr><td rowspan="8">1</td><td colspan="2">TestFailedThisOperationCycle</td><td>M</td><td>C1</td><td>TFTOC</td></tr>
<tr><td colspan="5">This bit shall indicate whether or not a diagnostic test has reported a testFailed result at any time during the current operation cycle (or that a testFailed result has been reported during the current operation cycle and after the last time a call was made to ClearDiagnosticInformation). Reset to logical '0' when a new operation cycle is initiated or after a call to ClearDiagnosticInformation.

If this bit is set to logical '1', it shall remain a '1' until a new operation cycle is started.</td></tr>
<tr><td colspan="2">Bit state after a successful ClearDiagnosticInformation service</td><td colspan="3">Logical ‘0’</td></tr>
<tr><td colspan="5">Reset to logical '0' after a call to ClearDiagnosticInformation.</td></tr>
<tr><td colspan="2">Bit state definition</td><td colspan="3">Test Equipment Display Text</td></tr>
<tr><td colspan="2">'0' = testFailed: result has not been reported during the current operation cycle or after a call was made to ClearDiagnosticInformation during the current operation cycle.</td><td colspan="3">DTC test has not failed this operation cycle</td></tr>
<tr><td colspan="2">'1' = testFailed: result was reported at least once during the current operation cycle.</td><td colspan="3">DTC test failed this operation cycle</td></tr>
<tr><td colspan="5">C1: Bit 1 (testFailedThisOperationCycle) is Mandatory if bit 2 (pendingDTC) is supported. Bit 1 (testFailedThisOperationCycle) is User Optional if bit 2 (pendingDTC) is not supported.</td></tr>
</table>

표 D-6. DTC Status Bit 1 TestFailedThisOperationCycle Definitions

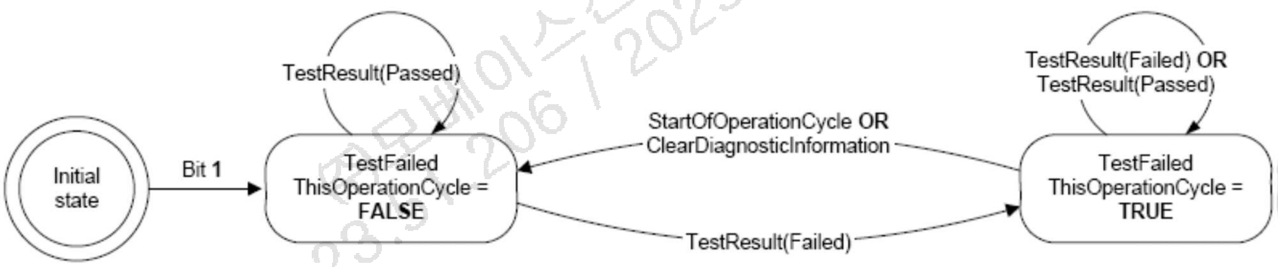


그림 D-2. DTC Status Bit 1 TestFailedThisOperationCycle Logic

Bit	Description	Cvt : Emission	Cvt : Non-Emission	Mnemonic
2	PendingDTC	M	U	PDTC
	This bit shall indicate whether or not a diagnostic test has reported a testFailed result at any time during the current or last completed operation cycle. The status shall only be updated if the test runs and completes. The criteria to set the pendingDTC bit and the TestFailedThisOperationCycle bit are the same. The difference is that the testFailedThisOperationCycle is cleared at the end of the current operation cycle and the pendingDTC bit is not cleared until an operation cycle has completed where the test has passed at least once and never failed. If the test did not complete during the current operation cycle, the status bit shall not be changed. For example, if a monitor stops running after a confirmed DTC is set, the pendingDTC must remain set = '1'. For an OBD DTC, a pending DTC is required to be stored after a malfunction is detected during the first driving cycle.			
	Bit state after a successful ClearDiagnosticInformation service	Logical '0'		
	Reset to logical '0' after a call to ClearDiagnosticInformation.			
	Bit state definition	Test Equipment Display Text		
	'0' = This bit shall be set to 0 after completing an operation cycle during which the test completed and a malfunction was not detected or upon a call to the ClearDiagnosticInformation service.	DTC test has not failed during the current or last completed operation cycle		
	'1' = This bit shall be set to 1 and latched if a malfunction is detected during the current operation cycle.	DTC test failed during the current or last completed operation cycle		

표 D-7. DTC Status Bit 2 PendingDTC Definitions

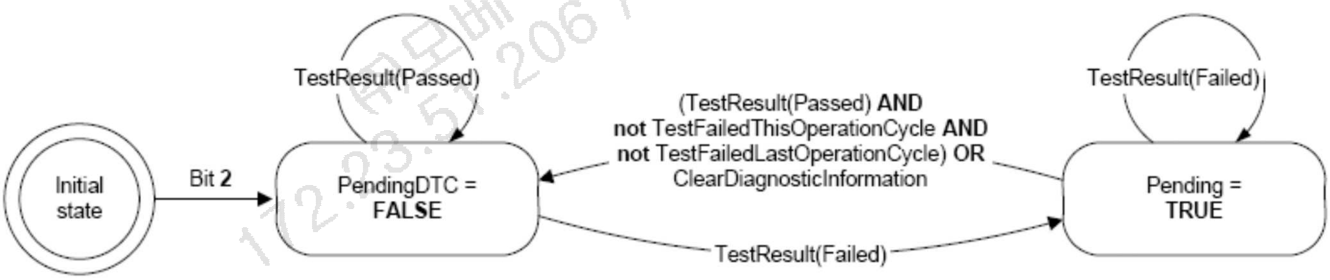


그림 D-3. DTC Status Bit 2 PendingDTC Logic

Bit	Description		Cvt : Emission	Cvt : Non-Emission	Mnemonic
3	ConfirmedDTC		M	M	CDTC
	This bit shall indicate whether a malfunction was detected enough times to warrant that the DTC is stored in long-term memory (e.g. pendingDTC has been set = '1' one or more times, depending on the DTC confirmation criteria). A confirmedDTC does not always indicate that the malfunction is present at the time of the request. (testFailed can be used to determine if a malfunction is present at the time of the request). Reset to logical '0' after a call to ClearDiagnosticInformation or after aging criteria has been satisfied (e.g., 40 engine warm-ups without another detected malfunction). Furthermore this bit is reset when the fault record associated with this DTC is overwritten by a newer DTC based upon vehicle manufacturer specific fault memory overflow requirements. DTC confirmation and aging criteria are defined by the vehicle manufacturer or mandated by On Board Diagnostic regulations.				
	Bit state after a successful ClearDiagnosticInformation service		Logical '0'		
	Reset to logical '0' after a call to ClearDiagnosticInformation.				
	Bit state definition		Test Equipment Display Text		
	'0' = DTC has never been confirmed since the last call to ClearDiagnosticInformation or after the aging criteria have been satisfied for the DTC (or DTC has been erased due to fault memory overflow).		A confirmed DTC is not stored in the ECU.		
	'1' = DTC confirmed at least once since the last call to ClearDiagnosticInformation and aging criteria have not yet been satisfied.		A confirmed DTC is stored in the ECU.		

표 D-8. DTC Status Bit 3 ConfirmedDTC Definitions

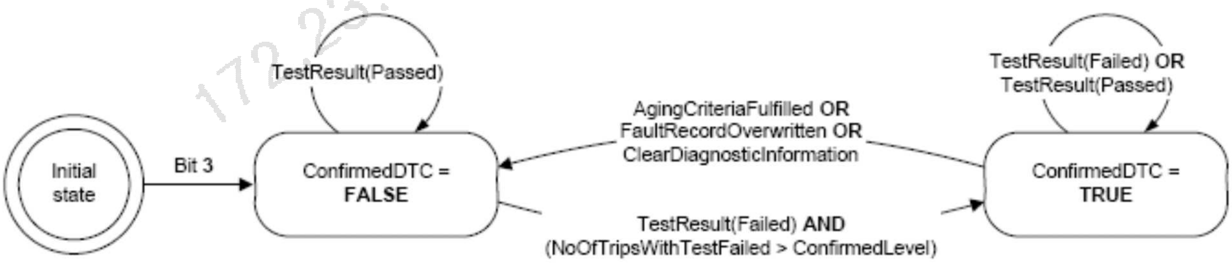


그림 D-4. DTC Status Bit 3 ConfirmedDTC Logic

Bit	Description		Cvt : Emission	Cvt : Non-Emission	Mnemonic
4	TestNotCompleteSinceLastClear		C2	C2	TNCSLC
	This bit shall indicate whether a DTC test has ever run and completed since the last time a call was made to ClearDiagnosticInformation. One ('1') shall indicate that the DTC test has not run to completion. If the test runs and passes or if the test runs and fails (e.g. testFailedThisOperationCycle = '1') then the bit shall be set to a '0' (and latched).				
	Bit state after a successful ClearDiagnosticInformation service		Logical '1'		
	Reset to logical '1' after a call to ClearDiagnosticInformation.				
	Bit state definition		Test Equipment Display Text		
	'0' = DTC test has returned either a passed or failed test result at least one time since the last time diagnostic information was cleared.		DTC test completed since the last code clear		
	'1' = DTC test has not run to completion since the last time diagnostic information was cleared.		DTC test not completed since the last code clear		
	C2: Bit 4 (testNotCompletedSinceLastClear) and bit 5 (testFailedSinceLastClear) shall always be supported together.				

표 D-9. DTC Status Bit 4 TestNotCompleteSinceLastClear Definitions

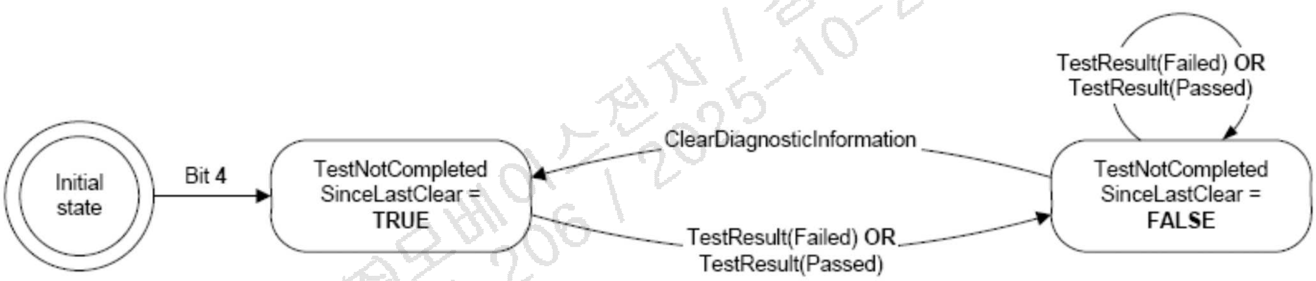


그림 D-5. DTC Status Bit 4 TestNotCompleteSinceLastClear Logic

Bit	Description		Cvt : Emission	Cvt : Non-Emission	Mnemonic
5	TestFailedSinceLastClear		C2	C2	TFSLC
	This bit shall indicate whether a DTC test has ever completed with a failed result since the last time a call was made to ClearDiagnosticInformation (i.e., this is a latched testFailedThisOperationCycle = '1'). Zero ('0') shall indicate that the test has not run or that the DTC test ran and passed (but never failed). If the test runs and fails then the bit shall remain latched at a '1'. Unlike confirmedDTC, this bit is not reset by aging-criteria or due to an overflow of the fault memory.				
	Bit state after a successful ClearDiagnosticInformation service		Logical '0'		
	Reset to logical '0' after a call to ClearDiagnosticInformation.				
	Bit state definition		Test Equipment Display Text		
	'0' = DTC test has not indicated a failed result since the last time diagnostic information was cleared.		DTC test never failed since last code clear		
	'1' = DTC test returned a failed result at least once since the last time diagnostic information was cleared.		DTC test failed at least once since last code clear		
	C2: Bit 4 (testNotCompletedSinceLastClear) and bit 5 (testFailedSinceLastClear) shall always be supported together.				

표 D-10. DTC Status Bit 5 TestFailedSinceLastClear Definitions

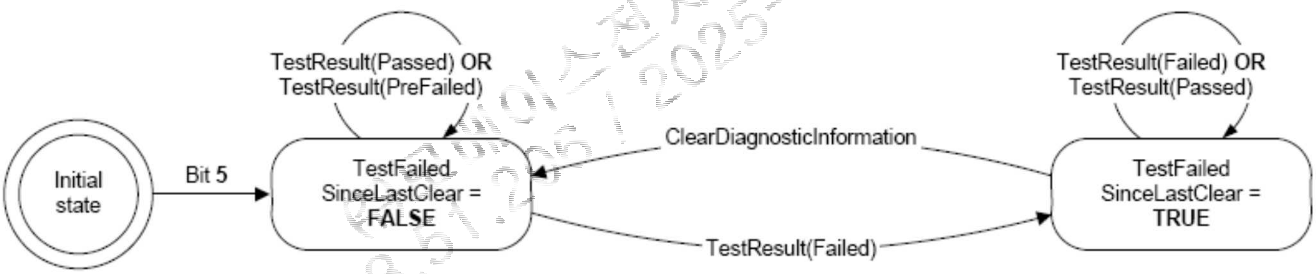


그림 D-6. DTC Status Bit 5 TestFailedSinceLastClear Logic

Bit	Description		Cvt : Emission	Cvt : Non-Emission	Mnemonic
6	TestNotCompleteThisOperationCycle		M	U	TNCTOC
	This bit shall indicate whether a DTC test has ever run and completed during the current operation cycle (or completed during the current operation cycle after the last time a call was made to ClearDiagnosticInformation). A logical '1' shall indicate that the DTC test has not run to completion during the current operation cycle. If the test runs and passes or fails then the bit shall be set (and latched) to '0' until a new operation cycle is started.				
	Bit state after a successful ClearDiagnosticInformation service		Logical '1'		
	Reset to logical '1' after a call to ClearDiagnosticInformation.				
	Bit state definition		Test Equipment Display Text		
	'0' = DTC test has returned either a passed or testFailedThisOperationCycle = '1' result during the current drive cycle (or since the last time diagnostic information was cleared during the current operation cycle).		DTC test completed this operation cycle		
	'1' = DTC test has not run to completion this operation cycle (or since the last time diagnostic information was cleared this operation cycle).		DTC test not completed this operation cycle		

표 D-11. DTC Status Bit 6 TestNotCompleteThisOperationCycle Definitions

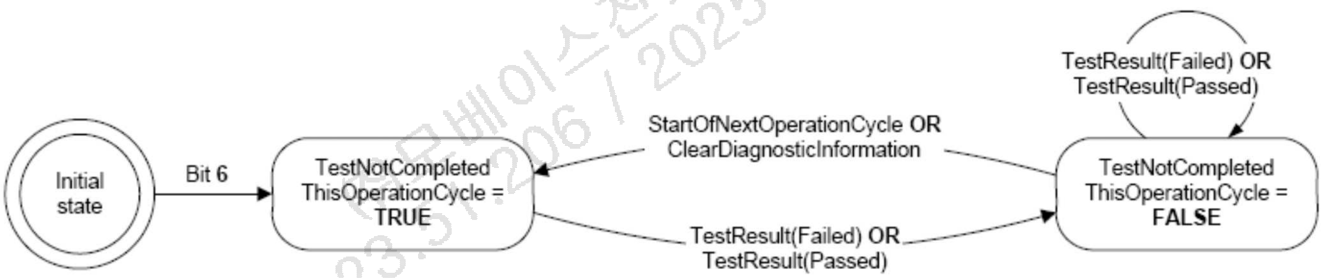
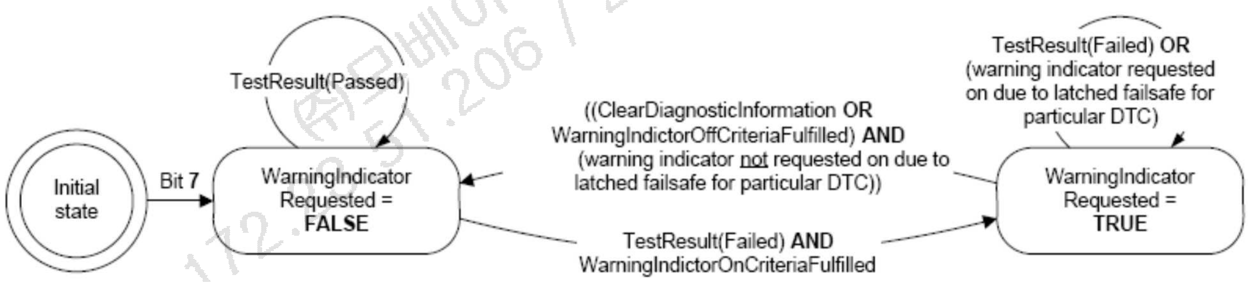


그림 D-7. DTC Status Bit 6 TestNotCompleteThisOperationCycle Logic

Bit	Description		Cvt : Emission	Cvt : Non-Emission	Mnemonic
7	WarningIndicatorRequested		M	M	WIR
	This bit shall report the status of any warning indicators associated with a particular DTC. Warning outputs may consist of indicator lamp(s), displayed text information, etc. If no warning indicators exist for a given system or particular DTC, this status shall default to a logic '0' state. Conditions for activating the warning indicator shall be defined by the vehicle manufacturer / implementation, but if the warning indicator is on for a given DTC, then confirmedDTC shall also be set to '1' (with the exception of a confirmed fault with a latched failsoft strategy as described below).				
	Bit state after a successful ClearDiagnosticInformation service	Logical ‘0’			
	Reset to a logical ‘0’ after a call to ClearDiagnosticInformation. Some ECUs may latch the failsoft strategy associated with a particular confirmed fault for the current operation cycle. If the warning indicator is still requested due to this latched failsoft following a call to ClearDiagnosticInformation, this bit shall not be cleared to a logical '0'. Rather, this bit shall remain set to logical '1' until the failsoft strategy is no longer active (e.g., test completes and passes). Additional reset conditions shall be defined by the vehicle manufacturer / implementation.				
	Bit state definition		Test Equipment Display Text		
	'0' = Server is not requesting warningIndicator to be active		DTC is currently not requesting warning indication		
	'1' = Server is requesting warningIndicator to be active		DTC is currently requesting warning indication		

표 D-12. DTC Status Bit 7 WarningIndicatorRequested Definitions



WarningIndicatorOnCriteriaFulfilled = warning indicator exists for given system and particular DTC AND (confirmedDTC = 1 OR vehicle manufacturer or implementation-specific warning indicator enable criteria are satisfied)

WarningIndicatorOffCritieriaFulfilled = TestResult(Passed) OR vehicle manufacturer or implementation-specific warning indicator disable criteria are satisfied

그림 D-8. DTC Status Bit 7 WarningIndicatorRequested Logic

D.3 DTCSeverityMask and DTCSevirity Bit Definitions

This section defines the mapping of the DTCSeverityMask / DTCSeverity parameters used with the ReadDTCInformation service. Every server shall adhere to the convention for storing bit-packed DTC severity information as defined in Table D-11. DTCSeverity byte definition and Table D-12. DTC severity bit definitions (bit 7-5).

The severity information is reported in a 1-byte value. Only the upper 3 bits (bit 7-5) of the 1-byte value are used to represent the DTC severity information. The remaining bits (bit 4-0) have to be set to zero (0). Based on this the following bit coding applies to the 3 bit DTC severity information contained in the 1-byte DTCSeverity parameter.

DTC Severity Byte							
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
3 Bit Severity Information			Reserved (To be set to zero)				

표 D-13. DTCSeverity byte definition

Bit 7-5	Description	Cvt	Mnemonic
000b	NoSeverityAvailable There is no severity information available.	M	NSA
001b	MaintenanceOnly This value indicates that the failure requests maintenance only.	M	MO
010b	CheckAtNextHalt This value indicates to the failure requires a check of the vehicle at the next halt.	M	CHKANH
011b	Reserved	M	
100b	CheckImmediately This value indicates to the failure requires an immediate check of the vehicle.	M	CHKI
101~111b	Reserved	M	

표 D-14. DTC severity bit definitions (bit 7-5)

D.4 DTC Format Structure

DTC High Byte			DTC Middle Byte		DTC Low Byte	
Bit 7-6	Bit 5-4	Bit 3-0	Bit 7-4	Bit 3-0	Bit 7-4	Bit 3-0
00 = powertrain (P) 01 = chassis (C) 10 = body (B) 11 = network (U)	00 = ISO/SAE controlled 01 = manufacturer controlled 10 = ISO/SAE controlled 11 = ISO/SAE controlled	1st Character Of Code .(Hex 0-F)	2nd Character of Code (Hex 0-F)	3rd Character of Code (Hex 0-F)	DTC Failure Category (refer to ISO15031-6)	DTC Failure Sub Type (refer to ISO15031-6)

표 D-15. Structure of Diagnostic Trouble Code

D.4.1 DTC Failure Category and Sub Type definition

Please refer to the latest SAE J2012-DA

D 5 DTCStatusAvailabilityMask

A byte whose bits are defined the same as statusofDTC and represents the status bits that are supported by the server.
Bits that are not support by the server shall be set to ‘0’. Each supported bit (indicated by a value of ‘1’) shall be implemented for every DTC supported by the server.

Annex. E Input Output Control Functional Unit Data Parameter Definitions

E.1 InputOutputControlParameter Definitions

HEX	Description	Cvt	Mnemonic
00	returnControlToECU This value shall indicate to the server that the client does no longer have control about the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier. - Number of controlState bytes in request: 0 - Number of controlState bytes in pos. response: depends on the dataIdentifier	U	RCTECU
01	resetToDefault This value shall indicate to the server that it is requested to reset the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier to its default state. - Number of controlState bytes in request: 0 - Number of controlState bytes in pos. response: depends on the dataIdentifier	U	RTD
02	freezeCurrentState This value shall indicate to the server that it is requested to freeze the current state of the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier. - Number of controlState bytes in request: 0 - Number of controlState bytes in pos. response: depends on the dataIdentifier	U	FCS
03	shortTermAdjustment This value shall indicate to the server that it is requested to adjust the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier in RAM to the value(s) included in the controlOption parameter(s). (e.g. set Idle Air Control Valve to a specific step number, set pulse width of valve to a specific value/duty cycle). - Number of controlState bytes in request: depends on the dataIdentifier - Number of controlState bytes in pos. response: depends on the dataIdentifier	U	STA
04~FF	Reserved	M	

표 E-1. InputOutputControlParameter Definition

Annex. F Remote Activation of Routine Functional Unit Data Parameter Definitions

'표 8-3. Request message routineIdentifier definition' 참조

Annex. G Address And Length Format Identifier Parameter Values

G.1 AddressAndLengthFormatIdentifier Definitions

This parameter is a one byte value with each nibble encoded separately. See below table.

AddressAndLengthFormatIdentifier							
High Nibble				Low Nibble			
7	6	5	4	3	2	1	0
Bytes used for MemorySize Parameter				Bytes used for MemroyAddress Parameter			

표 G-1. AddressAndLengthFormatIdentifier Definition

Annex. H Diagnostic CAN Identifiers

H.1 Definition of CAN Identifier for Enhanced Diagnostic Communication

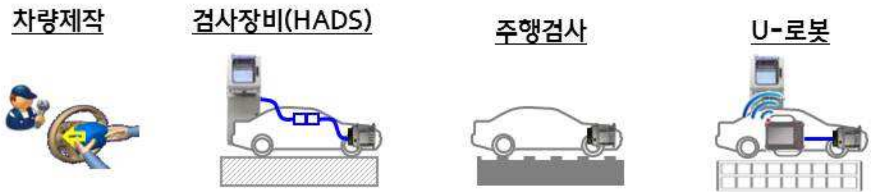
※The latest Diagnostic CAN Identifiers refer to the iDIMS and NIDS

CAN ID (HEX)	Description
	Functional Addressing
07DF	Functional request CAN ID sent by the external test equipment (All controllers must support this CAN Identifier)
	Physical Addressing
0580~05FF	HMC specific. See also iDIMS and NIDS ※Only for Third Generation of CAN (CAN-FD)
0700~07DE	HMC specific. See also iDIMS and NIDS
07E0	Physical request CAN ID to EMS (Engine Management System), FCU (Fuelcell Control Unit)
07E8	Physical response CAN ID from EMS (Engine Management System), FCU (Fuelcell Control Unit)
07E1	Physical request CAN ID to TCU (Transmission Control Unit)
07E9	Physical response CAN ID from TCU (Transmission Control Unit)
07E2	Physical request CAN ID to HCU (Hybrid Control Unit) / VCU (Vehicle Control Unit) / HCULDC (HCU & Low DC-DC Converter)
07EA	Physical response CAN ID from HCU (Hybrid Control Unit) / VCU (Vehicle Control Unit) / HCULDC (HCU & Low DC-DC Converter)
07E3	Physical request CAN ID to MCU (Motor Control Unit) / MHSG (Mild Hybrid Starter Generator) / F_MCU (Front MCU) / R_MCU (Rear MCU)
07EB	Physical response CAN ID from MCU (Motor Control Unit) / MHSG (Mild Hybrid Starter Generator) / F_MCU (Front MCU) / R_MCU (Rear MCU)
07E4	Physical request CAN ID to BMS (Battery Management System)
07EC	Physical response CAN ID from BMS (Battery Management System)
07E5	Physical request CAN ID to OBC (On Board Charger) : From LF/JF PHEV, AE/DE EV/PHEV ICCU (Integrated Charge Control Unit) : From Osc EV LP1
07ED	Physical response CAN ID from OBC (On Board Charger) : From LF/JF PHEV, AE/DE EV/PHEV ICCU (Integrated Charge Control Unit) : From Osc EV LP1
07E6	Physical request CAN ID to SCU_FF (SBW Control Unit) : From LX2 EVSCU_FF (EV SBW Control Unit) : EV SCU
07EE	Physical request CAN ID from SCU_FF (SBW Control Unit) : From LX2 EVSCU_FF (EV SBW Control Unit) : EV SCU
07E7	Physical request CAN ID to DCU (Doesing Control Unit) : Diesel Engine Only ESC(IEB) (Electronic Stability Control Integrated Electronic Brake)
07EF	Physical response CAN ID from DCU (Doesing Control Unit) : Diesel Engine Only ESC(IEB) (Electronic Stability Control Integrated Electronic Brake)
07F0~07F3	HMC specific. See also iDIMS and NIDS
07F4~07F6	ISO15765-2 Reserved
07F7~07F9	HMC specific. See also iDIMS and NIDS
07FA~07FC	ISO15765-2 Reserved
07FD~07FF	HMC specific. See also iDIMS and NIDS
17FF0001 ~17FFFFFE	HMC specific. See also iDIMS and NIDS

표 H-1. Definition of CAN Identifier for Enhanced Diagnostic Communication

Annex. I EOL (END OF LINE) Diagnostic Mode

I.1 EOL 공정 고장 코드 표출 이원화 (HADS, 최종검사 공정)



- [HADS 공정] 임시고장코드 표출 제외 (DTC Case #1)
- [최종검사 공정] 모든 고장코드 확인
- PT 가솔린/디젤 시스템 DTC 표출 기준 표준화 (Bit 2 Pending 개념 포함)
- UDS on CAN 기준 Pending DTC 개념 적용 시, 아래 Table 기준으로 적용한다.

DTC Case	Status Of Bit			DTC Status Mask (※ 결과값이 1 이상일 경우 해당 DTC 응답)		진단기 표출
	Bit3	Bit2	Bit0	HADS	최종검사	
	Confirmed	Pending	TestFailed	09	0D	
DTC Case #1	0	1	0	0 (미응답)	1	Pending DTC
DTC Case #2	0	1	1	1	2	Pending DTC
DTC Case #3	1	0	0	1	1	Historic DTC
DTC Case #4	1	1	0	1	2	Historic DTC
DTC Case #5	1	1	1	2	3	Active DTC

표 I-1. Meaning of DTC according to Status Of Bit and DTC Status Mask

I.2 EOL DTC 일괄 소거 (기억소거 장비)

ES95486-00 을 따르는 전 제어기의 DTC 일괄 소거 수행 시, 제어기와 장비간 통신 프로세스는 아래 사항을 준수한다.

I.2.1 CAN ID

각 제어기의 개별 작업 수행 시, 사양서에 명기된 제어기별 CAN ID 이용하여 통신한다. 일괄 소거 명령어 수행 시, Functional Request CAN ID 인 0x7DF 를 이용하여 통신한다.

I.2.2 EOL 표준 프로세스 (기억소거 장비 ↔ 제어기)

순서	프로세스	통신	비고
1	차량 전원 인가	-	작업자가 IG ON 실시
2	각 제어기 진단통신 시작	Tx : 02 10 03 Rx : 02 50 03	

3	… …		각 제어기 부가기능 수행 (코딩, 보정, 초기화 등)
4	각 제어기 진단통신 종료	Tx : 10 01 Rx : 50 01	
5	UDS on CAN 제어기 통신 시작	Tx : 02 10 03 Rx : 02 50 03	Tx CAN ID : 0x7DF Rx CAN ID : 제어기별 상이
6	UDS on CAN 제어기 DTC 일괄 소거 실시	Tx : 04 14 FF FF FF Rx : 01 54	↑
9	기억소거 작업 완료		장비 결과창에 OK/NG 표시 (공장별 상이)

표 I-2 EOL Standard Process of Batch Clear DTC

- 각 제어기의 개별 DTC 소거 및 부가기능 수행은 기존과 동일하게 수행한다. (1~4)
- 기억소거 장비에서 0x7DF CAN ID 로 통신 시작 및 DTC 소거 명령을 전송한다.

Annex. J RXSWIN requirement

J.1 RXSWIN Read

The EDT requests RXSWIN (RDBI 22 F18F) and the Central gateway requests a local RXSWIN of each ECU (RDBI 22 F1EF), stores the result, filters (remove redundancy) and transmits the result.

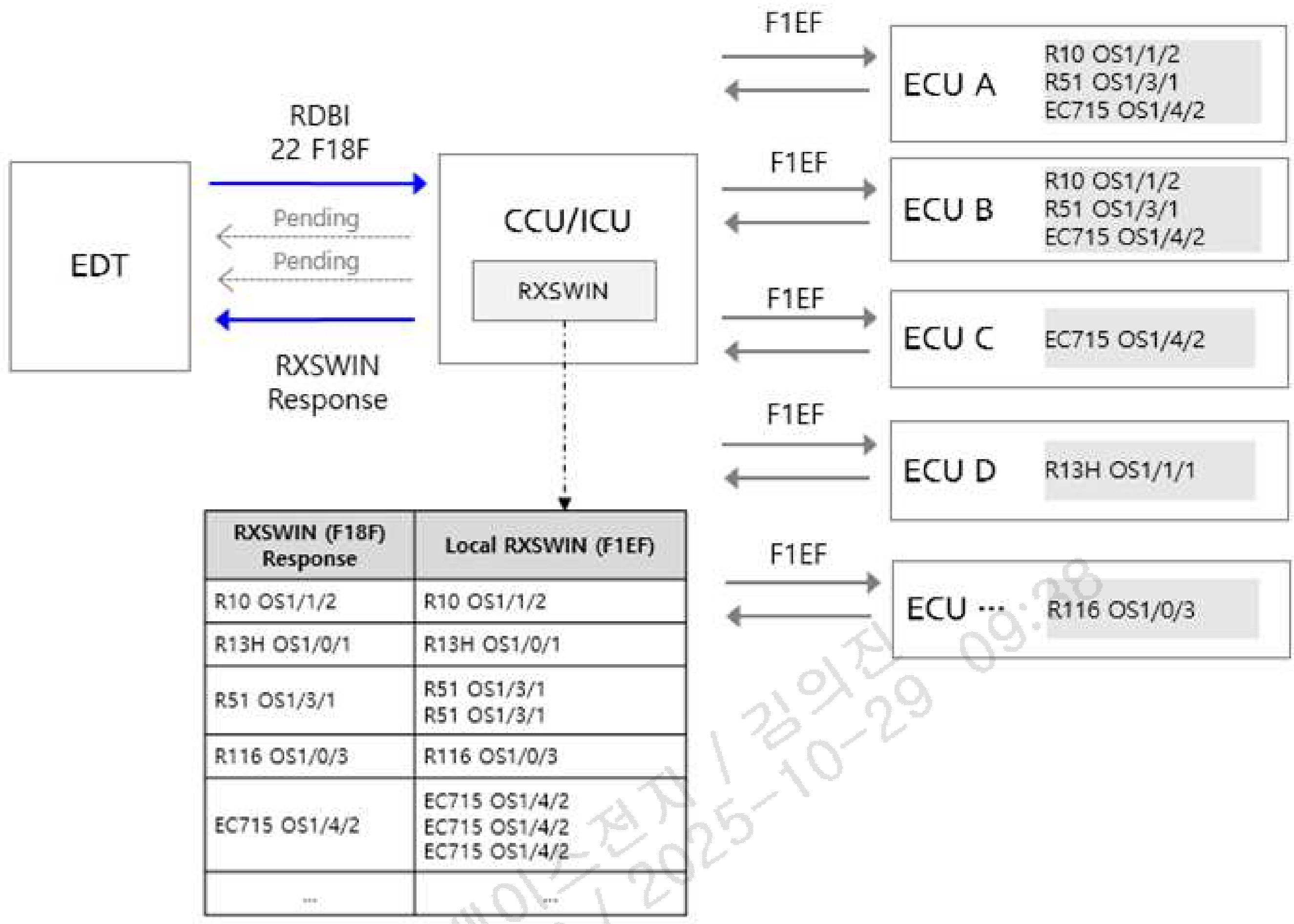


Figure J-7. RXSWIN Overview

Hex	Description	Cvt	Mnemonic
F18F	**RXSWIN** RXSWIN is information that combines local RXSWIN information into one RXSWIN.	C1	
F1EF	**Local RXSWIN** RXSWIN information which is in a ECU.	C2	
C1: Only the central gateway shall support this DID. C2: All ECUs related to RXSWIN shall support this Identifier.			

Table J-1. RXSWIN Identification

J.2 RXSWIN 요청 응답 시퀀스

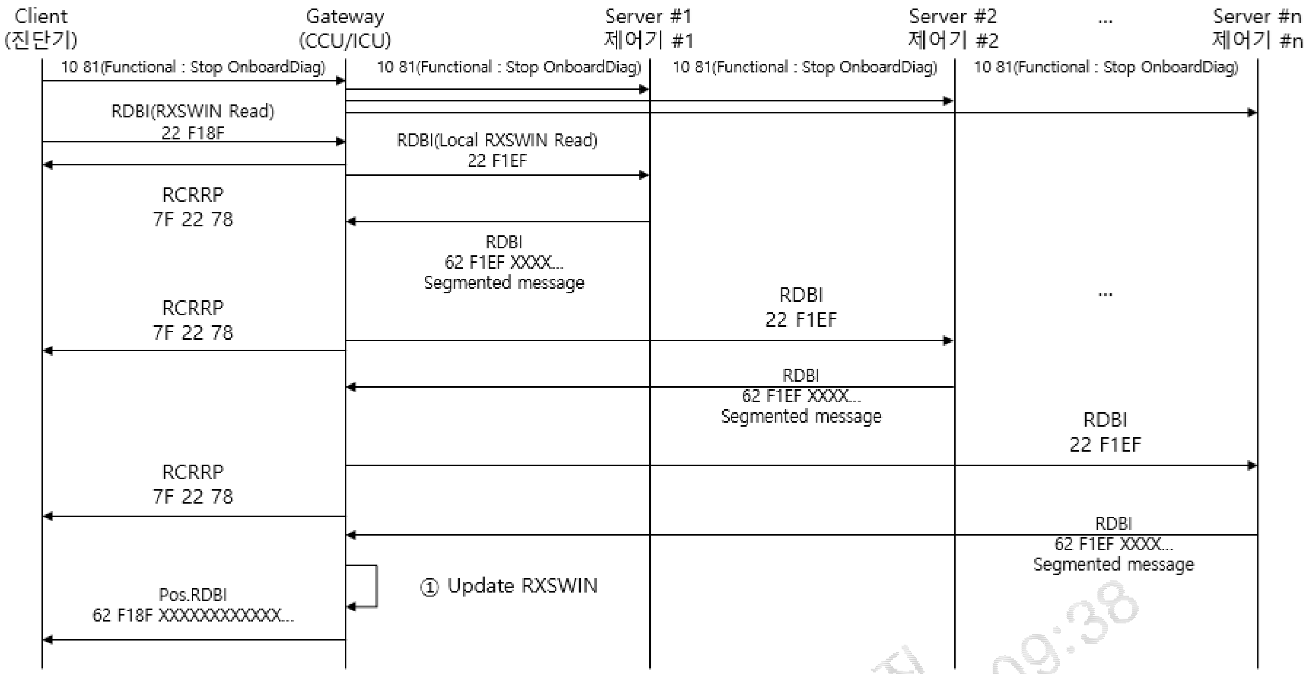


Figure J-8. RXSWIN gathering and read sequence caused by RDBI(F18F)

Exception : 10 81(Functional:Stop OnboardDiag) Although Functional addressing 10 81 causes Stop onboard diagnostics, Gateway can gather Local RXSWIN

*OnBoardDiag : Diagnostics from ECU to ECU (Client : ECU, Server : ECU)

When the diagnosis device requests RXSWIN in the IGN off state, Gateway does not request LocalRXSWIN but responds by referring to the RXSWIN table updated previously.

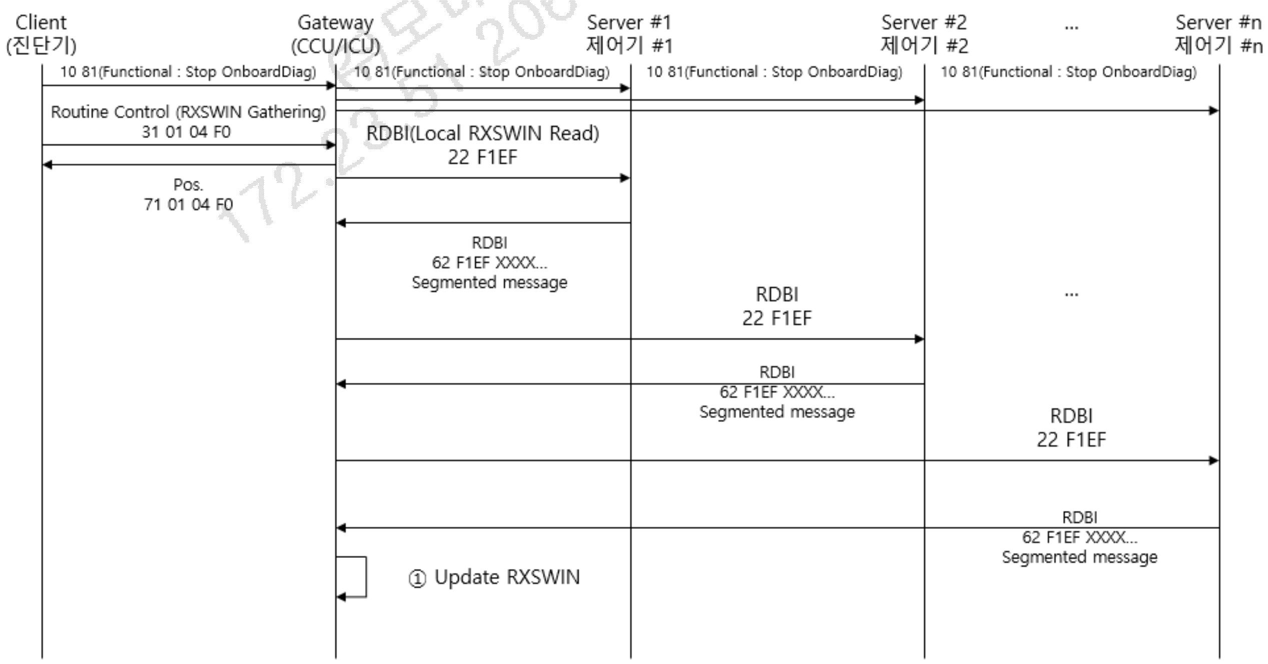


Figure J-9. Start RXSWIN gathering – routine control

The gateway can perform the gathering and reading separately. The gathering can be started by routine control($31). After the gateway finish gathering RXSWIN, RXSWIN shall be updated in NVM.

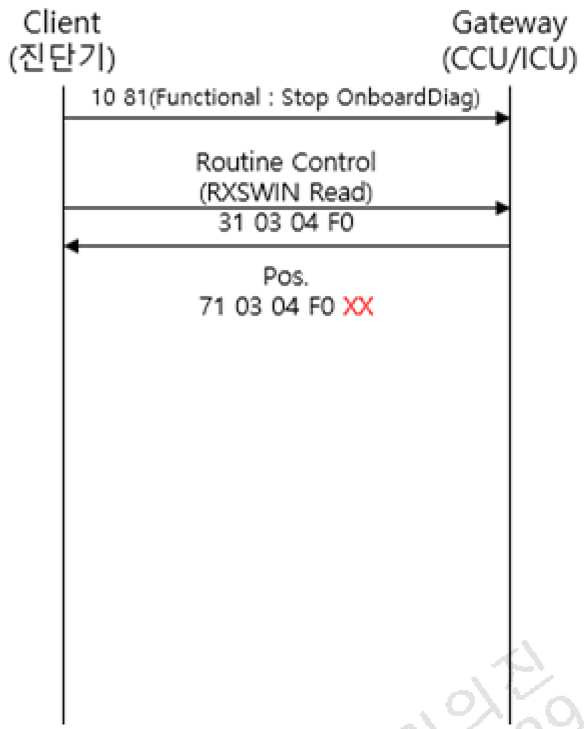


Figure J-10. Request routine result - Routine Control

Value	Routine Result	Description
0	Done	Routine has been finished correctly – RXSWIN gathering has been done
1	Collecting	Routine is running – Central Gateway is gathering RXSWIN
2	Aborted	Because of IGN off or other problems, Gathering has been aborted.

Table J-6. RXSWIN Routine result

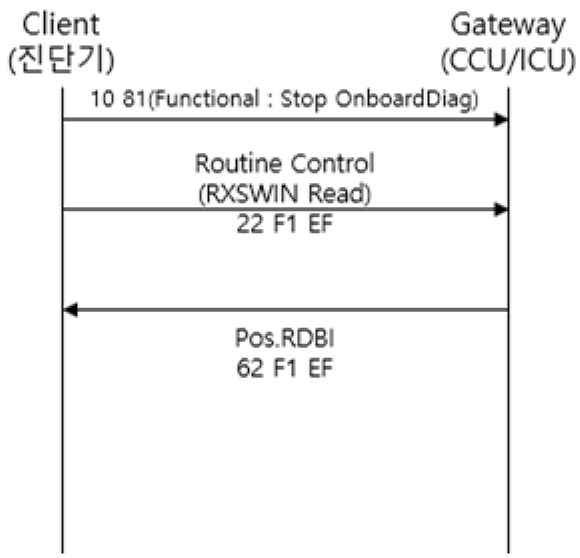


Figure J-11. Read local RXSWIN from the central gateway

If diagnostic tester requests RDBI with F1EF(local RXSWIN) to the central gateway, the central gateway doesn't gather RXSWIN and respond positive with already gathered RXSWIN which is stored in NVM.

If RXSWIN never has been gathered, the central gateway respond positive with only gateway own RXSWIN.

If the central gateway received RDBI with F1EF or F18F during the gathering RXSWIN by routine control, the central gateway shall respond positive with already gathered RXSWIN which is stored in NVM. And it doesn't affect gathering RXSWIN.

J.3 Local RXSWIN gathering example for master – slave ECU

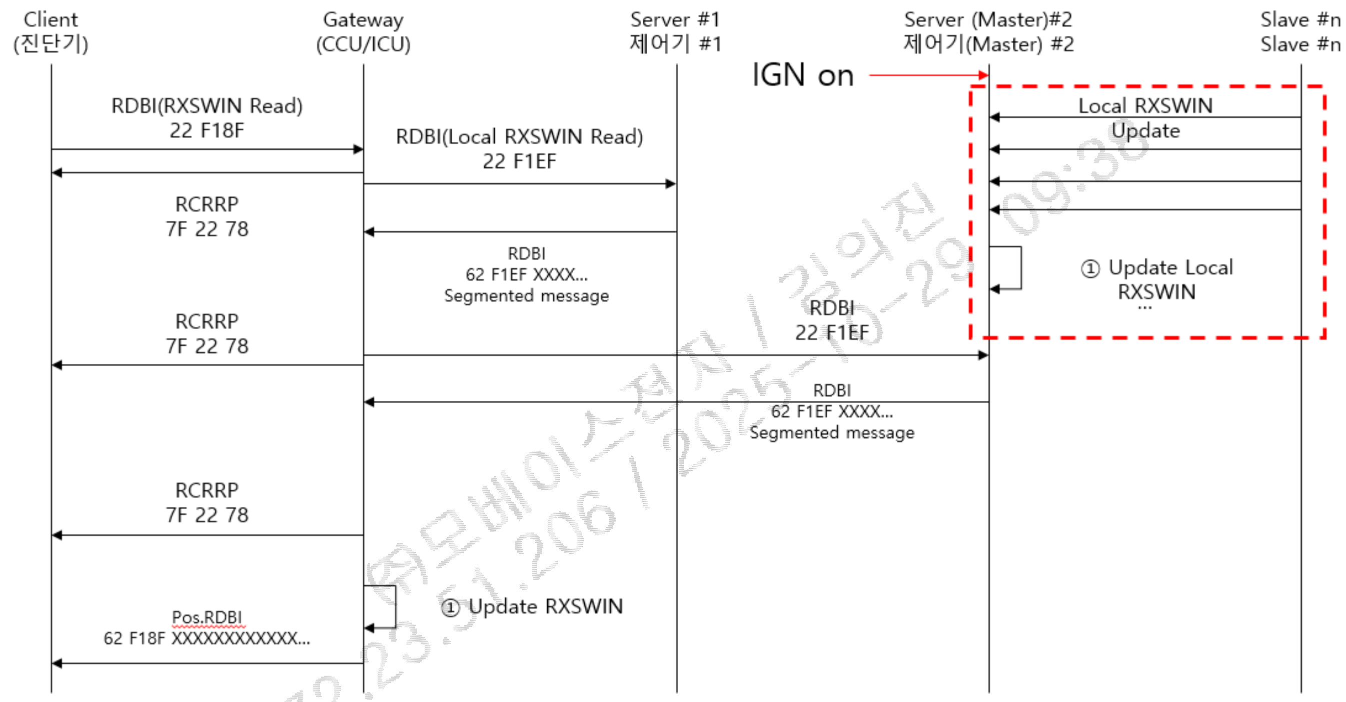


Figure J-12. Example of RXSWIN gathering – Master/Slave ECU

Master –Slave ECUs could update local rxwin after IGN on.

But This is not mandatory update sequence. Because The exchange of Rxswin between Master Slave is not scope of the Diagnostic specification.

J.4 Local RXSWIN Request Response format

Gate way Req.	HEX	22	F1	EF																											
	Description	SID	DID																												
ECU Resp.	HEX	62	F1	EF	XX	XX	0C	52	31	35	36	20	44	4E	38	2F	31	2F	31	0B	52	34	38	20	44	4E	38	2F	31	2F	30
	Description	SID	DID		REQ CAN ID		Length	RXSWIN ASCII STRING												Length	RXSWIN ASCII STRING										
								R	1	5	6		D	N	8	/	1	/	1		R	8	3		D	N	8	/	1	/	0

Table J-2. Example of Local RXSWIN Request Response format

IMPORTANT : XX XX is Request CAN ID to recognize where the response came from on the Gateway

Application layer

REQ CAN ID Example)
Diagnostic Request CAN ID 0x700
Diagnostic Response CAN ID :0x708
REQ CAN ID value shall be 0x700

REQ CAN ID Example2)
Diagnostic Request CAN ID 0x17FFCA00
Diagnostic Response CAN ID : 17FF00CA
REQ CAN ID value shall be 0x50CA (See also chapter 2.4.1 addressing. For RXSWIN, 29bit to 2byte CAN ID conversion is comply with DoIP logical address conversion rule even if the server uses ethdiag protocol

Length Example1)
R156 DN8/1/1
The length of RXSWIN string including space is 12bytes
Length value shall be 0x0C (Decimal 12)

Length Example2)
R83 DN8/1/1
The length of RXSWIN string is 11bytes
Length field shall be 0x0B (Decimal 11)

RXSWIN		Length	Reg. Authority	Reg. No.	Space	Vehicle Project	/Version	/Extension	RXSWIN Result (Length)String
Ex1)	RXSWIN Initial	0x13	EC	1008/2010	" "	CN7	/1	/0	(0x13)EC1008/2010 CN7/1/0
	RXSWIN Extension	0x13	EC	1008/2010	" "	CN7	/1	/1	(0x13)EC1008/2010 CN7/1/1
Ex2)	Type Gasoline	0x0B	R	83	" "	DN8	/1	/0	(0x0B)R83 DN8/1/0
	Type Diesel	0x0B	R	83	" "	DN8	/2	/0	(0x0B)R83 DN8/2/0

Table J-3. RXSWIN structure

J.5 RXSWIN Update Example

As is (RXSWWIN table – Central Gateway)

List No.	RXSWIN STRING														
0	R	1	5	6		D	N	8	/	1	/	1			
1	R	8	3		D	N	8	/	1	/	0				
2															
…															

Example) Local RXSWIN Response 1

	HEX	22	F1	EF

Gateway Req.	Description	SID	DID																													
ECU Resp.	HEX	62	F1	EF	07	00	0C	52	31	35	36	20	44	4E	38	2F	31	2F	31	0B	52	34	38	20	44	4E	38	2F	31	2F	30
	Description	SID	DID		REQ CAN ID		Length	RXSWIN ASCII STRING												Length	RXSWIN ASCII STRING											
								R	1	5	6		D	N	8	/	1	/	1		R	8	3		D	N	8	/	1	/	0

Example) Local RXSWIN Response 2

Gateway Req.	HEX	22	F1	EF																												
	Description	SID	DID																													
ECU Resp.	HEX	62	F1	EF	07	00	0C	52	31	35	36	20	44	4E	38	2F	31	2F	32	0B	52	34	38	20	44	4E	38	2F	31	2F	30
	Description	SID	DID		REQ CAN ID		Length	RXSWIN ASCII STRING												Length	RXSWIN ASCII STRING											
								R	1	5	6		D	N	8	/	1	/	2		R	8	3		D	N	8	/	1	/	0

To be (RXSWWIN table – Central Gateway)

List No.	RXSWIN STRING													
0	R	1	5	6		D	N	8	/	1	/	2		
1	R	8	3		D	N	8	/	1	/	0			
2														
…														

Example) RXSWIN Response (EDT-Central Gateway)

EDT Request	HEX	22	F1	8F																										
	Description	SID	DID																											
ECU Resp.	HEX	62	F1	8F	0C	52	31	35	36	20	44	4E	38	2F	31	2F	32	0B	52	34	38	20	44	4E	38	2F	31	2F	30
	Description	SID	DID		Length	RXSWIN ASCII STRING												Length	RXSWIN ASCII STRING											
						R	1	5	6		D	N	8	/	1	/	2		R	8	3		D	N	8	/	1	/	0

The gateway shall store the RXSWIN table in memory. The Local RXSWIN received from the ECU shall be updated to the latest value after removing redundancy.

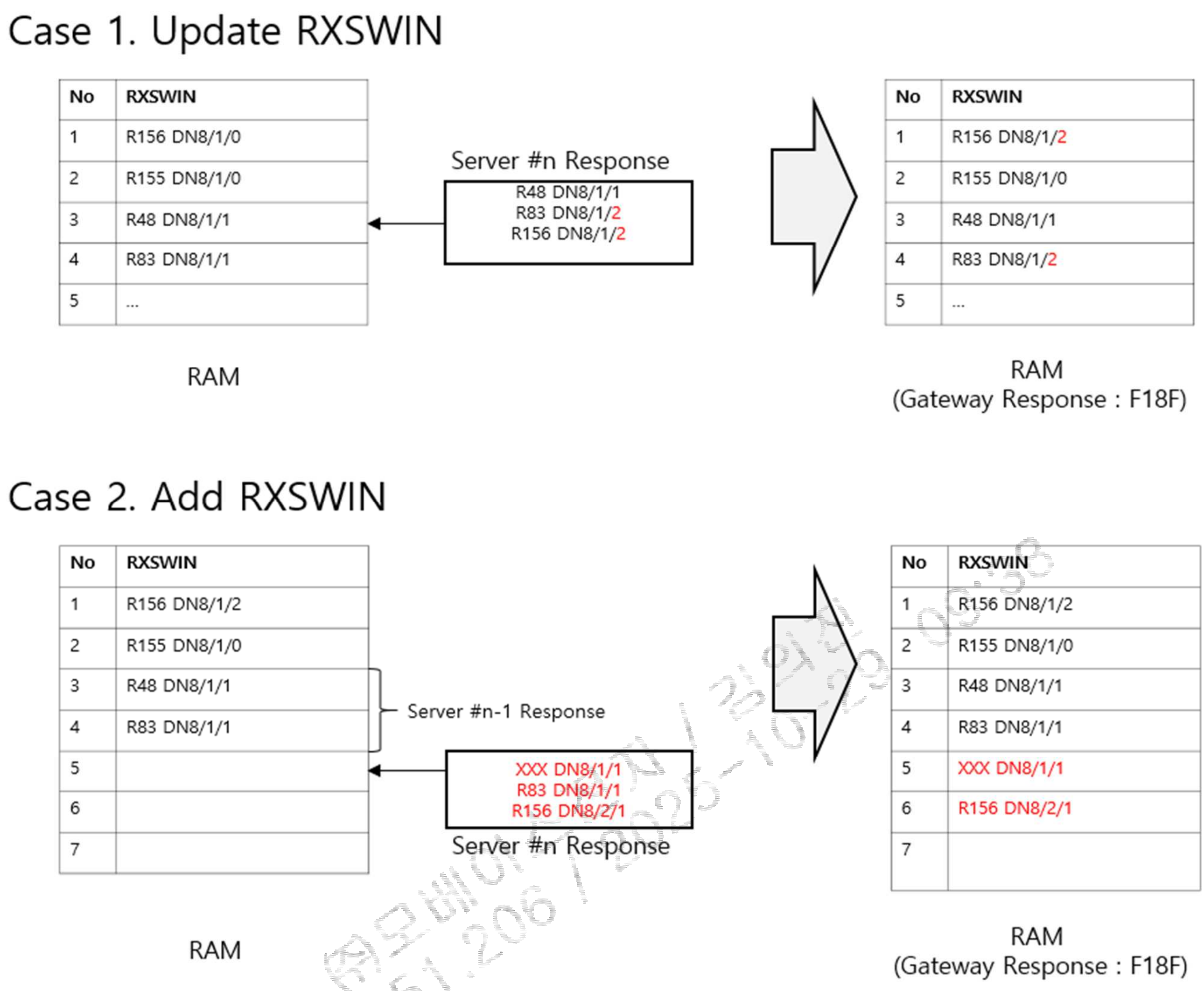


Figure J-13. RXSWIN memory handling in the Central gateway-1

NOTE : Each case is not sequential.

This is a example to show how to gathering RXSWIN on the central gateway side.

The meaning of Gateway Response in the picture is that Response of RXSWIN for DID F18F is made from the RAM which is result of gathering local RXSWIN.

If one or more fields (Reg. Authority / Reg.No / Project / Version), which are the local RXSWIN of server #n, are not the same as the gathered RXSWIN fields, RXSWIN shall be added. If each field between the gathered RXSWIN and the local RXSWIN response is the same except the extension field and the local RXSWIN extension value is greater than the gathered RXSWIN, the extension shall be updated.

Under the following conditions, the central gateway responds to the RXSWIN stored on the NVM.Under the following conditions, The central gateway respond RXSWIN stored in the NVM :

1) IGN off
2) DID is F1EF

Case 3. Finalize RXSWIN gathering

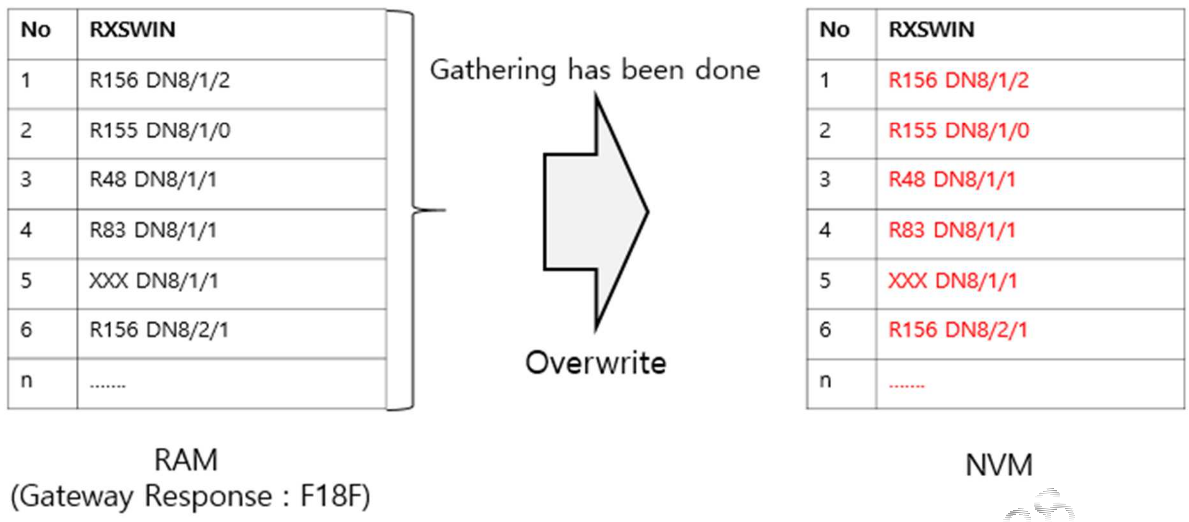


Figure J-14. RXSWIN memory handling in the Central gateway-2

After finish gathering RXSWIN to all servers, The gateway stores RXSWIN(RAM) in the NVM(overwrite).

J.6 RXSWIN Session Timer

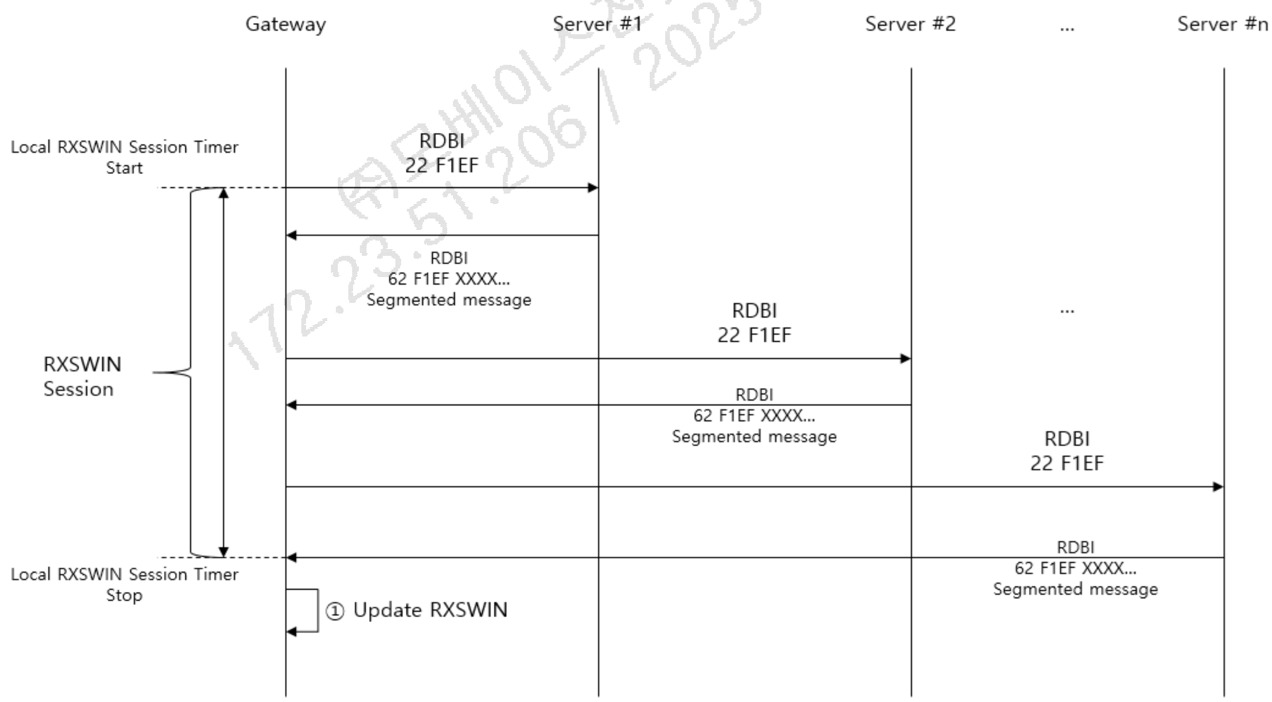


Figure J-15. RXSWIN Session timer sequence diagram

Timing parameter	Action	Condition
Local RXSWIN Session	Start	Gateway starts request of LocalRXSWIN

Local RXSWIN Session	Stop	The end of LocalRXSWIN response from All ECU which has to respond to the gateway.
		60seconds(+-3sec) elapsed from Timer start. (If the 60-second timer has been terminated but Gateway has not completed the RXSWIN gathering, Gateway shall stop gathering RXSWIN.)

Table J-4 RXSWIN Session

J.7 Timing Parameter and Retry logic

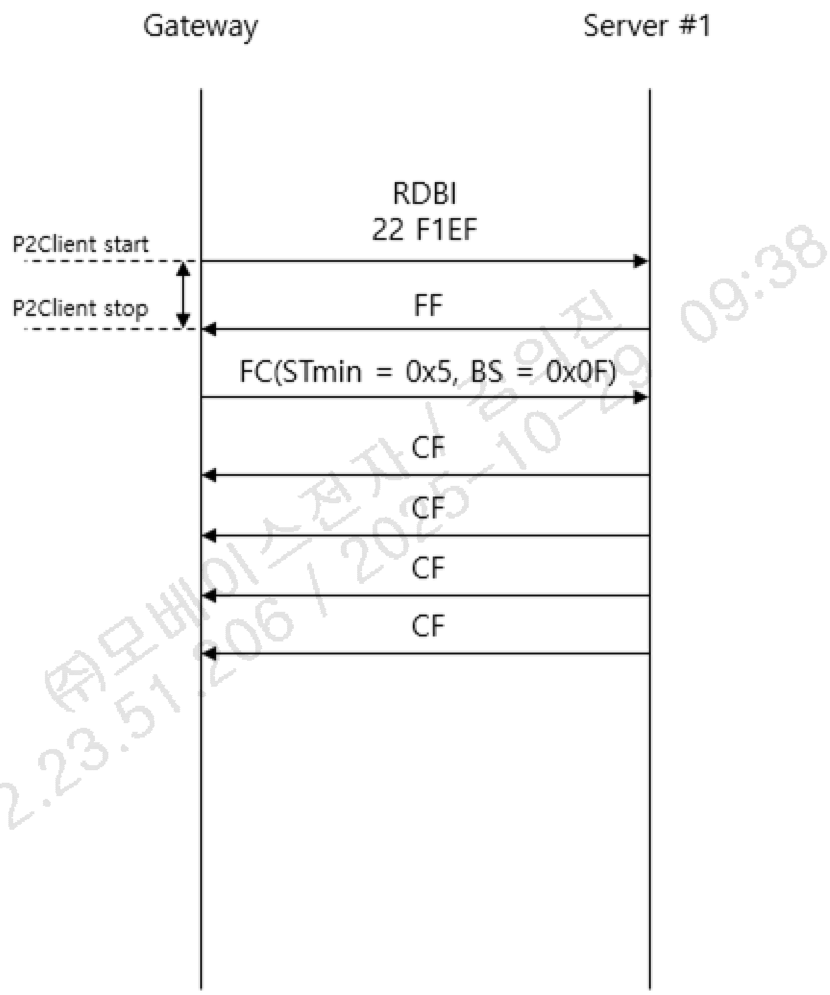


Figure J-16. RXSWIN Timing parameter diagram

Parameter	Time	Description
P2Server	50ms	ECU(Server) shall respond within 50ms
P2Client	200ms	Gateway shall await response from ECU(Server). If there is no response within P2Client, Gateway shall abort awaiting and send request to the next server(e.g. Server#2)

Table J-5 RXSWIN Timing parameter

Server shall not send RCRRP(requestCorrectlyReceivedResponsePending 0x78) for the RDBI request.

If there was P2 Client timeout or error, Gateway shall retry gathering RXSWIN after all request to the ECU has been completed.

e.g.) If there is no response from Server #3, Gateway shall proceed with RXSWIN request to the next server (Server #4).

After Gateway complete gathering RXSWIN from the last server (Server#n), Gateway can retry to gather RXSWIN for failed Server#3

Gateway shall not retry gathering RXSWIN if gateway receives NRC(Except Pending) from the Server(ECU).

When the gateway retries gathering RXSWIN, P2Client is constantly 200ms.

J.8 Error handling

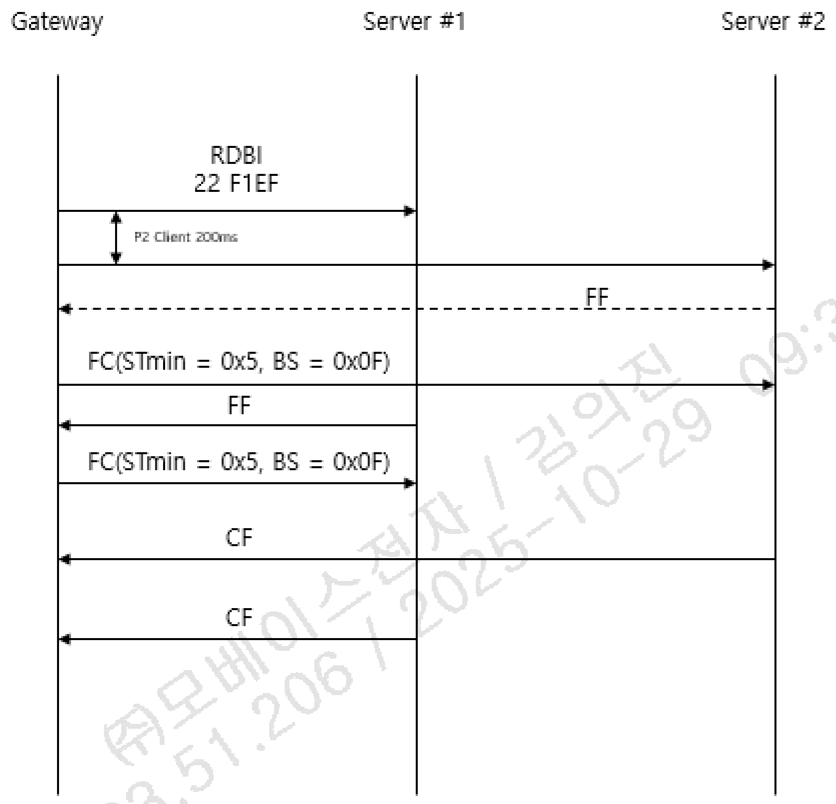


Figure J-17. RXSWIN error handling-1

In case Server #1 which is timedout responds to RDBI while Server #2 is sending RXSWIN, Gateway shall discard each response sent from Server#1 Server#2. And a retry is also required according to the J.7 Timing parameter and Retry logic.

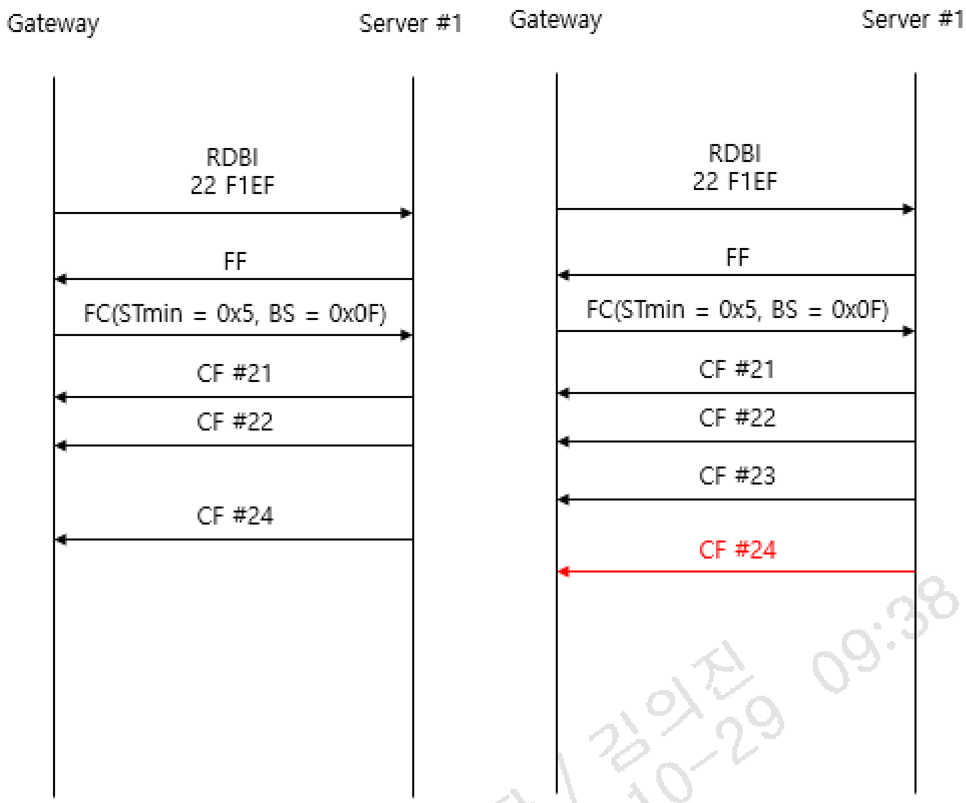


Figure J-18. RXSWIN error handling-2

In case a CF omission occurs, Gateway discard all responses. And a retry is also required according to the J.7 Timing parameter and Retry logic.

N_CR timeout is 200ms.

In case expected CF is not sent from Server(ECU) within 200ms(N_Cr), Gateway discard all responses. And a retry is also required according to the J.7 Timing parameter and Retry logic.

J.9 Addressing and DID requirement

Only a central gateway can have DID F18F.

According to the response behavior, ECU except central gateway shall respond with NRC31 in case client requests DID F18F with physical addressing.

According to the response behavior, No response is required in case client requests DID F18F with functional addressing to the ECU except central gateway.

Annex. K IVD Requirements

K.1 IVD 요구사항

aSIMS (또는 FST)를 활용하여 Secure Flash 펌웨어를 생성하는 제어기는 펌웨어 생성 시 별도로 제공되는 Hash 를 IVD 로 사용한다.(알고리즘 : SHA256)

IVD 계산 영역은 전체영역(전자서명 제외) 로 한다.

aSIMS (또는 FST)를 활용하지 않거나 별도의 IVD 계산 알고리즘을 갖는 제어기의 경우는 다음의 사항을 만족해야 한다. :

- 계산 알고리즘은 SHA256 으로 한다.
- SHA384, SHA512, SHA3 시리즈 사용시 허용되지만 HMC 에 통보 해야한다.
- 계산 영역은 전체 영역으로 하되 불가피한 경우, 형식 승인에 영향을 받지 않는 영역은 제외 가능

하다.

K.2 Hash 값 제공 및 검증 방식

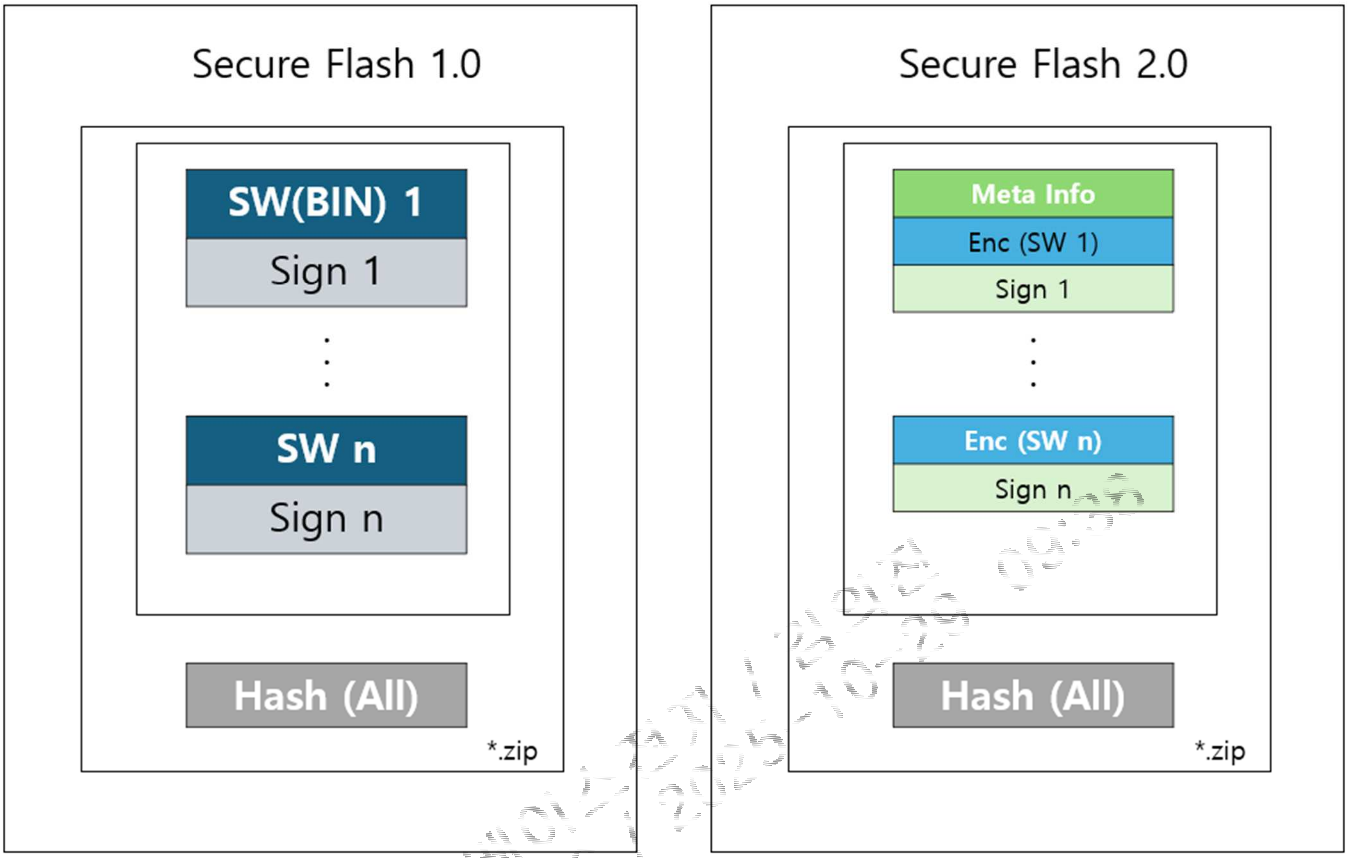


Figure K-1. Secure flash hash 값 제공

※ Max n =8 (aSIMS) or 10(FST)
※ SecureFlash 2.0 의 Signature 는 평문 펌웨어로 생성.
※ SUMS 활용을 위해 별도 Hash 제공.
※ Hash(all)을 제외한 영역이 Secure Flash 1.0/2.0 용 펌웨어 형태로 제어기에 전송되어 리프로그램 되어야 함

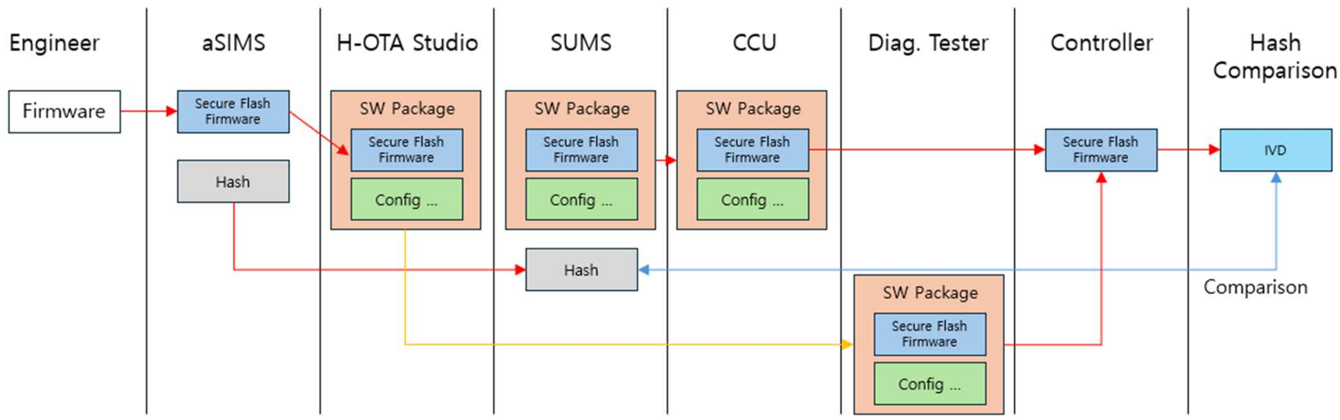


Figure K-2. Hash-IVD 검증 방법